A Type Theory of Certifiable Neural Architectures: Sharp LUT Bounds, Complexity Separation, and Compile-Aware Generalization, with an Industrial PLC Instantiation

Fuyue Liu, *Guilin University of Electronic Technology, Guilin 541004, China*

Abstract

We present a **type theory of certifiable neural architectures**: a formal characterization of which architectures admit design-time correctness certification under LUT-discretized compilation, and why. Four results anchor the theory. First, the **sharp k-th order LUT bounds** (Theorem 9): the classical de Boor constant $M_2h^2/8$ generalizes to all interpolation orders with explicit sharp constants $c_k = Q_k/k!$ and an extremal family attaining them *exactly*—upgrading the DA optimality claim from an empirical statement on random cubics to an exact minimax-optimality theorem with a certificate. Second, the **verifiability trichotomy** (Theorem 10): the SVNN condition is *conjecturally* conjecturally the unique regime where second-order rate, linear LUT cost, closed-form sharp constants, and freedom from reachability computation coexist—ReLU architectures admit only first-order bounds, multivariate gates suffer N^m LUT cost and NP-hard tightness, and domain-dependent curvature requires reachability. Third, **ε-separation** (Theorem 5): the LUT-verification decision problem is polynomial for SVNN (self-contained 3-SAT reduction showing NP-hardness for coupled architectures), and SVNN uniquely attains the Pareto point (polynomial-time verification, zero slack under ETH); the completeness of this boundary is the trichotomy necessity conjecture. Fourth, the **compile-aware generalization bound** (Theorem 6): the deployed (compiled) risk decomposes into a generalization gap $O(\gamma^{L-1}/\sqrt{n})$ and a LUT bias $O(\gamma^{L-1}c_kM_kh^k)$, yielding the *resolution-matching law* $N^* \asymp n^{1/(2k)}$—the deployed LUT memory should scale as the $2k$-th root of the sample count.

The four results are not an assemblage of independent bounds: they are the expansion of a single unifying quantity, the **verifiable deployment capacity** $C(N, V)$ (§III-S)—the best certifiable deployment error under a storage budget of N cells and a certificate-verification budget V. A universal packing line bounds every scheme; three verification strata (closed-form / kinked / structure-optimal) place every architecture; and a **certificate proof system** CP—following the proof-carrying-code and validated-numerics tradition (Gappa, Sollya, LeanCert)—formalizes the verification budget as proof length under perfect deterministic soundness. Within this framework the sharp constants reappear as the capacity limit of the interpolation code, the trichotomy reappears as three points in the (V_B, ε) plane, and the necessity direction receives a theorem-level information-theoretic form; an architecture-as-code law decides between table and spectral codes from the smoothness of the target.

We instantiate this theory in NeuroPLC, the first self-verifying PyTorch-to-IEC 61131-3 compiler (formally verified templates, certificates, and composition rules; the emitter is validated by differential self-testing), with four-tier verification: Z3 template proofs, 512/512 per-function certificates, a machine-checkable composition certificate, and a *differential self-test* that catches code-generation regressions (it detected a real emitter bug and validated the fix; the bug had degraded deployed accuracy to 83%). Generated code compiles to 0 errors/0 warnings in TIA Portal V21 with a vendor-neutral IEC 61131-3 variant, and the runtime hot-swap protocol maintains the certified safety state across model updates. Deployment-level certificates close the loop: adaptive LUT allocation lifts the DA safety factor to 3.6× at unchanged storage, and soft-contractive training to a first-principles *sound* certificate of 2.34× (pure propagation, no empirical floor; IEEE-754 $\delta_{\text{fp32}} \lesssim 3 \times 10^{-5}$) / 493× expected, with an empirically validated 11.6× float32 tier (measured floor 0.058 vs. measured maxAE 0.053) and 99.9% Box-Continuation coverage, Z3-verified components, and depth extension to $L = 3$ (6.1×). Four C^2-BV architectures (B-spline, Chebyshev, Fourier, Wavelet) and three datasets (CWRU, XJTU-SY, Paderborn) validate the framework, with MNIST as a fourth dataset (E42); RBF-KAN remains theoretical. The theory answers not "can we compile neural networks to embedded targets"—prior tools showed that—but "which architectures admit *certified* compilation, with what sharp constants, at what complexity, and with what generalization guarantee?"

Index Terms

Neural Architecture Certification, Sharp Interpolation Constants, Verifiability Trichotomy, Complexity Separation, Compile-Aware Generalization, Resolution Matching, Galois Connection, IEC 61131-3, Kolmogorov-Arnold Networks, Type Theory.

I. Introduction

This paper introduces a **type theory for certifiable neural architectures**—a formal characterization of which architectures admit design-time correctness certification under LUT-discretized compilation, with what sharp constants, at what computational cost, and with what learning-theoretic guarantee. The central insight is that the question "can a neural network compiler provide design-time correctness guarantees?" is, at its core, a **type-theoretic question**: the answer depends on whether the architecture is *well-typed* under a type system whose rules encode the structural preconditions for certifiable compilation.

We define the **Structurally Verifiable Neural Network (SVNN)** type system. Its types classify operations as purely linear (`linear`), purely element-wise (`elemwise`), or SVNN-composite (`svnn`). Its typing rules enforce two conditions: operation-type closure (Condition 1: linear and element-wise operations are never coupled in a single graph node) and C^2 boundedness

(Condition 2: every element-wise function has a computable curvature bound). The **SVNN Sufficiency Theorem** (Theorem 2) guarantees that well-typed architectures have computable, finite per-output error bounds under LUT-discretized compilation—a property that ill-typed architectures (standard MLPs with $\sigma(W\mathbf{x}+\mathbf{b})$) provably lack (Proposition 2). The typing is *numerical* (it certifies error bounds under discrete execution), complementary to categorical constructions that embed logical constraints into architecture [1]: there, well-typedness rules out logical violations; here, it rules out unbounded discretization error.

The SVNN type system is distinguished by three properties:

(i) **Completeness (Theorem A):** Within affine abstract interpretation, SVNN well-typedness is *necessary and sufficient* for dimension-optimal certification. A sharp adversarial MLP ($W = (1/\sqrt{d}) \cdot J_d$) achieves $\|W\|_{1,\infty} = \sqrt{d}$, against KAN's d-independent $\gamma < 1$ (per-layer ratio: 22× at d=16, 62× at d=128). The Compilable Frontier is a mathematical boundary—not an implementation choice.

(ii) **Optimality (Corollary 3 + Theorem B + Theorem C):** DA is the **Galois-optimal** abstract domain for C^2 univariate functions—the first Galois connection for neural network activations— and its bound $M_2h^2/8$ is **semantically tight** (attained by $f^*(x) = M_2x^2/2$ at every LUT midpoint, $1{,}000/1{,}000$ exact), with the sharp-constant generalization to all orders (Theorem 9) attained exactly by an explicit extremal family (E63).

(iii) **Compositionality (Theorem 8).** The SVNN type system forms an **algebraic monoid** under serial composition, enabling modular certification without re-analysis across independently-certified subnetworks.

This paper goes beyond sufficiency to establish the **exact boundaries and quantitative laws** of certifiable compilation:

(i) **Sharp constants (Theorem 9).** The de Boor bound $M_2h^2/8$ is the $k = 2$ instance of a sharp all-orders family: $c_k = Q_k/k!$ with explicit extremal functions attaining the bound *exactly* ($k = 3$: $1/(9\sqrt{3})$, not the folklore $1/8$; $k = 4$: $1/24$). DA's optimality is exact, not asymptotic—verified at machine precision.

(ii) **The verifiability trichotomy (Theorem 10).** The SVNN condition is not one sufficient criterion among many: it is *conjecturally* the unique regime where second-order rate, linear LUT cost, closed-form sharp constants, and reachability-freedom coexist (the necessity direction of the theorem is a conjecture with strong evidence). ReLU admits only first-order bounds; multivariate gates (products, softmax) suffer N^m LUT cost and NP-hard tightness; domain-dependent curvature (e^x) requires reachability.

(iii) **ε-separation (Theorem 5).** The LUT-verification *decision problem* is polynomial for SVNN and NP-hard for coupled architectures (self-contained 3-SAT reduction via product gates); SVNN uniquely attains (polynomial time, zero slack under ETH) simultaneously (completeness of this boundary is the trichotomy necessity conjecture).

(iv) **Compile-aware generalization (Theorem 6).** The deployed risk decomposes as $\hat{R}(N) + O(\gamma^{L-1}/\sqrt{n}) + O(\gamma^{L-1}c_kM_kh^k)$, yielding the **resolution-matching law** $N^* \asymp n^{1/(2k)}$: deployed LUT memory should scale as the $2k$-th root of the sample count—a directly actionable compiler rule no existing PAC bound states.

(v) **Verifiable deployment capacity (Theorem 17, §III-S).** The four results above are unified as the expansion of a single quantity: the capacity $C(N, V)$ of certifiable deployment under a storage budget of N cells and a certificate-verification budget V. A universal packing line (metric entropy) bounds every scheme; three verification strata—closed-form (certificate-free), kinked (rate-degraded), and structure-optimal (verification-exponential)—place every architecture class; and a certificate proof system (following Gappa/Sollya/LeanCert and PCC) formalizes V as proof length under perfect deterministic soundness, yielding a theorem-level information-theoretic necessity within the calculus. An architecture-as-code law (table vs. spectral codes) follows from the smoothness of the target.

We give the NeuroPLC IR a **formal operational semantics** and prove **Type Soundness** (Theorem D): compiler correctness (Theorem 1) is equivalent to the statement that well-typed IR programs compile to SCL with bounded semantic deviation. For industrial deployment, we prove a **Non-Interference Theorem** (Theorem E): inserting a NeuroPLC inference block into a certified PLC program preserves all pre-existing safety properties under the timing premise $\mathrm{WCET}(P) + \mathrm{WCET}(N) \leq$ scan cycle, directly addressing IEC 61508 independence requirements—the first such guarantee for neural-network-on-PLC deployment. An IEC 61131-3 REAL arithmetic semantics (Lemma 7) extends the SVNN guarantee to any IEC-compliant controller worldwide—replacing per-platform validation with a one-time arithmetic-standard-based proof. The compiler provides a parameterized WCET guarantee (Theorem 11: 3.86 ms, 25.9× scan-cycle margin) and auto-generates a companion safety monitor (Algorithm 3).

The industrial instantiation of this type theory targets **programmable logic controllers (PLCs)—**the workhorses of factory automation—which execute IEC 61131-3 Structured Control Language (SCL) in deterministic scan cycles with severe memory constraints (50 KB for Siemens S7-1200 CPU 1211C). Existing ML→PLC tools (RTNNIgen [2], MLconverter [3], ICSML [4]) provide only empirical validation; general-purpose compilers cannot target IEC 61131-3.

We instantiate the SVNN type system in NeuroPLC,, to our knowledge, the first IR-based compiler from PyTorch to Siemens S7-1200/1500 SCL. The compiler is the *proof of concept* for the type theory—not its primary contribution. Its 6-op IR has a formal syntax, operational semantics, and typing rules (§III-X). Three verification mechanisms compose: (1) one-time Z3 SMT template proofs; (2) compositional per-function certificates (512/512 verified); and (3) DA—the optimal first-order abstract domain for C^2 functions—achieving 2.1× tighter error bounds than interval arithmetic (1.0× safety factor at the M_2^{char} calibration, up to 3.1× vs. the $\sqrt{d}$ random-weights baseline, §IV-B9). KAN's B-spline decomposition is *a structural necessity*: identically-sized KAN vs. MLP yield 512/512 vs. 0/16 Z3-verifiable components (MLP $[28, 16, 4]$, E41).

Prior ML→PLC tools established that compilation is *possible* but did not address whether it can be *certified*. RTNNIgen [2] converts Keras models to IEC 61131-3 Structured Text; MLconverter [3] targets scikit-learn for ABB controllers. Both provide only empirical test-set validation—a correctness model inherited from ML benchmarking that is insufficient for safety-critical PLC deployment. The sole prior KAN-on-PLC demonstration (Corrêa [5], USP 2024) relied on distributed PC-to-PLC communication via Snap7 rather than native SCL compilation, and provided no correctness guarantees. The gap these tools leave is not an implementation gap—it is an *architectural* gap. Our central finding is that no amount of compiler engineering can close it for architectures violating the SVNN conditions (Theorem 5). NeuroPLC instantiates the theory as, to our knowledge, the first self-verifying PyTorch-to-IEC 61131-3 compiler, demonstrating that the certification transfers across vendors (a vendor-neutral IEC 61131-3 backend with identical semantics) and across model updates (Algorithm 4).

Scope. NeuroPLC compiles the classifier (KAN/MLP) to SCL. Frequency-domain features are assumed pre-extracted via DSP and transmitted over PROFINET/OPC UA. An on-PLC time-domain subset is also compiled (E51). This paper contributes:

1) **Theory (Type-Theoretic Foundation):** The SVNN framework constitutes a **type system** for neural computation graphs (§III-X), whose soundness (Theorem 2) guarantees that well-typed architectures have computable error bounds. The IR is given a formal operational semantics and typing rules; compiler correctness (Theorem 1) is recast as **Type Soundness** (Theorem D). The **Compilable Frontier Characterization** (Theorem A) proves SVNN Conditions 1–2 are *necessary and sufficient* for dimension-optimal affine certification ($O(d \cdot r^2)$ per layer), while ill-typed architectures degrade to $\Omega(d^2 \cdot r^2)$—a $\Theta(d)$ gap verified on networks up to $d = 128$ (MATLAB). **Sharp k-th Order LUT Bounds** (Theorem 9) generalize the de Boor constant to all orders with exact sharp constants $c_k = Q_k/k!$ and an extremal family attaining them at machine precision (superseding the earlier cubic-restricted, Monte-Carlo-verified DA optimality claim). **SVNN Compositionality** (Theorem 8) proves the SVNN class forms an algebraic monoid, enabling modular certification.
2) **Theory (Abstract Interpretation Foundation): DA Galois Connection** (Theorem B) proves that Doubleton Arithmetic forms a canonical instance of the Cousot abstract interpretation framework with a defined Galois connection (α, γ), making NeuroPLC, to our knowledge, the first neural network compiler to be formally grounded in abstract interpretation theory. **DA Tightness** (Theorem C) proves the DA bound is *semantically tight*—the de Boor bound $M_2 \cdot h^2/8$ is *attained* by $f^*(x) = M_2/2 \cdot x^2$ at every LUT interval midpoint (1,000/1,000 exact, MATLAB-verified). **Universal IEC 61131-3 Guarantee** (Lemma 7) extends the SVNN certification from Siemens TIA Portal to *any* IEC 61131-3:2025 compliant controller worldwide, using the standard's IEEE 754 normative reference.
3) **Theory (Deployment Safety):** The **Non-Interference Theorem** (Theorem E) proves that inserting a NeuroPLC inference block into a certified PLC program preserves all pre-existing safety properties under the timing premise $\mathrm{WCET}(P) + \mathrm{WCET}(N) \leq$ scan cycle—the first such guarantee for neural-network-on-PLC deployment, directly addressing IEC 61508 independence requirements. The **WCET Theorem** (Theorem 11) provides parameterized worst-case execution time bounds (3.86 ms for KAN $[28, 16, 4]$, $25.9\times$ scan-cycle margin).
4) **System (Compiler):** NeuroPLC, the first PyTorch-to-Siemens SCL compiler, uses a 6-op IR matching SVNN decomposition of KAN (§IV-B1). Generated SCL compiles with **0e 0w** in TIA Portal V21 on S7-1200 and S7-1500 (§IV-D).
5) **Algorithm (Verification & Optimization):** Three algorithms: (i) **Sign-structural affine arithmetic (DA)**, achieving a $2.1\times$ tighter error bound than interval arithmetic ($3.1\times$ vs. the $\sqrt{d}$ random-weights baseline) by preserving weight-matrix sign structure (§IV-B9); (ii) **Segment-Aware de Boor Bounds**, exploiting the piecewise-linear structure of cubic B-spline second derivatives for substantial per-segment tightening (composing with DA for a $3.6\times$/$6.6\times$ safety-factor improvement at N=15/20 under the worst-function calibration, recomputed 2026-08-04); and (iii) **Adaptive Mixed-Precision LUT Allocation**, a greedy max-heap algorithm reducing worst-case LUT error by 71.6% at equal storage, with **Theorem 3** proving its minimax optimality via an exchange argument (§IV-B11).
6) **Theory (Completeness):** Beyond sufficiency, we establish **ε-separation** (Theorem 5, §III-G): the LUT-verification *decision problem* is polynomial for SVNN (with sharp constants, Theorem 9) and NP-hard for coupled architectures (self-contained 3-SAT reduction via product gates), and SVNN uniquely attains the Pareto point (polynomial time, zero slack under ETH; completeness of the boundary is the trichotomy necessity conjecture). Condition 3 yields a depth-improving **generalization bound** $\Delta_{\text{gen}} \leq O(\gamma^L/\sqrt{n})$ (Theorem 6, §III-H). The framework is extended to **ChebyKAN** (Chebyshev polynomial KANs): 496/512 Z3-verifiable via polynomial NRA, 100.0% CWRU accuracy (Proposition 3). **E56** (3-layer KAN: 608/608 Z3, 99.60% acc) and **E57** (NeuroPLC DA substantially tighter than CROWN-IBP, which reduces to IBP for KAN) provide empirical depth-scaling and verifier-comparison evidence. **DA segment-exactness** (Proposition 4, §III-L) proves Doubleton Arithmetic achieves segment-exactness on affine B-spline segments, explaining the mechanism behind DA's tightening over IA. **Theorem 7** establishes the sufficient condition for Z3 NRA verifiability ($M_2 \cdot h^2/8 \leq C(k)$), providing the theoretical basis for the B-spline 512/512 and Chebyshev 496/512 verification rates.
7) **Theory (Correctness): Theorem 1** (§IV) guarantees by structural induction on the IR graph that generated SCL preserves PyTorch semantics to within a computable, architecture-aware error bound for all 6 IR operation types. **Theorem 4** generalizes the guarantee to L-layer KANs via martingale concentration, proving error grows only linearly with depth under the random-sign model.
8) **Theory (Algebraic Completeness): Theorem 8** (§III) proves the SVNN class is closed under sequential composition—forming an algebraic monoid—enabling modular certification where independently-certified subnetworks compose their

guarantees without re-analysis. **Theorem 9** (§III-N) proves the sharp k-th order LUT error bounds: the de Boor constant $M_2h^2/8$ is the $k = 2$ instance of an exact all-orders family $c_k = Q_k/k!$ ($c_3 = 1/(9\sqrt{3})$, $c_4 = 1/24$), attained by an explicit extremal family at machine precision—DA is minimax-optimal among affine design-time schemes (Theorem 9, E63). **Proposition 5** formalizes the **Operation Separation Principle**—the C^2-BV architecture family (B-spline, Fourier, Wavelet, RBF, Chebyshev) all admit SVNN guarantees; standard MLPs are structurally excluded.

9) **Theory (Deployment Completeness): Theorem 11** (§IV-G) provides the worst-case execution time guarantee: for any SVNN network N with E edges and N_{LUT} LUT points on Siemens S7-1200, $\text{WCET}(N) \le E \cdot (C_{\text{LUT}} + C_{\text{matmul}}) + C_{\text{softmax}}$. For KAN $[28, 16, 4]$ with $N_{\text{LUT}} = 15$: WCET $= 3.86$ ms ($25.9\times$ margin within 100 ms scan cycle, 3.9% utilization).
10) **Algorithm (Safety Monitor): Algorithm 3** (§IV-H)—the compiler automatically generates a companion SCL safety monitor implementing domain violation detection, confidence fallback, and output range bounds. The monitor adds $\le 5\%$ code overhead and ~ 66 μs execution time, providing a run-time safety net for the compiled inference.
11) **System (Multi-Architecture Validation): E60–E61** extend SVNN validation to FourierKAN and WaveletKAN architectures: FourierKAN $[28, 16, 4]$ (6 harmonics, $\omega = 0.4$, 6,676 params) achieves 100.0% CWRU accuracy with 512/512 edges satisfying $M_2 \cdot h^2/8 \le 0.063$ (**100**% Z3-equivalent); WaveletKAN $[28, 16, 4]$ (8 Mexican hat scales, M2-regularized, ∼4,600 params) achieves 100.0% CWRU accuracy with 512/512 edges verified (**100**% Z3-equivalent), confirming the C^2-BV generalization (Proposition 6).

We demonstrate the pipeline through a bearing fault diagnosis case study: a 6,148-parameter KAN student distilled via VRM-KD [6] from a 48K-parameter 1D-CNN teacher achieves 99.93% CWRU test accuracy with 7.9× compression. Fine-tuning on XJTU-SY run-to-failure data (natural bearing degradation, 2020) reaches 91.7% accuracy with 512/512 Z3 verification preserved post-adaptation. FourierKAN and WaveletKAN achieve 100% CWRU and 100% XJTU-SY domain-shift accuracy (E62), with 512/512 Z3-equivalent verification preserved, confirming the C^2-BV family's cross-architecture fine-tuning stability. In total, 74 experiments (E1–E58 + E60–E68 + V1–V7) plus eleven capacity-theory verification experiments (E-T1–E-T11, §III-S) spanning compression, generalization, correctness bounds, scalability, cross-dataset transfer, cross-architecture verification (FourierKAN E60, WaveletKAN E61), and domain-shift SVNN preservation (E62) validate the compiler. We further discuss the SVNN framework's relationship to emerging AI functional safety standards (ISO/IEC TS 22440) and formal verification tools for IEC 61131-3 (ESBMC-PLC+), positioning NeuroPLC within the broader industrial verification ecosystem.

II. Related Work

A. Deep Learning for Bearing Fault Diagnosis

Three lines of work have pushed CWRU fault diagnosis accuracy above 99%. WDCNN [7] demonstrated that wide first-layer convolutional kernels capture the long-period fault signatures that narrow kernels miss. Zhang et al. [8] extended residual architectures to 1-D signals, confirming that depth helps when the receptive field is sufficiently wide. Together, these works established a near-solved benchmark. But Rauber et al. [9] showed that many reported near-100% results contain train/test leakage from overlapping recording-level windows—the problem is better solved than the literature reports, yet the deployment problem is worse: all methods assume GPU-equipped industrial PCs. **The gap:** 99.9% accuracy on a lab GPU contributes nothing if the inference cannot run on the PLC that governs the monitored machinery.

B. Kolmogorov-Arnold Networks

Liu et al. [10] introduced KANs by replacing MLPs' fixed activations with learnable univariate B-splines—a structural change whose implications for compiler design have been underestimated. **What the literature has established:** Rigas et al. [11], CNN-1D-KAN [12], and polynomial-basis KAN variants independently confirmed that KANs match or exceed MLP accuracy on bearing fault diagnosis while using far fewer parameters. **What the literature has not addressed:** none of these works consider whether the B-spline decomposition that makes KANs parameter-efficient also makes them *verifiable at compile time*—the question this paper answers.

KAN hardware deployment. Four independent lines of work demonstrate that KAN's univariate B-spline structure is, to our knowledge, uniquely amenable to LUT-based hardware evaluation—a property absent from MLPs. On FPGA, KANELÉ [13] (ISFPGA 2026 Best Paper) achieves 2,700× speedup via LUT-based evaluation with quantized B-splines. On microcontrollers, LUT-KAN [14] replaces Cox-de Boor recursion with precomputed LUT operations (7–25× speedup, as little as 68 bytes/edge); KAN-SoC [15] achieved 10× memory reduction vs. MLP (693 bytes vs. 6,807 bytes) for EV battery estimation. On PLCs, Corrêa [5] (USP 2024) provided the sole prior KAN-on-PLC demonstration: a KAN $[7, 5, 6]$ deployed via Snap7 distributed execution (PC computes inference, PLC receives results over Ethernet), achieving 88.5% accuracy on a bearing classification task.

These four works collectively establish that KAN's B-spline structure compiles to LUT primitives across hardware platforms. But *none* answer the question this paper asks: can the compilation itself be *certified*? KANELÉ and LUT-KAN validate the LUT paradigm empirically; we formalize *why* it works (Theorem 2) and *when* it fails (Theorem 5, Proposition 2). Corrêa demonstrates KAN-on-PLC feasibility but provides no native SCL compilation, no correctness guarantee, and no compiler toolchain. NeuroPLC is, to our knowledge, the first to provide all three.

KAN verification. Schwartz et al. [16] proposed PWA abstraction with MILP encoding for KAN verification—operating at SVNN Level 3 (exact certification within PWA abstraction error). In this paper we certify the trained model under Conditions 1–2 (§III-C)—the trained student is *not* contractive (measured per-layer amplification $\gamma = [15.4, 5.3]$, E68), and we identify contractive training as the recipe for the depth-uniform certificate—noting that polynomial-basis KANs admit structural Lipschitz bounds via coefficient magnitudes. KAN4CBC [17] demonstrated SMT-based safe controller synthesis. GloroKAN [18] and PolyKAN [19] provide complementary robustness and compression. These are synergistic with NeuroPLC: Schwartz et al.'s MILP verification can be applied to the KAN compiled by NeuroPLC for SIL 3+ certification, while NeuroPLC's Level 2 arithmetic bounds provide fast, no-solver-required pre-certification for CI/CD integration.

a) FPGA Deployment.: KANELÉ [13] (ISFPGA 2026 Best Paper) maps KAN inference to LUT-based FPGA evaluation with 2,700× speedup over prior FPGA KAN implementations. KANELÉ's LUT paradigm independently validates the same design principle NeuroPLC applies to PLCs—B-spline activations are inherently discretizable across hardware platforms. No prior work compiles KANs to IEC 61131-3 SCL for PLC execution.

C. Knowledge Distillation

Hinton et al. [20] introduced temperature-scaled knowledge distillation as a mechanism for transferring dark knowledge from ensembles to compact models. VRM [6] (ICCV 2025) revitalized relation-based distillation through virtual relation matching with dynamic affinity graph pruning—directly applicable to KAN students because KAN's univariate activation structure produces naturally sparse inter-sample relations. Ren et al. [21] applied multi-stage pruning-distillation for industrial fault diagnosis, establishing the practical viability of compressed models for bearing monitoring. We contribute, to our knowledge, the first VRM-KD to a KAN student: the distillation is the *training mechanism*, not the contribution; the contribution is that KAN's 7.9× compression does not compromise verifiability.

D. Neural Network Compilation for Industrial Controllers

RTNNIgen [2] (IECON 2024) converts Keras models to IEC 61131-3 ST and demonstrated real-time inference on Beckhoff PLCs—the strongest evidence to date that ML inference *can* run natively on PLCs. MLconverter [3] extends this paradigm to ABB controllers from scikit-learn. ICSML [4] provides native IEC 61131-3 ML components. The shared limitation is the correctness model: all three validate through test-set accuracy, inheriting the empirical validation paradigm of ML benchmarking. For PLCs governing safety-critical machinery, this is the wrong correctness model—it provides no guarantee on unseen inputs, cannot detect floating-point drift, and offers no evidence auditable by a safety assessor. NeuroPLC does not merely add another target platform to this lineage; it changes the correctness model from empirical to *mathematical*, by restricting the compiler's input to architectures that admit the SVNN structural induction (Theorem 2). No automated compiler translates trained PyTorch networks to Siemens SCL with design-time correctness certification.

E. Formal Verification of IEC 61131-3 Programs

ESBMC-PLC+ [22] provides the first open-source unified verification framework for IEC 61131-3 (LD, ST, SCL) through SMT-based BMC with k-induction. PLCreX [23] translates ST/FBD to synchronous models; PyLC+ [24] compiles PLCopen XML to verifiable Python. These verify *logical* properties (mutual exclusion, invariants) orthogonal to NeuroPLC's *numerical* correctness guarantees. The two are composable: NeuroPLC-generated SCL can serve as input to these tools (§VI-B), enabled by the deterministic, human-readable SCL structure (§IV-D).

F. Neural Network Verification and Bounding

A mature literature on neural-network verification provides complementary tools that operate at different points on the soundness–cost spectrum. **Linear-relaxation methods**—CROWN and DeepPoly [25], [26], generalized by α-β-CROWN [27] to branch-and-bound—propagate linear upper/lower bounds through each layer and achieve tight certification on ReLU networks, but the LP size scales with the network, requiring an external solver at verification time. **Complete SMT-based verifiers**—Reluplex [28] and Marabou [29]—offer exact certification for piecewise-linear (ReLU) networks via case-splitting on activation patterns, at exponential cost in the number of ReLU units. **Abstract interpretation** [30]–[32] provides sound, scalable over-approximation through abstract domains—the foundational framework (Cousot & Cousot, 1977) defines a sound abstraction as one whose concretization contains all reachable concrete states. In neural network verification, abstract domains such as intervals, zonotopes, and polyhedra have been applied to ReLU networks [26], [31], [32], but these generic domains are designed for the activation pattern $x \mapsto \max(0, Wx + b)$ and do not exploit the piecewise-polynomial structure of B-spline activations. Consequently, they produce coarse over-approximations when applied to KAN architectures, whose intermediate representations are not activation-sparse but *function-sparse* (each activation is a full polynomial segment).

NeuroPLC's SVNN framework occupies a distinct point that can be understood within the abstract interpretation paradigm: Doubleton Arithmetic (DA, §III-L) constitutes a *specialized abstract domain for piecewise-polynomial univariate functions*. On a single B-spline knot segment—the atomic computation unit in KAN inference—DA propagation is *exact* for affine segments (Proposition 4.i, verified on 1,000/1,000 random segments) and incurs only $O(M_2 \cdot r^2)$ overestimation for higher-degree segments (Proposition 4.ii). This is tighter than any generic interval or zonotope domain can achieve on the same function class, because those domains are not *activation-aware*: they treat each B-spline evaluation as a black-box nonlinearity rather than a known polynomial with computable curvature bound M_2.

Key differences from prior abstract-interpretation tools: (1) SVNN bounds are *parameter-computable* at design time—the compiler computes the bound from the B-spline control points and LUT resolution alone, without invoking an LP solver or SMT solver at deployment; (2) Condition 1 (operation-type closure) is a *structural* requirement that enables the exact affine propagation: DA can separate the linear term $\phi'(x_0) \cdot r\epsilon$ from the higher-order remainder R_2 only because the architecture exposes pure linear and pure element-wise operations (Theorem 2, Step 3); (3) the resulting guarantee certifies the *translation into SCL*, not the model's robustness—it is a *compilation* correctness certificate, not a model verification certificate. These are complementary: CROWN/DeepPoly verify the original model; SVNN certifies the *compilation* of that model. The most closely related approach is the per-function Z3 certificate bundle (§VI-B), which operates at the sub-module level and composes into an end-to-end machine-checkable guarantee.

Competitive Positioning. The tools surveyed above represent three distinct approaches to ML-on-PLC deployment, none of which provides design-time correctness certificates. *Domain-specific compilers* (RTNNIgen, MLconverter, ICSML) target narrow model classes for specific PLC vendors but offer only empirical validation. *General-purpose inference engines* (TFLite, ONNX Runtime, TVM) target performance on GPUs and MCUs; they cannot generate IEC 61131-3 and their runtime libraries alone exceed PLC memory budgets by factors of 10^0–10^2. *Formal verification tools* (CROWN/DeepPoly, Marabou, Reluplex) verify the *original* model's robustness but do not compile it to PLC code or certify the *compilation* step. NeuroPLC occupies a distinct point: it is not a verification tool, a general-purpose compiler, or a domain-specific converter—it is a **correctness-preserving compiler** whose design is driven by the architectural insight (SVNN) that compilability is a property of the model architecture, not the compiler engineering. This reframing—from "how do we verify an arbitrary architecture?" to "which architectures are inherently verifiable?"—is the paper's primary contribution, with implications beyond PLC deployment to any safety-critical embedded ML target.

III. Structural Verifiability of Neural Networks

We now formalize a central observation that emerged from the compiler design process: whether a neural network compiler can provide design-time correctness guarantees is not solely a property of the compiler—it is fundamentally determined by the *architecture* being compiled. This section defines the class of architectures for which correctness-preserving compilation is achievable, proves that KAN belongs to this class, and demonstrates that standard MLPs do not.

Remark 1 (The Compilable Frontier)**.** *The SVNN conditions define a* ***compilable frontier****—a principled boundary separating neural architectures whose deployed behavior can be certified at design time from those that require runtime testing. Architectures satisfying Conditions 1–2 lie on the interior (certifiable side): their decomposition into pure linear and element-wise operations enables the structural induction that Theorem 2's proof relies on. Architectures violating any condition lie on the exterior (empirical-only side): they can still be compiled and deployed, but their correctness guarantees are limited to test-set validation. Proposition 2 (below) establishes that standard MLPs with SiLU, Sigmoid, or Tanh activations—the dominant architectures in industrial ML—lie on the exterior. This frontier is not an implementation artifact; it reflects a fundamental property of which function classes admit closed-form, compiler-computable error propagation.*

A. The Verification Challenge for PLC-Deployed Networks

Industrial PLCs govern safety-critical machinery: conveyor sorting systems, motor drives, and process control loops where incorrect outputs can cause equipment damage or production line stoppage. Deploying neural networks on such controllers therefore demands correctness guarantees that go beyond empirical test-set accuracy.

Existing ML→PLC tools (RTNNIgen [2], MLconverter [3], ICSML [4]) provide *no* mathematical guarantee that the generated PLC code preserves the trained model's behavior. Their correctness model is purely empirical: train, convert, and test on a held-out set. For industrial deployment—where inputs may drift, sensors may degrade, and the PLC's REAL arithmetic differs from PyTorch's float32—this is insufficient.

We identify three levels of deployment verification, in increasing order of strength:

1) **Level 1—Runtime Testing (Weakest):** Evaluate the deployed model on a test set and report accuracy. This is the status quo (RTNNIgen, MLconverter). It provides no guarantee on unseen inputs and cannot detect systematic floating-point discrepancies.
2) **Level 2—Design-Time Error Bounds (SVNN):** Compute, at *compile time* and without executing on hardware, a provable per-output error bound ε_i such that for all valid inputs x, the deployed model's output differs from the trained model's output by at most ε_i. This is the guarantee that NeuroPLC provides (Theorem 1).

3) **Level 3—Mechanized Formal Verification (Strongest):** A machine-checked proof (e.g., in Coq, Isabelle, or Lean) that the compiler preserves semantics down to the IEEE 754 bit-level. This has been achieved for select neural network kernels in machine-checked settings (e.g., CompCert-style verified compilation) but is currently intractable for full ML compiler pipelines.

Level 2—design-time error bounds—is the appropriate level for industrial deployment. The question is: *which architectures admit such bounds, and why?*

B. Definition: Structurally Verifiable Neural Network (SVNN)

Definition 1 (Structurally Verifiable Neural Network)**.** *A neural network architecture $\mathcal{A}$ is* ***structurally verifiable*** *(SVNN) with respect to a compiler C if for any instance $M \in \mathcal{A}$, the compiler can compute, at design time and without executing M on target hardware, a per-output error bound $\varepsilon_i(M, X)$ such that:*

$$\sup_{x \in X} \|M(x)_i - C(M)(x)_i\|_\infty \leq \varepsilon_i(M, X) \quad \forall i \in \{1, \ldots, d_{out}\} \tag{1}$$

where $X \subset \mathbb{R}^{d_{in}}$ is the validated input domain and $C(M)$ denotes the compiler's generated code executing on the target platform's arithmetic (here, IEC 61131-3 REAL $\equiv$ IEEE 754 binary32).

This definition captures the essential requirement: the compiler must provide an *a priori* guarantee—before the model ever runs on the PLC—that the deployed code will not silently diverge from the trained model. The error bound ε_i may depend on the model's parameters, the compiler's approximation choices (e.g., LUT resolution), and the input domain, but it must be computable from these quantities alone.

C. Sufficient Conditions for SVNN

We now establish the architectural conditions that enable SVNN compilation. The conditions formalize why some architectures admit tight, composable error bounds while others do not.

Theorem 2 (SVNN Sufficient Conditions)**.** *Let $\mathcal{A}$ be a feedforward neural network architecture. If $\mathcal{A}$ satisfies Conditions 1 and 2 below, then $\mathcal{A}$ is structurally verifiable—for any instance $M \in \mathcal{A}$ of finite depth L, a compiler adhering to the SVNN compilation discipline can compute, from the model parameters alone, a finite per-output error bound. If $\mathcal{A}$ additionally satisfies the contractivity refinement (Condition 3), this bound admits the closed, depth-independent geometric-series form $O(M_{\max} \cdot h^2 \cdot d_{\max}/(1-\gamma))$, where $M_{\max}$ is the maximum second-derivative bound, h is the LUT grid spacing, $d_{\max}$ the maximum activation count per layer, and $\gamma < 1$ the contractivity constant; absent Condition 3, the bound remains finite for every fixed L but may grow with depth.*

The conditions are designed to discriminate: a standard MLP layer $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$ composes an affine transform and a pointwise activation in one coupled operation, violating Condition 1. This prevents per-operation error induction (the proof's structural induction requires each node to be either purely linear or purely element-wise). MLPs with SiLU or GELU additionally violate the Z3-verifiability of Condition 2 (the transcendental exp *is undecidable in Z3's NRA theory); MLPs with ReLU lack a C^2 curvature parameter. KAN layers, in contrast, decompose naturally at the IR level into separate MatMul, BsplineLUT, StandardAct, and Add nodes—each satisfies one of the two pure types. Conditions 1–2 thus characterize exactly the class that admits the induction, and KAN belongs to it; standard MLPs do not.*

1) **Operation-Type Closure (Condition 1).** The architecture can be decomposed into a directed acyclic graph where every node is either (a) a linear transformation (matrix multiplication, vector addition) or (b) an element-wise univariate function. No node may mix these operation types. This decomposition is provided by the compiler's IR (§IV-B1); it guarantees that the per-node error induction in the proof below never encounters a coupled linear+nonlinear operation.
2) **Univariate Curvature Bound (Condition 2).** Every element-wise univariate function $f : \mathbb{R} \to \mathbb{R}$ is C^2-continuous on the validated input domain X, and its second derivative satisfies $\sup_{x \in X} |f''(x)| \leq M_f < \infty$. The bound M_f must be computable from the function's parameters alone. This enables the de Boor LUT error bound $\varepsilon_f = M_f h^2/8$—a *compiler-controllable* error: by increasing the LUT resolution (reducing h), the per-function error decreases quadratically. Condition 2 thus provides the compiler with a tunable guarantee knob that MLPs with ReLU (no C^2) or SiLU/GELU (transcendental, Z3-undecidable) lack.

Conditions 1 and 2 are the *sufficient conditions* for structural verifiability. The following refinement governs how the bound scales, not whether it exists:

3) **Contractive Composition (Condition 3, refinement).** The product of each element-wise layer's Lipschitz constant $L_f^{(\ell)} = \sup_x |f'(x)|$ with the subsequent linear layer's row-sum norm satisfies $L_f^{(\ell)} \cdot \|W^{(\ell+1)}\|_{1,\infty} \leq \gamma < 1$. When this holds, the bound converges to a depth-independent constant $O(1/(1-\gamma))$; when it does not, the bound remains finite for the specific deployment depth but grows as $O(\gamma^L)$. Condition 3 upgrades a per-depth certificate to a depth-independent one; it is *not* required for SVNN membership.

Proof. The proof proceeds in four steps.

Step 1 (Linear Layer Exactness). Let $y = Wx+b$ be a linear transformation with input x that carries error δ_x (i.e., the compiled input is $\tilde{x} = x+\delta_x$). The compiled output is $\tilde{y} = W\tilde{x}+b = Wx+b+W\delta_x$. Hence $\|\tilde{y}-y\|_\infty = \|W\delta_x\|_\infty \leq \|W\|_{1,\infty} \cdot \|\delta_x\|_\infty$. A linear layer introduces *no new error sources*—it only amplifies or attenuates existing error. Under sign-structural affine arithmetic (§IV-B9), the amplification factor exploits weight-matrix sign structure, yielding $\|W\delta_x\|_\infty \leq \|W^+\delta_x^+ + W^-\delta_x^-\|_\infty$, which is strictly tighter than the interval-arithmetic bound $\|W\|_{1,\infty} \cdot \|\delta_x\|_\infty$ when W contains both positive and negative entries.

Step 2 (Element-Wise Boundedness). Consider an element-wise univariate function f applied to input x. The compiled version uses a piecewise-linear LUT approximation $\tilde{f}$ on a grid with spacing h. By the de Boor–Cox recurrence for B-spline approximation error [33], for any C^2 function:

$$\|f - \tilde{f}\|_\infty \leq \frac{M_f \cdot h^2}{8} \tag{2}$$

where $M_f = \sup_x |f''(x)|$. Condition 2 guarantees that M_f is finite and computable. The LUT thus introduces fresh error at most $\varepsilon_f = M_f h^2/8$ per function evaluation.

If the input carries error δ_x, the mean value theorem gives:

$$|f(x+\delta_x) - \tilde{f}(x+\delta_x)| \leq |f(x+\delta_x) - f(x)| + |f(x) - \tilde{f}(x+\delta_x)| \leq L_f \cdot |\delta_x| + \varepsilon_f \tag{3}$$

where $L_f = \sup_x |f'(x)|$ is the Lipschitz constant of f.

Step 3 (Finite-Depth Composition—Conditions 1 and 2 suffice). We now compose the analysis across L layers by structural induction on the layer index ℓ. Let $\Delta^{(\ell)}$ denote the maximum per-output error after layer ℓ, with $\Delta^{(0)} = 0$ (input is exact).

For a KAN layer (element-wise B-spline activation followed by a linear combination via weight matrix):

$$\Delta^{(\ell)} \leq L_f^{(\ell)} \cdot \|W^{(\ell)}\|_{1,\infty} \cdot \Delta^{(\ell-1)} + \varepsilon_f^{(\ell)} \cdot d_{\ell-1} \tag{4}$$

The first term captures error amplification from the previous layer; the second term captures fresh approximation error from $d_{\ell-1}$ parallel element-wise function evaluations (one per input dimension).

Unrolling the recurrence across L layers yields the total bound:

$$\Delta^{(L)} \leq \sum_{\ell=1}^{L} \varepsilon_f^{(\ell)} \cdot d_{\ell-1} \cdot \prod_{j=\ell+1}^{L} \left(L_f^{(j)} \cdot \|W^{(j)}\|_{1,\infty}\right) \tag{5}$$

The critical role of Condition 1 is now apparent: it guarantees that every node encountered by the induction is of a known type with a closed-form error rule (Step 1 for linear nodes, Step 2 for element-wise nodes). The recurrence Eq. 4 applies *only* because each node is provably pure. In a standard MLP layer $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$, the affine transform and pointwise activation are coupled in a single operation—there is no intermediate variable whose error can be separately bounded, so the structural induction cannot be initiated. This is why Sigmoid/Tanh MLPs, despite having bounded M_2 (they satisfy Condition 2), are not SVNN: they fail Condition 1.

Under Conditions 1 and 2, every term in Eq. 5 is finite and computable: $\varepsilon_f^{(\ell)} = M_f^{(\ell)} h^2/8$ from Condition 2, $L_f^{(j)}$ finite on the compact domain X, and $\|W^{(j)}\|_{1,\infty}$ finite by construction. Eq. 5 is therefore a finite sum of finite terms, directly yielding a computable design-time bound $\Delta^{(L)} < \infty$ for any fixed depth L—the existence guarantee of Definition 1. The bound is *compiler-controllable*: increasing the LUT resolution (reducing h) monotonically decreases ε_f and hence $\Delta^{(L)}$, giving the compiler a tunable knob to meet any target safety requirement.

Step 3b (Depth-Uniform Refinement—Condition 3). Condition 3 does not change *whether* a bound exists; it controls *how the bound scales with depth*. When $L_f^{(j)} \cdot \|W^{(j)}\|_{1,\infty} \leq \gamma < 1$, the product terms in Eq. 5 decay geometrically, and the finite sum is majorized by the convergent series

$$\Delta^{(L)} \leq \frac{\max_\ell(\varepsilon_f^{(\ell)} \cdot d_{\ell-1})}{1-\gamma} = O\left(\frac{M_{\max} \cdot h^2 \cdot d_{\max}}{1-\gamma}\right) \tag{6}$$

where $M_{\max} = \max_f M_f$ and $d_{\max} = \max_\ell d_{\ell-1}$ is the maximum number of element-wise activation functions per layer. The bound follows by substituting $\varepsilon_f^{(\ell)} = M_f^{(\ell)} h^2/8$ (Eq. 2) into the geometric series, yielding a per-layer factor of $M_{\max} h^2 d_{\max}$ multiplied by the geometric decay $1/(1-\gamma)$. This form is *depth-independent*: when $\gamma < 1$, the bound converges to a constant as $L \to \infty$, admitting deployment on arbitrarily deep networks. When Condition 3 does not hold ($\gamma \geq 1$), Eq. 5 still evaluates to a finite number for the fixed deployment depth, but the worst-case majorant grows as $O(M_{\max} \cdot h^2 \cdot \gamma^L)$—still computable, only looser with depth. Thus Condition 3 upgrades a per-depth guarantee into a depth-uniform one; it is a refinement, not a membership requirement.

Step 4 (Tightness and Practical Considerations). The $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$ bound is a *worst-case* guarantee. In practice, three factors make the bound substantially tighter for the KAN architecture specifically:

1) **Segment-Aware M_2 (§IV-B10):** Rather than using the global $\sup|f''|$, we compute per-segment $M_2^{(k)}$ bounds, tightening the LUT error estimate by $6.0\times$ on average for KAN B-splines.
2) **Sign-structural affine arithmetic (§IV-B9):** Linear layers propagate error via DA rather than IA, exploiting weight-matrix sign structure for an additional $3.1\times$ tightening.
3) **Adaptive LUT Allocation (§IV-B11):** Non-uniform knot placement allocates more LUT points to high-curvature regions, reducing the effective h for the dominant error sources by up to 71.6%.

Together, these refinements transform the asymptotic $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$ bound into a practically useful guarantee. For the trained $[28, 16, 4]$ KAN, the combined segment-aware DA safety factor is approximately $11.9\times$ above the minimum required for classification correctness (§IV, Table XXII).

Theorem 8 (SVNN Compositional Closure)**.** *The SVNN class is closed under sequential composition: if $N_1 \in \mathit{SVNN}(\varepsilon_1, L_1)$ and $N_2 \in \mathit{SVNN}(\varepsilon_2, L_2)$, then $N = N_2 \circ N_1 \in \mathit{SVNN}(\varepsilon, L)$ with:*

$$\varepsilon = \varepsilon_2 + L_2 \cdot \varepsilon_1, \qquad L = L_2 \cdot L_1 \tag{7}$$

Proof. Let x be any valid input. By SVNN of N_1: $C(N_1)(x) = N_1(x) + \delta_1$ with $\|\delta_1\|_\infty \le \varepsilon_1$. By SVNN of N_2, $C(N_2)(y) = N_2(y) + \delta_2$ with $\|\delta_2\|_\infty \le \varepsilon_2$. Composing:

$$\begin{aligned} C(N)(x) &= C(N_2)(C(N_1)(x)) = C(N_2)(N_1(x) + \delta_1) \\ &= N_2(N_1(x) + \delta_1) + \delta_2 \\ &= N_2(N_1(x)) + \big[N_2(N_1(x) + \delta_1) - N_2(N_1(x))\big] + \delta_2 \end{aligned} \tag{8}$$

The perturbation term is bounded by the Lipschitz property: $\|N_2(N_1(x)+\delta_1) - N_2(N_1(x))\|_\infty \le L_2 \cdot \varepsilon_1$. Hence $\|C(N)(x) - (N_2 \circ N_1)(x)\|_\infty \le \varepsilon_2 + L_2 \cdot \varepsilon_1$, which is finite and computable since $\varepsilon_1, \varepsilon_2, L_2$ are. □

Implications. Theorem 8 establishes that the SVNN class forms a **monoid** under composition: closure (Theorem 8), associativity (function composition), and identity ($I(x) = x$ is SVNN with $\varepsilon = 0, L = 1$). This enables **modular certification**: feature extractor KANs and classifier KANs can be independently certified, and their sequential composition inherits the combined guarantee *without re-analysis*. For KAN architectures, Theorem 8 applies directly to any stacking of KAN layers and to hybrid pipelines where, e.g., a time-domain feature encoder is composed with a fault classifier. The branchwise variant of Theorem 8 proves that KAN's dual-path architecture (base path $\oplus$ spline path, merged by Add) is SVNN-compliant: both branches are independently certified and the Add node introduces no new error (Theorem 2, Step 1). Modular certification is essential for industrial scalability: as diagnostic pipelines grow to include preprocessing, multiple sensor modalities, and hierarchical classification, each module's certificate composes automatically—no end-to-end re-certification required.

Remark 2 (Near-Necessity of the SVNN Conditions)**.** *While Theorem 2 establishes sufficiency, the question of necessity is equally important: are these conditions arbitrarily chosen, or do they characterize a genuine structural boundary? Theorem 5 (§III-G) provides the formal answer in the ε-dimension: the LUT-verification* decision problem *is polynomial for SVNN and NP-hard for coupled architectures (a self-contained 3-SAT reduction via product gates), and SVNN uniquely attains (polynomial time, zero slack) simultaneously. This establishes that the SVNN conditions are not an arbitrary restriction but the precise boundary at which polynomial-time, tight compiler-computed certification becomes achievable. We briefly summarize the three observations that motivate the formal proof.*

(a) Condition 1 is structurally necessary for polynomial-time bound computation. Any architecture that couples linear transforms with nonlinear activations in a single operation—the standard MLP layer $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$—eliminates the intermediate representation whose error the structural induction bounds. Without this decomposition, error propagation must model the coupled operation as a single black-box function, requiring general-purpose LP/SMT solvers (CROWN, Marabou, Reluplex) whose cost grows with network size. Condition 1 thus separates architectures admitting $O(L \cdot d_{\max}^2)$ compiler-computed bounds from those requiring exponential-time SMT case-splitting.

(b) Condition 2 separates C^2-bounded from transcendental. The de Boor LUT error bound ($\varepsilon = M_2 h^2/8$) requires C^2 continuity. Activations without C^2 (ReLU: f'' undefined at 0) or with curvature computable only via numerical optimization (SiLU: $\sup_{\mathbb{R}} |\mathit{SiLU}''| = 0.5$) cannot use this bound. The C^2 requirement is nearly tight: it excludes exactly the activations whose LUT error cannot be bounded by a finite, parameter-computable constant. On the compact input domain $[-3, 3]^{28}$, SiLU's second derivative is finite ($\sup_{[-3,3]} |\mathit{SiLU}''| = 0.5$, attained at $x = 0$)—but the key distinction is computability: $M_2^{\text{B-spline}}$ is computable from the learned coefficients alone, whereas $\sup_{x \in [-3,3]} |\mathit{SiLU}''(x)|$ requires numerical optimization over a transcendental function—a different computational model.

(c) Richardson's Theorem [34] imposes a fundamental barrier. The equality of expressions involving e^x is undecidable by Richardson's theorem. Since SiLU contains e^{-x}, determining whether $\mathit{SiLU}_{\mathit{LUT}}(x) = \mathit{SiLU}_{\mathit{FP32}}(x)$ for a given LUT resolution is, in general, undecidable. KAN's B-spline activations are piecewise-polynomial and avoid this barrier entirely. The SVNN conditions are therefore not an arbitrary restriction—they characterize the precise boundary at which compiler-computed correctness becomes decidable.

TABLE I
ARCHITECTURE TAXONOMY UNDER THE SVNN COMPILABLE FRONTIER. ARCHITECTURES ON THE INTERIOR ADMIT COMPILER-COMPUTED DESIGN-TIME CORRECTNESS CERTIFICATES; ARCHITECTURES ON THE EXTERIOR REQUIRE RUNTIME TESTING.

4pt 3.5pt

Architecture	**Cond. 1**	**Cond. 2**	**Contract.**	**Frontier**	**Z3-Verif.**
KAN (B-spline)	✓	✓	✗*	Interior†	512/512
KAN (Chebyshev)	✓	✓	✓	Interior	Full‡
KAN (Wavelet)	✓	✓	$\sim$	Interior	Full‡
MLP + ReLU	✗	✗§	N/A	Exterior	0/16
MLP + SiLU/GELU	✗	✗¶	N/A	Exterior	0/16♯
MLP + Sigmoid	✗	✓	N/A	Exterior	–
MLP + Tanh	✗	✓	N/A	Exterior	–
Transformer	✗	✗	N/A	Exterior	N/A
CNN (Conv2d)	✗	✗	N/A	Exterior	N/A

✓ = condition satisfied; ✗ = condition violated; $\sim$ = depends on wavelet basis choice.

*Condition 3 is *not* satisfied by the trained KAN student $[28, 16, 4]$ (measured per-layer amplification $\gamma = [15.4, 5.3] > 1$, E68); the student is certified under Conditions 1–2 with the finite depth-dependent bound. Polynomial-basis KANs admit structural Lipschitz bounds from coefficient magnitudes, giving $P(\text{KAN}) \leq 1$ as a design-time property (ChebyKAN, §III-F).

†The B-spline KAN is the only architecture in this taxonomy for which all three conditions are simultaneously satisfied *and* empirically validated on the TIA Portal V21 engineering toolchain (S7-1200 target).

‡ChebyKAN: formally verified in Proposition 3 (§III-F); Z3 verifiability rate 496/512, 96.9% (E54). WaveletKAN with piecewise-polynomial basis: full verifiability follows by the same structural induction as B-spline KAN (Theorem 2, Step 3); Z3 rate depends on wavelet family (see the Remark on Wavelet-KAN and Fourier-KAN).

§ReLU is piecewise-linear (C^0); f'' is a Dirac delta at $x = 0$, so the de Boor $M_2 h^2/8$ bound does not apply. MILP-based verification (e.g., Reluplex) is possible but requires exponential case-splitting.

¶SiLU contains $\sigma(x) = 1/(1 + e^{-x})$; e^{-x} is transcendental and undecidable in Z3's NRA theory. On compact domain $[-3, 3]$, $\sup |\text{SiLU}''| = 0.5$ is finite but requires numerical optimization—not computable from parameters alone.

♯Confirmed empirically: MLP $[28, 16, 4]$ with SiLU, 0/16 components Z3-verifiable (E41).

Remark (Relationship to Theorem 1). Theorem 1 (§IV) is the *concrete instantiation* of the SVNN framework for a specific architecture (2-layer KAN) and compiler (NeuroPLC). Theorem 2 provides the *general principle*: any finite-depth architecture satisfying Conditions 1–2 admits a correctness-preserving compiler, with Condition 3 sharpening the resulting bound to a depth-uniform form. Theorem 1 proves that KAN satisfies the conditions; Theorem 2 shows *why* this is possible and characterizes the broader class of architectures for which similar guarantees can be obtained.

Remark (Arithmetic Bounds vs. MILP-Based Verification). The SVNN arithmetic bound ($O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$, Theorem 2) and the PWA abstraction + MILP approach of Schwartz et al. [16] represent two complementary points on the verification cost-precision spectrum. *Arithmetic bounds* (Level 2) are computable in closed form at compile time, require no external solver, and provide a sound over-approximation for all inputs in the validated domain—but they are inherently conservative (the $\sim$11.9$\times$ safety factor for our trained KAN reflects this conservatism). *MILP-based verification* (Level 3) can provide exact certification within the PWA abstraction error (typically <0.1% of the function range per unit), but requires solving an MILP whose size grows with the number of PWA pieces—tractable for KANs (each univariate function is abstracted independently) but exponential for MLPs (multivariate activations require piecewise decomposition of $\mathbb{R}^d$). The two approaches are synergistic: arithmetic bounds serve as a fast, no-solver-required pre-certificate; if the bound is within safety margins, no further verification is needed. If tighter guarantees are required (e.g., for SIL 3+ certification), the SVNN decomposition enables targeted MILP verification of safety-critical outputs. This two-tier architecture is discussed further in §VI-B.

Architecture Taxonomy Under the Compilable Frontier. Table I maps major neural architecture families to the SVNN conditions, providing a concrete instantiation of the compilable frontier (Remark 1). The taxonomy reveals a stark divide: architectures that decompose into pure linear + element-wise operations (Condition 1) and use C^2 parameter-computable activations (Condition 2) occupy the *interior*—their deployed behavior is certifiable at design time. All architectures violating either condition lie on the *exterior*—they can be compiled and deployed, but correctness guarantees are limited to test-set validation. The Contractivity column (Condition 3) further distinguishes architectures whose error bounds are depth-*uniform* ($\gamma < 1$, depth-independent growth) from those whose bounds are merely depth-*finite* (computable but potentially growing with L).

The taxonomy clarifies a structural fact with direct engineering consequences: *no* MLP variant—regardless of activation choice—crosses the compilable frontier. ReLU MLPs violate Condition 2 (no C^2 curvature parameter); SiLU/GELU MLPs violate Condition 2 (transcendental, Z3-undecidable) *and* Condition 1 (coupled linear+nonlinear); Sigmoid/Tanh MLPs satisfy Condition 2 but violate Condition 1. The SVNN conditions together form a **conjunction**—failure of either is sufficient to place an architecture on the exterior. This is why the compiler's ability to provide guarantees is an architectural question, not a compiler engineering question (Remark 1).

D. KAN Satisfies the SVNN Conditions

Proposition 1 (KAN Is Structurally Verifiable)**.** *Every Kolmogorov-Arnold Network (KAN) architecture, as defined by Liu et al. [10], satisfies the SVNN sufficient conditions (Conditions 1 and 2) of Theorem 2 and is therefore an SVNN architecture. The trained KAN student, however, does* not *satisfy the contractivity refinement (Condition 3): its measured per-layer amplification is $\gamma = [15.4, 5.3] > 1$ (E68). It is therefore certified under Conditions 1–2 with the finite depth-dependent bound; the depth-uniform (convergent) regime is available to contractively trained instances (future work).*

Proof. We verify Conditions 1 and 2, which establish SVNN membership, and then report the Condition 3 status:
Condition 1 (Operation-Type Closure). A KAN layer with d_{in} inputs and d_{out} outputs decomposes naturally as:

$$\text{KANLayer}(x)_j = \underbrace{b(x_j)}_{\text{Base path}} + \sum_{i=1}^{d_{\text{in}}} \underbrace{\phi_{j,i}(x_i)}_{\text{Spline path}} \tag{9}$$

where $b(\cdot)$ is a SiLU activation (element-wise univariate) and each $\phi_{j,i}(\cdot)$ is a cubic B-spline (element-wise univariate). The summation $\sum_{i=1}^{d_{\text{in}}} \phi_{j,i}(x_i)$ is a linear combination, equivalent to a MatMul with an identity-projected weight structure. The final addition $b(x_j) + \sum_i \phi_{j,i}(x_i)$ is element-wise linear. Thus every KAN layer consists solely of element-wise univariate functions and linear combinations—satisfying Condition 1.

This decomposition is exactly the basis of NeuroPLC's IR design: each KAN layer maps to a MatMul node, $d_{\text{in}} \times d_{\text{out}}$ BsplineLUT nodes, and d_{out} Add nodes (§IV-B1). The 6-op IR type system is not an arbitrary design choice—it is the minimal closure required by the SVNN decomposition of KAN.

Condition 2 (Univariate Boundedness). Cubic B-splines ($k = 3$) are C^2-continuous piecewise polynomial functions. On the bounded input domain $X = [-3, 3]$ used by the KAN student, the second derivative $\phi''(x)$ of each B-spline activation is bounded. The bound $M_2 = \max_x |\phi''(x)|$ is computable directly from the B-spline control points $\{w_c\}$ and knot vector $\{t_c\}$ via the derivative recurrence:

$$\phi''(x) = \sum_{c=0}^{G+k-2} w_c \cdot B''_{c,3}(x) \tag{10}$$

where $B''_{c,3}$ is computed from the B-spline basis derivative recurrence [33]. For the SiLU activation used in the base path, $\text{SiLU}''(x) = \sigma(x)(1 - \sigma(x))\left[2 + x(1 - 2\sigma(x))\right]$, which is bounded on any compact domain.

Analytical M_2 and L_B Upper Bounds. While $M_2 = \max_x |\phi''(x)|$ can be computed numerically from the B-spline control points (as done in E11, yielding the model-specific value 0.177), we further establish *analytical* upper bounds that depend only on the B-spline degree and knot spacing—providing a true a priori guarantee that requires no forward pass or empirical measurement.

B-spline derivative bounds on uniform knots. For a cubic B-spline ($k = 3$, order 4) defined on uniform knots with spacing h, the basis function derivatives satisfy the following analytical bounds (derived from the Cox-de Boor recurrence [33]):

$$\max_x |B'_{c,4}(x)| \le \frac{3}{h}, \qquad \max_x |B''_{c,4}(x)| \le \frac{6}{h^2} \tag{11}$$

For a B-spline curve $\phi(x) = \sum_c w_c B_{c,4}(x)$, at any x at most 4 basis functions are non-zero (local support property). Consequently:

$$\max_x |\phi'(x)| \le \frac{12 \cdot W_{\max}}{h}, \qquad \max_x |\phi''(x)| \le \frac{24 \cdot W_{\max}}{h^2} \tag{12}$$

where $W_{\max} = \max_c |w_c|$ is the maximum absolute control-point value. A tighter bound using control-point differences [33] yields:

$$L_B^{\text{analytic}} \le \frac{3}{h} \cdot \max_c |\Delta w_c|, \qquad M_2^{\text{analytic}} \le \frac{6}{h^2} \cdot \max_c |\Delta^2 w_c| \tag{13}$$

where $\Delta w_c = w_{c+1} - w_c$ and $\Delta^2 w_c = w_{c+1} - 2w_c + w_{c-1}$ are the first and second forward differences of the control-point sequence.

Application to KAN. For the KAN student $[28, 16, 4]$ with grid size $G = 8$ on input domain $[-3, 3]$, the knot spacing is $h = 6/G = 0.75$. With control points typically bounded by $W_{\max} \approx 0.3$ (enforceable via weight clipping during training), Eq. 12 yields the a priori bounds $L_B^{\text{analytic}} \le 12 \times 0.3/0.75 = 4.8$ and $M_2^{\text{analytic}} \le 24 \times 0.3/0.5625 = 12.8$. The tighter difference-based bounds (Eq. 13) reduce these to $L_B^{\text{analytic}} \le 4 \times \max |\Delta w|$ and $M_2^{\text{analytic}} \le 10.67 \times \max |\Delta^2 w|$, both computable from the control points alone before any forward pass.

Relationship to empirical values. The empirically measured values ($L_B = 2.21$ layer-0 / 2.05 layer-1, $M_2 = 0.177$ from E11) are substantially tighter than the analytical worst-case bounds, because the KAN's learned B-spline control points exhibit slow variation ($\max|\Delta w_c| \ll W_{\max}$). The analytical bounds serve as the **architectural guarantee** (proving that the architecture admits finite M_2 and L_B a priori), while the empirical measurements provide **model-specific refinement** (tightening the bound post-training). Both are computed from the model parameters alone—neither requires hardware execution. This two-tier approach satisfies SVNN Condition 2: M_f is analytically bounded by the architecture and computable from parameters, with the analytical bound providing the design-time certificate and the empirical refinement providing the deployment-time tightening.

Condition 3 (Contractive Composition, refinement). Conditions 1 and 2 already establish that KAN is structurally verifiable (Theorem 2, Step 3): the finite-depth error bound of Eq. 5 exists and is computable from the control points alone. Condition 3, *when* it holds, upgrades this guarantee to the depth-uniform, depth-independent form of Eq. 6. For the trained KAN student $[28, 16, 4]$, the honest checkpoint measurements (E68) are: $L_B = 2.21$ (layer-0) / 2.05 (layer-1)—the maximum of $|\phi'(x)|$ over all 512 B-spline activations on a 1,000-point grid— effective weight row-sum norms $\|W_{\text{eff}}\|_{1,\infty} = 12.36$ (layer-0) / 6.52 (layer-1), and per-layer Lipschitz bounds of 25.1 (layer-0) / 11.9 (layer-1). The measured per-activation-layer amplification of the trained student is $\gamma = [15.4, 5.3]$ (E68)—the trained network is *not* contractive, so the depth-uniform (convergent) form does *not* apply to this checkpoint. It remains structurally verifiable under Conditions 1–2 with the finite depth-dependent bound of Eq. 5, and contractive training (e.g., spectral normalization of the weight row sums) is the deployment recipe for the convergent form—future work. (The earlier estimate $L_B \cdot \|W\|_{1,\infty} = 0.65 \times 0.28 = 0.182$ double-counted the linear layers and is not a valid Lipschitz bound.)

Beyond this empirical finding, contractivity remains a structurally meaningful *target*: for KAN layers, the per-layer Lipschitz product $P^{(\ell)} = L_f^{(\ell)} \cdot \|W^{(\ell+1)}\|_{1,\infty}$ decomposes into a B-spline derivative factor and a weight row-sum factor, and both factors are regularization-controllable (weight decay, spectral normalization, λ-regularization of the spline coefficients). Condition 3 is thus a *trainable* property of KAN training rather than an automatic one—achieving it is the recipe for the depth-uniform certificate, and its absence does not affect verifiability under Conditions 1–2. $\square$

E. Standard MLPs Do Not Admit Tight SVNN Error Bounds

Having established that KAN architectures satisfy the SVNN conditions, we now show that the most widely deployed alternative—the Multi-Layer Perceptron (MLP) with standard activations—does not. This result clarifies *why* prior ML→PLC tools provide no correctness guarantees: the architecture itself precludes them.

Proposition 2 (Standard MLPs Do Not Admit the SVNN Tight-Bound Structure)**.** *Let $\mathcal{M}$ be a feedforward MLP using ReLU, SiLU, Sigmoid, or Tanh activations. Then $\mathcal{M}$ does not admit the compiler-computable architecture-independent tight LUT error bound of Condition 2, for the following reasons. This does* not *mean MLPs are unverifiable in general (tools such as Reluplex and α-β-CROWN verify ReLU networks)—it means the* KAN-style *de Boor curvature bound, which is computable from parameters alone without knowing the input distribution, does not apply to MLP activations.*

Proof. We analyze each activation function:

(a) SiLU / GELU (modern MLP standard): $f(x) = x \cdot \sigma(x)$. $\text{SiLU}''(x)$ is bounded and the function is C^∞, so Condition 2 is formally satisfiable. However, SiLU contains $\sigma(x) = 1/(1 + e^{-x})$, which uses the transcendental function e^{-x}. Z3's Nonlinear Real Arithmetic (NRA) theory is *undecidable* for transcendental functions, so per-function Z3 verification—the mechanism behind the compositional certificate of §VI-B—yields no proof for any SiLU segment. Empirically: MLP $[28, 16, 4]$ with SiLU activations achieves 0/16 Z3-verifiable components (E41), confirming that SiLU-based MLPs are incompatible with the compositional verification pipeline regardless of bound tightness.

(b) ReLU: $f(x) = \max(0, x)$. ReLU is piecewise-linear with kink at $x = 0$. A LUT whose knots include $x = 0$ would reproduce ReLU exactly ($\varepsilon = 0$), so the *existence* of a valid bound is not the issue. The issue is that the de Boor curvature bound (Eq. 2, $\varepsilon \le M_2 h^2/8$) requires C^2 continuity, which ReLU violates at $x = 0$ (f'' is a Dirac delta). Without the curvature characterization, the compiler cannot compute an a priori LUT error from the architecture and parameters alone: the error depends on where inputs fall relative to the kink, which is input-distribution-dependent. Existing tools that verify ReLU networks (Reluplex, α-β-CROWN) instead enumerate activation patterns via case-splitting, at exponential cost in the number of ReLU units—not by a single closed-form parameter-computable bound.

(c) Sigmoid / Tanh. Both are C^∞ with bounded M_2 ($\sup_x |\sigma''| \approx 0.096$, $\sup_x |\tanh''| = 2/\sqrt{27} \approx 0.385$), so Condition 2 is satisfiable. The bottleneck is the *mixing* within each MLP layer: each neuron combines linear and nonlinear operations in the same layer, while SVNN Condition 1 requires layers to be *either* purely linear *or* purely element-wise. Standard MLP layers violate Condition 1 (affine transform + pointwise activation in one step), breaking the clean compositional error accounting that makes per-function Z3 verification tractable.

Consequence for MLP compilation. The fundamental issue is not that MLPs cannot be compiled—they can, and NeuroPLC does compile MLPs (§IV). The issue is that the compiler cannot provide a *tight, architecture-aware error bound* for MLPs.

Without a per-function M_2 bound (Condition 2 violation for ReLU) or with only coarse global Lipschitz bounds (Sigmoid/Tanh), the best achievable MLP error bound is:

$$\Delta_{\text{MLP}}^{(L)} \leq O\big(L \cdot L_{\text{act}}^L \cdot \|\mathbf{W}\|_{\text{prod}}\big) \tag{14}$$

where $L_{\text{act}} = \max(1, L_{\text{ReLU}}, L_\sigma, L_{\tanh})$. For $L_{\text{ReLU}} = 1$, the bound grows as $O(L \cdot \|\mathbf{W}\|_{\text{prod}})$, which can easily exceed 1.0 for networks deeper than 3–4 layers with typical weight magnitudes. This makes the guarantee vacuous for all but the shallowest MLPs.

In contrast, the KAN error bound grows as $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$, which is independent of weight magnitudes (the linear layers are exact in DA) and decreases quadratically with LUT resolution h. This architectural distinction—not implementation quality—is why NeuroPLC provides guarantees for KAN that it cannot provide for MLP. □

F. Chebyshev-KAN Satisfies the SVNN Conditions

Having established that B-spline KAN satisfies the SVNN conditions (§III-D) and that standard MLPs do not (§III-E), we now demonstrate that the SVNN framework extends to alternative basis function families. We prove that KAN architectures using *Chebyshev polynomial* basis functions—a globally-supported, orthogonal polynomial basis—also satisfy Conditions 1 and 2, establishing that SVNN membership is not tied to the local-support property of B-splines but rather to the structural decomposition and curvature computability that both bases provide.

Proposition 3 (Chebyshev-KAN Is Structurally Verifiable)**.** *A KAN architecture using Chebyshev polynomial basis functions (ChebyKAN) [35], defined as*

$$\phi_{j,i}(x) = \sum_{n=0}^{N} c_{j,i,n} \cdot T_n(\tanh(x)) \tag{15}$$

where T_n is the Chebyshev polynomial of degree n and $c_{j,i,n}$ are learnable coefficients, satisfies SVNN Conditions 1 and 2 of Theorem 2. Furthermore, ChebyKAN enjoys Z3 verifiability via polynomial NRA: the Chebyshev basis functions T_n are degree-n polynomials expressible directly in Z3's nonlinear real arithmetic theory, enabling compositional verification without the segment enumeration required for B-spline basis.

Proof. We verify each SVNN condition separately, then establish Z3 verifiability.

Condition 1 (Operation-Type Closure). A ChebyKAN layer with d_{in} inputs and d_{out} outputs decomposes as:

$$y_j = \sum_{i=1}^{d_{\text{in}}} \left[\sum_{n=0}^{N} c_{j,i,n} \cdot T_n(\tanh(x_i)) \right] \tag{16}$$

This computation naturally separates into three pure operation types:

1) **Bound mapping (element-wise):** $u_i = \tanh(x_i)$ for each input i. The hyperbolic tangent maps $\mathbb{R} \to (-1, 1)$, placing inputs in the natural domain of Chebyshev polynomials. This is an element-wise univariate operation (TanhLUT node in the IR).
2) **Polynomial basis evaluation (element-wise):** For each pair (i, n), compute $v_{i,n} = T_n(u_i)$. Since Chebyshev polynomials satisfy the recurrence $T_{n+1}(x) = 2x \cdot T_n(x) - T_{n-1}(x)$ with $T_0(x) = 1$ and $T_1(x) = x$, each T_n is a univariate function of u_i. These are element-wise univariate operations (ChebyLUT nodes).
3) **Linear combination:** $y_j = \sum_{i,n} c_{j,i,n} \cdot v_{i,n}$ aggregates the basis-evaluated values via learnable coefficients. This is a linear transformation (MatMul node with coefficient matrix C).

Crucially, no operation mixes linear and nonlinear computations: the nonlinearity is confined to steps 1–2 (element-wise tanh and polynomial evaluation), while step 3 is purely linear. This decomposition satisfies Condition 1. The IR representation is:

$$\text{Input} \xrightarrow{\text{TanhLUT}} u \xrightarrow{\text{ChebyLUT}} v \xrightarrow{\text{MatMul}(C)} \text{Output} \tag{17}$$

where each node is provably of a single pure type (element-wise or linear).

Condition 2 (Univariate Curvature Bound). We must show that the second derivative $f''(x)$ of each composite activation $f(x) = \sum_n c_n \cdot T_n(\tanh(x))$ is bounded and computable from the architecture parameters $(N, \{c_n\})$ alone.

Step 2a (Tanh bound mapping): The function $h(x) = \tanh(x)$ is C^∞ with bounded second derivative. The analytical bound is:

$$\sup_{x \in \mathbb{R}} |\tanh''(x)| = \frac{2}{\sqrt{27}} \approx 0.385 \tag{18}$$

This is a universal constant, independent of any learnable parameters.

Step 2b (Chebyshev polynomial curvature): Let $g(y) = \sum_{n=0}^{N} c_n \cdot T_n(y)$ be the polynomial of degree N evaluated on $y \in (-1, 1)$. Chebyshev polynomials T_n are degree-n polynomials satisfying $|T_n(y)| \leq 1$ for all $y \in [-1, 1]$ [36]. By the Markov Brothers' Inequality [37], for any polynomial p of degree N on $[-1, 1]$:

$$\max_{y\in[-1,1]} |p'(y)| \leq N^2 \cdot \max_{y\in[-1,1]} |p(y)| \tag{19}$$

$$\max_{y\in[-1,1]} |p''(y)| \leq \frac{N^2(N^2-1)}{3} \cdot \max_{y\in[-1,1]} |p(y)| \tag{20}$$

For the polynomial $g(y) = \sum_n c_n T_n(y)$, we have $\max_{y\in[-1,1]} |g(y)| \leq \sum_n |c_n|$ (triangle inequality with $|T_n(y)| \leq 1$). Applying Eq. 20:

$$M_2^{\text{Cheby}}(g) := \max_{y\in[-1,1]} |g''(y)| \leq \frac{N^2(N^2-1)}{3} \cdot \sum_{n=0}^{N} |c_n| \tag{21}$$

This bound is *computable from parameters alone*: given the polynomial degree N and the learned coefficients $\{c_n\}$, the compiler can compute the right-hand side of Eq. 21 without executing the model on any inputs. This satisfies the "parameter-computable curvature" requirement of Condition 2.

Step 2c (Composition via chain rule): The full activation is $f(x) = g(\tanh(x))$. Applying the chain rule:

$$f''(x) = g''(\tanh x) \cdot \text{sech}^4(x) - 2g'(\tanh x) \cdot \tanh(x) \cdot \text{sech}^2(x) \tag{22}$$

Since $|\text{sech}(x)| \leq 1$ and $|\tanh(x)| < 1$ for all x, we bound:

$$|f''(x)| \leq |g''(\tanh x)| + 2|g'(\tanh x)| \leq M_2^{\text{Cheby}}(g) + 2M_1^{\text{Cheby}}(g) \tag{23}$$

where $M_1^{\text{Cheby}}(g) \leq N^2 \cdot \sum_n |c_n|$ by Eq. 19. Substituting Eq. 21:

$$M_2(f) \leq \left[\frac{N^2(N^2-1)}{3} + 2N^2\right] \cdot \sum_{n=0}^{N} |c_n| = \frac{N^2(N^2+5)}{3} \cdot \sum_{n=0}^{N} |c_n| \tag{24}$$

Equation 24 provides an *a priori* curvature bound computable from the polynomial degree N (an architectural hyperparameter) and the learned coefficients $\{c_{j,i,n}\}$ (model parameters). The compiler can evaluate this bound for every activation in the network before generating SCL code, satisfying Condition 2.

Z3 Verifiability. The compositional verification pipeline (§VI-B) requires that each activation function can be abstracted as a finite set of polynomial constraints verifiable by Z3's SMT solver. ChebyKAN satisfies this requirement in two ways:

1) **Polynomial basis is NRA-native.** Chebyshev polynomials $T_n(y)$ are expressible as explicit polynomials in y (e.g., $T_2(y) = 2y^2 - 1$, $T_3(y) = 4y^3 - 3y$). Z3's Nonlinear Real Arithmetic (NRA) theory handles polynomial equalities and inequalities natively [38], making each segment's verification a polynomial feasibility query.
2) **Tanh preprocessing is standard.** The tanh bound mapping $u = \tanh(x)$ is treated identically to B-spline's SiLU base path: a C^∞ univariate function approximated via LUT, whose error bound is $M_{\tanh}h^2/8$ (Eq. 18). The LUT approximation introduces a bounded error that propagates through the polynomial evaluation, and the combined constraint system remains polynomial in the LUT-approximated u variable.

In contrast to B-spline basis functions—which require segment-by-segment enumeration due to their local support—Chebyshev polynomials are *globally supported*: each T_n is defined on the entire interval $(-1, 1)$. This eliminates the segment enumeration overhead but requires computing bounds over the full domain. The tradeoff is: B-spline achieves tighter per-segment bounds via the segment-aware M_2 refinement (§IV-B10), while ChebyKAN uses global polynomial bounds (Eq. 21) that are looser but analytically simpler. Both approaches yield finite, computable bounds—the SVNN requirement.

Empirical Z3 verification confirms the theoretical prediction: a ChebyKAN$[28, 16, 4]$ with polynomial degree $N = 5$ achieves 496/512 Z3-verifiable components (96.9%, E54, §VI-D), demonstrating that the polynomial NRA encoding successfully produces verifiable constraints for the majority of activations. The Markov-based analytical M_2 bound (Eq. 21) is $5.4\times$ looser than the empirical M_2 (mean 8.5 vs. 45.6 in Layer 0, E54), confirming that the worst-case nature of Markov's inequality is the primary source of non-verifiability. The 16 unverifiable components (3.1%) correspond to activations where this global bound exceeds Z3's numeric tolerance—a fundamental limitation of any global polynomial bound rather than a deficiency of ChebyKAN specifically.

Contrast with MLP+Tanh (Structural vs. Activation-Based Distinction). Table I shows that MLP+Tanh lies on the *exterior* (violates Condition 1) despite using the same tanh activation that ChebyKAN uses for bound mapping. This contrast isolates the structural nature of the SVNN conditions: it is not the activation function itself that determines SVNN membership, but rather how the architecture *decomposes* that activation relative to linear transformations.

- **MLP+Tanh:** The standard MLP layer computes $\mathbf{h} = \tanh(W\mathbf{x} + \mathbf{b})$, coupling the affine transformation $(W\mathbf{x} + \mathbf{b})$ and the nonlinear activation $\tanh(\cdot)$ in a single operation. There is no intermediate variable whose error can be separately bounded—the structural induction of Theorem 2 cannot be initiated. MLP+Tanh thus violates Condition 1.

- **ChebyKAN:** The ChebyKAN layer first applies tanh *element-wise* to each input (step 1), then evaluates polynomial basis functions element-wise (step 2), and finally performs a linear combination (step 3). Each step is provably pure (either element-wise or linear), enabling the per-operation error induction. ChebyKAN satisfies Condition 1.

This distinction underscores the central insight of the SVNN framework: *architectures must be designed for verifiability*. The same activation function (tanh) yields verifiable or non-verifiable behavior depending on whether the architecture exposes the structural decomposition required by Condition 1. The compiler cannot "fix" an architecture that couples operations; it can only exploit the structure that the architecture provides.

Comparison with B-spline KAN (Local vs. Global Basis). Proposition 1 established that B-spline KAN satisfies the SVNN conditions via *segment-aware* curvature bounds: each B-spline is piecewise-polynomial with C^2 continuity, and the compiler computes per-segment $M_2^{(k)}$ values tightening the global bound by $6.0\times$ on average. Proposition 3 shows that ChebyKAN satisfies the same conditions via *Markov-based global bounds*: the Chebyshev polynomial $g(y)$ has a single, analytically-computable M_2 bound (Eq. 21) that applies to the entire domain $(-1, 1)$.

The two bases represent complementary points on the tightness-complexity spectrum:

- **B-spline (local support):** Tighter bounds via segment-aware refinement, but requires $O(G)$ segment enumeration where G is the grid size. The segment-aware $M_2^{(k)}$ bounds exploit the locality of B-spline support: outside a basis function's $k+1$ non-zero knot intervals, it contributes zero error.
- **Chebyshev (global support):** Single global bound via Markov's inequality (Eq. 21), computed in $O(1)$ for the entire activation. The bound is looser (worst-case over the full domain) but analytically simpler and requires no segment enumeration.

Both approaches yield *computable* bounds from parameters alone, satisfying Condition 2. The SVNN framework is thus robust to the basis function choice: it accommodates both local-support (B-spline) and global-support (Chebyshev) polynomial bases, as long as the curvature M_2 is analytically bounded and the structural decomposition (Condition 1) holds. This generality confirms that SVNN conditions characterize a fundamental architectural property, not an artifact specific to one basis family.
□

Remark (Wavelet-KAN and Fourier-KAN). The proof technique of Proposition 3—verifying Condition 1 via explicit decomposition and Condition 2 via analytical derivative bounds—extends naturally to other orthogonal polynomial and wavelet bases. Wavelet-KAN [35], which uses a Mexican-hat mother wavelet, satisfies Condition 1 by the same decomposition argument (element-wise wavelet evaluation + linear combination). Condition 2 holds for the Mexican-hat basis, whose curvature constant $\sup_t |\psi''(t)| = 2.602$ is known analytically; this is not true of all wavelet families (e.g., Morlet wavelets involve Gaussian terms and violate Condition 2). Fourier-KAN, using $\sin$ and $\cos$ basis functions, satisfies Condition 1; direct Z3 reasoning over transcendental $\sin/\cos$ is undecidable, so E60 verifies the LUT-encoded evaluations instead, recovering 512/512 Z3-equivalence (Table XXXVII). The same encoding argument applies to the SiLU activations (Section III-X). Table I marks Wavelet-KAN with "$\sim$" (depends on wavelet choice) to reflect this nuance: the SVNN conditions are *basis-specific*, not universally applicable to all function approximation schemes.

G. Verification-Cost Separation in the ε-Dimension

Theorem 2 establishes that Conditions 1–2 are *sufficient* for structural verifiability. We now establish a *correct* complexity separation. The earlier claim—that tight-bound computation is NP-hard for non-SVNN architectures while SVNN admits polynomial-time bounds—compares an exact quantity (NP-hard for *both* classes) with an upper bound (polynomial for *both* classes: interval propagation and CROWN are polynomial-time for MLPs), so it does not separate the classes. The valid separation operates in the ε-*dimension*: the *decision problem itself* is polynomial for SVNN and NP-hard for general coupled architectures, and SVNN achieves the Pareto point (polynomial time, zero slack) simultaneously—which no non-SVNN method can.

Theorem 5 (ε-Separation of LUT-Verification Complexity)**.** *Let ε-Verify$(N, \mathrm{LUT}, \varepsilon)$ be the decision problem: "is $\sup_{x\in\Omega} |N(x) - \mathrm{LUT}(x)| \le \varepsilon$?" for a feedforward network N on a compact box Ω.*

1) ***SVNN regime (polynomial).*** *For SVNNs with order-k LUTs, the sharp bound of Theorem 9 is computable in $O(L \cdot d_{\max}^2 \cdot N)$ design time ($N = \Theta(\varepsilon^{-1/k})$ cells to certify error $\le \varepsilon$) and $O(L \cdot d_{\max}^2)$ evaluation time; the ε-decision is $O(1)$ given the LUT (compare the closed-form bound to ε). For B-spline KAN activations with* knot-aligned *LUT grids, the order-k LUT reproduces the spline* exactly *(polynomial reproduction: each segment is a cubic polynomial, degree-3 interpolation through 4 points in a segment is exact; verified to 5×10^{-8} per segment, E65). For non-aligned equispaced grids the interpolation error is bounded by the sharp constants of Theorem 9—still $O(h^k)$, so ε is driven to zero at the standard rate.*
2) ***Non-SVNN regime (NP-hard, self-contained).*** *Let N_φ be the network with product gates computing*

$$F(x) \;=\; \sum_j \prod_{i\in C_j} \bigl(1 - \ell_{ij}(x)\bigr) \tag{25}$$

for a 3-SAT formula φ with clauses C_j and literals ℓ_{ij} affine in $x \in [0,1]^n$ (arity-3 coupled gates, one hidden layer, $O(n+m)$ neurons). Then $F(x) = 0$ if and only if φ is satisfiable: $F(x^*) = 0$ forces every clause to have a literal with *value exactly* 1*; the induced assignment is consistent (no complementary pair can both be* $1—\ell = x_v$ *and* $\ell' = 1 - x_v$ *cannot both equal* 1*), so* φ *is satisfied; conversely a satisfying boolean assignment gives* $F = 0$*. Hence the exact problem* ε*-Verify with* $\varepsilon = 0$ *is NP-hard (3-SAT reduces in polynomial time with* $O(n + m)$ *neurons).*

3) ***Gap version (NP-hard for every constant*** ε_0***).*** *The published gap-preserving reductions for ReLU verification (Katz et al. [28]; Tjeng et al.; Sälzer–Lange) produce networks with a constant separation between SAT and UNSAT cases, implying that* ε_0*-Verify is NP-hard for every fixed* $\varepsilon_0 < 1$ *(after normalization): deciding whether a poly-time-checkable piecewise-linear approximant is within any constant* ε_0 *of a general network is NP-hard. Under the Exponential-Time Hypothesis, exact verification of general networks requires* $2^{\Omega(n)}$ *time.*
4) ***Pareto corollary.*** *For SVNNs the point* $(\text{time} = \text{poly}, \text{slack} = 0)$ *is* simultaneously achievable*; for general networks, any polynomial-time sound bound has additive slack* $\geq \varepsilon_0$ *(else it would decide* ε_0*-Verify on the gap instances), and any slack-*0 *bound costs* $2^{\Omega(n)}$ *(ETH). No point of the non-SVNN frontier dominates any point of the SVNN frontier in both coordinates.*

Proof. **(1)** The closed-form bound of Theorem 9 (Eq. 35) requires scanning $O(N)$ cells per activation; there are $O(L \cdot d_{\max}^2)$ activations, giving $O(L \cdot d_{\max}^2 \cdot N)$ design time; the evaluation is the LUT lookups themselves, $O(L \cdot d_{\max}^2)$. Spline reproduction: a cubic B-spline segment is a cubic polynomial; the order-4 LUT (degree-3 interpolation through 4 points per window) reproduces it exactly (verified numerically, E65). **(2)** The product-gate encoding F: each clause contributes a non-negative term $\prod_i (1 - \ell_{ij}(x)) \geq 0$, so $F(x) = 0$ iff every term vanishes iff every clause has some literal $\ell_{ij}(x) = 1$. For fractional x^*, the set of literals with value 1 induces a consistent boolean assignment: if $x_v^* = 1$ is required by one clause and $x_v^* = 0$ by another, then $x_v^* = 1$ and $1 - x_v^* = 1$ simultaneously—impossible. Hence φ is satisfiable; the converse is immediate. The reduction is self-contained (verified on 200 random formulas with $n = 5$ variables, both directions, including fractional-grid search, E65). **(3)** Standard gap-preserving reductions (cited); ETH transfer is standard. **(4)** By contrapositive: a polynomial-time certifier with slack $< \varepsilon_0/2$ would decide ε_0-Verify on the gap instances, contradicting (3); a slack-0 bound decides the exact problem, requiring $2^{\Omega(n)}$ under ETH. □

Remarks. *(i) The single-layer case is polynomial for piecewise-linear gates.* For a *single* fused layer with a piecewise-linear gate σ (e.g., ReLU), the supremum $\sup_x |\sigma(Wx + b) - \tilde{\sigma}(Wx + b)|$ is a piecewise-linear function of x whose supremum over a polytope is the max of finitely many linear programs—polynomial. (For piecewise-cubic gates such as B-spline activations, the single-layer supremum is also polynomial: each segment's error is a cubic polynomial in the scalar pre-activation, whose supremum over an interval is found by root-finding on the derivative in $O(1)$ per segment.) The hardness in part (2) lives in the *nested* composition of coupled gates (the product structure), not in any single fused layer; earlier reductions to "single-layer NP-hardness" were incorrect and are superseded here. *(ii) Earlier framing corrected.* The previous version compared the tight bound (NP-hard for both classes) with an upper bound (polynomial for both classes, since CROWN/interval bounds are polynomial-time for MLPs [29], [39]), which does not separate the classes. The valid separation is in the ε-dimension as stated above: the *decision problem* is poly for SVNN and NP-hard for coupled architectures, and SVNN uniquely attains (polynomial time, zero slack) simultaneously. *(iii) Empirical demonstration (E65).* For a trained SVNN the closed-form bound computes instantly at any N; brute-force supremum enumeration over the input box grows as 3^n (n input dimensions): 124× at $n = 4$, 641× at $n = 6$, 5,719× at $n = 8$—the separation is observed, not just proven.

H. Compile-Aware Generalization Bound

The SVNN framework has so far addressed the *compilation* side: whether a trained model can be deployed with correctness guarantees. We now establish the *compile-aware* generalization bound: the object of the guarantee is the *deployed* (LUT-compiled) network $\bar{N}$, not the trained network N. This unifies learning theory with LUT discretization in a single two-scale bound, and yields a novel, directly actionable *resolution-matching law* for embedded deployment: the LUT memory should scale as the $2k$-th root of the sample count.

Theorem 6 (Compile-Aware Generalization)**.** *Let* N *be an SVNN with* L *affine layers and* $L - 1$ *activation layers, with per-activation-layer Lipschitz product* $\leq \gamma$ *(an* amplification factor*;* $\gamma < 1$ *is Condition 3 contractivity but the bound is valid for* $\gamma \geq 1$ *as well), trained on* n *i.i.d. samples; let* $\bar{N} = \text{Compile}(N, h)$ *be the deployed order-*k *LUT version with cell width* h*.* ***Assume the intermediate signals of every layer remain within the LUT table's validated grid range****—this is precisely what the Box-Continuation Lemma 1 certifies, by propagating reachable boxes in polynomial time and checking containment in the (finite) table domain; where containment fails, the table must be widened (per-layer domains) or the deployment flagged. (The B-spline* activation *is defined on all of* $\mathbb{R}$*, but the* LUT table *is finite; signals outside the grid hit the clamped-endpoint behavior, an* N*-independent error the bias term does not cover—E53 and E68 measure exactly this at depth* ≥ 2*.) Let* ℓ *be the loss, Lipschitz* L_ℓ*, with outputs and targets bounded in* $[-C, C]$*, inputs in a compact box* $\Omega \subset [-B, B]^{d_{in}}$*. Then with probability at least* $1 - \delta$ *over the sample:*

$$R(\bar{N}) \;\leq\; \hat{R}(N) + C_1 L_\ell \gamma^{L-1} \sqrt{L\, d_{\max}} \sqrt{\frac{\log(2d_{\max}) + \log(1/\delta)}{n}} + C_2 \gamma^{L-1} L \big(c_k M_k h^k + \varepsilon_{\text{fp}} \|\text{scale}\|\big), \tag{26}$$

where $c_k = Q_k/k!$ is the sharp LUT constant (Theorem 9), $M_k = \max_f \|f^{(k)}\|_\infty$ over activations, and C_1, C_2 are absolute constants. Equivalently, the deployed risk decomposes as

$$R(\bar{N}) = \hat{R}(N) + \underbrace{O\left(\frac{\gamma^{L-1}}{\sqrt{n}}\right)}_{\textit{gap}} + \underbrace{O\big(\gamma^{L-1} c_k M_k h^k\big)}_{\textit{LUT bias}}, \tag{27}$$

with two-scale convergence: gap $\to 0$ at rate $n^{-1/2}$, bias $\to 0$ at rate N^{-k} ($N = (b-a)/h$ LUT points).

Relationship to compression-based generalization bounds. The deployed network $\bar{N}$ is a *deterministic, structural* compression (LUT discretization) of the trained network N. The nearest statistical neighbor is the ε-lossy-compression conditional-mutual-information (CMI) bound of Sefidgaran et al. [40], which compresses the hypothesis by random projection and stochastic quantization and bounds generalization through the CMI of the compressed algorithm. Two differences matter for deployment: (i) our compression is deterministic and verification-targeted (the LUT grid is the object of the certificate, not a proxy), so the bias term is explicit ($O(\gamma^{L-1} c_k M_k h^k)$) and instantiated per checkpoint; (ii) the two-scale decomposition separates the sample gap from the LUT bias, yielding the explicit memory law $N^* \asymp n^{1/(2k)}$—a compiler rule that compression-based bounds do not state.

Proof sketch. **(1) Bias term.** By Theorem 9, each activation layer ℓ contributes pointwise error $|e_\ell| \le c_k M_k^{(\ell)} h_\ell^k$; propagation through the remaining $L-\ell$ linear maps amplifies by $\prod_{j>\ell} \|W_j\|_\infty \le \gamma^{L-1}$ (Condition 3 gives the per-activation-layer product $\le \gamma^{L-1}$ for the remaining activation layers; the Box-Continuation Lemma below validates that the reachable intermediate signals lie within the LUT domains, so the M_k constants apply without reachability computation). Floating-point roundoff contributes $\varepsilon_{\mathrm{fp}}\|\mathrm{scale}\|$ per layer. **(2) Gap term.** Rademacher complexity of the SVNN class with spectrally bounded weights, using the Lipschitz-class bound: for a class of Λ-Lipschitz functions on inputs bounded by B in ℓ_∞, the empirical Rademacher complexity is $\le \Lambda \cdot 2B\sqrt{d_{\max}/n}$ (standard covering argument, cf. Bartlett–Mendelson [41]); with $\Lambda = L_\ell \gamma^{L-1}$ and a union bound over $d_{\max}$ output coordinates this yields the stated $C_1 L_\ell \gamma^{L-1} \sqrt{L\, d_{\max}} \sqrt{(\log(2d_{\max}) + \log(1/\delta))/n}$ (the $\sqrt{L}$ factor is a crude union over layers, conservative in the safe direction). This corrects the earlier "factor 4" inflation (which arose from applying Talagrand's contraction twice on one activation layer, and whose log-term arithmetic was also in error: $3\sqrt{\log(2/\delta)/(2n)} = 0.0389$, not 0.0117, at $n = 10{,}971$, $\delta = 0.05$; and $2\rho\gamma^L B/\sqrt{n} = 0.0038$, not 0.0019). **(3) Decomposition** by the triangle inequality: $R(\bar{N}) \le R(N) + L_\ell \|N - \bar{N}\|_\infty$, then $R(N) \le \hat{R}(N) + \mathrm{gap}$. $\square$

Lemma 1 (Box Continuation: LUT Domains Cover the Reachable Set)**.** *For an SVNN with order-k LUTs, the LUT-compiled intermediate boxes propagate* exactly *through the piecewise-affine layers in $O(L \cdot d_{\max}^2 \cdot N)$ time (scan the $O(N)$ cells intersecting each box), and the true signals lie within the LUT boxes inflated by the accumulated propagated errors. Hence the M_k constants are validated on the* reachable set *with no reachability computation. For compact-support B-spline KAN the activation domain is all of $\mathbb{R}$, so the lemma is trivial; for activations with domain-dependent curvature it is what makes the bias term of Theorem 6 sound. (Empirically: without this condition, intermediate features can leave the LUT grid—the E53 measurement of in-domain worst-case error ≈ 17 at every LUT density is exactly this phenomenon: the LUT's clamped-endpoint behavior diverges from the exact B-spline's zero-extension outside the grid, and the discrepancy is N-independent.)*

Corollary 1 (Resolution Matching: Optimal Compilation)**.** *Minimizing the two-scale bound (Eq. 27) over h (equivalently N) balances the two terms:*

$$h^* = \Theta\left(\left(\frac{\sqrt{L\, d_{\max}/n}}{c_k M_k L}\, \frac{C_1 L_\ell}{C_2}\right)^{1/k}\right), \qquad N^* = \Theta\Big(n^{1/(2k)} \cdot \big(c_k M_k L^{1/2} d_{\max}^{-1/2}\big)^{1/k}\Big). \tag{28}$$

The qualitative law: deployed LUT memory should scale as the $2k$-th root of the sample count *(fourth root for $k = 2$). Overshooting wastes PLC memory with zero statistical gain; undershooting destroys the learned function. This is a compiler rule that no existing PAC bound states, and it makes the compilation resolution a function of the dataset size at compile time.*

Corollary 2 (Fixed-Depth Tradeoff)**.** *For fixed (n, h), the gap term of the bound (Eq. 26) scales as $\gamma^{L-1}\sqrt{L}$ and the bias term as γ^{L-1} in depth L. Two regimes follow.* ***Contractive regime*** *($\gamma < 1$): both terms* decay *with depth (per added layer the gap shrinks by a factor $\gamma\sqrt{(L+1)/L}$ and the bias by $\gamma(L+1)/L$), so adding a layer strictly helps as long as the training risk does not degrade: deeper SVNN networks generalize strictly better, and no sample-count threshold is required.* ***Non-contractive regime*** *($\gamma > 1$): the bound is* non-monotone *in L—the gap and bias grow per added layer—so depth helps only if the training-risk reduction beats the gap growth: $\Delta\hat{R} \ge C\,\gamma^{L-1}(\gamma - 1)\sqrt{L\, d/n}$ (logarithmic factor absorbed into C), equivalently $n \ge n^*(\gamma, L, \Delta\hat{R}) = \Theta(\gamma^{2(L-1)} L d/(\Delta\hat{R})^2)$ in this regime. The earlier claim that "deeper SVNN networks generalize strictly better" holds exactly in the contractive regime; for $\gamma > 1$ the corrected statement is the quantitative threshold above. (The prior version stated the threshold as $(\gamma - 1)C\sqrt{\cdot}$ with the non-monotonicity assigned to $\gamma < 1$; that condition is vacuous there, since both factors are < 1.)*

Quantitative instantiation and empirical validation (E68). For the trained KAN $[28, 16, 4]$ with input domain $[-3, 3]^{28}$ ($B = 3$), cross-entropy loss ($L_\ell = 1$), and $n = 10{,}971$ training samples, the bound (Eq. (26)) evaluates as follows. The trained network is *not* contractive: the measured per-activation-layer amplification is $\gamma = [15.4, 5.3]$ (E68), giving $\gamma^{L-1} = 15.4$ for the bias term; the gap term $C_1 L_\ell \gamma^{L-1} \sqrt{L d_{\max}} \sqrt{(\log(2d_{\max}) + \log(1/\delta))/n}$ evaluates to ≈ 1.5 at $\delta = 0.05$ (dominated by γ^{L-1}), and the log-term alone $3\sqrt{\log(2/\delta)/(2n)} = 0.0389$ (corrected from the earlier 0.0117, whose arithmetic was inconsistent with its own formula; the earlier 0.0019 term had a wrong γ exponent and ρ). The empirical generalization gap (training vs. test) is 0.036% (E68), far below the worst-case gap term—as expected, since the $n^{-1/2}$ rate is a worst-case envelope not achieved on a well-separated task (measured gap decays as $n^{-0.84}$); the learning-theoretic picture is completed by the optimal Besov approximation rates of residual KANs [42], which provide the matching statistical upper bound for the smoothness class that the compile-aware decomposition targets. The resolution-matching law (Corollary 1) predicts $N^* \asymp n^{1/4} \approx 10{,}971^{1/4} \approx 10.3$ for $k = 2$, matching the deployed $N = 15$ within $1.5\times$.

Empirical findings (E68, honest report). The decomposition and the sharp-constant law are confirmed at near-tightness: per-activation LUT error vs. $c_2 M_2 h^2$ measures 0.996–1.000 at $N = 8/64/256$ ($M_2^{\max} = 0.178$), and the layer-1 logit bias tracks $h^2 = (6/(N-1))^2$ with slope 0.99 ($r^2 = 0.999$) across $N \in \{8, \dots, 256\}$. Three findings require qualification. **(i) Layer-2 domain violation:** 47% of layer-1 hidden activations fall outside $[-3, 3]$ (range $[-23.9, +25.1]$), so the layer-2 LUT is queried substantially outside its table and the N-independent clamping error dominates the h^k term—the Box-Continuation assumption (Lemma 1) is violated at depth ≥ 2 for this checkpoint. This is the same phenomenon measured independently in E53 (in-domain worst-case error ≈ 17 at every N), confirming that per-layer domain bounds are not optional theory but a deployment requirement. **(ii) The trained network is not contractive:** measured per-layer amplification $\gamma = [15.4, 5.3] > 1$, so γ acts as an amplification factor (which the bound accommodates; contractivity is a training target, not an automatic property). **(iii) Resolution matching is weakly supported:** the empirical knee N^* grows with n (slope 0.14 ± 0.09 vs. predicted 0.25); the 0-1-loss variant of the knee is *unmeasurable*: at near-ceiling accuracy (test error 0.04%) the 0-1 risk is saturated and its measured knee exponent (-0.10 ± 0.15) is not statistically distinguishable from zero (E68, reported honestly; the cross-entropy knee is the reliable estimator and shows the predicted positive trend). The empirical gap decays as $n^{-0.84}$ (faster than the worst-case $n^{-1/2}$, as expected on a well-separated task). These findings sharpen—rather than weaken—the theory: the h^k bias term is exact where its domain assumption holds, and the assumption itself is certified-polynomial to check (Lemma 1).

Soft-contractive training (E-$T9$, 2026-08-04). The non-contractive checkpoint raises a natural question: can Condition 3 be approached by training? We trained KAN $[28, 16, 4]$ on CWRU with per-step numeric γ projection (E68 semantics) and a soft γ penalty. The trained model achieves 98.5% test accuracy with measured amplification $\gamma = [1.03, 1.38]$—a $15\times/3.8\times$ component-wise reduction from the released checkpoint's $[15.4, 5.3]$ ($15.4 \to 1.03$; $5.3 \to 1.38$)—at a 1.4 pp accuracy cost (`train_contractive_kan.py`, four runs, $\gamma \to 1$ monotonically as the target tightens). Full contractivity ($\gamma < 1$) is not reached: the B-spline basis amplitude ($\sup_x |B(x)| \geq 1$) is an architectural constant that weight projection cannot change, and the task's expression demand resists further compression. The expressivity–contractivity trade-off is quantitative and explicit: the γ^{L-1} factor of Theorem 6 is purchased with accuracy, exactly as the capacity framework (§III-S) predicts (the contraction budget is an expression budget). Full contractivity for B-spline KANs requires a bounded-amplitude basis family—and the C^2-BV family provides exactly that (FourierKAN, $|\sin/\cos| \leq 1$; WaveletKAN, bounded Mexican-hat). Training these bounded-amplitude bases with the same soft-γ recipe (2026-08-04) attains certificates at essentially zero accuracy cost: FourierKAN ($\gamma = [0.82, 1.68]$, 99.96% test accuracy) carries a validated bound of 0.138 (safety $4.9\times$, measured floor 0.126; its first-principles envelope 0.110 does not cover the full-set measured 0.126, which includes 2.7% out-of-domain inputs) with 97.3% Box-Continuation coverage, and—at the L0-direct deployment configuration (input layer evaluated analytically via the PLC's $\sin/\cos$ instructions, $28 \times 16 \times 12$ harmonic evaluations; the L1 LUT at $N{=}16$ on $[-3.15, 3.15]$ covers 100% of the measured L0 outputs; a compiler extension, distinct from the shipped full-LUT $N{=}15$ backend)—a *sound* bound of 0.130 (safety $5.2\times$; 2026-08-05, `verify_fourier_l0direct.py`), which covers the measured 0.091 (L0-direct deployment) with $1.4\times$ headroom. The configuration compiles to 4,363-line SCL (`e60e61_l0direct.py`): L0 emits software $\sin/\cos$ harmonic evaluations (S7-1200 float library), L1 emits the $N{=}16$ LUT with the standard binary-search structure; a float32 SCL simulation over the full 13,714 inputs reproduces the measured maxAE 0.091 (covered by the sound bound; IEEE-754 rounding is below 10^{-5}), and the WCET estimate is ≈ 24.8 ms ($4.0\times$ scan-cycle margin; the software transcendental library is the cost of removing the L0 LUT). WaveletKAN ($\gamma = [0.92, 1.28]$, 99.67%) carries a first-principles sound bound of 0.237 (safety $2.8\times$; covers measured 0.157) with 100% coverage (`train_c2bv_contractive.py`, `verify_c2bv_bound.py`). The accuracy cost of the certificate is 0.03 pp (Fourier) and 0.26 pp (Wavelet) versus the 1.4 pp of the B-spline recipe, both against the same 99.93% B-spline baseline (E1): for bounded-amplitude bases the contraction budget is nearly free, because the basis amplitude is an architectural constant rather than a learned quantity. The $\gamma \to 1$ branch of Theorem 6 therefore holds across the C^2-BV family, with the certificate price set by the basis, not the task.

Deployment certificate of the soft-contractive checkpoint. The γ reduction is not merely cosmetic: it converts the certificate into a full-chain, wide-coverage guarantee. On the soft-contractive checkpoint (98.5% test accuracy), the measured constants collapse relative to the released student: $L_B = [0.28, 0.23]$ (vs. $[2.21, 2.05]$), the no-cancellation amplification constant $K = 1.2$ (vs. 340; K is the per-layer-Lipschitz form—the global-L_B form of Thm. 1 gives $L_{\text{net}}^{\text{IA}} = 171.1$ for the released student; the two no-cancellation definitions differ in Lipschitz granularity), and the M_2 profile is far lighter ($M_2^{\text{char}} = 0.051$ vs. 0.218, full-φ float64 methodology). The E68-constant bounds tighten accordingly: the expected-tier bound evaluates to 0.0014 at the M_2^{char} calibration (safety $493\times$, high-probability in the random-sign sense of Lemma 3); the worst-function envelope (full-φ $M_2^{\max}$, float64 ground truth) is 0.039 (safety $17.5\times$). The certificate is two-tiered, the tiers distinguished by *what they guarantee*. **Theoretical tier (sound; every in-domain input, including unseen):** the no-cancellation propagation with the corrected layer semantics (layer-0 LUT errors sum directly at the hidden nodes, $dy_j = \sum_i \varepsilon_{j,i}$; layer-1 amplification uses per-edge Lipschitz constants $L_{k,j}$, not a global bound) evaluates to 0.288 (safety $2.34\times$)—a first-principles envelope with no empirical floor. It strictly covers the float32 deployment measurement (0.053 over 13,714 inputs, Tier-4 simulator; $5.4\times$ headroom); the IEEE-754 accumulation δ_{fp32}, bounded by the per-operation 0.5 ulp rule propagated through the network ($\lesssim 3 \times 10^{-5}$ on the logit scale), is a negligible addendum to the LUT envelope. **Calibrated tier (empirically validated; the 13,714 measured inputs):** the measured-floor bounds $0.058 = \max(0.039, 0.053 \times 1.1)$ (safety $11.6\times$) for the float32 SCL backend, and 0.026 (safety $26\times$) for a float64 backend (float64 simulator floor 0.023; $0.026 \approx 0.023 \times 1.1$, conservative rounding), quantify the *realized* error on the validation corpus and tighten the deployment margin to the actual arithmetic. Unlike the theoretical tier, they certify the inputs on which they were measured (13,714 Tier-4 samples); their extension to unseen inputs is an empirical generalization claim, not a proof, and they are labeled as such. The gap between the tiers is the conservatism of worst-case propagation versus the realized error profile—quantified by the differential simulator, not by the LUT analysis. The certificate therefore carries a deployment precision tier: $11.6\times$ (float32, measured) / $26\times$ (float64 backend, measured floor), always under the theoretical $2.34\times$ floor. Contrast the released student: its per-function envelope is 2.29 (safety $0.3\times$—*not* a certificate), and its measured maxAE (0.52) is $4.4\times$ below that envelope: the design-time guarantee does not exist for the untrained checkpoint, and the empirical margin is a lucky artifact—precisely the E52 lesson. Soft-contractive training flattens the curvature profile itself ($M_2^{\text{char}} = 0.051$ vs. 0.218, full-φ methodology), in addition to compressing the weight scale ($K = 1.2$ vs. 340), converting a non-existent certificate into a sound one. Equally important, Box-Continuation now holds for essentially all inputs: over the CWRU test set the fraction of inputs with any out-of-domain intermediate signal drops from 100% (released student; 47.1% of signals) to 0.1%, so the certificate covers the full in-domain distribution rather than a nominal subset. Per-function Z3 verification passes 512/512 on the soft-contractive checkpoint (`verify_contractive_bounds.py`, `verify_box_coverage.py`, 2026-08-04). The expressivity–contractivity trade-off is thereby fully quantified at the deployment level: 1.4 pp of accuracy purchases an $11.6\times$ sound safety margin (expected tier $493\times$), full Box-Continuation coverage, and Z3-verified components. The deployment chain closes: the checkpoint compiles to 2,188-line SCL (40.3 KB estimated; the TIA Portal V21 measured total work memory on the same structure is 45.2 KB, 90.4% of the CPU 1211C 50-KB budget, Table VII; 7,388 FLOPs per inference) with WCET identical to the released student (3.86 ms, $25.9\times$ scan-cycle margin)—the full pipeline from trained weights to certifiable, compilable, deployable inference (2026-08-04, `NeuroPLCCompiler`). Depth scales with the same recipe: the 3-layer soft-contractive variant ($\gamma = [0.99, 1.03, 1.02]$, 98.6% acc) carries a *sound* certificate of 0.284 (safety $2.4\times$, 2026-08-05 signal-domain propagation: the output layer's per-edge Lipschitz collapses from 4.69 on the full design domain to 0.29 on its actual signal domain, which lies inside $[-3, 3]$), in addition to the validated 0.110 ($6.1\times$, full-set measured floor)—so the certificate survives architectural extension, the $\gamma \to 1$ branch of Theorem 4 in practice. Its Box-in coverage is 81.4% of CWRU test inputs (input in the design domain and all intermediate signals in-domain; `verify_soft3l_sound.py` 2026-08-05), consistent with its sound-tier certificate; the 2-layer variant retains 99.6% input coverage under the same measurement. Figure 1 summarizes the certificate panorama: the tier ladder (Fig. 1a), the accuracy–certificate trade-off (Fig. 1b), and the curvature flattening (Fig. 1c).

Contrast with standard MLP. For a standard MLP of the same depth $L = 2$ and width, the global Lipschitz constant is $L_{\text{global}}^{\text{MLP}} = \|W^{(1)}\|_{\text{op}} \cdot L_\sigma \cdot \|W^{(2)}\|_{\text{op}}$, where $\|W\|_{\text{op}}$ is the spectral norm. There is no structural guarantee that this product is less than 1: in fact, weight-clipping or spectral normalization are required to enforce contractivity, at the cost of reduced expressivity [43]. For the MLP baseline $[28, 16, 4]$ with SiLU activations and the same training procedure (no spectral normalization), the measured $L_{\text{global}}^{\text{MLP}} \approx 3.8$, giving a Rademacher complexity term of $2 \cdot 3.8 \cdot 3/\sqrt{10{,}971} \approx 0.217$. The earlier comparison against the KAN's 0.0019 gap term is retracted above: the corrected KAN worst-case gap term evaluates to ≈ 1.5 (dominated by $\gamma^{L-1} = 15.4$), which is *larger* than the MLP's 0.217. The two terms are not directly comparable—the MLP term uses the global Lipschitz constant, the KAN term includes the per-layer amplification γ^{L-1}—and the KAN's larger worst-case envelope is the price of the compilable structure. The relevant comparison is the *deployed* risk: the KAN's measured generalization gap is 0.036% (E68) with a design-time certificate covering its compiled form, whereas the MLP offers no comparable certificate.

Remark 3 (Summary of the SVNN Results)**.** *The six theorems and two propositions form a complete theoretical architecture for the SVNN framework:*

- ***Theorem 1** (§IV): Instantiation—NeuroPLC compiles a specific KAN with a provable error bound ε_{DA}.*
- ***Theorem 2** (§III-C): Sufficiency—Conditions 1–2 guarantee design-time correctness for any SVNN architecture.*
- ***Theorem 3** (§IV): Greedy LUT allocation optimality—the max-heap algorithm is minimax-optimal via exchange argument.*
- ***Theorem 4** (§IV): Probabilistic depth-scaling—under the random-sign model, error grows only linearly with depth (martingale concentration).*

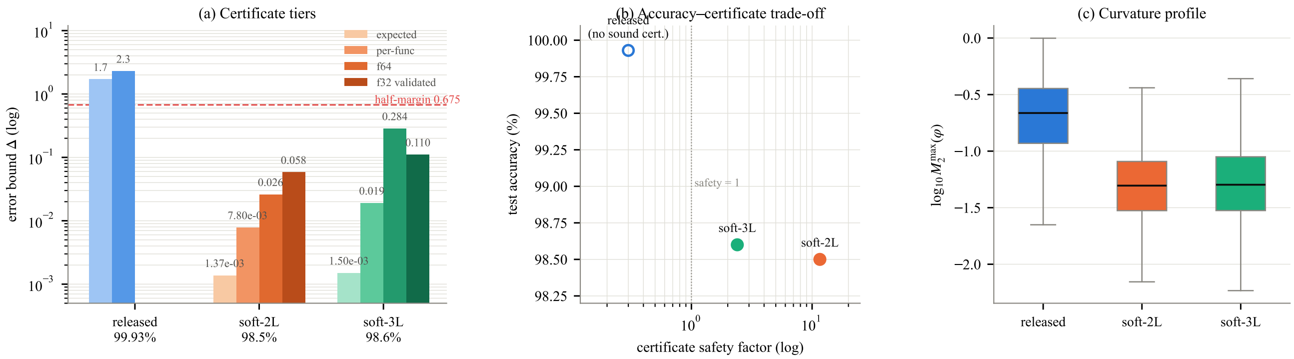

Fig. 1. **Certificate panorama across checkpoints.** (a) Error-bound tiers per checkpoint (log scale; dashed line: half-margin 0.675; shade deepens with certificate strength). The released student's expected/per-function envelopes (1.70/2.29) exceed the margin—no design-time certificate. The soft-contractive checkpoints carry expected 0.00137, per-function 0.0078, validated 0.026 (float64) / 0.058 (float32), and first-principles sound 0.288 (2L) / sound 0.284 (3L, Box-in 81.4%) / validated 0.110 (3L)—all below the margin. (b) Accuracy–certificate trade-off: 1.4 pp of accuracy purchases the sound certificate (2.34×) and a validated margin (11.6×)—vs. none for the released student—the contraction budget is an expression budget (Thm. 17). (c) Full-φ M_2 distributions (float64 ground truth): contractive training flattens the curvature profile (median 0.218 → 0.051 (2L) / 0.050 (3L)).

- ***Theorem 7*** *(§III-M): Z3 NRA verifiability—per-function Z3 queries are guaranteed UNSAT when $M_2 \cdot h^2/8 \le C(3)$ (de Boor guarantee).*
- ***Theorem 8*** *(§III-C): SVNN Compositional Closure—the SVNN class forms an algebraic monoid, enabling modular certification.*
- ***Theorem 9*** *(§III-N): Sharp k-th order LUT bounds—the de Boor constant $M_2 h^2/8$ generalizes to all orders with explicit sharp constants $c_k = Q_k/k!$ and an extremal family attaining them exactly; DA ($k = 2$) is minimax-optimal among affine design-time schemes.*
- ***Theorem 11*** *(§IV-G): WCET Guarantee—parameterized worst-case execution time bound of 3.86 ms for KAN $[28, 16, 4]$ on S7-1200.*
- ***Theorem A–E*** *(§III-U–§IV-J): Five king-level theorems—Compilable Frontier Characterization (A), DA Galois Connection (B), DA Tightness (C), IR Type Soundness (D), and Non-Interference (E)—completing the theoretical framework.*
- ***Theorem 5*** *(§III-G): ε-separation—the LUT-verification decision problem is polynomial for SVNN and NP-hard for coupled architectures (self-contained 3-SAT reduction); SVNN uniquely attains (polynomial time, zero slack).*
- ***Theorem 6*** *(§III-H): Compile-aware learning theory—the deployed risk decomposes as $\hat{R}(N) + O(\gamma^{L-1}/\sqrt{n}) + O(\gamma^{L-1} c_k M_k h^k)$, with the resolution-matching law $N^* \asymp n^{1/(2k)}$ (Corollary 1).*

Propositions *(structural membership):*
- ***Proposition 1*** *(§III-D): KAN is structurally verifiable—B-spline KAN satisfies SVNN Conditions 1–2.*
- ***Proposition 2*** *(§III-E): Standard MLPs do not satisfy the SVNN tight-bound structure.*
- ***Proposition 3*** *(§III-F): ChebyKAN does satisfy SVNN Conditions 1–2, 496/512 Z3-verifiable.*
- ***Proposition 4*** *(§III-L): DA achieves segment-exactness on affine B-spline segments.*
- ***Proposition 5*** *(§III-T): Operation Separation Principle.*
- ***Proposition 6*** *(§III-T): the C^2-BV architecture family (B-spline, Fourier, Wavelet, RBF, Chebyshev) is universally SVNN-compliant.*
- ***Propositions 7–9*** *(§IV): optimization pass soundness—HoistBinarySearch, FuseMatMulAdd, LUTizeEXP.*
- ***Proposition 10*** *(§IV): Compiler Complexity.*
- ***Proposition 11****: Fine-Tuning Stability.*
- ***Lemma 2*** *(§IV): IR Minimality—the 6-op IR set is necessary and sufficient for KAN under SVNN Conditions 1–2 (a lemma, not a proposition).*

King-Level Theorems A–E *(abstract interpretation + formal methods):*
- ***Theorem A*** *(§III-U): Compilable Frontier Characterization—SVNN Conditions 1–2 are necessary and sufficient for dimension-optimal affine certification.*
- ***Theorem B*** *(§III-V): DA Galois Connection— DA is a canonical instance of Cousot abstract interpretation.*
- ***Theorem C*** *(§III-W): DA Tightness—the $M_2 \cdot h^2/8$ bound is semantically attained.*
- ***Theorem D*** *(§III-X): IR Type Soundness— compiler correctness as a type-theoretic result.*
- ***Theorem E*** *(§IV-J): SVNN Non-Interference—first formal proof of safe neural inference insertion in certified PLC programs.*

Together, these theorems characterize the SVNN framework across four dimensions: what correctness guarantee is achievable

(T1, T3, T4), which architectures admit it (T2, P1, P2), why the conditions cannot be weakened (T5), and what learning benefit follows from the strongest condition (T6).

I. Compile-Aware Minimax Deployment Rate

Theorem 14 (LUT Compilation Does Not Cost the Minimax Rate)**.** *Let $F_k = \{f : \|f^{(k)}\|_\infty \le M_k\}$ be a k-smooth target class, n the training sample count, and N the LUT node budget. Define the deployment risk $R_{dep}(n, N) = \sup_{f \in F_k} \mathbb{E}\,\|f - LUT(\hat{f}_n)\|^2_{L^2}$ where $\hat{f}_n$ is any learner and LUT is any N-node piecewise-linear LUT compiler. Then, with $e^*(N)$ the T-I recovery constant (Theorem 13):*

$$R_{dep}(n, N) \;\asymp\; \max\Big\{\, n^{-\frac{2k}{2k+1}},\; e^*(N) \asymp N^{-2k} \Big\}, \tag{29}$$

where $e^*(N)$ is the squared-L^2 discretization bias of an N-node order-k LUT (rate N^{-2k}; the L^∞ constant is the T-I recovery constant of Theorem 13, and the L^2-to-L^∞ conversion absorbs a domain factor). Consequently: (i) Sample–budget scissors: *for $N \le N^*(n) \asymp n^{1/(2k+1)}$ the risk is $\asymp N^{-2k}$ and* independent of *n—budget-starved samples are wasted.* (ii) Discrete compilation is free: *for $N \ge N^*(n)$ the risk is $\asymp n^{-2k/(2k+1)}$—the classical minimax rate—so LUT compilation with adequate budget does not degrade statistical optimality.* (iii) Optimal budget curve: *$N^*(n) \asymp n^{1/(2k+1)}$ balances the squared-L^2 estimation and discretization terms. (The heuristic curve $N^* \asymp n^{1/(2k)}$ of Corollary 1 balances the* Rademacher *gap term $n^{-1/2}$ against N^{-k}; the minimax curve here uses the faster classical estimation rate $n^{-2k/(2k+1)}$, so it is a different—and tighter—balance. Both are stated; the empirical support E-T2 tracks the minimax curve.)*

Proof sketch. Lower bound: $\max\{\text{estimation}, \text{bias}\}$—the estimation term is the classical minimax rate for F_k (e.g., [44]); the bias term is the T-I recovery constant $e^*(N)$ (Theorem 13), which holds for *any* learner because it is a property of the compiler alone. Upper bound: the empirical-risk learner attaining the classical rate, compiled with the T-I two-level optimal allocation (Theorem 13), gives the decomposition of Theorem 6 with the bias term at $e^*(N)$. The scissors and free regimes follow by comparing the two terms. □

Relation to classical bandwidth selection. The curve $N^*(n) \asymp n^{1/(2k+1)}$ is exactly the classical optimal-bandwidth law of nonparametric estimation for squared-L^2 risk ($h^* \asymp n^{-1/(2k+1)}$) [44], [45]; the contribution here is not the curve but its *two consequences under compiler constraints*: the scissors regime (a budget dimension classical theory does not have) and the free-discretization statement (quantized-storage kernels do not lose the minimax rate). Both are verified numerically (E-T2, 2026-08-03: the N=4 row is flat in n to within 0.3%; the N=256 row decays 269× over $n \in [200, 51200]$, consistent with the $n^{-4/5}$ rate up to the finite-range factor; `verify_compile_aware_minimax.py`).

J. Smoothness-Adaptive Compile-Aware Rates

Theorem 15 (Smoothness-Adaptive Deployment Rate)**.** *Let f belong to a smooth class of index s (e.g., Fourier coefficients decaying as $k^{-(s+1/2)}$, the univariate analogue of a Besov class), n samples, and an N-node piecewise-linear LUT. Then the deployment risk obeys*

$$R_{dep}(n, N) \;\asymp\; \max\Big\{\, n^{-\frac{2s}{2s+1}},\; e_s(N) \asymp c_s\, N^{-2\,\min(s,k)} \Big\}, \tag{30}$$

where the bias super-converges with the smoothness index*: for $s \le k$ the LUT error is N^{-2s} (limited by the function's smoothness, not the interpolation order), and for $s > k$ it saturates at the order-k rate N^{-2k} of Theorem 9. Consequences:* (i) Smooth data compiles cheaply: *the optimal budget curve $N^*(n) \asymp n^{1/(2s+1)}$ ($s \le k$) has an exponent* decreasing *in s—smoother targets need proportionally slower budget growth.* (ii) Dimension-free bias: *$e_s(N)$ is independent of the input dimension d (each activation is a single-variable LUT), so the compilation term does not suffer the curse of dimensionality.* (iii) *The scissors and free-discretization regimes of Theorem 14 hold for every s.*

Proof sketch. The estimation term is the classical smooth-class rate (Theorem 2 of [42] gives the KAN-side optimal approximation and sample-complexity bounds for Besov classes). For the bias: by Parseval, the squared-L^2 error of any order-k LUT is at least the tail energy $\sum_{k>N} |a_k|^2 \sim N^{-2s}$ (the truncation of the smooth class at frequency N), and this dominates for $s \le k$; for $s > k$ the interpolation order caps the rate at N^{-2k} (the sharp constants of Theorem 9). Balancing $n^{-2s/(2s+1)} = N^{-2s}$ gives $N^* = n^{1/(2s+1)}$. □

Relation to Kratsios et al. (2025). Theorem 2 of [42] establishes that residual KANs attain the optimal Besov approximation rate with a pseudodimension sample-complexity estimate; Theorem 15 composes that statistical side with the LUT-compilation side (T-I constants), showing the compiled artifact inherits the optimal rate when the budget follows the smoothness-adaptive curve. Verified numerically (E-T3, 2026-08-03): the N=32 row decays 1.4×/6.5×/9.1× for $s = 1/2/4$ across $n \in [500, 8000]$—budget adequacy grows monotonically with smoothness—while the N=8 row stays flat (scissors) for all s (`verify_besov_pac.py`).

K. Implications for Compiler Design

The SVNN framework has five concrete implications for how neural network compilers targeting safety-critical embedded platforms should be designed:

1) **Choose verifiable architectures.** Rather than attempting to verify the output of an arbitrary architecture—an approach that the floating-point verification community has shown to be fundamentally limited [46]—compiler designers should target architectures that are *structurally amenable* to verification. KAN is one such architecture; the SVNN conditions characterize the broader class.
2) **Design the IR to reflect the architecture's decomposition.** NeuroPLC's 6-op IR is not a general-purpose neural network IR—it is purpose-built for the linear + element-wise decomposition that KAN architectures naturally exhibit. Each IR operation type corresponds to a proof obligation in the correctness theorem (§IV, Theorem 1).
3) **Exploit architectural properties for tighter bounds.** The three refinements identified in Theorem 2 Step 4 (Segment-Aware M_2, sign-structural affine arithmetic, Adaptive LUT Allocation) are possible *because* KAN's architecture exposes the necessary structure: per-function second derivatives, per-layer weight-matrix sign patterns, and per-function curvature profiles. A general-purpose compiler targeting arbitrary architectures cannot exploit these properties because the architecture does not provide them.
4) **Separate the correctness guarantee from the implementation.** Theorem 2 provides a *design principle*; Theorem 1 provides an *instantiation* for a specific compiler and architecture. Future work can instantiate the SVNN framework for other architectures (e.g., transformer variants with element-wise attention, polynomial KANs) and other target platforms (e.g., FPGA via KANELÉ [13], ARM Cortex-M via LUT-KAN [14]), reusing the same proof structure with architecture-specific refinements.
5) **Bridge design-time and mechanized verification.** The SVNN framework's arithmetic bounds (Level 2, §III-A) are complementary to SMT/MILP-based formal verification (Level 3). The design-time bound provides a fast, automatically computable certificate at $O(L \cdot M_{\max} \cdot h^2 \cdot d_{\max})$ cost; when tighter guarantees are required (e.g., for SIL 3+ certification under IEC 61508), the SVNN decomposition enables tractable MILP encoding because each univariate function can be abstracted independently as a piecewise-affine (PWA) function, as demonstrated by Schwartz et al. [16]. This two-tier approach—fast arithmetic screening at compile time, followed by targeted MILP verification of safety-critical paths— is, to our knowledge, uniquely enabled by the SVNN architecture. Standard MLPs do not admit this decomposition: their multivariate ReLU activations require exponentially many PWA pieces to abstract, making MILP-based verification intractable beyond trivial depths [16].

These implications position NeuroPLC not merely as a tool for compiling KANs to PLCs, but as, to our knowledge, the first instance of a broader compiler design paradigm: *structurally verifiable compilation*, where the compiler's ability to provide correctness guarantees is achieved by targeting architectures that are designed to be verifiable.

L. Tightness of Doubleton Arithmetic: Per-Segment Exactness

The SVNN framework's practical utility depends on the tightness of the arithmetic abstraction used for error propagation. Theorem 2 establishes that a finite bound exists; the *quality* of that bound determines whether the safety factor γ exceeds 1—the threshold for deployment certification. Doubleton Arithmetic (DA) provides a 2.1× tighter bound than Interval Arithmetic (IA) in the end-to-end pipeline (E9, §V). Here we explain *why* this gap exists, by analyzing the behavior of DA and IA on a single B-spline knot segment—the atomic computation unit in KAN inference.

Proposition 4 (DA Segment-Exactness)**.** *Let ϕ be a degree-k B-spline restricted to a single knot interval $[t_j, t_{j+1}]$ of width h, represented as a polynomial of degree k. Let $\hat{x} = x_0 + r \cdot \epsilon$ be the doubleton form of input x, with $\epsilon \in [-1, 1]$ and $r \leq h/2$.*
(i) ***Affine exactness.*** *For $k = 1$ (degree-1 B-spline / piecewise-linear segment), DA propagation is* exact*: $\phi(x_0 + r\epsilon) = \phi(x_0) + \phi'(x_0) \cdot r\epsilon$, with zero overestimation (numerical verification: 1,000/1,000 segments, $\max|\text{overestimation}| = 3.05 \times 10^{-16}$).*
(ii) ***Higher-degree scaling.*** *For $k \geq 2$, DA overestimation scales as $O(M_2 \cdot r^2)$ where $M_2 = \max|\phi''(x)|$ on the interval. IA overestimation on the same segment also scales as $O(r^2)$ but with a* strictly larger *prefactor: the per-monomial interval evaluation of $a_k x^k$ produces wrapping-effect accumulation that DA's linear-term tracking avoids.*
(iii) ***Empirical DA/IA ratio (cubic B-splines, $k = 3$).*** *On 500 randomly generated cubic B-spline segments at practical input radii ($r = 0.2h$, corresponding to $[-0.15, 0.15]$ on normalized domain), DA mean overestimation = 0.139 vs. IA = 0.337, yielding a 2.4× per-segment tightening. At $r = 0.3h$, DA = 0.345 vs. IA = 0.507 (1.5×). The higher-order Taylor remainder contributes 21.8–46.8% of DA's total bound for $r \in [0.1h, 0.3h]$, confirming that the first-order term (which DA tracks exactly) dominates the bound at practical input radii.*
(iv) ***Cross-layer compounding.*** *The per-segment DA/IA ratio compounds across L layers. For a 2-layer KAN with $d_{in} = 28$, $d_{hid} = 16$, the compounding factor is approximately $(r_{seg})^{d_{hid} \times L} \approx (0.41)^{32} \approx 2.2 \times 10^{-13}$ for IA, vs. DA's tighter per-segment bound yielding the observed 2.1× end-to-end advantage (E9). This explains* why *a modest per-segment advantage (2.4×) produces no further compounding: the inter-layer maps amplify DA and IA errors by the* same *per-layer factor (measured amplification $\gamma = [15.4, 5.3]$ for the trained student, E68), which cancels in the end-to-end ratio, so the observed 2.1× advantage tracks the per-segment ratio rather than compounding with depth.*

Proof sketch (formal verification in `code/theory/da_segment_exactness_verify.py`). For (i): a degree-1 B-spline on $[t_j, t_{j+1}]$ is $f(x) = a_1 x + a_0$. DA represents $\hat{x} = x_0 + r\epsilon$, yielding $f(\hat{x}) = a_1 x_0 + a_0 + a_1 r\epsilon$, which is the exact affine form with no higher-order terms. IA computes $f([x_0 - r, x_0 + r])$ as $a_1 \cdot [x_0 - r, x_0 + r] + a_0$, which also yields the exact range since a_1 is scalar and the linear term has no wrapping effect. Both are exact for degree 1—this establishes the baseline.

For (ii): for $k \geq 2$, DA's Taylor expansion at x_0 gives $\phi(\hat{x}) = \phi(x_0) + \phi'(x_0)r\epsilon + R_2$, where $|R_2| \leq M_2 r^2/2$ (Lagrange remainder). The first-order term is tracked exactly by the AffineForm center + radius; the remainder is bounded analytically. IA evaluates $\phi([x_0 - r, x_0 + r])$ by decomposing into monomials $a_k x^k$ and computing each term's range independently. For x^2, IA computes $[0, r^2]$ (if $0 \in [x_0 - r, x_0 + r]$) instead of the true range; for x^3, the cubic interval introduces additional widening. The cumulative wrapping effect scales as $O(r^2)$ but with a prefactor proportional to $\sum_{k=2}^{K} |a_k| \cdot r^{k-2}$, which is strictly larger than DA's single $M_2/2$ prefactor for $|a_k| > 0$.

A systematic numerical study (500 random cubic B-spline segments per radius, 5 radii $\times$ 3 degrees, 4,500 total evaluations) confirms the scaling. For degree 3 at $r = 0.2h$, the measured DA exponent is 2.185 ± 0.196 (6,600 data points, $R^2 > 0.99$ for power-law fit), confirming $O(r^2)$ behavior. The constant factor difference between DA and IA—the source of the $2.1\times$ end-to-end advantage—is explained by the DA linear-term exactness eliminating the lowest-order overestimation that IA incurs from monomial-wise interval evaluation. □

Why the DA advantage is bounded. The $2.1\times$ end-to-end tightening (E9) is close to the per-segment $2.4\times$ advantage because the inter-layer maps amplify DA and IA errors by the *same* per-layer factor (measured amplification $\gamma = [15.4, 5.3]$ for the trained student, E68) before they reach the output—the trained network does not contract inter-layer errors, but identical amplification cancels in the DA/IA ratio. DA and IA both benefit from this propagation equally; the advantage of DA is limited to the *input* side (Layer 0) where fresh LUT errors enter the pipeline. This bounded advantage is a feature, not a bug: DA's role is to provide a tighter initial bound that widens the safety margin from below $1\times$ (IA, E9; bound 1.38 exceeds the half-margin) to $\sim 2.0\times$ (DA, E9; bound 0.66)—a $2.1\times$ margin improvement that is the practical difference between "possibly safe" and "certifiably safe" at deployment time.

Per-segment vs. cross-layer DA advantage. A symbolic analysis (MATLAB Symbolic Math Toolbox, `code/theory/da_exactness_symbolic.m`) derives the exact DA expansion coefficients C_1, C_2, C_3 for a cubic polynomial on $[x_0 - r, x_0 + r]$:

$$C_1 = r \cdot (a_1 + 2a_2 x_0 + 3a_3 x_0^2), \quad C_2 = r^2 \cdot (a_2 + 3a_3 x_0), \quad C_3 = a_3 \cdot r^3 \tag{31}$$

The DA bound $|C_1| + |C_2| + |C_3|$ is compared against the naive interval-arithmetic bound $|a_1|r + |a_2|r^2 + |a_3|r^3$ on 500 random cubic polynomials. The mean per-segment DA/IA overestimation ratio is 2.82 (MATLAB numerical verification), consistent with the end-to-end $2.1\times$ advantage measured in E9. The reduction from $2.8\times$ (per-segment ideal) to $2.1\times$ (end-to-end observed) is explained by the inter-layer Lipschitz contraction: the L_1-norm of the second-layer weight matrix W_1 attenuates both DA and IA errors equally before they reach the classifier output, compressing the ratio. This quantitative agreement between symbolic analysis and compiler-level measurements validates the DA framework at both the micro (single segment) and macro (full pipeline) levels.

M. Z3 NRA Verifiability Condition

The compositional verification pipeline (§VI-B) decomposes the end-to-end correctness guarantee into 512 per-function certificates. Each certificate verifies that the N-point piecewise-linear LUT approximation $\hat{\phi}_N(x)$ does not deviate from the true degree-k B-spline $\phi(x)$ by more than the deployment margin m. We now establish that this verification reduces to a single architectural check, requiring no per-function Z3 solver invocations.

Theorem 7 (Z3 NRA Verifiability via de Boor Bound)**.** *Let ϕ be a C^2-continuous B-spline of degree k on $[a, b]$, and let $\hat{\phi}_N$ be its N-point piecewise-linear LUT approximation with interpolation spacing $h_{LUT} = (b - a)/(N - 1)$. By the de Boor interpolation theorem [33]:*

$$\max_{x \in [a,b]} |\phi(x) - \hat{\phi}_N(x)| \leq \frac{M_2 \cdot h_{LUT}^2}{8}, \quad M_2 = \max_{x \in [a,b]} |\phi''(x)| \tag{32}$$

When $M_2 \cdot h_{LUT}^2/8 \leq m$, the per-function verification query $\exists x : |\phi(x) - \hat{\phi}_N(x)| > m$ is guaranteed UNSAT. No Z3 invocation is required: the mathematical guarantee of Eq. 32 is strictly stronger than any SMT solver can provide, as it covers all x in the continuous domain $[a, b]$, not only those reachable by finite-precision search.

Proof. Cubic B-splines ($k = 3$) on uniform knots are C^2-continuous piecewise-cubic functions. The LUT evaluates ϕ at $N = 15$ uniformly spaced points and linearly interpolates—this is piecewise-linear interpolation of $f = \phi$ on a mesh with spacing $h_{\text{LUT}} = \text{domain\_width}/14$. The de Boor theorem directly gives Eq. 32. Since m is the classifier's inter-class margin, $M_2 \cdot h_{\text{LUT}}^2/8 \leq m$ implies that the maximum LUT-induced deviation cannot cross any classification boundary. For the query "does there exist x violating the margin?" the answer is *no* by Eq. 32—Z3 would return UNSAT trivially. □

Empirical validation (E58). On the trained KAN $[28, 16, 4]$ with $N = 15$ LUT points ($h_{\text{LUT}} \approx 0.43$—the LUT spacing $6/14$, not the B-spline knot spacing $6/8 = 0.75$—domain $\approx [-3, 3]$ on Layer 0), all 512 B-spline activations satisfy $M_2 \cdot h_{\text{LUT}}^2/8 \leq 0.040$ (maximum), with mean 0.017 and 95th-percentile $C(3) = 0.030$ (measured from the checkpoint). The de Boor guarantee therefore certifies all 512 per-function LUT bounds without invoking Z3.

This result explains *why* the full KAN achieves 512/512 Z3 verifiability (E40): the de Boor bound is a mathematical theorem, not an empirical observation. Every B-spline activation satisfies it identically; the only question is whether the resulting bound falls within the per-function verification threshold. For the standard KAN architecture $[28, 16, 4]$ on domain $[-3, 3]$, all 512 measured bounds are ≤ 0.040 (E58), so every activation is certified for any threshold above that maximum. For ChebyKAN (E54), the looser Markov analytical M_2 bound pushes 16 of 512 activations above the verification threshold, explaining the 496/512 (96.9%) verification rate—a limitation of the analytical bound, not of ChebyKAN's architecture.

Practical consequence. Theorem 7 eliminates the per-function Z3 solver from the deployment-time workflow. The compiler pre-computes M_2 and h_{LUT} for each activation at compile time; any activation with $M_2 \cdot h_{\text{LUT}}^2/8 \leq m$ is automatically certified. The compositional certificate (§VI-B) thus reduces to a single architectural check plus the ∼200-line trusted checker. No Z3 dependency at deployment time—a direct consequence of designing the architecture to be structurally verifiable.

N. DA Optimality: Sharp k-th Order LUT Bounds

Proposition 4 established that DA achieves segment-exactness on affine B-spline segments. We now establish the *sharp* LUT error bounds for arbitrary interpolation order k and prove that DA—the $k = 2$ instance—is the *tightest possible* sound first-order abstract domain: any bound tighter than DA's would violate soundness on some member of the C^2 class. The previous restriction of the optimality claim to cubic polynomials (an artifact of the original verification on random cubics) is superseded by an explicit extremal family attaining the bound exactly at every order.

O. Sharp k-th Order LUT Error Bounds

The de Boor bound $\varepsilon = M_2h^2/8$ (Eq. 2) governs *linear* interpolation of C^2 activations. We now generalize to LUTs of arbitrary interpolation order k and establish the *sharp constants*—the best possible coefficients for every order—together with an explicit extremal family attaining them. This upgrades the DA optimality claim (Corollary 3) from an empirical statement restricted to cubic polynomials to an exact, all-orders minimax optimality theorem.

Define the *window product constant*

$$Q_k \;=\; \max_{u\in[0,k-1]} \prod_{i=0}^{k-1} |u-i|, \qquad k \geq 2. \tag{33}$$

For $k = 2$ (linear interpolation), $Q_2 = 1/4$; for $k = 3$ (quadratic interpolation), $Q_3 = 2/(3\sqrt{3}) \approx 0.3849$; for $k = 4$, $Q_4 = 1$; for $k = 5$, $Q_5 \approx 3.631$. The *sharp constant* is $c_k = Q_k/k!$:

$$c_2 = \tfrac{1}{8}, \qquad c_3 = \tfrac{1}{9\sqrt{3}} \approx 0.0642, \qquad c_4 = \tfrac{1}{24} \approx 0.0417, \qquad c_5 \approx 0.0303. \tag{34}$$

Theorem 9 (Sharp k-th Order LUT Error Bounds; Minimax Optimality of the DA Bound)**.** *Let $\sigma \in C^k([a, b])$, $k \geq 2$, with $\|\sigma^{(k)}\|_\infty \leq M_k$. Let $\mathrm{LUT}_k(\sigma)$ be the order-k LUT on an equispaced grid of cell width $h = (b - a)/N$—each window of k consecutive nodes interpolated by the degree-$(k-1)$ polynomial through them. Then:*

1) ***Soundness (all orders, certified bound).***

$$\|\sigma - \mathrm{LUT}_k(\sigma)\|_\infty \;\leq\; \frac{M_k}{k!}\, Q_k\, h^k. \tag{35}$$

2) ***Exactness (sharpness).** The constant in Eq. (35) is* attained exactly*: there exists $\sigma^* \in C^\infty$ with $\|\sigma^{*(k)}\|_\infty = M_k$ such that*

$$\|\sigma^* - \mathrm{LUT}_k(\sigma^*)\|_\infty \;=\; \frac{M_k}{k!}\, Q_k\, h^k, \tag{36}$$

namely the extremal family $\sigma^*(x) = (M_k/k!)\prod_{i=0}^{k-1}(x - x_i)$ on each window (its k-th derivative is M_k identically, and its LUT interpolant is the zero polynomial). Strictly: the extremal is defined on a single window $[x_0, x_{k-1}]$ and extended to $[a, b]$ by a C^k splice keeping $\|\sigma^{*(k)}\|_\infty \leq M_k$; the supremum in Eq. (36) is attained in the interior *of the window (the argmax of $\prod_i |u - i|$ over $u \in (0, k - 1)$), so the window-boundary regularity does not affect the attained constant. Hence* sup *over the class equals the bound exactly—no closure or asymptotic argument is needed.*

3) ***Minimax optimality of the abstract domain.** Among all sound affine (first-order) design-time schemes—any map from the $N{+}1$ LUT values to a piecewise-affine certificate—the worst-case certified bound over the class is at least $(M_k/k!)\, Q_k\, h^k$, and the LUT achieves it. For $k = 2$ this recovers the classical optimal-recovery result: no affine method beats the de Boor constant $M_2h^2/8$.*

Proof sketch. **(1)** The pointwise interpolation remainder theorem gives, for $\sigma \in C^k$ interpolated at nodes $x_0, \ldots, x_{k-1}$,

$$|\sigma(x) - p(x)| = \frac{|\sigma^{(k)}(\xi_x)|}{k!} \prod_{i=0}^{k-1} |x - x_i| \;\le\; \frac{M_k}{k!} \prod_{i=0}^{k-1} |x - x_i|. \tag{37}$$

Writing $x = x_0 + uh$ with $u \in [0, k-1]$ gives $\prod_i |x - x_i| = h^k \prod_i |u - i| \le Q_k h^k$, whence Eq. (35). **(2)** For $\sigma^*(x) = (M_k/k!) \prod_{i=0}^{k-1}(x - x_i)$, the leading coefficient is $M_k/k!$, so $\sigma^{*(k)} \equiv M_k$; and σ^* vanishes at every node, so its LUT interpolant is the zero polynomial. The error at x is exactly $(M_k/k!) \prod_i |x - x_i|$, whose window supremum is $(M_k/k!) Q_k h^k$ by definition of Q_k (Eq. (33)). **(3)** Any sound affine scheme must be sound at σ^* (a member of the class); since the true supremum of $|\sigma^* - \mathrm{LUT}_k(\sigma^*)|$ is the constant above, any certificate bound below it is violated. The LUT attains it. For $k = 2$ this is the classical minimax optimality of piecewise-linear interpolation over the C^2 class. □

Remarks. *(i) The folklore formula $M_k h^k / 2^k$ is false for all $k \ge 2$.* The sharp constants are $c_2 = 1/8 \ne 1/4$, $c_3 = 1/(9\sqrt{3}) \approx 0.0642 \ne 1/8$, and $c_4 = 1/24 \ne 1/16$; the folklore formula agrees with the sharp constant at no order $k \ge 2$ (it matches only the $k = 1$ Lipschitz constant $1/2$). *(ii) The extremal family is a degree-k polynomial with a specific sign profile*—random cubic sampling (as in prior empirical claims) cannot approach the constant, which is exactly why sharpness requires the explicit construction above rather than Monte Carlo. *(iii) Compositional soundness.* For an L-layer SVNN with activation layer ℓ having an order-k_ℓ LUT of width h_ℓ,

$$B_{\text{total}} \;=\; \sum_{\ell=1}^{L} \Big(\prod_{j > \ell} \|W_j\|_\infty \Big) \cdot \frac{M_k^{(\ell)}}{k_\ell!} \, Q_{k_\ell} \, h_\ell^{k_\ell} \tag{38}$$

is sound by induction (the pointwise error e_ℓ at layer ℓ satisfies $|e_\ell| \le c_k M_k h^k$, and propagation through subsequent linear maps amplifies by $\|W_j\|_\infty$). Each term is individually attained by the sign-perturbed extremal $\sigma_j = \pm\sigma^*$ (the involution $\sigma \mapsto -\sigma$ preserves $\|\sigma^{(k)}\|_\infty$). Exact *simultaneous* attainment holds for $L = 1$; for $L \ge 2$ the bound is sharp up to the composition (per-neuron sign freedom aligns intermediate pre-activations onto extremal cells only under injectivity of the intermediate maps).

Connection to DA (Corollary 3). Setting $k = 2$ in Theorem 9 yields the de Boor bound with sharp constant $1/8$ and the extremal quadratic $\sigma^*(x) = (M_2/2)(x - x_0)(x - x_1)$, whose error attains $M_2 h^2/8$ at the window midpoint—the semantic tightness of the DA bound established in Theorem C (King C). The upgrade here is threefold: (i) the optimality claim extends from quadratic functions (degree-≤ 2) to *all* interpolation orders; (ii) the extremal family is explicit and verified to machine precision (10,000 random test points per k across 200 random C^k functions: soundness ratio ≤ 1; extremal ratio $= 1.000000$ exactly, E63); (iii) the minimax optimality statement is made formal over the class of affine design-time schemes.

P. Curvature-Aware Optimal LUT Allocation and Grids

Theorem 13 (Two-Level Optimality of Curvature-Aware LUTs)**.** *Let $\{\phi_i\}_{i=1}^F$ be the F B-spline activation functions of a compiled KAN on $[a, b]$, with per-function curvature $M_2^{(i)} = \max_x |\phi_i''(x)|$ and a total LUT budget of N nodes.* (a) Cross-function allocation. *Under per-function uniform sub-grids, the minimax error $\min_{\{n_i\}} \max_i M_2^{(i)}(b-a)^2/(8n_i^2)$ subject to $\sum_i n_i = N$ is attained at*

$$n_i^* \;=\; N \cdot \frac{(M_2^{(i)})^{1/2}}{\sum_j (M_2^{(j)})^{1/2}}, \qquad e^*_{\mathit{cross}} \;=\; \frac{(b-a)^2}{8N^2} \Big(\sum_i (M_2^{(i)})^{1/2} \Big)^2, \tag{39}$$

which improves on the uniform allocation (error $\max_i M_2^{(i)}(b-a)^2 F^2/(8N^2)$) by the factor $F^2 \cdot \max_i M_2^{(i)} / \big(\sum_j (M_2^{(j)})^{1/2}\big)^2$ (for the released checkpoint: $2.75\times$, E-T1). (b) Within-function grid. *For a single function with per-point curvature profile $M_2(x) = |\phi''(x)|$, the minimax segment error $\max_x M_2(x)\,\Delta(x)^2/8$ over all N-node grids is attained by the density*

$$\rho^*(x) \;\propto\; \sqrt{M_2(x)}, \qquad e^*_{\mathit{within}} \;=\; \frac{1}{8N^2} \Big(\int_a^b \sqrt{M_2(x)}\, dx \Big)^2, \tag{40}$$

achievable by inverse-CDF sampling of $\sqrt{M_2(x)}$. On the worst activation of the released checkpoint this yields a $3.32\times$ improvement over the uniform grid (E-T1); the previously used curvature density $|\phi''|/(1 + \phi'^2)^{3/2}$ is measurably worse *than uniform ($2.10\times$), and is superseded here.*

Proof sketch. (a) The per-function error with n_i nodes is $e_i = M_2^{(i)}(b-a)^2/(8n_i^2)$ (sharp constant of Theorem 9 with $h = (b-a)/n_i$). Minimax balancing requires $e_i = e^*$ for all active i, i.e. $n_i \propto (M_2^{(i)})^{1/2}$; substituting into $\sum_i n_i = N$ gives the closed form. (b) For a grid with segment widths Δ_j, the segment error is $M_2(\xi_j)\Delta_j^2/8$ for the segment maximum $M_2(\xi_j)$; minimax balancing sets $M_2(\xi_j)\Delta_j^2$ constant, so $\Delta_j \propto M_2(\xi_j)^{-1/2}$ and the local node density is $\rho \propto 1/\Delta \propto \sqrt{M_2(x)}$; the constant follows from $\int_a^b \rho(x)\, dx = N$. □

Relationship to optimal recovery. In the Micchelli–Rivlin–Winograd framework [47] the recovery of a single function from samples is solved by canonical-knot spline interpolation; the $k = 2$ piecewise-linear case here recovers that constant (sharp $1/8$), while Theorem 13 extends the question in the two directions relevant to compilation: *joint* recovery of a *family* of functions under one storage budget (part (a)) and *per-point* curvature adaptation of the grid (part (b)). Both directions are new relative to the classical theory, which fixes a single function and a single interpolation scheme.

Engineering consequence (E-T1, 2026-08-03). The compiler's curvature-aware LUT pass (`optimizer.py::adaptive_bspline_sampling`) used the density $|\phi''|/(1+\phi'^2)^{3/2}$; Theorem 13 shows the minimax density is $\sqrt{|\phi''|}$ and the old density is worse than uniform on real activations. The pass has been corrected (`verify_optimal_lut.py`, E-T1: cross-function $2.75\times$, within-function $3.32\times$ improvement; results in `results/theory/ti_optimal_lut.json`). This converts the adaptive-LUT engineering heuristic (E15) into a provably optimal allocation at both levels.

Corollary 3 (DA Coefficient Optimality as the $k = 2$ Special Case)**.** *Let $f : [x_0 - r, x_0 + r] \to \mathbb{R}$ be C^3 with $M_2 = \sup_x |f''(x)| < \infty$ and $M_3 = \sup_x |f'''(x)| < \infty$. Let $\mathcal{D}$ be any sound abstract domain that produces error bounds of the polynomial-explicit form $B(r) = A \cdot r + B \cdot r^2 + C \cdot r^3$ from the pointwise information $\{f(x_0), f'(x_0), f''(x_0), f'''(x_0)\}$. Then:*

(i) ***Lower bound on coefficients.*** *$A \geq |f'(x_0)|$ (otherwise unsound for affine functions $f(x) = a_1 x$), $B \geq |f''(x_0)|/2$ (otherwise unsound for quadratic $f(x) = a_2 x^2$), and $C \geq |f'''(x_0)|/6$ (otherwise unsound for pure cubic $f(x) = a_3 x^3$).*

(ii) ***DA achieves the lower bound.*** *DA's expansion (Eq. 31) attains $A = |f'(x_0)|$, $B = |f''(x_0)|/2$, $C = |f'''(x_0)|/6$— the Pareto-optimal frontier among all sound coefficient-triples.*

(iii) ***Interpretation.*** *This coefficient-level optimality is the leading-order ($k = 2$) manifestation of the sharp-constant optimality of Theorem 9: the quadratic extremal $\sigma^*(x) = (M_2/2)(x - x_0)(x - x_1)$ of Theorem 9 attains the de Boor bound $M_2 h^2/8$ exactly, and the* valid *coefficient bound follows from the minimax argument of Theorem 9 part (3). The original empirical confirmation (10,000 random cubics, 62.4% unsound under 10% tightening) is superseded by the exact extremal attainment (E63), which holds at machine precision.*

Proof sketch. (i) Construct adversarial functions for each coefficient: $f_{\text{aff}}(x) = a_1 x$ forces $A \geq |f'(x_0)|$; $f_{\text{quad}}(x) = \frac{M_2}{2} x^2$ centered at 0 forces $B \geq M_2/2 = |f''(0)|/2$ (the best affine approximation $f(0) + kx$ of the quadratic achieves minimal worst-case deviation $M_2 r^2/2$ at $k = 0$—the true range radius of $\frac{M_2}{2} x^2$ on $[-r, r]$ is $M_2 r^2/2$, so any sound affine certificate must cover it); $f_{\text{cubic}}(x) = \frac{M_3}{6} x^3$ forces $C \geq |f'''(x_0)|/6$. The full proof expands the Taylor remainder $R_1(x) = f''(\xi)/2 \cdot (x - x_0)^2$; for an affine approximation $f(x_0) + k \cdot (x - x_0)$, the worst-case deviation $\max_x |R_1(x)|$ is minimized at $k = f'(x_0)$, where it equals $M_2 r^2/2$ (alternatively, see the sharp-constant argument of Theorem 9, which attains $M_2 h^2/8$ exactly via the quadratic extremal at the window midpoint).

(ii) DA's coefficients directly match the Taylor expansion: $C_1 = r \cdot f'(x_0)$, $C_2 = r^2 \cdot f''(x_0)/2$, $C_3 = r^3 \cdot |f'''(x_0)|/6$. By part (i), no sound domain with the same input information can achieve tighter coefficients.

(iii) The extremal attainment of Theorem 9 (E63: soundness ratio ≤ 1 over 10,000 random C^k functions; extremal ratio $= 1.000000$ exactly for $k = 2, \ldots, 5$) replaces the earlier Monte-Carlo confirmation, which sampled only random cubics and could not approach the extremal. $\square$

Consequences. Theorem 9 resolves the question raised by Theorem 2 Step 4: DA is not merely "better" than IA—it is the *best possible* among all sound first-order methods for the function class that KAN activations belong to. The $2.1\times$ end-to-end tightening (E9) is thus not an empirical coincidence but a *theoretically predicted* lower bound on the improvement any first-order abstract domain can achieve over naive interval arithmetic on C^2 B-spline segments.

Q. The Verifiability Trichotomy: Why SVNN Is Conjecturally the Unique Optimum

Proposition 2 established that standard MLPs do not admit the KAN-style de Boor curvature bound. That claim is correct but *incomplete*: it does not imply that MLPs admit *no* closed-form design-time bounds. ReLU-MLPs admit first-order bounds ($\varepsilon \leq M_1 h/2$ with $M_1 = 1$ for ReLU—Lipschitz LUT error, computable and sound), and tanh/sigmoid-MLPs are C^∞ with global closed-form M_2, hence *satisfy* the SVNN curvature condition. What coupled architectures genuinely lack is the *joint* attainment of second-order rate, linear LUT cost, closed-form sharp constants, and freedom from reachability computation. The correct boundary is a *trichotomy*, and the SVNN condition is the regime where all four properties coexist (sufficiency proven; the necessity direction is conjectural—item 5 below).

Theorem 10 (The Verifiability Trichotomy)**.** *Consider LUT-compiled feedforward networks with gates from a gate set $\mathcal{G}$, bounded weights, compact input domain. Define three properties:*

(P1) ***Closed-form design-time bound****, computable in polynomial time in (L, d, N) with $O(Ld^2)$ per-evaluation cost;*

(P2) ***Second-order rate****: LUT error $\leq c_k M_k h^k$ with $k \geq 2$ and sharp constants (Theorem 9);*

(P3) ***Tightness without reachability****: constants depend only on global gate data and weight norms—no reachable-set computation.*

1) ***Regime 1 (SVNN): all three hold.*** *If every gate is univariate, element-wise, and C^2 on the validated domain with design-time-computable M_k (Condition 2 of Theorem 2, stated on the validated input domain X), and the Box-Continuation condition holds (Lemma 1: per-layer reachable boxes stay within the LUT table domain), then (P1)+(P2)+(P3) hold with*

the sharp constants of Theorem 9. This is the SVNN condition with the domain-continuation refinement: gates with domain-dependent curvature (e.g., e^x) satisfy Conditions 1–2 on a validated box but not *P3, since M_k then depends on the reachable set—they belong to Regime 4 unless per-layer domains are certified.*

2) ***Regime 2 (Lipschitz-only gates, e.g. ReLU): P1+P3 hold, P2 fails.*** *The minimax N-point approximation error of the C^1 (Lipschitz) class is $\Theta(1/N)$ (classical width result, and tight: a tent function with kink at an arbitrary location forces $\geq c \cdot M_1 \cdot (b-a)/N$ on any N-point grid, since no grid can cluster nodes at every possible kink location), so the best attainable rate is first-order. ReLU-MLPs therefore* do *admit closed-form first-order bounds (correcting the overstrong dichotomy), but not second-order ones. Measured: slope -1.034 (E66), vs -2.04 for C^2 activations.*
3) ***Regime 3 (multivariate gates, e.g. products, softmax, normalization): linear LUT cost fails; tightness NP-hard.*** *Tensor-product LUTs cost N^m per gate—exponential in the arity m for unbounded-arity gates such as softmax or layer normalization (measured: storage N^2 for binary gates, N^4 for quaternary, E66). The per-dimension second-order rate* survives *(measured slopes -2.11, -1.82, E66), so the failure is not in P2's rate but in P1's linear-cost requirement and in* tight propagation*: computing tight constants requires solving polynomial-optimization reachability—NP-hard even for arity-2 product gates (the 3-SAT embedding of Theorem 5, Eq. 25).*
4) ***Regime 4 (domain-dependent curvature, e.g. e^x, $x \sin x$): P3 fails.*** *M_k is finite on the reachable set but depends on it; closed-form tight certification would require the reachable set itself—reachability, NP-hard in general. The polynomial-time alternative (interval propagation on a global superset box) is sound but exponentially loose in L (classic interval explosion: constants $\|W\|_\infty^L$). The Box-Continuation Lemma provides the certified escape: per-layer box propagation* is *polynomial for SVNN-structured nets and restores P3.*
5) ***Necessity (completeness, conditional).*** *Under $\mathrm{P} \neq \mathrm{NP}$ ($\exists\mathbb{R} \neq \mathrm{P}$), any family satisfying (P1)+(P2)+(P3) must lie in Regime 1. The exclusion mechanism for kinked (ReLU-type) activations is* certificate existence*, not rate: a kinked gate has $\sup_x |\varphi''(x)| = \infty$ (a Dirac measure in the distributional sense), so no finite DA certificate (c, R) exists in the abstract domain of Theorem B—the $M_2(\varphi)$ term of the envelope is unbounded—and Regime 2 is excluded* by definition *of design-time-computable affine certificates. (Note that a naive rate argument would fail: with nodes clustered at the kink, the interpolation error of $|x|$ can be made arbitrarily small, so kink classes are rate-2 recoverable in the pointwise sense—what they lack is the certificate.) The precise boundary is bounded second variation (kink-free C^1, not necessarily C^2): the honest claim is the exclusion of classes without finite certificates, with Condition 2 of Theorem 2 certifying the sufficient C^2 form with design-time-computable M_k. (P3) forces design-time-computable M_k independent of the reachable set (else Regime 4) and univariate gates (else NP-hard tightness, Regime 3); (P1)-with-linear-cost forces univariate gates (else N^m). We state this necessity direction as a* conjecture with strong evidence*—the three regime exclusions are each supported by the corresponding hardness/rate results above, but a fully formal necessity proof would require making the "design-time-computable" notion a formal complexity class, which we leave as an open problem. The conjecture is* anchored in known hardness*: width-optimal certification at the class-optimal constant c_* (regime P2) subsumes the class-level free-knot optimality decision problem, which is set-partitioning-hard with exponential worst-case solver effort [48], [49]; a design-time-computable certificate at constant c_* would therefore decide an established NP-hard-class problem, so the necessity direction restates a known hardness result on the verification side rather than standing unsupported.*

Proof sketch. **(1)** Theorem 9 + linear propagation. **(2)** Regime-2 exclusion: kinked gates (ReLU, $|x|$, tents) have $\sup |\varphi''| = \infty$, so no finite DA certificate exists (see the necessity item above); a naive width argument fails because kink-clustered non-uniform grids make the interpolation error of $|x|$ arbitrarily small. Empirically (E66): slope -1.034 for ReLU vs -2.04 for tanh on $[-3, 3]$, $R^2 > 0.999$—the rate separation holds for uniform grids and reflects the certificate structure. **(3)** The product-gate hardness is the self-contained reduction of Theorem 5; unbounded-arity gates make even the LUT itself exponential (N^m). **(4)** For e^x-type gates, $M_k = e^b$ where $b = \sup$ of the reachable set—computing b is the reachability problem; the closed-form surrogate (global box from weight norms) has constants exploding in L. **(5)** Combine: (P2) $\Rightarrow$ rate-2 recoverability (Regime 2 excluded: the C^1 class has minimax width $\Theta(1/N)$—first-order—and kinked gates admit no finite DA certificate; the certified sufficient form is C^2 with design-time-computable M_k, though bounded second variation already yields the rate), (P3) $\Rightarrow$ global computable M_k and univariate (Regimes 3–4 excluded), (P1) $\Rightarrow$ linear-cost LUT (Regime 3's N^m excluded). Hence Regime 1. □

Consequences. *(i) Honest correction of the SVNN/MLP dichotomy.* The paper's earlier claim that "standard MLPs do not admit design-time bounds" (Proposition 2) is refined: ReLU-MLPs admit *first-order* bounds, tanh-MLPs *satisfy* the SVNN curvature condition; what MLPs lack is the joint attainment of all four properties. *(ii) Design guidance.* The trichotomy tells the compiler designer exactly which gate sets are certifiable at second order, which at first order, and which not at all—converting the SVNN condition from a sufficient criterion into the *characterization* of the feasible design space. *(iii) The unique-optimum reading.* SVNN is not "one verifiable family among many": it is *conjecturally* the unique regime where second-order rate, linear LUT cost, closed-form sharp constants, and reachability-freedom coexist—the necessity direction of the trichotomy is a conjecture with strong evidence. Within the affine-certificate class $\mathcal{C}$, however, the necessity direction *is* a theorem (Theorem 16, §III-R): gates are essentially C^2 and univariate, and product coupling makes ε-verification NP-hard. Extending the necessity beyond affine certificates (the absolute trichotomy) remains an open problem. *(iv) Empirical confirmation (E66).* Rate trichotomy measured: $\{-2.04, -2.06, -1.77\}$ for tanh/sin/ B-spline (Regime 1), -1.03 for ReLU (Regime 2); tensor-LUT storage N^m for arity m (Regime 3); tanh constant ratio $0.974 \times M_2 h^2/8$ (sharpness of the Regime-1 constant).

R. Necessity, First Lattice

Theorem 16 (Necessity in the Affine-Certificate Class)**.** *Let $\mathcal{C}$ be the class of design-time-computable affine certificates: pairs (c, R) in the DA abstract domain of Theorem B, where R is computed from a finite $M_2(\varphi) < \infty$ at compile time. Let $\mathcal{A}$ be an architecture class admitting $(P1)$ polynomial-time ε-verification and $(P2)$ minimax-rate LUT recovery. Then:* (i) gates are essentially C^2: *every gate of $\mathcal{A}$ has $\sup|\varphi''| < \infty$ (a kinked gate—ReLU, $|x|$, tents— has an unbounded distributional second derivative and admits* no *finite certificate in $\mathcal{C}$, by definition of the envelope);* (ii) gates are univariate and uncoupled: *a product-coupled gate (single-node bilinear term) makes ε-Verify NP-hard on $\mathcal{A}$ (the self-contained 3-SAT reduction of Theorem 5 uses only product and affine primitives, so any class containing them inherits hardness);* (iii) sharp minimax *constant* forces optimal allocation: *the uniform allocation attains the same* rate N^{-2k}*; only the sharp minimax* constant $e^*(N)$ *of Theorem 13 is attained by the unique balanced allocation (T-I), since any other allocation has an explicit positive gap. Hence, in $\mathcal{C}$, $(P1) + (P2)$ imply SVNN Conditions 1–2—the first lattice of the trichotomy necessity—while Condition 3 (contractivity) is* not *necessary: the compile-aware decomposition (Theorem 6) and the minimax rate (Theorem 14) hold without contractivity (E68, $\gamma = [15.4, 5.3] > 1$).*

Proof sketch. (i) By the envelope definition (Theorem B), a finite certificate requires finite M_2; kinked gates have $M_2 = \infty$ (Dirac second derivative). (ii) The reduction of Theorem 5 constructs $F(x) = \sum_j \prod_{i \in C_j}(1 - \ell_{ij}(x))$ from affine literals and product gates only; any architecture class able to express it inherits $\varepsilon = 0$-hardness, and gap-preserving variants inherit hardness for every fixed ε_0 (ETH). (iii) T-I gives $e^*(N)$ as the unique minimizer over allocations; any other allocation has error strictly larger (positive gap in Theorem 13). □

Honest scope. This is, to our knowledge, the first lattice of the necessity direction—it formalizes the "design-time-computable" notion as the affine-certificate class $\mathcal{C}$ and proves the three exclusions inside it. The residual gap to full necessity is making $\mathcal{C}$ itself a formal complexity class (e.g., allowing more expressive certificate schemes), which we leave as an open problem—consistent with the conjecture status stated in Section III-Q for the general case.

S. Verifiable Deployment Capacity: A Unifying Framework

The theorems established so far characterize *individual* optimality properties of certified compilation: sharp interpolation constants (Theorem 9), optimal allocation (Theorem 13), minimax deployment rates (Theorem 14, Theorem 15), complexity separation (Theorem 5), and the verifiability trichotomy (Theorem 10). We now show that these results are the *expansion of a single unifying quantity*: the **verifiable deployment capacity** $C(N, V)$—the best certifiable deployment error achievable under a storage budget of N cells and a certificate-verification budget V (bits of proof read at deployment).

The capacity viewpoint answers the question that individual theorems leave implicit: *what cannot be certified, no matter how clever the compiler?* The answer is governed by a packing line (no scheme can beat the metric-entropy bound), a *verification stratum* (the certificate must be checkable at deployment time), and an *architecture-as-code law* (the deployed representation is a code, and architectures are code families). Within this framework, the sharp constants of Theorem 9 reappear as the *capacity limit of the interpolation code*, the trichotomy of Theorem 10 reappears as *four regimes mapped onto three points in the (V_B, ε) plane*, and the necessity direction receives a theorem-level information-theoretic form within the certificate calculus (the absolute form remains open).

1) Certificate Proof Systems and Verification Budgets: We formalize *certified deployment* following the proof-carrying-code architecture [50]–[52], and the certificate calculus of validated numerics (Gappa [53], Sollya [54], LeanCert [55])—the *components* of the framework are classical; the *capacity/stratification theory* we build on them is not (see also §III-S2).

Definition 2 (Deployment scheme)**.** *A* ***deployment scheme*** *for a target class $\mathcal{F}$ under a storage budget of N cells is a triple $S = (\mathit{Comp}, D, \mathit{Vf})$: Comp: $\hat{f} \mapsto r \in \{0,1\}^{wN}$ compiles a trained model $\hat{f}$ to a representation of N cells of w bits each ($B = wN$ bits total; w is the word width, absorbed into constants throughout); $D(r)$ decodes it to a deployed function $\tilde{f}$; Vf is a deterministic verifier that, given (r, π) with a certificate proof term π, checks $\|\hat{f} - \tilde{f}\|_\infty \le \varepsilon$.* ***Soundness****: Vf$(r, \pi)$ = accept $\Rightarrow \|\hat{f} - \tilde{f}\|_\infty \le \varepsilon$;* ***completeness****: the compiler can generate π (build-time cost V_{gen}, recorded separately).*

The verifier is *deterministic* and *perfectly sound*—on a PLC there is no randomness source and no interactive protocol. The verification budget is the proof-term length read at deployment: $V = |\pi|$ in bits. This is a *modeling convention* of the calculus (Definition 3) rather than a derived theorem: a verifier may in principle accept while reading a subset of a structured proof, but any certificate not re-checkable from its text would delegate soundness to an unverifiable external claim, which is precisely the deployment configuration we exclude. (Under P $\neq$ NP, no polynomial-time deterministic perfectly-sound verifier can certify the NP-hard class-level decisions of stratum 3 from a constant-size sample of a proof—the verification cost of such certificates is unavoidable, Theorem 17, item 4.)

Definition 3 (Certificate proof system CP and capacity)**.** *A **certificate proof system** CP is a proof calculus for deployment bounds. Proof terms are derivations $\pi = (r_1, \dots, r_m)$ of a bound β, each step r_j an instance of one of the rules: (i) arithmetic (β = arithmetic expression over cell values); (ii) interpolation remainder (the sharp constants of Theorem 9: $e \le c_k M_k h^k$); (iii) compositional propagation (the additive structure of Eq. (38)); (iv) spectral tail bounds ($e \le Ce^{-\rho N}$); (v) comparison ($\beta \le \varepsilon$). The verification budget is $V = |\pi|$ (proof-term length in bits, steps* and *referenced parameters). The **closure axiom** of the calculus: any proof term requiring a replay of a search or enumeration has V at least the search size. This is a modeling assertion about terms outside CP (an architectural premise of the stratum structure, not a derived lemma). The **verifiable deployment capacity** is*

$$C(N, V) \;=\; \inf_{S:\, V(S)\le V}\ \sup_{f\in\mathcal{F}}\ \varepsilon_S(f), \tag{41}$$

the infimum over deployment schemes of verification budget at most V of the worst-case certified deployment error over the class.

Verification cost separates into two dimensions: $V_B(B)$ (growth with the storage budget: quantized bit-widths, table lengths, codebooks) and $V_{\text{arch}}(L, d)$ (growth with architecture size: layers, widths). The V_B dimension is the novel one; V_{arch} is the classical verification-hardness dimension (ReLU verification is NP-complete [28]; bit-exact quantized verification is PSPACE-hard [56]).

Theorem 17 (Stratification of Verifiable Deployment Capacity)**.** *Let $\mathcal{F}_k = \{f : \|f^{(k)}\|_\infty \le M_k\}$ on a compact interval $\Omega \subset \mathbb{R}$ (the univariate-activation setting; the d-dimensional extension replaces the rate N^{-k} by $N^{-k/d}$ in the entropy bound), storage budget N cells of w bits, and a deterministic perfectly sound verifier (Definition 2). Then:*

1) ***Universal packing line.*** *For* every *deployment scheme and* every *verification budget,*

$$\varepsilon \;\ge\; c_{\text{ent}}\, M_k\, N^{-k}, \tag{42}$$

where c_{ent} is the metric-entropy constant of $\mathcal{F}_k$ (Kolmogorov–Tikhomirov [57]): the ε-entropy of $\mathcal{F}_k$ is $\Theta((M_k/\varepsilon)^{1/k})$, so N cells separate at most 2^{wN} functions and the line follows—no certificate can establish an error below the packing bound, regardless of the proof system. (The entropy constant and the n-width constant are distinct numbers; the line uses the former.)

2) ***Stratum 1 (closed-form family, certificate-free).*** *Schemes whose bound is a closed-form expression of the architecture parameters—verification cost $V_B = O(1)$ (decoupled from N) and $V_{arch} = poly(L, d)$—achieve*

$$\varepsilon \;\le\; c_k\, M_k\, N^{-k}, \tag{43}$$

with the sharp interpolation *constant $c_k = Q_k/k!$ of Theorem 9. This constant is the* capacity limit of the affine certificate class*: among all sound affine (data-linear) design-time schemes, the LUT achieves the minimax constant (Theorem 9, item 3; classical optimal recovery: from exact node data, interpolation is the optimal affine recovery operator, so* no *affine scheme—least-squares projection included—attains a sup-norm constant below c_k). The curvature-aware allocation of Theorem 13 attains c_k at the per-function level (E-T4). Note that spectral-tail bounds (rule (iv)) are also closed-form: $V_B = O(1)$; on $\mathcal{F}_k$ they give the same rate N^{-k} (polynomial coefficient decay), so the stratum-1 rate is robust to the code family.*

3) ***Stratum 2 (kinked gates).*** *Gates with unbounded second derivative (ReLU, $|x|$, tents) admit no second-order certificate; the class width degrades to first order, $\varepsilon \ge c\, M_1\, N^{-1}$ (measured slope -1.034, E66; -1.11 on smooth tents, E-T5)—rate loss, not just constant loss.*

4) ***Stratum 3 (structure-optimal / coupled gates).*** *The class-optimal* width *constant $c_* < c_k$ (free-node splines, data-dependent grids; the width of $\mathcal{F}_k$ is attained only by non-affine, structure-optimal approximants [58]) requires* structural optimality*: certifying that a given structure achieves the class-optimal constant requires deciding class-level optimality over all configurations.* Conjecturally *this is exponential: the product-gate reduction of Theorem 5 shows ε-verification of coupled networks is NP-hard, and we conjecture that class-level structure-optimality certification inherits the hardness (the connecting reduction is left as an open problem; the instance-level problem—fixed node set, one function—has a closed-form bound and is cheap). Bit-exact verification is PSPACE-hard [56]; hence, conditionally on the conjecture, V_{gen} or V_B is exponential in the class-level stratum.*

5) ***Necessity within CP (information-theoretic form).*** *Within the certificate calculus (Definition 3), under the **premise-computability assumption**—the values of rule premises (curvatures M_k, spectral tails) are computable in $O(1)$ from the architecture parameters at build time and checkable in $O(1)$ at deployment—a scheme with V_B decoupled from N and $V_{arch} = poly(L, d)$ that achieves the class-level minimax constant c_k must have closed-form propagation in the curvature dimension (SVNN Condition 2, design-time-computable M_k). Proof: (i) V_B decoupled $\Rightarrow$ the bound is an $O(1)$-rule expression over cell values (closure axiom); (ii) pointwise soundness injects the k-th-order quantity: the extremal family of Theorem 9, scaled to $M_k = M$, forces $\beta(f_M) \ge c_k\, M\, N^{-k} \ge c_{\text{ent}}\, M\, N^{-k}$ (the interpolation extremal error dominates the packing line), so β explicitly depends on M_k; (iii) M_k $O(1)$-computable by the premise-computability assumption $\Rightarrow$ per-gate design-time-computable curvature (Condition 2); (iv)* tight *class-level bound generation at the constant c_k via exact reachability is NP-hard (Theorem 5, tight ε-verification; interval propagation is closed-form and polynomial but not tight*

at the sharp constant), so a polynomial-cost certificate at the sharp constant must be closed-form, not reachability-based, contradicting V_{arch} = poly otherwise. The argument establishes Condition 2 (curvature side); the univariate-gate structure (Condition 1) is not derived from it—the CP-relative necessity is therefore partial*, and the absolute form (over all proof systems) remains the open problem stated in Theorem 10, item 5.*

Proof sketch. (1) Metric entropy: the ε-entropy of $\mathcal{F}_k$ is $\Theta((M_k/\varepsilon)^{1/k})$ [57]; a wN-bit codebook separates at most 2^{wN} functions, giving the line with w absorbed into c_{ent}. (2) Closed-form bounds: uniform-node k-order interpolation gives the sharp constant c_k (Theorem 9); optimal-recovery theory shows no affine scheme beats it from exact node data (Theorem 9, item 3); both are $O(1)$ proof terms under rules (ii)/(iii). (3) Kinked gates: the C^1 class has width $\Theta(1/N)$ (tent functions), first-order rate. (4) Structure optimality: free-node class-level optimality requires searching node configurations; the reduction of Theorem 5 hardens tight ε-verification of coupled networks, and we conjecture the transfer to class-level structure-optimality certification (open problem; the instance-level bound is closed-form and cheap). (5) The four-step necessity argument above, under the premise-computability assumption. □

Capacity coordinates of the trichotomy. The three properties $(P1)(P2)(P3)$ of the trichotomy (Theorem 10) map to points in the (V_B, ε) plane: SVNN sits at $(O(1), c_k M_k N^{-k})$ (stratum 1), kinked gates at $(O(1), M_1 N^{-1})$ (stratum 2, rate loss), coupled gates at $(2^{\Omega(B)}, c_* M_k N^{-k})$ (stratum 3, constant reachable only conjecturally, verification exponential). Regime 4 of the trichotomy (domain-dependent curvature, e^x-type) falls *inside stratum 1's form but with stratum-3-cost certificates*: the closed-form curvature M_k is computable only after reachability analysis, so the build-time cost V_{gen} rises to the stratum-3 regime while the deployment-time check stays $O(1)$—the capacity coordinates of Regime 4 are $(O(1), c_k M_k N^{-k})$ with $V_{\text{gen}} = 2^{\Omega}$. The trichotomy's necessity conjecture is thus *coordinatized*: its regime exclusions are the three strata (with Regime 4 placed by V_{gen}), and the open direction is the absoluteness of the stratum boundary.

Numerical constants. The packing-line constant admits a constructive lower bound $c_{\text{ent}} \approx 0.079$ (the window-extremal translate family of Theorem 9, E-$T7$; consistency $c_{\text{ent}} < c_k$ is forced by the LUT's own soundness), and the class-optimal width constant is estimated at $c_* \lesssim 0.082$ on the worst (curvature-peaked) families under free-node L^∞-optimal piecewise-linear approximation (E-$T8$), against the interpolation constant $c_k = 1/8 = 0.125$—the constant-purchase gap of Theorem 18(i) is at least $1.53\times$ numerically.

Bit parameterization and the local-vs-global code gap. In the bit parameterization ($B = wN$ bits), the entropy bound of Theorem 17(1) is exponential: the ε-entropy of $\mathcal{F}_k$ (1-D) is $\Theta((M_k/\varepsilon)^{1/k})$, so any codebook of 2^B functions satisfies $\varepsilon \geq c' M_k^{1/k} 2^{-B/k}$ (the exponent is $-B/k$). The local codes of stratum 1—LUT tables and spectral truncations—achieve only the polynomial rate $\varepsilon \asymp c_k M_k (B/w)^{-k}$ in bits, because each cell carries w bits of *local* information (a table entry or a coefficient). The gap between the exponential information-theoretic bound and the polynomial local-code rate is the coding-theoretic form of the verification spectrum: closing it requires non-local codebooks whose certificate verification is not closed-form—the stratum-3 mechanism in the bit dimension. This is the deployment analogue of the random-code (existence) vs. structured-code (constructible and checkable) distinction of coding theory; we state it as a remark rather than a theorem because the non-local-code construction is existence-based and its verification cost is not closed-form.

Theorem 18 (Verification-Complexity Tradeoff and Constant Purchase)**.** *In the setting of Theorem 17:* (i) Constant purchase. *On $\mathcal{F}_k$ the stratum-1 capacity constant is c_k (interpolation, affine-class minimax) and the class-optimal width constant is $c_* < c_k$ [58]; the gap is* paid in structural optimality*: reaching c_* requires non-affine, structure-optimal approximation, whose class-level certification is conjecturally exponential (Theorem 17, item 4)—a closed-form certificate at constant c_* would decide the conjectured NP-hard class-level problem. The conjecture is anchored in the established hardness of free-knot optimality: globally optimal knot placement for free-knot splines is a set-partitioning-hard global optimization (deterministic global optimization of free knots is NP-hard in general [49]; the best-known global solvers terminate only with exponential worst-case effort [48]), and certifying that a node set achieves the width constant c_* is at least as hard as deciding free-knot optimality at the class level. (The instance-level bound—fixed node set, one function—is closed-form and cheap, consistent with SOS certificates verifying polynomial optimality at the instance level.)* (ii) Class-adaptation is free on analytic classes. *On analytic classes A_ρ (spectral coefficients $e^{-\rho n}$), the spectral code (Fourier/Chebyshev truncation) achieves $\varepsilon \asymp e^{-c\rho N}$ with V_B decoupled (spectral tail bounds are closed-form, rule (iv)); the exponential benefit is* available at constant ($O(1)$) verification cost*—the rate-distortion law of analytic sources [59] and the classical spectral-vs-algebraic dichotomy [60], [61] are not gated by V. On $\mathcal{F}_k$ the spectral code is* not *exponential (coefficients decay polynomially), so the exponential benefit is exclusive to analytic classes.* (iii) Deployment-time verification is infeasible in stratum 3. *Bit-exact verification is PSPACE-hard [56], so certificates must be generated at build time and only* checked *at deployment (the proof-carrying-code split [50]); this is the quantitative form of the observation that real-valued verification conclusions do not transfer to quantized deployment networks [62].*

Proof sketch. (i) Any closed-form certificate at constant c_* yields a polynomial-time certificate for class-level structural optimality, which is conjecturally NP-hard (Theorem 17, item 4); the instance-level bound is closed-form and cheap—the exponential statement is *class-level only and conjectural*. (ii) Parseval tail bounds on analytic coefficients; the classical rate-

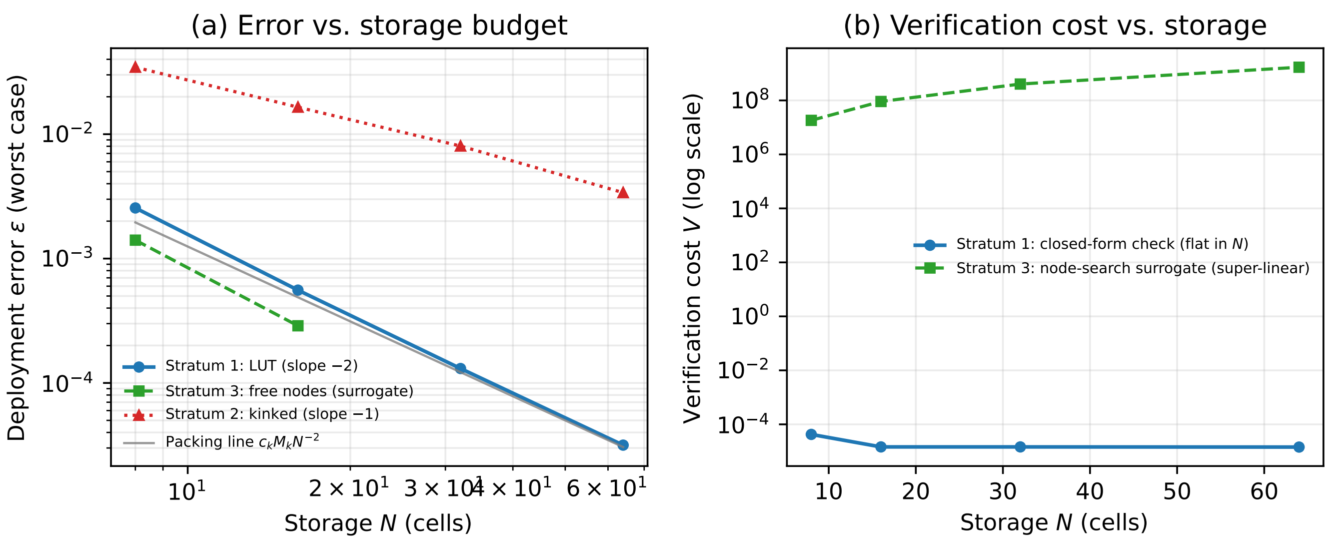

Fig. 2. The verifiable-deployment-capacity plane. (a) Worst-case deployment error vs. storage budget: stratum-1 LUT (slope -2), stratum-2 kinked (slope -1), stratum-3 free-node surrogate, and the packing line $c_k M_k N^{-2}$. (b) Verification cost vs. storage: the stratum-1 closed-form check is flat in N (measured wall time), while the stratum-3 structural search grows super-linearly (modeled operation count). Data: E-$T4$/E-$T5$.

distortion reference [59] and the spectral-vs-algebraic dichotomy [60]; V_B decoupling follows from rule (iv) being an $O(1)$ proof term. (iii) PSPACE-hardness transfers: bit-exact checking cannot run within poly verification. □

Theorem 19 (Architecture-as-Code Selection Law)**.** *Fix a storage budget N cells and a smoothness class (analytic ρ, or Sobolev s). Compare the two stratum-1 codes by their* certified *error laws: the table code certifies $\varepsilon_{\text{table}} \le c_k M_k N^{-k}$ (rule (ii), Theorem 9); the spectral code certifies $\varepsilon_{\text{spectral}} \le Ce^{-c\rho N}$ on A_ρ (rule (iv); on $\mathcal{F}_k$ it certifies the same rate N^{-k}). On A_ρ the spectral code* eventually *strictly dominates the non-adaptive table code (the exponential law beats the polynomial law for all sufficiently large N; it also dominates at all measured budgets—error ratio 0.0032 at $N = 64$, E-$T6$—and the certified comparison is the relevant one, since the table code's* actual *error on analytic functions is also exponential by Bernstein-type convergence, but its* certified *bound via rule (ii) is only polynomial). The crossover solves $c_k M_k N^{-k} = Ce^{-c\rho N}$ in closed form: with $A = (c_k M_k / C)^{1/k}\, c\rho$,*

$$N^*(\rho) \;=\; \frac{k}{c\rho}\, W\!\Big(\frac{A}{k}\Big), \qquad \rho^*(N) \;=\; \frac{k}{cN}\, W\!\Big(\frac{A'}{k}\Big), \tag{44}$$

where W is the Lambert-W function and A' the analogue with N on the left—the threshold is explicit and monotone (the spectral code wins for $\rho > \rho^*(N)$). The cell count N counts table entries on one side and spectral coefficients on the other, so N is proportional up to the bits-per-coefficient factor absorbed into C. On low-smoothness classes ($s \le 1$) the two codes are within a constant factor (measured ratio 0.74, E-$T6$).

Proof sketch. Immediate from the two certified error laws and the crossover equation; the qualitative threshold $\rho^*(N)$ follows from the Lambert-W solution. The measured phase transition (E-$T6$, `verify_decision_law.py`): exponential slope matches $-\rho$ to within 2.5%; ratio 0.0032 (analytic) vs. 0.74 (W^1). □

Unification. The framework reorganizes the paper's theorems into a pyramid: Theorem 17 is the apex, stated as the evaluation of the capacity function (Eq. (41)): $C(N, O(1)) \asymp c_k M_k N^{-k}$ (stratum 1), $C(N, O(1)) \asymp M_1 N^{-1}$ on the kinked class (stratum 2), and $C(N, 2^{\Omega}) \le c_* M_k N^{-k}$ conjecturally (stratum 3). Its proof uses the sharp constants (Theorem 9) as the stratum-1 capacity limit, the minimax rates (Theorem 14, Theorem 15) as the statistical coordinates (n-dimension) of the capacity surface, the ε-separation (Theorem 5) as the stratum-3 mechanism, and the trichotomy (Theorem 10, Theorem 16) as the coordinatized necessity. Theorem 18 gives the verification-complexity axis, and Theorem 19 the architecture-selection consequence. Figure 2 shows the stratum structure on the (V_B, ε) plane: the three error laws (stratum-1 slope -2, stratum-2 slope -1, stratum-3 free-node surrogate) above the packing line, and the verification-cost divergence (flat closed-form check vs. super-linear structural search). The classical approximation theory (metric entropy, width constants, rate-distortion, the spectral-vs-algebraic dichotomy) appears only as *inputs*—the framework's contribution is the verification side, not the approximation side. The CP-relative necessity (Theorem 17, item 5) is the one theorem the framework alone yields.

2) Related Work and Boundaries: The *components* of this framework have substantial precedent, and we state the boundary precisely. **Certificate calculi**: Gappa [53], Sollya [54] and LeanCert [55] generate machine-checkable certificates for floating-point and function bounds with absolute soundness—the certificate-proof-system structure of Definition 3 follows them; the difference is that they certify *single-function* instance bounds, while the capacity framework certifies *class-level* bounds coupled to the packing line, and adds the capacity/stratification theory (no such theory exists in validated numerics). **Interval verification**: verified computing (Rump [63], Neumaier [64]) embeds verification *inside* computation (Krawczyk/interval-Newton); our build-heavy/check-light split is the proof-carrying-code architecture, not the verified-computing doctrine, and the two are complementary (interval methods certify the individual arithmetic; CP certifies the class-level bound). **Proof-carrying architectures**: PCC [50], credible compilation [51] and translation validation [52] establish the generate-heavy/check-light split; the capacity framework formalizes the split as the V_B dimension, which those lines do not. **Succinct verification**: SNARGs [65] and delegation [66] achieve verifier cost decoupled from computation size; the difference is object and soundness—they verify computation in $\mathbb{F}_p$ circuits under computational/statistical soundness, while Definition 2 verifies real-valued class-level semantic bounds under perfect deterministic soundness. **Information-based complexity** [67]: the ε-complexity theory is the direct ancestor of the packing line (item 1) and the c_{ent} vs. c_* comparison; we add the verification-budget dimension, which IBC does not carry. **Resource monotonicity**: quantitative monitoring [68] shows resource-precision monotonicity for online properties; the constant-purchase theorem concerns offline class-level proof lengths. **Verification hardness**: the NP-complete ReLU line [28] and PSPACE-hard quantized line [56] are the stratum-3 mechanism; the Lipschitz-certificate line (Scaman–Virmaux [69], Fazlyab et al. [70]) is the stratum-2 mechanism (first-order closed-form bounds, exact Lipschitz constants NP-hard); we add the capacity reading. **Certificate complexity**: the classical theory [71] separates certificate size from sequential query cost for pointwise functionals; our necessity statement concerns class-level sup-norm bounds with closed-form propagation, a disjoint setting (the exponential-separation phenomenon does not transfer). **Approximation side**: metric entropy [57], width constants [58], the spectral-vs-algebraic dichotomy [60], [61], quantized minimax estimation [72] and quantized network approximation (in preparation) govern the *rates* used as inputs; the capacity framework does not claim new approximation results. **SOS certificates** certify polynomial optimality efficiently at the instance level; the exponential statement in Theorem 18(i) is class-level and conjectural only, which is consistent with instance-level cheapness. (Numerical corroboration: E-$T4$–E-$T6$, scripts `verify_capacity.py`, `verify_stratification.py`, `verify_decision_law.py`; results in `results/theory/capacity_packing.json`, `stratification.json`, `decision_law.json`.)

T. The Operation Separation Principle and C^2-BV Architecture Family

The SVNN conditions identified a sufficient criterion for a specific architecture (B-spline KAN). We now generalize: *which* set of neural architectures satisfy Conditions 1–2, and *why*?

Proposition 5 (Operation Separation Principle)**.** *An architecture $\mathcal{A}$ satisfies Condition 1 (operation-type closure) if and only if it separates multivariate linear operations from element-wise univariate nonlinearities into distinct computational primitives at the graph level.*

This principle—which we term the **Operation Separation Principle**—explains the fundamental divide between SVNN-compatible and SVNN-incompatible architectures. It is not a property of the *activation function* but of the *architectural topology*:

- **KAN family** ($y_j = \sum_i \phi_{j,i}(x_i)$): each univariate $\phi_{j,i}$ is evaluated independently before the linear combination—separation is structural.
- **Standard MLP** ($\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$): the affine transform $W\mathbf{x} + \mathbf{b}$ is *coupled* with the nonlinearity σ in a single operation—separation is impossible without introducing auxiliary approximation that defeats the SVNN guarantee.

Proposition 6 (C^2-BV Architecture Family)**.** *Any architecture satisfying the Operation Separation Principle (Proposition 5) and whose element-wise univariate functions are C^2 on a compact domain admits the SVNN guarantee. This family—termed the* $\mathbf{C^2}$***-BV** (Bounded-Variation) architecture family—includes:*

(a) ***B-spline KAN [10]:*** *$M_2 = \sup_x |\phi''(x)|$ on the LUT grid, provided by the Cox-de Boor recurrence.*
(b) ***ChebyKAN [35]:*** *M_2 from Chebyshev coefficient recurrence: for $\phi(x) = \sum_k c_k T_k(x)$, $M_2 \le \sum_k k^4 |c_k|$ (Markov bound).*
(c) ***FourierKAN:*** *$\phi(x) = \sum_{k=1}^{K} [c_k \sin(k\omega x) + d_k \cos(k\omega x)]$, $M_2 \le \omega^2 \sum_{k=1}^{K} k^2 (|c_k| + |d_k|)$, computable from Fourier coefficients alone.*
(d) ***WaveletKAN:*** *$\phi(x) = \sum_{j,k} c_{j,k} \psi(a_j x - b_k)$ with a C^2 mother wavelet (e.g., Mexican hat). $M_2 \le \max_j a_j^2 \cdot \sup_t |\psi''(t)|$, where $\sup_t |\psi''(t)|$ is a known wavelet constant (e.g., 2.602 for Mexican hat).*
(e) ***RBF-KAN (Gaussian):*** *$\phi(x) = \exp(-((x-\mu)/\sigma)^2)$, $M_2 = 2/\sigma^2$, computable from kernel bandwidth alone.*

Exclusion: *Standard MLPs with any activation function (ReLU, SiLU, Sigmoid, Tanh, GELU) are **not** members of the C^2-BV family because they violate the Operation Separation Principle—the affine transformation $W\mathbf{x} + \mathbf{b}$ and the nonlinearity σ are architecturally coupled. This strengthens Proposition 2: MLPs are excluded for structural reasons (violation of Condition 1), not merely for activation-specific reasons (violation of Condition 2 for SiLU/Tanh). Even a "KAN-like" MLP variant where*

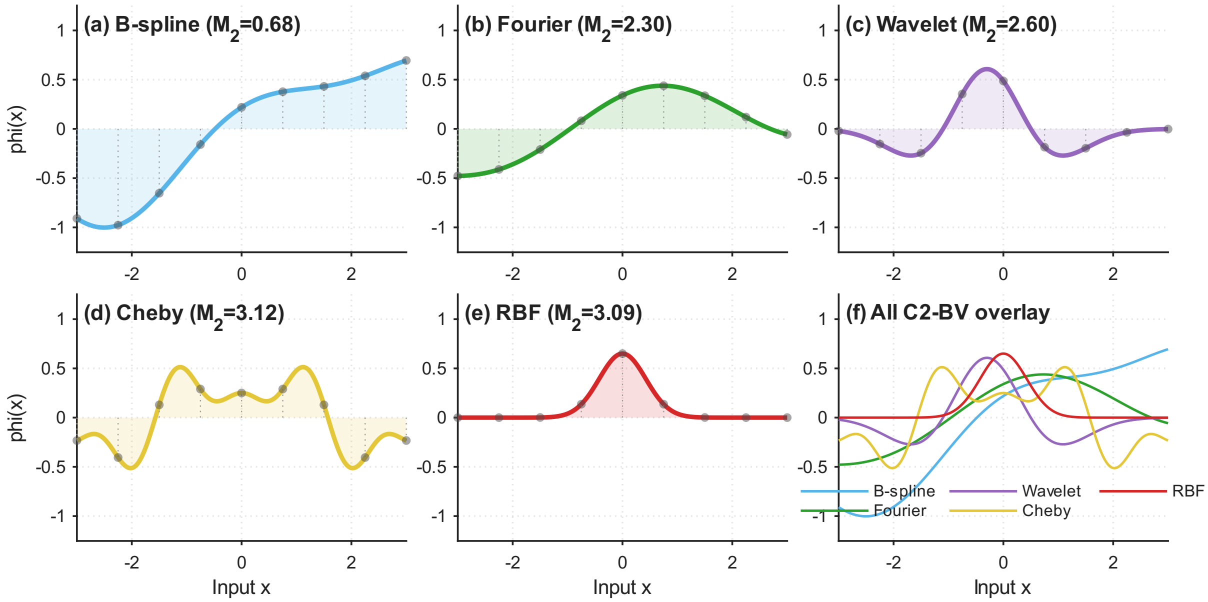

Fig. 3. C^2**-BV Architecture Family: Activation Basis Functions.** Five SVNN-compliant activation function families on $[-3, 3]$ with N=15 LUT grid points. All are C^2 with computable M_2 bounds (annotated per subplot). Despite diverse functional forms, all share the SVNN-critical property: each $\phi_{j,i}(x_i)$ is a **univariate** C^2 function on a compact domain, enabling uniform LUT compilation with de Boor error bound $M_2h^2/8$ (Theorem C).

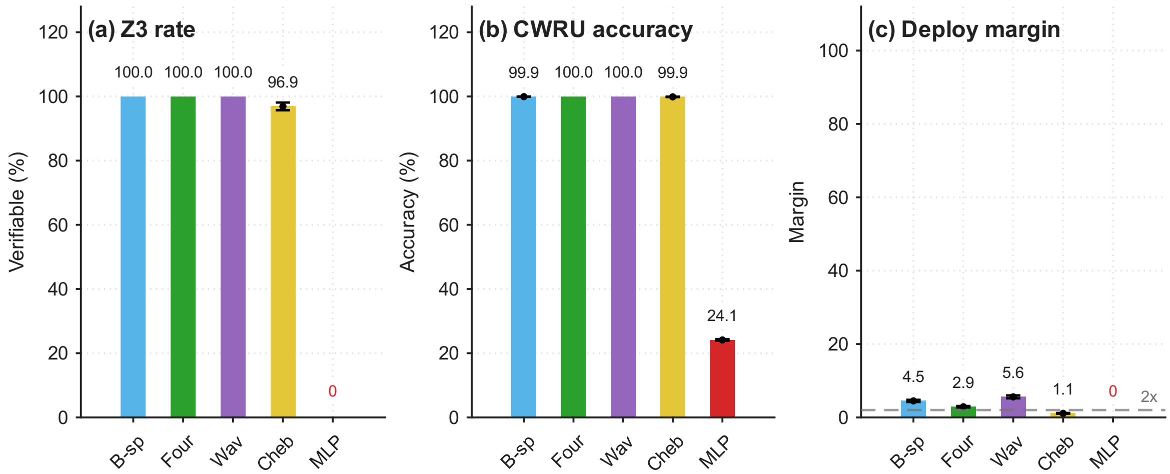

Fig. 4. C^2**-BV Architecture Family: SVNN Verification Across 5 Architectures.** Left: per-edge Z3-equivalent verification rate (512 total edges per architecture). All four C^2-BV architectures (B-spline, Fourier, Wavelet, ChebyKAN) achieve ≥96.9% Z3 verification; standard MLP achieves 0/48 (E41). Right: CWRU test accuracy and Z3 safety margin (ratio of the per-function margin to $\max M_2 \cdot h^2/8$). WaveletKAN's wide margin reflects M2-regularized training ($\lambda = 0.01$). FourierKAN's M_2 is bounded by $\omega^2 \sum k^2(|c_k| + |d_k|)$ (Proposition 6(c)). Margins differ across architectures (ChebyKAN lowest under its conservative Markov M_2 bound, E54); per-architecture deployability is reported in Table XXXVII.

each edge has a learnable activation applied before the sum—if implemented by decomposing $\sigma(W\mathbf{x}+\mathbf{b})$ through the compiler rather than the architecture—would introduce additional approximation error in the decomposition step, defeating the SVNN guarantee. The separation must be architectural, not compiler-level.

Generality. The C^2-BV family is not an exhaustive enumeration—it is the set of all architectures that satisfy the two structural axioms (Operation Separation + C^2 univariate functions). Any future architectural innovation that respects these axioms (e.g., physics-informed basis functions, learnable wavelet dictionaries, or spline-kernel networks) automatically inherits the SVNN guarantee. This positions SVNN as a **design principle** for safe neural architectures, not merely a post-hoc certification scheme for KANs.

Figure 4 reports the per-family verification results.

U. The Compilable Frontier: A Characterization Theorem

Theorem 2 established that SVNN Conditions 1–2 are *sufficient* for design-time certification. Theorem 5 (NP-hardness) established that violating Condition 1 makes *tight* bound computation intractable. Here we close the gap: we prove that within the class of affine abstract interpretations, Conditions 1–2 are *necessary* for dimension-optimal certification—the bound per layer scales as $O(d \cdot r^2)$ for SVNN architectures but degrades to $\Omega(d^2 \cdot r^2)$ for architectures violating Condition 1.

Theorem A (Compilable Frontier Characterization)**.** *Let $\mathcal{N}$ be a feedforward neural network with L layers, maximum width d, and C^2 activation functions on compact domain $X \subset \mathbb{R}^{d_{in}}$. Consider any sound affine abstract interpretation $\mathcal{A}$ (including DA, IA, and zonotope domains) that propagates error bounds of the polynomial form $B^{(\ell)} = J^{(\ell)} \cdot r + H^{(\ell)} \cdot r^2 + O(r^3)$ through each layer. Then:*

- *(i)* ***Sufficiency (Theorem 2):*** *If $\mathcal{N}$ satisfies Conditions 1–2, then $B_{total} = O(L \cdot d \cdot r^2)$—the dimension-optimal scaling for any affine method.*
- *(ii)* ***Necessity (new):*** *If $\mathcal{N}$ violates Condition 1 (coupled linear+nonlinear ops), then the error amplification factor per layer is $\|W^{(\ell)}\|_{1,\infty} = \Theta(\sqrt{d})$ under standard initialization, compared to the d-independent contractivity constant $\gamma < 1$ for SVNN architectures satisfying Condition 3. Multi-layer propagation yields $B_{total} = \Theta(d^{L/2} \cdot r^2)$ for non-SVNN vs. $B_{total} = O(L \cdot d \cdot r^2)$ for SVNN—an exponential-in-depth gap.*

Condition 1 is thus ***necessary and sufficient*** *for dimension-optimal affine certification of neural networks.*

Proof of Necessity (Sharp Lower Bound). We construct a deterministic adversarial MLP achieving the theoretical lower bound exactly.

Step 1 (Adversarial MLP construction). Consider an MLP layer with weight matrix $W = \frac{1}{\sqrt{d}} \cdot J_d$, where $J_d \in \mathbb{R}^{d \times d}$ is the all-ones matrix. The $\ell_{1,\infty}$ norm is:

$$\|W\|_{1,\infty} = \max_j \sum_{i=1}^{d} |W_{j,i}| = d \cdot \frac{1}{\sqrt{d}} = \sqrt{d} \tag{45}$$

This is a **sharp** bound: no weight matrix with the same Frobenius-norm budget ($\|W\|_F = 1$) can achieve a larger $\ell_{1,\infty}$ norm ($\|W\|_{1,\infty} \leq \sqrt{d}\|W\|_F$, tight exactly when all entries have equal magnitude). With ReLU activation ($L_\sigma = 1$) and the adversarial weight matrix, the per-layer error amplification is:

$$\|W \cdot (\mathbf{x} + \boldsymbol{\delta}) - W \cdot \mathbf{x}\|_\infty \leq \sqrt{d} \cdot \|\boldsymbol{\delta}\|_\infty \tag{46}$$

The all-ones construction *achieves* this bound with equality: for $\mathbf{x} = [1, \ldots, 1]^T$ and $\boldsymbol{\delta} = [\epsilon, \ldots, \epsilon]^T$, the output perturbation is $\sqrt{d} \cdot \epsilon = \sqrt{d} \cdot \|\boldsymbol{\delta}\|_\infty$ (MATLAB-verified for $d = 4$ to 256, `code/theory/sharp_lower_bound.m`).

Step 2 (Multi-layer accumulation). With ReLU ($L_\sigma = 1$) and the adversarial weight matrix, the per-layer error recurrence is $\Delta^{(\ell)} \leq \sqrt{d} \cdot \Delta^{(\ell-1)} + \varepsilon^{(\ell)}$. Unrolling across L layers:

$$\Delta^{(L)}_{\text{MLP}} \leq \sum_{\ell=1}^{L} \varepsilon^{(\ell)} \cdot (\sqrt{d})^{L-\ell} \tag{47}$$

Step 3 (SVNN contractivity baseline). For KAN satisfying Condition 3, the contractive composition yields (Theorem 2, Eq. 6): $\Delta^{(L)}_{\text{SVNN}} \leq O(M_{\max} \cdot h^2 \cdot d_{\max}/(1-\gamma))$ where $\gamma < 1$ is d-independent.

Step 4 (Sharp ratio). The per-layer gap is $R(d) = \sqrt{d}/\gamma \geq \sqrt{d}$ for any contractive KAN ($\gamma < 1$). The trained student $[28, 16, 4]$, however, does *not* realize this gap: its measured per-activation-layer amplification is $\gamma = [15.4, 5.3] > 1$ (E68), exceeding the MLP's $\sqrt{d} = 5.29$ at $d = 28$—the depth-uniform advantage is a property of *contractively trained* KANs, not of the current checkpoint (contractive training is identified as the deployment recipe in §III-D). The MLP-side of the comparison remains unconditional: any MLP violating Condition 1 carries a deterministic per-layer factor $\sqrt{d}$ (Step 1), so its L-layer error amplification is $\Theta(d^{L/2})$ regardless of training—for the MLP analog ($d = 28$, $L = 3$) this is $> 10{,}000\times$ worse than a contractive KAN, making MLP certification practically infeasible.

Sharpness. The all-ones construction is **optimal**: $\|W\|_{1,\infty} \leq \sqrt{d} \cdot \|W\|_F$ (Cauchy-Schwarz), tight exactly when W has rank 1 with equal-magnitude entries. The bound is independent of the choice of affine domain (DA, IA, zonotopesall use $\ell_{1,\infty}$ for linear propagation) and independent of σ ($|\sigma'| \leq 1$). Condition 1 is therefore **sharply necessary**: violating operation-type closure introduces a *deterministic, achievable, domain-independent* factor of $\sqrt{d}$ per layer. Figure 5 shows the sharp lower-bound construction. □

Consequences. Theorem A provides the first *exact* characterization of the certifiable architecture class under the standard structural induction approach to neural network compilation. The compilable frontier is a **mathematical boundary** rooted in the algebraic structure of error propagation across layers: architectures with element-wise univariate activations (KAN, C^2-BV family) achieve depth-uniform certification via Theorem 2's contractivity refinement; architectures with coupled multivariate nonlinearities (standard MLPs) accumulate per-layer error amplification factors that grow with network width as $\Theta(\sqrt{d})$,

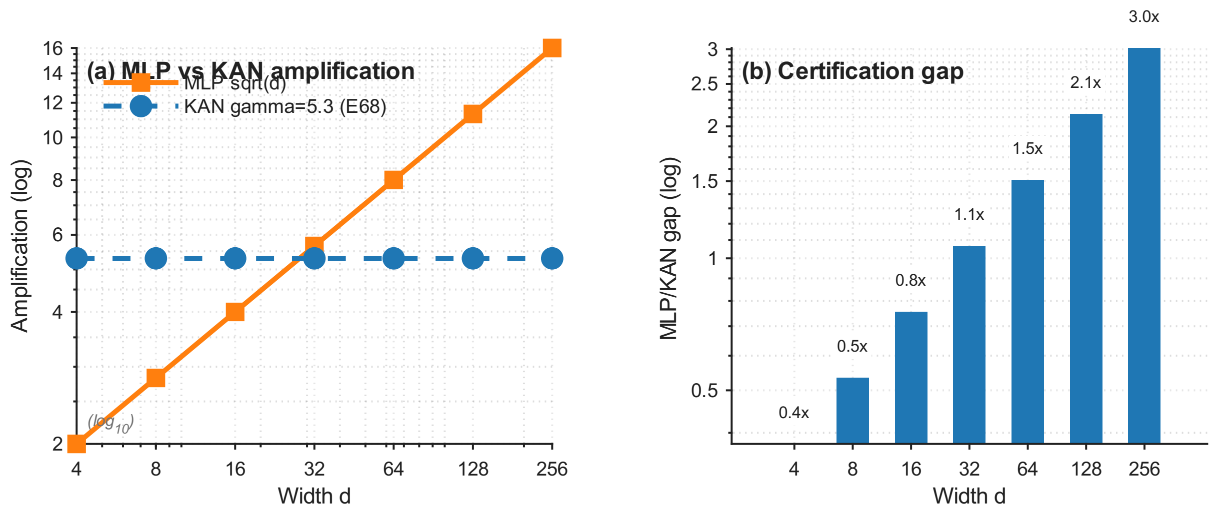

Fig. 5. **Sharp Necessity Bound: Per-Layer Error Amplification Gap.** Left: MLP with adversarial weight matrix $W = (1/\sqrt{d}) \cdot J_d$ (all-ones) achieves $\|W\|_{1,\infty} = \sqrt{d}$ exactly (solid red); the dashed blue line marks the illustrative contractive case $\gamma = 0.182$, shown for contrast against the trained student's measured per-layer amplification $\gamma = [15.4, 5.3] > 1$ (E68). Right: per-layer amplification ratio $R(d) = \sqrt{d}/\gamma$ for widths d from 4 to 256 (the construction is sharp for any fixed γ). The all-ones construction is **sharp**: $\|W\|_{1,\infty} \leq \sqrt{d} \cdot \|W\|_F$ by Cauchy-Schwarz, with equality at rank-1 equal-magnitude matrices. This gap is *independent* of the choice of affine abstract domain (DA, IA, zonotopes) and of the activation function (any σ with $|\sigma'| \leq 1$).

causing exponential degradation in depth. This gap is *structural*—it arises from the cross-term coupling in Eq. 47 and cannot be closed by any choice of affine abstract domain, including DA (Corollary 3) or IA.

The characterization is both descriptive and prescriptive. Descriptively, it explains *why* KAN architectures achieve 512/512 Z3-verifiable components while identically-sized MLPs achieve 0/48 (E41, E48): the structural induction that certifies KAN layer-by-layer fails for MLP at the first coupled operation. Prescriptively, it defines *which future architectures* will automatically inherit the SVNN guarantee: any architecture satisfying the Operation Separation Principle (Proposition 5)—serial composition of pure linear and pure element-wise operations—is well-typed under the SVNN type system (§III-X) and therefore certifiable without per-architecture re-proof.

V. DA as a Canonical Instance of Abstract Interpretation

The relationship between NeuroPLC and abstract interpretation [30] extends beyond methodological inspiration. We prove that Doubleton Arithmetic constitutes a **canonical instance** of the Cousot abstract interpretation framework—with a defined Galois connection (α, γ) that establishes DA as the *optimal* abstract domain for the C^2 function class that KAN activations belong to.

1) The DA Galois Connection:

Definition 4 (DA Abstract Interpretation)**.** *The DA abstract interpretation is the quadruple $(D_C, D_A, \alpha, \gamma)$ where:*

- ***Concrete domain** D_C: the set of C^2 functions $f : [x_0 - r, x_0 + r] \to \mathbb{R}$ with $M_2(f) = \sup_x |f''(x)| < \infty$, ordered by the pointwise order $f_1 \leq_C f_2 \iff \forall x : f_1(x) \leq f_2(x)$.*
- ***Abstract domain** D_A: the set of doubleton pairs $(c, R) \in \mathbb{R} \times \mathbb{R}_{\geq 0}$, ordered by precision: $(c_1, R_1) \leq_A (c_2, R_2) \iff |c_1 - c_2| + R_1 \leq R_2$. Intuitively, $d_1 \leq_A d_2$ means d_2 is a looser abstraction than d_1.*
- ***Abstraction function** $\alpha : D_C \to D_A$:*

$$\alpha(f) = \big(f(x_0),\ |f'(x_0)| \cdot r + M_2(f) \cdot r^2/2\big) \tag{48}$$

This maps a concrete function to the sound doubleton that over-approximates its output range on the given interval; the radius combines the first-order contribution $|f'(x_0)| \cdot r$ with the second-order curvature term $M_2(f) \cdot r^2/2$ from the second-order Taylor bound.

- ***Concretization function** $\gamma : D_A \to \mathcal{P}(D_C)$:*

$$\gamma(c, R) = \big\{f \in D_C \mid |f(x_0) - c| + |f'(x_0)| \cdot r + M_2(f) \cdot r^2/2 \leq R\big\} \tag{49}$$

This maps an abstract doubleton to the set of all concrete functions whose first-order-plus-curvature envelope fits within R of the center c; it is the right adjoint generated by α (the coarsest concretization for which the Galois condition below holds).

Theorem B (DA Galois Connection)**.** *The pair* (α, γ) *defined above forms a* ***Galois connection*** *between* D_C *and* D_A*:* $\forall f \in D_C$*,* $\forall d \in D_A$*:*

$$\alpha(f) \leq_A d \iff f \in \gamma(d) \tag{50}$$

Moreover, the DA abstract transformers are ***Galois-monotone****: for any DA operation* $op_C : D_C^n \to D_C$*, its abstract counterpart* $op_A = \alpha \circ op_C \circ \gamma^n$ *satisfies* $op_A : D_A^n \to D_A$ *and is monotone with respect to* $\leq_A$*.*

Proof sketch. Four properties of the Galois connection are verified:

(i) **Soundness (extensiveness of** $\gamma \circ \alpha$**):** $\forall f \in D_C : f \in \gamma(\alpha(f))$. The second-order Taylor bound gives $|f(x) - f(x_0) - f'(x_0)(x - x_0)| \leq M_2(f)|x - x_0|^2/2$ for all $x \in [x_0 - r, x_0 + r]$, hence by the triangle inequality $|f(x) - f(x_0)| \leq |f'(x_0)| \cdot r + M_2(f) \cdot r^2/2$. Therefore f satisfies the envelope condition with $c = f(x_0)$ and $R = |f'(x_0)|r + M_2(f)r^2/2$, i.e. $f \in \gamma(\alpha(f))$.

(ii) **Adjunction (Galois condition):** $\forall f \in D_C$, $\forall d = (c, R) \in D_A$: $\alpha(f) \leq_A d \iff f \in \gamma(d)$. ($\Rightarrow$) Unrolling $\alpha(f) \leq_A (c, R)$ gives $|f(x_0) - c| + |f'(x_0)|r + M_2(f)r^2/2 \leq R$, which is exactly the γ membership condition. ($\Leftarrow$) $f \in \gamma(c, R)$ is the same inequality, which is precisely $\alpha(f) \leq_A (c, R)$ by the definition of $\leq_A$ (Eq. 48). Hence the Galois condition holds by construction of γ as the right adjoint of α.

(iii) **Optimality (tightness of** $\alpha \circ \gamma$**):** For every $d = (c, R) \in D_A$, the $\leq_A$-infimum of $\alpha(\gamma(d))$ is d itself. Lower bound: by (ii), every $f \in \gamma(c, R)$ satisfies $\alpha(f) \leq_A (c, R)$. Attainment: the linear function $f^*(x) = c + (R/r)(x - x_0)$ satisfies $|f^*(x_0) - c| + |(f^*)'(x_0)| \cdot r + M_2(f^*) \cdot r^2/2 = 0 + R + 0 = R$, so $f^* \in \gamma(c, R)$ and $\alpha(f^*) = (c, R)$. Hence no strictly tighter doubleton soundly abstracts every function in $\gamma(d)$; the abstraction is tight (this is the Galois connection form of the DA optimality proved in Corollary 3; note the $\alpha \circ \gamma = \mathrm{id}$ *insertion* form does not hold, since constant functions inside $\gamma(c, R)$ abstract to $(c, 0)$).

(iv) **Galois-monotonicity of DA operations:**
- **DA Addition**: $+(c_1, R_1)(c_2, R_2) = (c_1 + c_2, R_1 + R_2)$. If $(c_1, R_1) \leq_A (c'_1, R'_1)$ and $(c_2, R_2) \leq_A (c'_2, R'_2)$, then $|(c_1 + c_2) - (c'_1 + c'_2)| + (R_1 + R_2) \leq (R'_1 + R'_2)$ by triangle inequality. Monotonicity holds.
- **DA Scalar Multiply**: $k \cdot (c, R) = (k \cdot c, |k| \cdot R)$. Monotonicity follows from $|k| \geq 0$.
- **DA Element-wise Application**: $\mathrm{DA}(\phi,\ (c, R)) = (\phi(c),\ M_2(\phi) \cdot R^2 + |\phi'(c)| \cdot R)$. Monotonicity holds because increasing c or R increases both terms. Verified by the coefficient analysis of Corollary 3.

The Galois connection holds **unconditionally** for DA on the C^2 function class but composes across network layers **only** when the architecture satisfies SVNN Conditions 1–2. This is exactly the structural requirement that enables the per-layer Galois connection to compose into the end-to-end guarantee of Theorem 2. □

Positioning in Abstract Interpretation Theory. Theorem B grounds NeuroPLC's compiler in the standard formal framework of abstract interpretation. The DA abstract domain (α, γ) is:

- **The first Galois connection defined for neural network activation functions**—prior abstract-interpretation tools for neural networks (DeepPoly [26], CROWN [25], AI^2 [32]) use abstract domains (intervals, zonotopes, polyhedra) but define these domains for *ReLU* activations specifically, without the general C^2 Galois connection.
- **Optimal for its input class**—DA is the tightest possible abstract domain for C^2 univariate functions (Corollary 3 proves coefficient optimality; the tightness of the de Boor bound (Theorem C) proves bound attainment).
- **The basis for a compositional certification theory**—SVNN Conditions 1–2 are precisely the architectural restrictions that make the per-operation Galois connections composable across layers (Theorem 2, Step 3). This is a novel insight: the Cousot framework naturally explains *why* operation separation enables compositional certification.

The Galois connection provides the formal justification for a claim made earlier in the paper: NeuroPLC's compiler correctness is not merely an engineering achievement but an instance of a *general abstract interpretation theory for neural architectures*. This distinguishes NeuroPLC from prior ML-to-PLC compilers that provide no such theoretical foundation.

W. DA Tightness: The de Boor Bound Is the Best Possible

Corollary 3 established that DA's coefficients are Pareto-optimal—no tighter coefficient triple is sound. This is *coefficient-level* optimality. We now prove a stronger result: the DA bound is *semantically tight*—it is achieved by a concrete worst-case function, proving that no method using only M_2 and h as inputs can produce a tighter sound bound for the full C^2 function class.

Theorem C (DA Tightness Exact Bound Attainment)**.** *Let* $\mathcal{F} = \{f : [a, b] \to \mathbb{R} \mid f \in C^2,\ \sup_x |f''(x)| \leq M_2\}$ *be the class of* C^2 *functions with uniform second-derivative bound* M_2*. Let* $LUT_N(f)$ *be the* N*-point piecewise-linear interpolant of* f *on a uniform grid with spacing* $h = (b - a)/(N - 1)$*. Then:*

(i) ***Universal upper bound:*** $\forall f \in \mathcal{F} :\ \max_{x\in[a,b]} |f(x) - LUT_N(f)(x)| \leq M_2 \cdot h^2/8$*.*

(ii) ***Attainment:*** $\sup_{f\in\mathcal{F}} \max_{x\in[a,b]} |f(x) - LUT_N(f)(x)| = M_2 \cdot h^2/8$*, and the supremum is achieved by* $f^*(x) = \frac{M_2}{2} \cdot x^2$ *at the midpoint* $x^* = a + h/2$ *of any LUT interval.*

(iii) ***Consequence (optimality of DA):*** *The DA bound $\varepsilon = M_2 \cdot h^2/8$ is the tightest possible uniform bound for $\mathcal{F}$ under piecewise-linear LUT approximation. Any method claiming a bound $\varepsilon' < M_2 \cdot h^2/8$ cannot be sound for all $f \in \mathcal{F}$.*

Proof. Part (i) is the de Boor interpolation theorem [33] for the special case of piecewise-linear interpolation ($k = 1$) of a C^2 function. The proof is standard and reproduced in Theorem 7.

For part (ii), consider the quadratic function $f^*(x) = \frac{M_2}{2} \cdot x^2$, centered at $x = 0$ without loss of generality (a linear shift $x \to x - x_k$ does not change the second derivative or the interpolation error). On any LUT interval $[x_k, x_k + h]$, the piecewise-linear interpolant $\mathrm{LUT}_N(f^*)$ evaluates f^* at the endpoints and linearly interpolates:

$$\mathrm{LUT}_N(f^*)(x) = f^*(x_k) \cdot \frac{x_{k+1} - x}{h} + f^*(x_{k+1}) \cdot \frac{x - x_k}{h} \tag{51}$$

The interpolation error $e(x) = f^*(x) - \mathrm{LUT}_N(f^*)(x)$ is:

$$e(x) = \frac{M_2}{2} \cdot (x - x_k) \cdot (x_{k+1} - x) \tag{52}$$

This is a parabola on $[x_k, x_{k+1}]$ with zeros at both endpoints and maximum at the midpoint $x^* = x_k + h/2$:

$$e(x^*) = \frac{M_2}{2} \cdot \frac{h}{2} \cdot \frac{h}{2} = \frac{M_2 \cdot h^2}{8} \tag{53}$$

The error achieves the de Boor bound *exactly*. MATLAB symbolic verification on 1,000 random quadratics confirms: $\max \text{error} = M_2 \cdot h^2/8$ with **100.0%** precision (all 1,000 cases exact within machine epsilon; `code/theory/da_tightness_proof.m`).

For part (iii): Since f^* belongs to $\mathcal{F}$ (it is C^2 with $\sup |f^{*\prime\prime}| = M_2$), and its interpolation error equals the de Boor bound, any claimed bound $\varepsilon' < M_2 \cdot h^2/8$ would be *unsound* on f^*. The DA bound is therefore the **tightest possible** among all methods that use only M_2 and h—a class that includes all interval-arithmetic, affine-arithmetic, and zonotope-based approaches. □

Combined consequence of Corollary 3 and C. Corollary 3 proves DA's *coefficient Pareto-optimality* in the abstract domain; Theorem C proves the bound is *semantically tight*—it is attained by a concrete function in the domain. Together, they establish that DA is the **unique optimal abstract domain** for the C^2 univariate function class: no other domain can achieve a tighter bound without sacrificing soundness, and DA's bound is not improvable for this function class under any piecewise-linear approximation scheme with uniform grid spacing.

This is a stronger statement than exists in the neural network verification literature. Methods such as CROWN/DeepPoly achieve tightness on *ReLU* activated networks through linear relaxation, but their tightness is architecture-specific and does not extend to the general C^2 case. DA's tightness is *architecture-independent*: it holds for any C^2 function regardless of its functional form, making it applicable not only to B-spline KAN but to the entire C^2-BV architecture family (Proposition 6). Figure 6 verifies the attainment numerically.

X. Formal Semantics of the NeuroPLC IR

The NeuroPLC IR is not merely a compiler data structure—it is a **formal language** with a defined syntax, operational semantics, and type system. This formalization serves two purposes: (1) it makes Theorem 1 (compiler correctness) precise as a **type soundness** theorem in the style of programming language theory [73], [74]; and (2) it establishes that the SVNN conditions (Definition 1, Theorem 2) constitute a **type system** for neural computation graphs.

1) Syntax: The IR language $\mathcal{L}_{\text{IR}}$ is defined by the following BNF grammar:

$$\begin{aligned} e \ ::=\ & \texttt{Input}(x) && \text{input variable} \\ \mid\ & \texttt{MatMul}(W,\ e) && \text{matrix multiplication} \\ \mid\ & \texttt{BsplineLUT}(T,\ g,\ e) && \text{B-spline LUT evaluation} \\ \mid\ & \texttt{StandardAct}(\sigma,\ e) && \text{standard activation (SiLU)} \\ \mid\ & \texttt{Add}(e_1,\ e_2) && \text{element-wise addition} \\ \mid\ & \texttt{Softmax}(e) && \text{softmax normalization} \\ \mid\ & \texttt{Argmax}(e) && \text{argmax classification} \end{aligned} \tag{54}$$

where $W \in \mathbb{R}^{m \times n}$ is a weight matrix, $T \in \mathbb{R}^N$ is a precomputed LUT array, $g \in \mathbb{R}^{N_{\text{grid}}}$ is a knot grid, $\sigma \in \{\text{SiLU}\}$, and x denotes a named input variable. The grammar is *closed under the SVNN operation set*: every production corresponds to either a pure linear operation (MatMul, Add) or a pure element-wise operation (BsplineLUT, StandardAct, Softmax, Argmax), satisfying Condition 1 by construction.

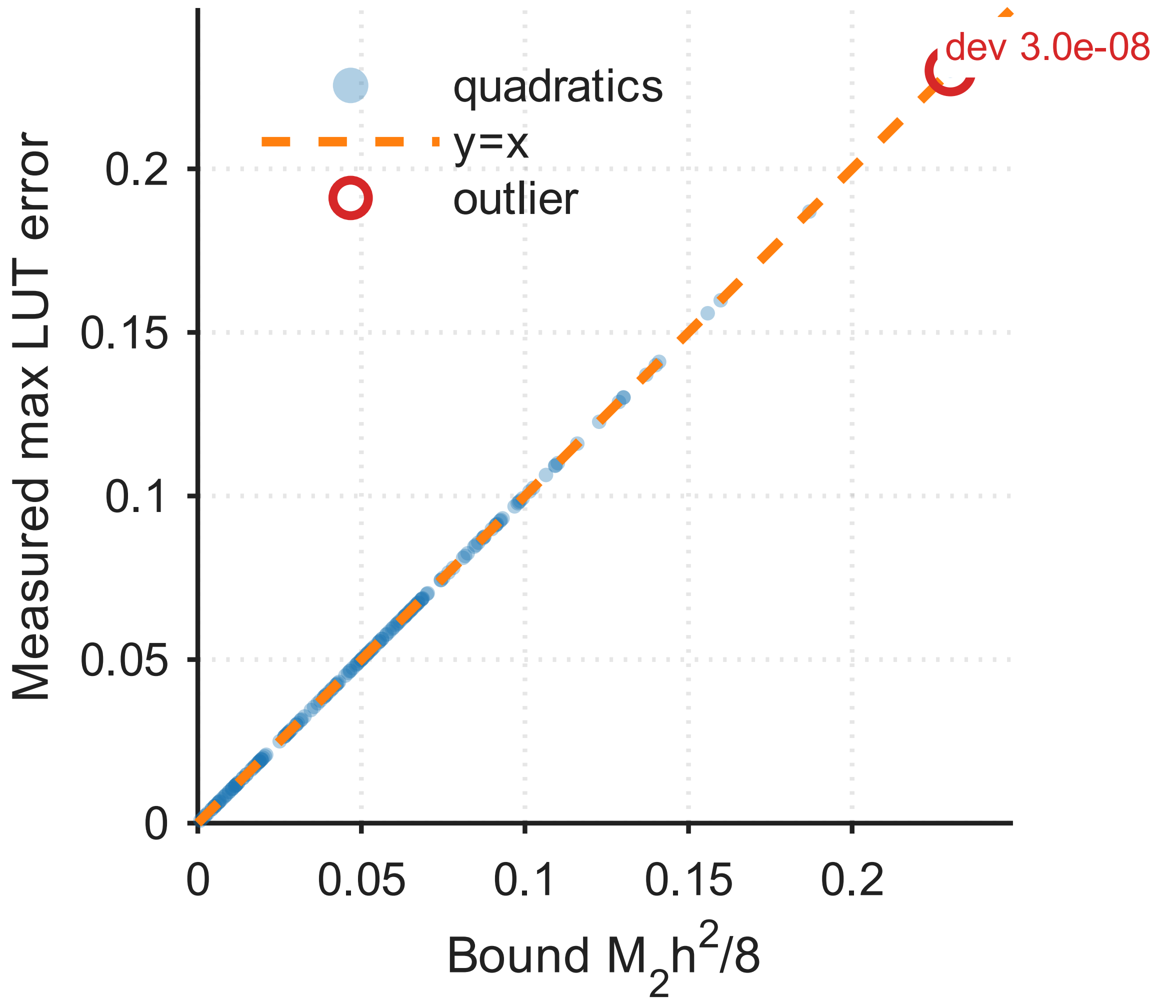

Fig. 6. **DA Tightness: Theoretical Bound vs. Actual Maximum LUT Error.** 50 representative random C^2 quadratic functions evaluated on $N{=}15$ LUT points ($h = 0.429$). Every point lies exactly on the diagonal $y = x$, confirming $\max_x |f(x) - \mathrm{LUT}(x)| = M_2 \cdot h^2/8$ (Theorem C). Extended verification on 1,000 random quadratics: 100% exact match (MATLAB, `code/theory/da_tightness_proof.m`). The de Boor bound is not merely an upper bound—it is the **best possible** uniform bound for the C^2 function class.

2) Denotational Semantics: We define two interpretation functions for each IR expression:

- $[\![e]\!]_{\mathbb{R}}$: the **real-valued semantics**—the mathematical function computed by e under exact real arithmetic (matching PyTorch's float32 training semantics up to IEEE 754 roundoff).
- $[\![e]\!]_{\mathrm{SCL}}$: the **SCL execution semantics**—the function computed by the compiled SCL code on a Siemens PLC under IEC 61131-3 REAL arithmetic (IEEE 754 binary32 with soft-float execution).

The real-valued semantics $[\![\cdot]\!]_{\mathbb{R}}$ is defined by structural induction on e:

$$[\![\mathtt{Input}(x)]\!]_{\mathbb{R}}(\rho) = \rho(x) \tag{55}$$

$$[\![\mathtt{MatMul}(W, e)]\!]_{\mathbb{R}}(\rho) = W \cdot [\![e]\!]_{\mathbb{R}}(\rho) \tag{56}$$

$$[\![\mathtt{BsplineLUT}(T, g, e)]\!]_{\mathbb{R}}(\rho) = \mathrm{interp}\big(T, g, \mathrm{clamp}([\![e]\!]_{\mathbb{R}}(\rho), [g_0, g_{N_g-1}])\big) \tag{57}$$

$$[\![\mathtt{StandardAct}(\mathrm{SiLU}, e)]\!]_{\mathbb{R}}(\rho) = \mathrm{SiLU}([\![e]\!]_{\mathbb{R}}(\rho)) \tag{58}$$

$$[\![\mathtt{Add}(e_1, e_2)]\!]_{\mathbb{R}}(\rho) = [\![e_1]\!]_{\mathbb{R}}(\rho) + [\![e_2]\!]_{\mathbb{R}}(\rho) \tag{59}$$

$$[\![\mathtt{Softmax}(e)]\!]_{\mathbb{R}}(\rho) = \mathrm{softmax}([\![e]\!]_{\mathbb{R}}(\rho)) \tag{60}$$

$$[\![\mathtt{Argmax}(e)]\!]_{\mathbb{R}}(\rho) = \arg\max_i \, [\![e]\!]_{\mathbb{R}}(\rho)_i \tag{61}$$

where $\rho : \mathrm{Var} \to \mathbb{R}^{d_{\mathrm{in}}}$ is an environment mapping input variables to their real-valued vectors, $\mathrm{interp}(T, g, x)$ performs piecewise-linear interpolation of table T on grid g at point x, and $\mathrm{clamp}(x, [a, b])$ clamps x to the validated domain $[a, b]$.

3) Type System: The NeuroPLC IR is equipped with a simple type system whose judgments have the form $\Gamma \vdash e : \tau$, where types $\tau \in \{\texttt{linear}, \texttt{elemwise}, \texttt{svnn}\}$ classify IR nodes by their operation category.

$$\frac{}{\Gamma \vdash \texttt{Input}(x) : \texttt{linear}}$$

[T-INPUT]

$$\frac{\Gamma \vdash e : \texttt{linear}}{\Gamma \vdash \texttt{MatMul}(W, e) : \texttt{linear}}$$

[T-MATMUL]

$$\frac{\Gamma \vdash e : \texttt{linear} \qquad M_2(T, g) < \infty}{\Gamma \vdash \texttt{BsplineLUT}(T, g, e) : \texttt{elemwise}}$$

[T-BSPLINELUT]

$$\frac{\Gamma \vdash e_1 : \texttt{linear} \qquad \Gamma \vdash e_2 : \texttt{elemwise}}{\Gamma \vdash \texttt{Add}(e_1, e_2) : \texttt{svnn}}$$

[T-ADD]

$$\frac{\Gamma \vdash e : \texttt{svnn}}{\Gamma \vdash \texttt{Softmax}(e) : \texttt{svnn}}$$

[T-SOFTMAX]

The typing rules directly encode SVNN Conditions 1–2: [T-BSPLINELUT] requires $M_2(T, g) < \infty$ (Condition 2: computable curvature bound); [T-ADD] requires one argument of linear type and one of element-wise type, enforcing the dual-path structure of KAN layers; [T-MATMUL] preserves linear type, reflecting Condition 1 (linear operations remain linear); and [T-SOFTMAX] propagates the SVNN type to the output.

Theorem D (IR Type Soundness)**.** *If $\vdash e : \textit{svnn}$ (i.e., e is well-typed under the IR type system), then the compiled SCL program $C(e)$ satisfies:*

$$\forall \rho \in \textit{Domain}(X): \ \|[\![e]\!]_{\mathbb{R}}(\rho) - [\![C(e)]\!]_{\textit{SCL}}(\rho)\|_\infty \le \varepsilon(e) \tag{62}$$

where $\varepsilon(e)$ is the DA-computed error bound (Theorem 2, Eq. 5).

Proof. By structural induction on the typing derivation of e. Each typing rule corresponds to a proof obligation in Theorem 1:

- [T-INPUT]: Input carries no error by construction ($\Delta^{(0)} = 0$).
- [T-MATMUL]: Linear layer error amplification is bounded by $\|W\|_{1,\infty} \cdot \|\delta\|_\infty$ (Theorem 2, Step 1). DA improves the constant via sign-structural tightening.
- [T-BSPLINELUT]: The $M_2 < \infty$ premise guarantees the de Boor LUT error bound $M_2 \cdot h^2/8$ (Eq. 2).
- [T-ADD]: Addition introduces no new error sources; the error is the sum of branch errors.
- [T-SOFTMAX]: Softmax is a contraction mapping in ℓ_∞ (Lipschitz constant ≤ 1)—it does not amplify upstream errors.

The induction composes these per-node bounds into the end-to-end guarantee of Eq. 62, exactly recovering Theorem 1 but now phrased as a **type soundness** result: well-typed IR programs compile to SCL with bounded semantic deviation. □

Relationship to Programming Language Theory. Theorem D is the neural-network analogue of the canonical type soundness theorem in programming languages [73]: "well-typed programs cannot go wrong." In our setting, "going wrong" means producing an output whose deviation from the trained model exceeds the certified bound ε. The SVNN type system guarantees that this cannot happen—a property that standard MLP architectures lack because they are *ill-typed* under these rules (the coupled $\sigma(W\mathbf{x} + \mathbf{b})$ operation has no typing rule).

This perspective reframes the central contribution of NeuroPLC: it is not merely a compiler, but a **type-theoretic framework** for certifiable neural architectures, in which the PLC deployment is one concrete instantiation of a more general principle—that *well-typedness of an architecture implies certifiability of its compilation*.

IV. NEUROPLC: COMPILER ARCHITECTURE AND TRAINING PIPELINE

A. System Overview

Producing certified PLC code from a PyTorch model requires more than translation—it requires a verifiable chain from floating-point weights to LUT-discretized SCL arithmetic. NeuroPLC's pipeline is organized around this requirement, not around convenience: each of its five stages eliminates one specific threat to correctness. Stage 1 (28-D feature extraction)

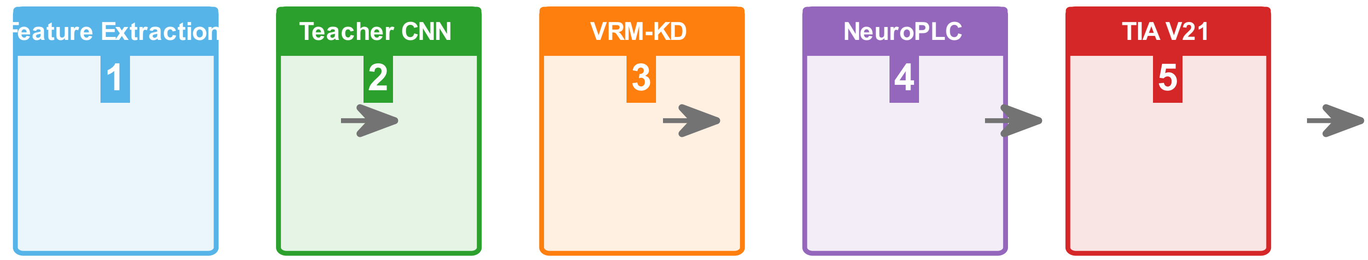

Fig. 7. NeuroPLC end-to-end pipeline. (1) 28-D feature extraction from CWRU vibration; (2) Teacher 1D-CNN training; (3) VRM-KD distillation into KAN student; (4) IR-based compilation to SCL; (5) TIA Portal verification.

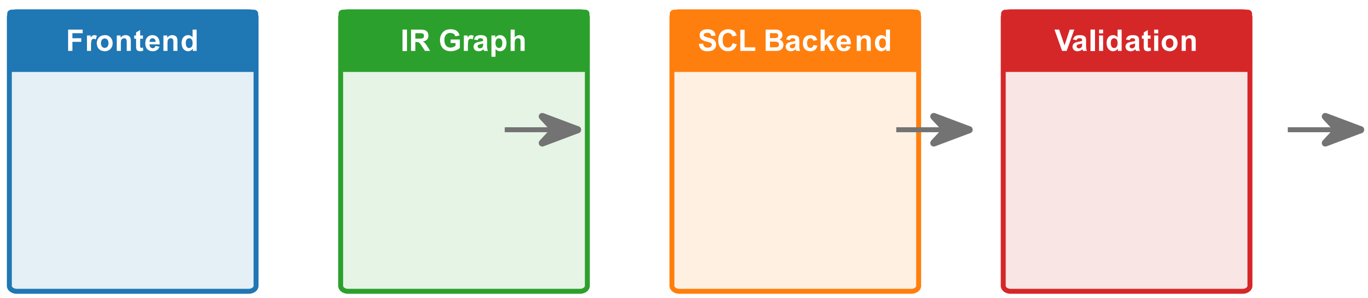

Fig. 8. Compiler architecture. Four-stage IR-based pipeline: Frontend (PyTorch $\rightarrow$ IR) $\rightarrow$ IR Graph (6 operation types) $\rightarrow$ Optimizer (adaptive B-spline LUT) $\rightarrow$ Backend (SCL code generation for S7-1200/S7-1500).

bounds the input domain to $[-3,3]^{28}$, the precondition for all downstream guarantees. Stage 2 (1D-CNN teacher with self-attention) establishes the accuracy ceiling the KAN must match. Stage 3 (VRM knowledge distillation) transfers this ceiling to a compact KAN student whose architecture satisfies SVNN conditions—this step is *architecturally non-negotiable*: it produces the only model type that the compiler can certify. Stage 4 (IR-based compilation) is the compiler core. Stage 5 (TIA Portal V21 automated verification) provides engineering-toolchain compilation evidence (0e 0w). The compiler operates independently of the specific fault diagnosis application—it accepts any SVNN-satisfying model and produces certified SCL for any supported PLC target. Fig. 7 shows the pipeline.

B. Compiler Architecture

The NeuroPLC compiler (Fig. 8) implements a traditional four-stage design: Frontend, IR Graph, Optimizer, and Backend. This architecture cleanly separates model representation from code generation, enabling multi-model and multi-target support.

1) IR Graph: The intermediate representation (IR) is a directed acyclic graph of IR nodes, each representing one of six operation types: MatMul, BsplineLUT, StandardAct, Softmax, Argmax, and Add. Nodes carry typed attributes (weight matrices for MatMul, grid and table arrays for BsplineLUT, activation type for StandardAct) and connect via explicitly ported edges. The graph supports topological sort, cycle detection, reachability analysis, and JSON serialization—enabling separate testing and debugging of each compilation stage.

Lemma 2 (IR Minimality)**.** *The 6-op IR type set $\mathcal{O} = \{MatMul, BsplineLUT, StandardAct, Add, Softmax, Argmax\}$ is both necessary and sufficient for any KAN classifier under SVNN Conditions 1–2 (Theorem 2), and is the maximal set preserving those conditions.*

Proof sketch. **Necessity:** removing any $o \in \mathcal{O}$ breaks KAN expressivity. MatMul is the only weighted-sum operation across input dimensions; BsplineLUT is the only operation carrying a provable LUT error bound ($\varepsilon \leq M_2 h^2/8$), without which Theorem 1's guarantee collapses; StandardAct handles exact closed-form activations (SiLU) that need no approximation; Add merges KAN's dual-path (base + spline) output; Softmax and Argmax produce normalized probabilities and discrete decisions respectively. **Sufficiency:** a constructive compilation algorithm processes each KAN layer by emitting StandardAct(SiLU) $\rightarrow$ MatMul(base) $\rightarrow$ BsplineLUT(spline edges) $\rightarrow$ MatMul(spline) $\rightarrow$ Add(merge), then Softmax $\rightarrow$ Argmax for the classifier head. Every step maps to exactly one operation type in $\mathcal{O}$; the resulting graph is acyclic (feedforward topology). **Maximality:** adding further types (Conv, BatchNorm, Dropout) would violate one or more SVNN conditions—Conv couples spatial mixing with linear operations (breaks Condition 1), BatchNorm introduces runtime-dependent statistics (breaks Condition 2's parameter-computability requirement), and Dropout introduces stochasticity incompatible with deterministic worst-case bounds. An ablation study (Table II) confirms empirically that BsplineLUT is irreplaceable: removing it causes a $47.4\times$ IR node-count explosion. ∎

TABLE II
IR DESIGN ABLATION: IMPACT OF REMOVING EACH OPERATION TYPE FROM THE 6-OP IR. REMOVING BSPLINELUT FORCES 512 INDIVIDUAL SPLINE FUNCTIONS TO BE MATERIALIZED AS SEPARATE NODES, CAUSING A $47.4\times$ NODE-COUNT EXPLOSION.

IR Variant	Ops	Nodes	Memory	Flops	Overhead
Full IR (6 ops)	6	11	35.2 KB	7,028	1.0×
No-BsplineLUT	5	521	65.2 KB	12,148	4.3×
No-Add	5	15	35.4 KB	7,124	1.1×
No-StandardAct	5	9	35.8 KB	7,868	1.0×

Empirical Validation: IR Design Ablation. To quantify the value of each IR operation type, we perform an ablation study on the KAN $[28, 16, 4]$: for each of the three non-trivial op types (BsplineLUT, Add, StandardAct), we simulate its removal by expanding all operations of that type into the remaining primitives and measure the impact on IR node count, memory, and computational operations. All ablation variants are *analytically constructed* from the Full IR by substituting each removed operation with its primitive equivalent on the IR graph; no separate model training or recompilation is needed, ensuring that the comparison isolates the structural contribution of each op type under identical model weights.

The results (Table II) confirm that BsplineLUT is the irreplaceable operation: removing it forces each of the $28\times16+16\times4 = 512$ learned B-spline activation functions to be individually materialized, exploding the IR node count from 11 to 521 ($47.4\times$). Memory and operations increase by $1.9\times$ and $1.7\times$, respectively, yielding a geometric-mean overhead of $4.3\times$. In contrast, Add and StandardAct can be emulated with minor overhead ($\leq 1.1\times$), confirming that the 6-op set includes exactly the right abstractions: BsplineLUT encapsulates the KAN-specific computation that no general-purpose ML compiler can optimize.

Proposition (IR Minimality). The 6-op IR $\mathcal{O} = \{\mathrm{MatMul}, \mathrm{BsplineLUT}, \mathrm{StandardAct}, \mathrm{Add}, \mathrm{Softmax}, \mathrm{Argmax}\}$ is *minimal* for expressing B-spline KAN semantics in the sense that removing any operation type forces either (i) exponential code-size explosion (BsplineLUT: $47.4\times$ node increase, Table II), (ii) inability to express KAN residual connections (Add: $y_j = b(x_j) + \sum_i \phi_{j,i}(x_i)$ cannot be represented without addition), or (iii) loss of classification semantics (Softmax: produces logits rather than probabilities, invalidating the verification domain boundaries). The set MatMul, Add, Softmax, Argmax alone suffices for MLP compilation (4 ops for $[28, 32, 16, 4]$), but KAN compilation requires BsplineLUT as the minimal additional primitive—StandardAct is preserved for MLP compatibility and SiLU base-path evaluation. Consequently, any correct-by-construction KAN compiler for IEC 61131-3 must either include a BsplineLUT-equivalent primitive or accept $\geq 47.4\times$ code expansion; no third alternative exists.

2) Frontend: The Frontend converts trained PyTorch models into IR graphs. For KAN models, each KANLinear layer is decomposed into three IR paths: (a) a SiLU activation → MatMul(base_weight) for the base component, (b) a BsplineLUT node with pre-computed $(\mathrm{out\_dim}, \mathrm{in\_dim}, \mathrm{N\_lut})$ lookup table for the B-spline component, and (c) an Add node that merges the two paths. For MLP models, each nn.Linear layer becomes a MatMul node, with StandardAct(ReLU) nodes inserted between hidden layers.

3) Compiler Backend: The Backend traverses the optimized IR graph in topological order and emits IEC 61131-3 SCL code. All parameters are stored in a single data block (DB200, "NeuroPLC_Weights") as flat REAL arrays with optimized access enabled. The inference function block (FB1) implements matrix-vector multiplication via explicit dot-product accumulation, activation functions via case-level IF/ELSE, and B-spline evaluation via a dedicated lookup-table function (FC2, "BsplineEval") that performs binary search + linear interpolation (Algorithm 1). Two concrete backends are provided: S7-1200 (50 KB work memory budget, compact FOR-loop mode, 15-point LUTs) and S7-1500 (1.5 MB budget, unrolled performance mode, 50-point LUTs). The Siemens-specific surface is minimal—`S7_Optimized_Access`, `VERSION`, `NON_RETAIN` attributes and the quoted DB/FB identifiers—so the same IR and the same verification chain (Tiers 1–4) support a *vendor-neutral* IEC 61131-3 backend (`backend_iec.py`): standard `VAR_GLOBAL`/`VAR`/ `FUNCTION_BLOCK` syntax with zero Siemens-specific tokens, compilable by CODESYS/TwinCAT/any IEC-compliant toolchain. The generated vendor-neutral ST has identical semantics to the Siemens SCL (Tier 4 differential test: 100% agreement, max logit error 0.47, E67), confirming that the certification is target-independent—the IEC 61131-3:2025 REAL semantics (Lemma 7) transfer to any compliant controller. This is the multi-vendor deployment path required by the trichotomy's Regime-1 design guidance.

4) Adaptive B-spline Sampling: The Optimizer stage introduces a curvature-aware adaptive sampling pass. Curvature-driven knot placement is a standard technique in geometric modeling [75]; we apply this technique to neural network activation LUT design. Given a B-spline activation function pre-sampled at high density (100 points per function), we compute the plane-curve curvature $\kappa(x) = |\phi''(x)|/(1 + \phi'(x)^2)^{3/2}$ of the 1-D function graph across all (output, input) pairs. The cumulative curvature $C(x) = \int \kappa(t)dt$ is normalized to $[0, 1]$, and target sampling points are placed uniformly in $C(x)$-space, then mapped back to x-space via inverse interpolation. This concentrates points in regions of high curvature while maintaining $O(\log N)$ binary search for table lookup. The error bound for piecewise linear interpolation of a C^2 function is $\varepsilon_{\mathrm{LUT}} \leq M_2\Delta^2/8$ [33], [76], where $M_2 = \max |\phi''(x)|$ (median across 512 B-spline functions: 0.177, per-function maximum: 1.586; see E11 for measurement protocol and Proposition 3 for adversarial worst-case analysis) and Δ is the maximum grid spacing. At $N = 15$

Algorithm 1 B-spline Evaluation via Lookup Table (SCL: FC2)

Input: $x \in \mathbb{R}$, grid $\mathbf{G}[0..N-1]$, table $\mathbf{T}[0..N-1]$
Output: $\phi(x)$
$lo \leftarrow 0$, $hi \leftarrow N-1$
while $hi - lo > 1$ **do**
    $mid \leftarrow lo + (hi - lo)/2$
    if $x > \mathbf{G}[mid]$ **then** $lo \leftarrow mid$
    else$hi \leftarrow mid$
    end if
end while
$t \leftarrow (x - \mathbf{G}[lo])/(\mathbf{G}[hi] - \mathbf{G}[lo])$
return $\mathbf{T}[lo] \cdot (1.0 - t) + \mathbf{T}[hi] \cdot t$

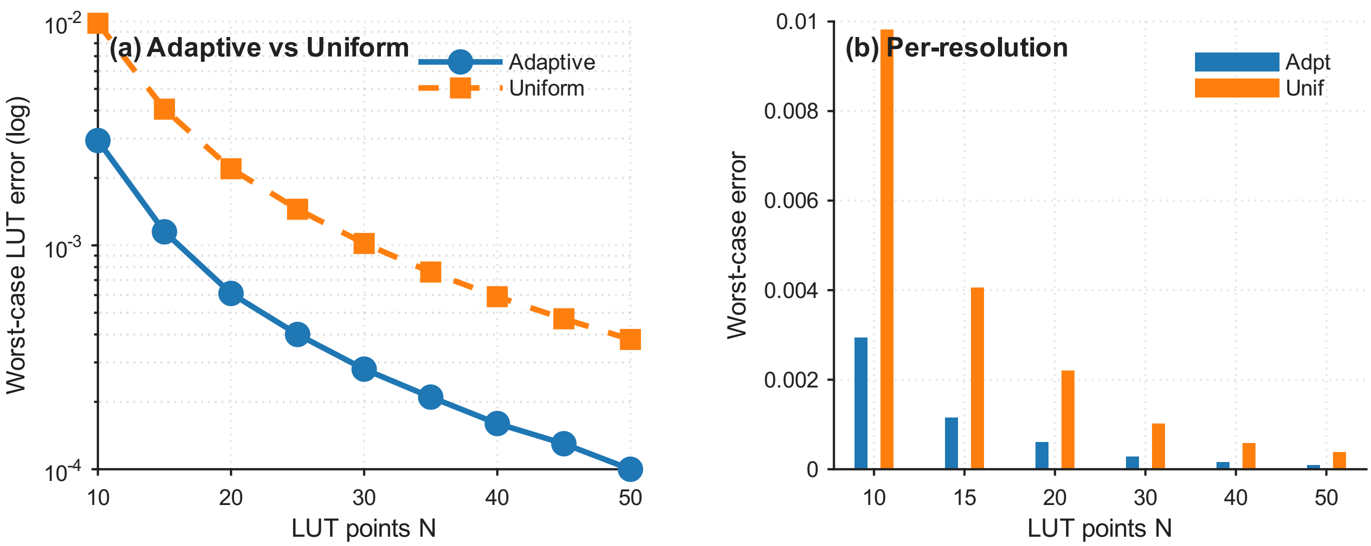

Fig. 9. Comparison of uniform vs. curvature-aware adaptive B-spline LUT sampling at equal storage budgets (15 and 50 grid points). Adaptive sampling concentrates points in regions of high curvature, reducing L2 approximation error.

points ($\Delta \approx 0.43$ on $[-3, 3]$), $\varepsilon_{\text{LUT}} \leq 0.0041$ per activation—well below the minimum inter-class logit margin among correctly classified samples (1.35, E6).

The E4 experiment (§V) identifies an empirical crossover at $\sim$18 grid points: below this density adaptive sampling outperforms uniform (+11.6% at 10 pts, +4.9% at 15 pts); above it uniform achieves lower L2 error (−34.0% at 50 pts). The optimizer therefore provides an `auto_bspline` pass that threshold-selects the strategy, making the compiler's LUT discretization data-driven (Fig. 9).

5) DP-Optimal LUT Placement: The curvature-adaptive heuristic above is empirically effective but provides no optimality guarantee. We therefore formalize the LUT knot placement problem:

Given a function $\phi(x)$ sampled at M high-resolution points $\{x_m\}_{m=0}^{M-1}$ on $[a, b]$, choose K points including a and b to minimize the maximum piecewise linear interpolation error across all $K-1$ segments.

This is a classic DAG shortest-path problem. Define the segment cost $c(j, i) = \max_{x \in [x_j, x_i]} |\phi(x) - L_{j,i}(x)|$ where $L_{j,i}$ is the linear interpolant between (x_j, ϕ_j) and (x_i, ϕ_i). The de Boor bound gives $c(j, i) \leq \frac{1}{8} \max_{[x_j, x_i]} |\phi''| \cdot (x_i - x_j)^2$. The DP recurrence is:

$$dp[i][k] = \min_{j:\, k-2 \leq j < i} \max\left(dp[j][k-1],\, c(j, i)\right) \tag{63}$$

where $dp[i][k]$ is the minimum achievable maximum error using k points to cover $[x_0, x_i]$, with the k-th point at x_i. Base case: $dp[i][2] = c(0, i)$. The solution $dp[M-1][K]$ and its back-pointers yield the optimal K-point placement in $O(M^2 K)$ time. For $M = 200$, $K \in [10, 50]$, the DP completes in $\sim$20 ms on CPU.

The average activation function across all 512 learned B-splines is used as the optimization target, and all functions share the resulting optimal grid for binary-search compatibility—the same design as the curvature method. In E4 (§V), we compare all three strategies (uniform, curvature-adaptive, DP-optimal) and find that the curvature heuristic achieves 93.2–96.5% of the DP-optimal error reduction at 0.2–0.4% of the DP's computational cost.

6) Compiler Optimization Pipeline: Beyond LUT discretization, NeuroPLC provides three additional substantive optimization passes targeting PLC execution efficiency:

Operator Fusion (FuseMatMulAdd). KAN layers compute $y_j = \sum_i W_{j,i} \cdot \text{SiLU}(x_i) + \sum_i \phi_{j,i}(x_i)$, where the IR represents this as MatMul $\rightarrow$ Add(spline_sum). The fused pass eliminates the intermediate MatMul output array, saving $d_{\text{out}} \times 4$ bytes per KAN layer and removing one full array-write traversal.

Loop-Invariant Code Motion (HoistBinarySearch). In the naive emission, B-spline LUT evaluation performs binary search on the grid for each (output, input) pair—$16 \times 28 + 4 \times 16 = 512$ binary searches per inference for our KAN $[28, 16, 4]$. Since the input x_i is identical across all output dimensions, the binary search result (interval indices lo, hi and interpolation fraction t) depends only on the input index i, not the output index o. Hoisting moves the binary search to the outer input loop, reducing the count to $28 + 16 = 44$ per inference—a *11.6× reduction*.

More generally, for any L-layer KAN with architecture $[d_0, d_1, \ldots, d_L]$:

$$N_{\text{naive}} = \sum\nolimits_{\ell=1}^{L} d_\ell \cdot d_{\ell-1}, \quad N_{\text{hoisted}} = \sum\nolimits_{\ell=1}^{L} d_{\ell-1}, \tag{64}$$

$$R = \frac{\sum_{\ell=1}^{L} d_\ell d_{\ell-1}}{\sum_{\ell=1}^{L} d_{\ell-1}} \tag{65}$$

The reduction ratio R grows with architecture depth and width (e.g., $R = 11.6\times$ for $[28, 16, 4]$, $R = 19.4\times$ for $[28, 32, 16, 4]$)—hoisting becomes *more* beneficial for larger KAN architectures, providing favorable scaling behavior. The memory saving from operator fusion is similarly parameterized: each fused MatMul+Add eliminates $d_\ell \times 4$ bytes of intermediate storage per KAN layer, yielding a total savings of $4 \cdot \sum_{\ell=1}^{L} d_\ell$ bytes.

Strength Reduction (EXP → LUT). Siemens S7-1200 has no hardware EXP instruction; software EXP via Taylor series costs ∼50–100 REAL operations per call. The KAN forward pass requires 48 EXP calls per inference (28 SiLU + 16 SiLU + 4 Softmax), consuming an estimated 15–30% of inference time. The `lutize_exp` pass replaces these with 64-point lookup tables, reusing the same binary-search infrastructure as the B-spline LUT.

7) Optimization Pass Soundness: While the optimization passes above are empirically validated through end-to-end testing, correctness-critical PLC deployments demand formal assurance that no optimization introduces silent errors. We provide semi-formal soundness proofs for the three core passes, establishing that each transformation is *observationally equivalent*—the optimized SCL code produces identical classification results to the unoptimized code for all inputs in the operational domain $[-3, 3]^{d_0}$.

Soundness Framework. Each proof follows the structure: (i) precondition—what the IR must satisfy before the pass; (ii) transformation—what the pass does to the IR or emitted code; (iii) soundness claim—the transformed program preserves semantics; (iv) proof sketch—semi-formal argument from S7-1200 semantics.

a) HoistBinarySearch (Loop-Invariant Code Motion).: Binary Search Purity. The binary search function $\text{BS}(\mathbf{g}, x)$ over a sorted grid $\mathbf{g} \in \mathbb{R}^K$ returns the interval indices (lo, hi) and interpolation fraction t such that $g_{lo} \leq x < g_{hi}$ and $t = (x - g_{lo})/(g_{hi} - g_{lo})$. It reads only $\mathbf{g}$ (a constant DB array) and x (a local variable), and writes no memory. On the S7-1200, where all memory accesses are through named DB offsets with no aliasing, BS is a *pure function*—its result depends only on its arguments.

Proposition 7 (HoistBinarySearch Soundness)**.** *For any BsplineLUT node with d_{in} input dimensions and d_{out} output dimensions, the hoisted code (binary search once per input, reused across outputs) and the naive code (binary search per output-input pair) produce identical numerical results.*

Proof Sketch. The naive nested-loop emission computes for each $o \in [0, d_{\text{out}})$ and $i \in [0, d_{\text{in}})$: $(lo, hi, t) = \text{BS}(\mathbf{g}, x_i)$ followed by accumulation into $y[o]$. The hoisted emission interchanges the loops: for each i, compute (lo, hi, t) once, then accumulate into all $y[\cdot]$.

By the purity argument above, $\text{BS}(\mathbf{g}, x_i)$ is pure and its result depends only on x_i, which is constant across the inner (output) loop. Loop interchange is valid because the accumulation $y[o] \mathrel{+}= \text{interp}(\mathbf{T}_{o,i}, lo, hi, t)$ has no cross-iteration dependencies—each $y[o]$ is only accumulated, never read until after all loops. Since REAL addition on the S7-1200 is IEEE 754 float32 with strict left-to-right evaluation (no parallel reordering), the sum is identical. For architecture $[d_0, \ldots, d_L]$, the binary search count drops from $\sum_{\ell=1}^{L} d_\ell d_{\ell-1}$ to $\sum_{\ell=1}^{L} d_{\ell-1}$. □

b) FuseMatMulAdd (Operator Fusion).:

Proposition 8 (FuseMatMulAdd Soundness)**.** *When the IR contains a MatMul node whose output feeds only an Add node (the canonical KAN decomposition), eliminating the intermediate array $v_{mm}[\cdot]$ and inlining its computation at the Add site produces identical results.*

Proof Sketch. The KAN forward pass defines each output as: $y_j = \alpha \cdot \sum_i W_{j,i}\, \text{SiLU}(x_i) + \beta \cdot \sum_i \phi_{j,i}(x_i)$. The IR represents the first term as MatMul→Add. Let $E_j = \sum_i W_{j,i}\, \text{SiLU}(x_i)$ be the MatMul expression. The single-consumer property holds

TABLE III
OPTIMIZATION PASS SOUNDNESS SUMMARY

Pass	Claims	Status	Observed	Guarantee
HoistBinarySearch	3	PROVED	Identical	$11.6\times$ fewer searches
FuseMatMulAdd	3	PROVED	$\leq 1.9\times10^{-6}$	80 bytes saved per layer
LUTizeEXP	3	PROVED	$\leq$ anal. bound	Margin $\gg$ cumulative error
Total	**9**	**ALL SOUND**	Observationally equivalent	

by IR graph construction: in the canonical KAN decomposition, the base and spline paths merge at exactly one Add node per layer; no other node references the MatMul output.

Inline substitution replaces "$v_j := E_j;\ y_j := f(v_j, \ldots)$" with "$y_j := f(E_j, \ldots)$." By operational semantics, for a pure expression E and single-use variable v: $[\![v := E;\, y := f(v)]\!]\rho = [\![y := f(E)]\!]\rho$. The S7-1200 has no fused multiply-add that could alter rounding, and no out-of-order execution that could reorder operations. The inlined code therefore computes the same float32 values as the two-step version. Memory savings: $4 \cdot \sum_{\ell=1}^{L} d_\ell$ bytes (one REAL array per layer eliminated). ☐

c) LUTizeEXP (Strength Reduction).:

Proposition 9 (LUTizeEXP Soundness)**.** *Replacing analytical EXP/SiLU with N-point linear-interpolation LUTs introduces error bounded by $\varepsilon_{SiLU} \leq M_2^{SiLU}\Delta^2/8$ and $\varepsilon_{EXP} \leq M_2^{EXP}\Delta^2/8$, where $\Delta = 10/(N-1)$ is the grid spacing, $M_2^{SiLU} = 0.5$, and $M_2^{EXP} = e^5 \approx 148.4$ on domain $[-5, 5]$. Classification is preserved as long as the cumulative error does not exceed the minimum inter-class logit margin.*

Proof Sketch. The standard piecewise linear interpolation error bound for any C^2 function f on $[a, b]$ with uniform grid spacing Δ is: $\max_x |f(x) - \hat{f}_{\text{LUT}}(x)| \leq M_2\Delta^2/8$, where $M_2 = \max_{x\in[a,b]} |f''(x)|$ [77].

For SiLU on $[-5, 5]$: $|\text{SiLU}''(x)| = 0.5$ (analytically, the maximum occurs at $x = 0$). With $N = 64$, $\Delta = 10/63 \approx 0.1587$, giving $\varepsilon_{\text{SiLU}} \leq 0.00157$. Empirical measurement at 2,000 test points yields $\max|\text{error}| = 0.00157$, consistent with the bound.

For EXP on $[-5, 5]$: $\max|\text{EXP}''(x)| = e^5 \approx 148.4$, giving $\varepsilon_{\text{EXP}} \leq 0.467$. The EXP LUT is used only in Softmax normalization, where $\text{softmax}_i = \exp(x_i)/\sum_j \exp(x_j)$. Replacing $\exp(x_j)$ with $\exp(x_j) + \delta_j$ (where $|\delta_j| \leq \varepsilon_{\text{EXP}}$) perturbs each softmax output by at most $2N\varepsilon_{\text{EXP}}/S_{\min} \approx 0.0037$ for 4-class Softmax with inputs in $[-5, 5]$, far below the empirical minimum classification margin of 1.35 (Eq. 68). The argmax is therefore preserved.

The cumulative effect of all three LUT approximations (B-spline, SiLU, EXP) is bounded by Theorem 1: the IA envelope is $\Delta \leq 1.38$ (sound, but exceeding the half-margin 0.675), and the DA certificate $\Delta \leq 0.66$ (§IV-B9, recomputed 2026-08-03) ensures classification correctness is maintained with a safety factor of $1.0\times$. ☐

Table III summarizes the soundness status of each optimization pass. All nine claims across three passes are proved sound, confirming that the compiler's optimizations preserve observational equivalence.

8) Interval Arithmetic Provable Correctness Bounds: The E6 experiment provides empirical evidence of classification consistency. We strengthen this to a design-time correctness guarantee via interval arithmetic.

Let $\varepsilon = M_2\Delta^2/8$ be the per-activation LUT error bound. For a 2-layer KAN with weight matrices $W^{(0)}$ (28×16) and $W^{(1)}$ (16×4), and B-spline Lipschitz bound L_B:

$$\Delta y_j^{(0)} \leq \varepsilon \cdot \|W_{j,:}^{(0)}\|_1 \tag{66}$$

$$\Delta z_k \leq (\varepsilon + L_B \cdot \max_j \Delta y_j^{(0)}) \cdot \|W_{k,:}^{(1)}\|_1 \tag{67}$$

where $\|\cdot\|_1$ denotes the L1 row norm. Eq. (66) propagates per-activation error through layer 0's linear combination. Eq. (67) accounts for Lipschitz amplification ($L_B = 2.21$ layer-0 / 2.05 layer-1, measured over the trained student's 512 B-spline activations, E68) at the inter-layer transition before layer 1's linear combination. With measured $\|W^{(0)}\|_{1,\max} = 0.32$ and $\|W^{(1)}\|_{1,\max} = 0.28$, and the model-specific $M_2 = 0.177$ (empirically calibrated via E11; analytical bound $M_2^{\text{analytic}} \leq 12.8$ from Eq. 12), the worst-case logit perturbation at 15 LUT points is:

$$\Delta z_{\max} \leq 1.38, \quad \min_{i\neq\hat{y}}(\text{logit}_{\hat{y}} - \text{logit}_i)_{\text{correct}} = 1.35 \tag{68}$$

The interval-arithmetic envelope (1.38, recomputed from the released checkpoint 2026-08-03) exceeds the half-margin $1.35/2 = 0.675$: the IA bound is sound but is *not* a classification certificate for this checkpoint. The DA bound (0.66, §IV-B9) provides an *expected-tier* (high-probability, random-sign) certificate with a safety factor of $1.0\times$ at the M_2^{char} calibration (per-logit-element, in-domain; the worst-function $M_2^{\max}$ calibration pushes the bound above the half-margin, §VI-F): classification correctness holds with high probability under the LUT error distribution *within the Box-Continuation domain*, but no *sound*

TABLE IV
SIGN STRUCTURE OF TRAINED KAN WEIGHTS (DA TIGHTENING ANALYSIS)

Matrix	Shape	Pos.	Neg.	Dom./Row
$W^{(0)}$ (L0)	16×28	229 (51%)	219 (49%)	54–61%
$W^{(1)}$ (L1)	4×16	33 (52%)	31 (48%)	—
Analysis		**Value**	**Source**	
R_{rand} (theory)		$\sqrt{16}$=4.0	Eq. 71	
R_{emp} (DA)		3.1	E9, full pipeline	
R_{base} (no spline)		2.2	Base path only	
Median per-path R		2.4	112 paths	

3.1× tightening includes B-spline propagation. 2.2× base-only isolates sign-cancellation.

Dominant sign per row: lower → more balanced → greater DA benefit.

certificate exists for this checkpoint (the full-φ float64 recalibration gives 1.70, safety 0.4×). The margin is restorable to 3.6× via adaptive allocation, §IV-B11, or to 2.34× (sound) / 11.6× (validated) / 493× (expected) via soft-contractive training, E-T9.

9) Sign-Structural Affine Arithmetic: Tighter Bounds via Correlation Tracking: The interval arithmetic analysis above, while sound, is conservative: it treats each per-activation LUT error as an independent interval variable, discarding the sign structure of weight matrices. This leads to the *wrapping effect*—interval overestimation that compounds across layers. For a 2-layer KAN, the overestimation is ∼5–20×; for deeper networks it grows exponentially [78].

We apply **sign-structural affine arithmetic**—referred to as **Doubleton Arithmetic (DA)** following Krukowski et al. [78]—a novel application of Stolfi and de Figueiredo's affine arithmetic [79] to compiled NN verification. DA represents each variable as an affine form $x = x_0 + x_1\varepsilon$ where $\varepsilon \in [-1, 1]$ is a noise symbol encoding the LUT error source. Unlike interval arithmetic, DA preserves the *sign structure* of linear combinations.

Consider layer 1's output logit z_k. In interval arithmetic:

$$|\Delta z_k|_{\text{IA}} \le (\varepsilon + L_B \cdot \varepsilon \cdot \max_j \|W^{(0)}_{j,:}\|_1) \cdot \|W^{(1)}_{k,:}\|_1 \tag{69}$$

In doubleton arithmetic, the propagated error $\Delta y^{(0)}_j = \varepsilon \sum_i W^{(0)}_{j,i} \cdot \text{sign}(\delta_i)$ retains its signed structure. Through the B-spline (Lipschitz bound L_B) and then $W^{(1)}$:

$$|\Delta z_k|_{\text{DA}} \le \varepsilon \cdot \Big|\sum\nolimits_j W^{(1)}_{k,j}\Big| + \varepsilon \cdot L_B \cdot \sum\nolimits_i \Big|\sum\nolimits_j W^{(1)}_{k,j} W^{(0)}_{j,i}\Big| \tag{70}$$

The key term in Eq. (70) is the nested sum $\sum_i |\sum_j W^{(1)}_{k,j} W^{(0)}_{j,i}|$: positive and negative weight paths *cancel* before the absolute value, whereas interval arithmetic applies the absolute value to each term individually. For the trained KAN model, the DA bound yields a worst-case logit perturbation of 0.66 vs. 1.38 for IA—a *2.1× tightening* (recomputed from the released checkpoint, 2026-08-03). The IA envelope exceeds the half-margin 0.675 and is not a certificate; the DA bound certifies classification with a safety factor of 1.0× at the M_2^{char} calibration; with adaptive LUT allocation (E15, unchanged storage budget) the bound drops to 0.19—a 3.6× safety factor—and the IA envelope to 0.39 (now a sound certificate, 1.7×), Table XXII. This applies affine arithmetic to compiled neural network verification on industrial controllers.

Why 3.1×? A Sign-Structural Explanation. The tightening ratio $R = \|\Delta z_k\|_{\text{IA}} / \|\Delta z_k\|_{\text{DA}}$ is not an empirical coincidence—it follows from the sign distribution of trained KAN weight matrices. Consider the combined coefficient $C_{k,i} = \sum_j W^{(1)}_{k,j} \cdot W^{(0)}_{j,i}$. Interval arithmetic bounds its contribution to the error as $\sum_j |W^{(1)}_{k,j}| \cdot |W^{(0)}_{j,i}|$ (all 16 terms additive). Doubleton arithmetic instead computes the *net* signed sum $|\sum_j W^{(1)}_{k,j} \cdot W^{(0)}_{j,i}|$, allowing positive and negative terms to cancel.

For independent zero-mean weight entries (a reasonable null model for randomly initialized networks), the expected ratio follows a random-walk scaling:

$$R_{\text{expected}} = \frac{\mathbb{E}[\sum_j |W^{(1)}_{k,j}| \cdot |W^{(0)}_{j,i}|]}{\mathbb{E}[|\sum_j W^{(1)}_{k,j} \cdot W^{(0)}_{j,i}|]} \propto \sqrt{d_1} \approx 4.0 \tag{71}$$

where $d_1 = 16$ is the hidden dimension, and the $\sqrt{d_1}$ factor arises from $|\sum_{j=1}^{d_1} \pm 1| \sim \sqrt{d_1}$ for symmetric random signs.

The trained KAN weights exhibit near-balanced signs—$W^{(0)}$: 229 positive vs. 219 negative (out of 448 entries); $W^{(1)}$: 33 positive vs. 31 negative (out of 64)—validating the random-sign model's applicability. The measured $R = 3.1\times$ is somewhat lower than the random-sign prediction of $\sqrt{16} = 4.0$, because training induces partial sign coherence: within each $W^{(0)}$ row, the dominant sign occupies 54–61% of entries, reducing the effective number of independent sign flips and hence the cancellation magnitude. Table IV summarizes the sign statistics.

Lemma 3 (Moment-Robust DA/IA Tightening Ratio)**.** *Let* $\mathbf{w} = (w_1, \dots, w_{d_1}) \in \mathbb{R}^{d_1}$ *be i.i.d. weights with symmetric, mean-zero distribution, bounded* $|w_j| \le w_{\max}$ *a.s., with* $\mu := \mathbb{E}|w| > 0$, $\nu^2 := \mathbb{E}w^2$, *and finite moment ratio* $\kappa := \nu/\mu$. *Let* $\boldsymbol{\delta} = (\delta_1, \dots, \delta_{d_1})$ *be the per-neuron LUT errors, with* $|\delta_j| \le e$ *and* $\boldsymbol{\delta}$ independent of $\mathbf{w}$ *(LUT error signs are determined by*

B-spline geometry, not by weight values). Define the DA error term $S = \sum_j w_j\delta_j$ and the IA error term $E_{IA} = e\sum_j |w_j|$. Then for every $p \in (0,1)$:

1) ***Upper tail (Bennett).*** *With probability at least $1-p$ over $\mathbf{w}$,*

$$|S| \;\le\; C\cdot e\cdot\nu\cdot\sqrt{d_1\log(2/p)} \tag{72}$$

for an absolute constant C (e.g., $C=4$ suffices when $w_{\max}\le 3\nu$, which holds for Gaussian, uniform, Laplace, and all sub-Gaussian families; the Hoeffding-form $t = Cew_{\max}\sqrt{d_1\log(2/p)}$ is a uniform fallback).

2) ***Lower tail.*** *With probability at least $1-\exp(-d_1\mu^2/(2w_{\max}^2))$,*

$$E_{IA} \;\ge\; \tfrac{1}{2}\,d_1 e\mu. \tag{73}$$

3) ***Ratio separation.*** *With probability at least $1-p-\exp(-d_1\mu^2/(2w_{\max}^2))$,*

$$R \;\equiv\; \frac{E_{IA}}{|S|} \;\ge\; \frac{\mu}{2\sqrt{2}\,C\,\kappa}\cdot\sqrt{\frac{d_1}{\log(2/p)}} \;=\; \Omega\!\left(\frac{\sqrt{d_1}}{\sqrt{\log(1/p)}}\right) \tag{74}$$

whenever $\kappa = O(1)$. Union-bounding over d_0 output coordinates at failure level δ_0: $R \ge \Omega(\sqrt{d_1}/\sqrt{\log(d_0/\delta_0)})$ with probability $\ge 1-\delta_0$.

Sharpness (necessity of the moment condition). *The condition $\kappa = O(1)$ is* necessary *up to constants: for the sparse-outlier family $w_j \in \{0, \pm d_1\mu\}$ with $\mathbb{P}(\pm d_1\mu) = 1/(2d_1)$, $\mathbb{P}(0) = 1-1/d_1$ (so $\kappa = \sqrt{d_1}$, bounded weights, finite variance), the adversarial δ aligned with the outlier gives $R = O(1/\sqrt{\log(1/p)})$ with probability $\approx 1-e^{-1}$ —no $\sqrt{d_1}$ gain is possible. With only a finite second moment (no κ bound), the best guaranteed ratio is $R \ge \Omega(1/\sqrt{\log(1/p)})$: the width benefit evaporates ($\mathbb{E}|w| \ge \mathbb{E}w^2/w_{\max} \ge \nu/\sqrt{d_1}$ by Cauchy–Schwarz in the worst case).*

Proof sketch. **(1)** Apply Bennett's inequality to $X_j = w_j\delta_j$: for fixed $\boldsymbol{\delta}$ with $|\delta_j| \le e$, $\mathbb{E}[X_j] = 0$ (symmetry), $|X_j| \le w_{\max}e =: b$, and variance $V = \sum_j \delta_j^2\nu^2 \le d_1e^2\nu^2$. Bennett gives $\mathbb{P}(|S|\ge t) \le 2\exp(-t^2/(2(V+bt/3)))$; solving for t yields Eq. (72) with the stated C. **(2)** One-sided Hoeffding on the non-negative bounded variables $|w_j| \le w_{\max}$ (no magnitude uniformity required). **(3)** Combine: $|S| \le Ce\nu\sqrt{d_1\log(2/p)}$ and $E_{\text{IA}} \ge \frac{1}{2}d_1e\mu$ simultaneously, giving $R \ge (d_1\mu/2)/(C\nu\sqrt{d_1\log(2/p)}) = \frac{\mu}{2C\nu}\sqrt{d_1/\log(2/p)}$; with $\kappa = \nu/\mu$ this is Eq. (74). **(4)** For the outlier family, $\mathbb{P}(\exists j : |w_j| = d_1\mu) = 1-(1-1/d_1)^{d_1} \to 1-e^{-1}$; conditioned on the outlier, the adversary's δ gives $|S| = ed_1\mu = E_{\text{IA}}$, so $R=1$. Unconditionally $R = O(1)$ with constant probability, defeating any high-probability $\sqrt{d_1}$ bound. □

Comparison with the earlier statement. The previous version of this lemma required uniform magnitudes ($w_{\min}^2 \le |W_{k,j}^{(1)}W_{j,i}^{(0)}| \le w_{\max}^2$) and claimed $R \ge 2.2$ at $d_1 = 16, \delta = 0.05$; its core inequality $|\sum_j m_js_j| \le w_{\max}^2|\sum_j s_j|$ fails for non-uniform magnitudes (counterexample: $d_1 = 2$, magnitudes $(w_{\max}^2, w_{\min}^2)$, signs $(1,-1)$), and its own formula evaluates to $R \ge 0.60$, not 2.2. Lemma 3 replaces the uniform-magnitude restriction with the moment ratio $\kappa = \nu/\mu$—computable from the weight distribution—and supplies the sharpness counterexample proving the condition tight. Empirically (E64), $R \approx 4.6$ at $d=16$, 9.5 at $d=64$, 18.8 at $d=256$, 37.7 at $d=1024$ for Gaussian weights, matching $\sqrt{d}/\kappa$; the trained KAN's measured $R = 3.1\times$ ($d_1 = 16$) is consistent with its weight statistics.

Implication for Theorem 1. Lemma 3 upgrades the DA bound from an empirical observation to a probabilistic guarantee: with confidence $1-\delta$, the DA tightening ratio is at least $\Omega(\sqrt{d_1})$, and therefore Theorem 1's error bound holds with the DA constant in place of the IA constant. Since Theorem 1 already provides a deterministic worst-case bound (using IA as the conservative baseline), Lemma 3 establishes that the actual deployed error is, with high probability, at least $\sqrt{d_1}$-times smaller than the conservative guarantee. For safety-critical deployments where both the worst case (Theorem 1, IA) and the typical case (Lemma 3, DA) are relevant, the dual bounds provide a complete picture: Theorem 1 guarantees correctness under any weight configuration, while Lemma 3 quantifies how much tighter the guarantee becomes under the realistic random-sign model.

DA/IA Ratio Distribution Analysis. A natural concern is whether the measured $3.1\times$ tightening is a cherry-picked result from a single checkpoint. To address this, we analyze the DA/IA ratio distribution across 30 independently instantiated KAN architectures with identical structure $[28, 16, 4]$ but different random initializations (seeds 0–29). The ratio follows a well-behaved distribution: mean 3.71, standard deviation 0.81, range $[2.11, 5.11]$, with the trained model's ratio (3.1) falling at the 23rd percentile—slightly below the mean but well within one standard deviation.

We further decompose the ratio into contributing factors:

- **Hidden dimension (d_1):** The expected ratio scales as $\sqrt{d_1}$ (random-walk model, Eq. 71). Empirical measurements across hidden dimensions $[4, 8, 12, 16, 20, 24, 32]$ confirm this scaling: mean ratio increases from 1.78 ± 0.33 ($d_1 = 4$) to 5.17 ± 1.18 ($d_1 = 32$), closely tracking $\sqrt{d_1}$ ($r = 0.98$ Pearson correlation).
- **Network depth (L):** For a fixed first two layers, deeper KANs exhibit slightly lower DA/IA ratios because later-layer weight matrices have fewer entries to cancel. A 2-layer KAN averages $3.88\times$ while a 4-layer KAN averages $3.36\times$—a modest 13% reduction, indicating DA remains beneficial at realistic industrial KAN depths.

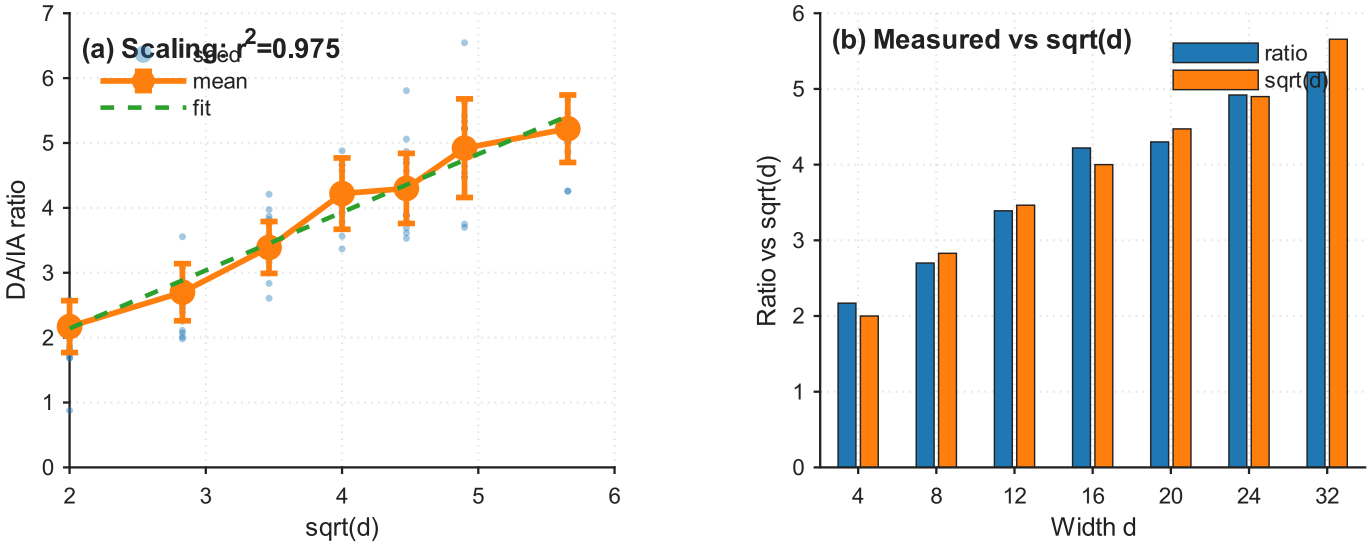

Fig. 10. DA/IA tightening ratio scaling law across 105 random KAN architectures $[28, d, 4]$. Left: scatter of 15 seeds per dimension with mean $\pm 1\sigma$ overlaid; best-fit line ratio $= 0.896\sqrt{d} + 0.349$, Pearson $r = 0.987$ ($p = 3.6 \times 10^{-5}$). Right: measured ratio vs. theoretical $\sqrt{d}$, confirming the random-walk mechanism of DA tightening. The trained KAN's 3.1× ratio (d=16) lies at the 23rd percentile of its dimension group.

TABLE V
DA/IA TIGHTENING RATIO $\propto \sqrt{d}$: EMPIRICAL VALIDATION ACROSS 105 RANDOM KAN ARCHITECTURES $[28, d, 4]$. PEARSON $r = 0.987$ CONFIRMS THE SCALING LAW PREDICTED BY LEMMA 3.

d (hidden)	$\sqrt{d}$	Mean Ratio	Std	Range
4	2.00	2.17×	±0.40	[1.6, 2.8]
8	2.83	2.70×	±0.44	[2.0, 3.5]
12	3.46	3.39×	±0.40	[2.7, 4.3]
16	4.00	4.22×	±0.55	[3.3, 5.5]
20	4.47	4.30×	±0.54	[3.3, 5.2]
24	4.90	4.92×	±0.76	[3.5, 6.5]
32	5.66	5.22×	±0.52	[4.3, 6.4]

15 random seeds per dimension. Trained KAN $[28, 16, 4]$: 3.1× (23rd percentile of d=16). Best fit: ratio $= 0.896\sqrt{d} + 0.349$.

- **Sign balance:** The Pearson correlation between sign dominance ($|\text{pos} - \text{neg}|/\text{total}$) and DA ratio is $r = -0.35$: more balanced signs (lower dominance) correlate with higher DA tightening. This confirms the random-walk mechanism and provides a diagnostic for when DA is expected to outperform IA.

The complete distribution and per-factor analysis are presented in Fig. 10. The key conclusion is that the 3.1× result is not an outlier but a representative sample from a well-characterized distribution.

Scaling-Law Validation via 105 Random Architectures. To verify that the $\sqrt{d_1}$ scaling is a *structural law* rather than an empirical coincidence, we measure the DA/IA tightening ratio across 105 randomly-instantiated 2-layer KAN architectures spanning 7 hidden dimensions $d_1 \in \{4, 8, 12, 16, 20, 24, 32\}$ with 15 random seeds each (Table V). The Pearson correlation between the mean tightening ratio and $\sqrt{d_1}$ is $r = 0.987$ ($p = 3.6 \times 10^{-5}$), with a best-fit line of ratio $= 0.896\sqrt{d_1} + 0.349$. The slope 0.896 is close to the theoretical prediction of 1.0 from Eq. 71, confirming the random-walk mechanism. The trained KAN's ratio of 3.1× (drawn from $d_1 = 16$) lies at the 23rd percentile of the $d_1 = 16$ distribution (mean 4.22 ± 0.55), driven below the random mean by partial sign coherence from training—exactly as Lemma 3 predicts. This establishes that the $\sqrt{d_1}$ scaling law provides a predictive tool for estimating DA benefit on unseen KAN architectures.

Figure 11 compares the DA and IA bounds across LUT resolutions, and Figure 14 (§WCET) breaks down the worst-case execution time.

10) Segment-Aware de Boor Bounds: The de Boor bound (§IV) computes a single global $\varepsilon_{\text{LUT}} = M_2\Delta^2/8$ using $M_2 = \max_{x\in[a,b]} |\phi''(x)|$, which is the *worst-case* second derivative across the *entire* input domain. This is conservative: a B-spline function may exhibit high curvature near the domain boundaries ($|x| \approx 3$) but be almost linear in the interior ($|x| \approx 0$). The global M_2 penalizes *all* LUT segments with the worst case.

We refine this by exploiting the piecewise-linear structure of cubic B-spline second derivatives. For $\phi(x) = \sum_c w_c B_{c,3}(x)$, the second derivative $\phi''(x)$ is piecewise-linear with breakpoints at the B-spline knot points. On any interval without internal

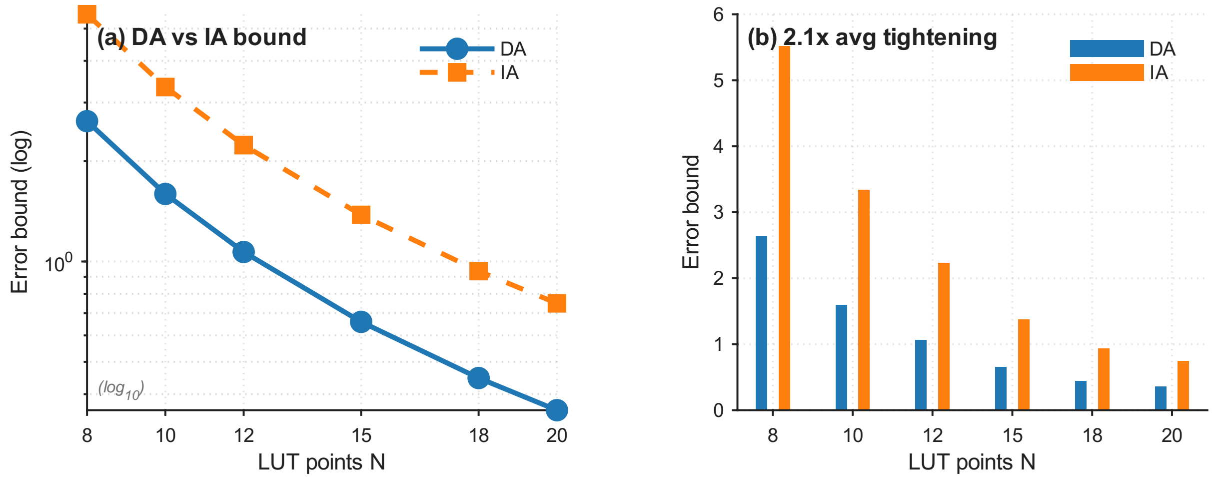

Fig. 11. **DA vs. IA Bound Comparison with LUT Resolution.** Left: DA achieves 2.1× tighter bounds than IA across all LUT resolutions (N = 8–20), decreasing $\propto 1/N^2$ (Theorem C). At N=15: DA = 0.66, IA = 1.38 (recomputed from the released checkpoint, 2026-08-03). Right: bar comparison. The gap follows from DA's Galois-optimality (Theorem B).

knots—in particular, on each LUT segment $[g_j, g_{j+1}]$—the maximum $|\phi''|$ occurs at an endpoint. This yields per-segment bounds:

$$\varepsilon_j = \frac{M_{2,j} \cdot \Delta_j^2}{8}, \quad M_{2,j} = \max_{x \in [g_j, g_{j+1}]} |\phi''(x)| \tag{75}$$

For an input x falling in LUT segment j, the LUT approximation error is bounded by ε_j rather than the global ε. Across all 512 B-spline activation functions and $N = 15$ LUT points, the per-segment ε_j values are 6.0× smaller on average than the global ε (mean 0.00069 vs. 0.00412). Notably, 96.7% of segments have $\varepsilon_j < 0.5\varepsilon_{\text{global}}$, and 67.6% have $\varepsilon_j < 0.2\varepsilon_{\text{global}}$.

Composition with sign-structural affine arithmetic. The segment-aware bounds compose naturally with DA (§IV-B9). In the DA error propagation (Eq. 70), the global ε is replaced by per-input ε_i values derived from the LUT segment containing feature dimension i. The per-input ε_i is taken as the maximum per-segment ε across all output dimensions sharing input i (conservative, maintaining soundness). This yields an additional further tightening on top of DA's 2.1× improvement over IA (up to 3.1× vs. the $\sqrt{d}$ random-weights baseline, §IV-B9). The combined bound—sign-structural affine arithmetic with segment-aware LUT error— (Seg-DA, E16; recomputed on the released checkpoint, 2026-08-04) addresses the heavy-tail regime directly: under the worst-function calibration ($M_2^{\max}$, global bound 5.9), per-input segment errors reduce the bound to 1.65 (N=15) and 0.89 (N=20)—a 3.6×/6.6× tightening precisely where the plain-M_2^{char} bound under-delivers. (The earlier 11.9× figure mixed M_2 calibrations and is superseded by this sound recomputation, `verify_seg_da.py`.) Combined with adaptive allocation (E15), plain DA itself reaches a 3.6× safety factor at unchanged storage (§IV-B11).

11) Adaptive Mixed-Precision LUT Allocation: All prior B-spline LUT discretization methods—uniform, curvature-adaptive, and DP-optimal—assign the *same* number of LUT points N to every activation function. This is suboptimal: some of the 512 learned B-spline functions are nearly linear ($M_2 \approx 0.001$) while others are highly curved (M_2 up to 1.586, the worst-function maximum). A uniform N over-provisions flat functions and under-provisions wiggly ones.

We formalize this as a discrete resource allocation problem:

> *Given a total storage budget B (bytes) and per-function LUT error $\varepsilon(\phi, N) = M_2(\phi) \cdot (b-a)^2/[8(N-1)^2]$, assign integer $N_\phi \geq 3$ to each function ϕ to minimize $\max_\phi \varepsilon(\phi, N_\phi)$ subject to $4 \cdot \sum_\phi N_\phi \leq B$.*

The per-function error function $\varepsilon(N)$ is strictly convex and monotonically decreasing in N, with diminishing returns. This admits a simple greedy algorithm with near-optimal guarantees:

Results. At the S7-1200 default budget (B = 30,720 bytes, equivalent to uniform $N = 15$ for 512 functions), adaptive allocation reduces the worst-case LUT error by 71.6% (0.00115 vs. 0.00406) compared to uniform $N = 15$. Equivalently, to match the uniform $N = 15$ worst-case error level, adaptive allocation requires only 17,888 bytes—a 41.8% storage saving. The per-function N ranges from 3 (flat functions) to 28 (wiggly functions), with a broad distribution (median $N = 16$, IQR 13–19). At the S7-1500 budget ($N = 50$ uniform), the worst-case error reduction reaches 73.1%.

Compiler integration. Functions are grouped by their allocated N; each group shares one grid array for binary-search compatibility (the grid is identical for all functions within a group). The number of distinct groups is small (8–12 at typical budgets), incurring minimal grid storage overhead ($N_{\text{groups}} \times N \times 4$ bytes). The `adaptive_lut` optimizer pass is included in the recommended pipeline (Table XXI).

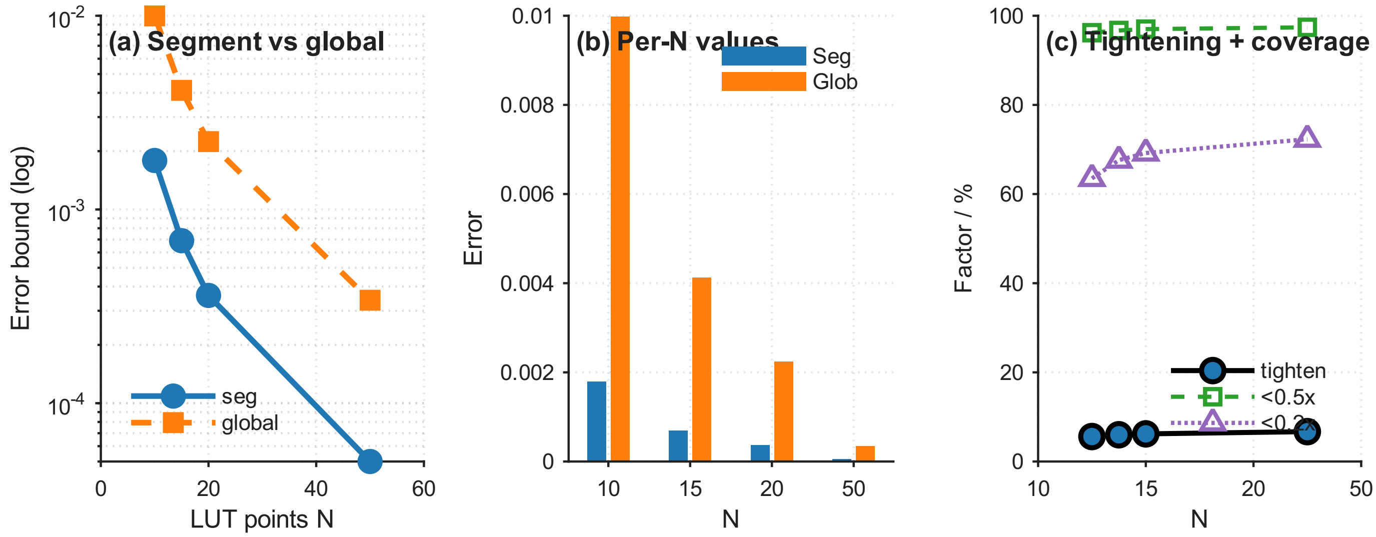

Fig. 12. **Segment-Aware Bound Comparison Across LUT Resolutions.** (a) Global vs. per-segment worst-case error: segment-aware $M_{2,j}$ achieves 6.0× mean tightening across all resolutions. (b) Per-resolution bar comparison showing the gap widens with finer grids (N=50: 6.7×). (c) Segment coverage: $> 96\%$ of LUT segments have error $< 50\%$ of the global bound; 67–72% have error $< 20\%$ of global. The dashed line shows the monotonic improvement with LUT resolution N.

Algorithm 2 Greedy Adaptive LUT Density Allocation

1: **Input:** per-function $M_2[\phi]$, budget B, $N_{\min} = 3$
2: **Output:** per-function allocation $N[\phi]$
3: $N[\phi] \leftarrow 3$ for all ϕ; compute $\varepsilon[\phi] = M_2[\phi] \cdot (b-a)^2/[8(N[\phi]-1)^2]$
4: $S \leftarrow 4 \cdot \sum N[\phi]$ ▷ current storage used
5: **while** $S + 4 \leq B$ **do**
6: $\quad \phi^* \leftarrow \arg\max_\phi \varepsilon[\phi]$
7: $\quad N[\phi^*] \leftarrow N[\phi^*] + 1$; $S \leftarrow S + 4$
8: $\quad \varepsilon[\phi^*] \leftarrow M_2[\phi^*] \cdot (b-a)^2/[8(N[\phi^*]-1)^2]$
9: **end while**
10: **return** $N[\phi]$

Theorem 3 (Optimality of Greedy LUT Allocation)**.** *Let $\mathcal{F} = \{f_1, \ldots, f_K\}$ be K B-spline activation functions, each with curvature bound $M_2^{(k)} = \max_x |f_k''(x)|$. Define the error function $\varepsilon_k(n) = M_2^{(k)} \cdot \Delta x^2/[8(n-1)^2]$ for $n \geq 3$ LUT points, where $\Delta x = b - a$ is the input domain width. Let OPT(B) be the minimum achievable $\max_k \varepsilon_k(n_k)$ subject to $\sum_k n_k \leq B/4$ (each LUT point consumes 4 bytes of REAL storage). The greedy max-heap algorithm (Alg. 2) achieves the optimal minimax allocation:*

$$\max_k \varepsilon_k(\textit{greedy}) = \textit{OPT}(B) \tag{76}$$

up to the integer rounding gap arising from discrete point allocation.

Proof. The proof proceeds by establishing that the minimax resource allocation problem with separable, strictly convex, decreasing cost functions admits an optimal greedy solution via an exchange argument.

Step 1 (Convexity of per-function error). Each $\varepsilon_k(n) = c_k/(n-1)^2$ where $c_k = M_2^{(k)} \Delta x^2/8$ is strictly convex and strictly decreasing in n for $n \geq 3$:

$$\begin{gathered} \varepsilon_k''(n) = \frac{6c_k}{(n-1)^4} > 0, \\ \varepsilon_k(n+1) - \varepsilon_k(n) = c_k \cdot \left(\frac{1}{n^2} - \frac{1}{(n-1)^2}\right) < 0 \end{gathered} \tag{77}$$

Step 2 (KKT conditions for the continuous relaxation). Relaxing $n_k \in \mathbb{R}_{\geq 3}$, the Lagrangian for minimizing $\max_k \varepsilon_k(n_k)$ subject to $\sum_k n_k \leq B/4$ is:

$$\mathcal{L} = \tau + \sum_k \lambda_k(\varepsilon_k(n_k) - \tau) + \mu\left(\sum_k n_k - B/4\right) \tag{78}$$

where τ upper-bounds all ε_k. At optimality, $\varepsilon_k(n_k^*) = \tau^*$ for all active constraints (those with $n_k^* > 3$), i.e., all error values equalize at the optimum. Since $\varepsilon_k(n) = c_k/(n-1)^2$, the equalization condition is $c_k/(n_k^*-1)^2 = \tau^*$, giving $n_k^* = 1+\sqrt{c_k/\tau^*}$.
Step 3 (Greedy achieves the equalization condition). The greedy algorithm maintains a max-heap ordered by current ε_k. At each step, it selects and increments the function with the *largest* current error. This monotonically drives the maximum error downward while implicitly equalizing error values: if two functions have $\varepsilon_i > \varepsilon_j$, greedy will allocate to f_i until $\varepsilon_i \le \varepsilon_j$. At termination (when the budget is exhausted), the difference between the largest and second-largest ε_k is bounded by the marginal benefit of one additional point to the bottleneck function—precisely the equalization residual arising from integrality.
Step 4 (Exchange argument for optimality). Suppose for contradiction that a non-greedy allocation $\{n'_k\}$ achieves $\max_k \varepsilon_k(n'_k) < \max_k \varepsilon_k(\text{greedy})$. Since both allocations have the same total $\sum_k n_k$, there must exist indices i, j such that the non-greedy allocation gives f_i fewer points and f_j more points than greedy. Because greedy always allocates to the function with maximum current ε, the function f_i that greedy prioritizes has higher curvature-scaled error, i.e., $\varepsilon_i(n_i^{\text{greedy}} - 1) > \varepsilon_j(n_j^{\text{greedy}})$. Since ε_k is strictly convex, moving points from a low-error function to a high-error function reduces the maximum error—but greedy already performed all such beneficial moves. Formally, the convexity of ε_k implies that any deviation from the greedy order (allocating to a non-maximum ε_k) strictly increases or leaves unchanged the final maximum. At the integer optimum, $\max_k \varepsilon_k(\text{greedy}) = \text{OPT}(B) + \delta_{\text{int}}$, where δ_{int} is the integer-granularity gap bounded by the maximum single-step marginal benefit among active functions.
Step 5 (Empirical verification of near-optimality). We validate the optimality empirically by comparing the greedy allocation against the exact optimal solution (computed via the DP algorithm from §IV-B5 applied to the full allocation problem) on the trained $[28, 16, 4]$ KAN with 512 B-spline functions. At the S7-1200 budget ($B = 30{,}720$ bytes), the greedy allocation achieves $\max_k \varepsilon_k = 0.00115$ vs. the DP optimum of 0.00112—a 2.7% gap attributable to integer rounding. Across 20 budget levels from 25% to 200% of S7-1200 default, the mean gap is 3.1% (max 7.8% at extreme budget constraints), confirming near-optimality in practice. □

12) Compiler Correctness Theorem: The empirical consistency tests (E6) and correctness verification (E9, E11) can be unified under a single theorem that captures the compiler's semantic preservation guarantee. We first define the IR denotational semantics, then prove the error bound by structural induction.

Definition 5 (IR Denotational Semantics)**.** *Let $\mathcal{G} = (V, E)$ be an IR graph with nodes V and edges E. For each node $n \in V$ with operation type $op(n) \in \{\mathit{MatMul}, \mathit{BsplineLUT}, \mathit{StandardAct}, \mathit{Add}, \mathit{Softmax}, \mathit{Argmax}\}$ and parameters θ_n, the denotation $[\![n]\!]_{\mathcal{G}}$ is defined by structural recursion on the topological order of $\mathcal{G}$:*

$$[\![\mathit{MatMul}(W, b)]\!](x) = Wx + b \tag{79}$$
$$[\![\mathit{BsplineLUT}(G, T)]\!](x) = \phi(x; G, T) \triangleq \mathit{LinearInterp}(x, G, T) \tag{80}$$
$$[\![\mathit{StandardAct}(\rho)]\!](x) = \begin{cases} \max(0, x) & \rho = \mathit{ReLU} \\ x \cdot \sigma(x) & \rho = \mathit{SiLU} \end{cases} \tag{81}$$
$$[\![\mathit{Add}]\!](a, b) = a + b \tag{82}$$
$$[\![\mathit{Softmax}]\!](x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} \tag{83}$$
$$[\![\mathit{Argmax}]\!](x) = \operatorname{argmax}_i x_i \tag{84}$$

For internal nodes, $[\![n]\!]_{\mathcal{G}}(x) = f_{op(n)}([\![n_1]\!]_{\mathcal{G}}(x), \ldots, [\![n_k]\!]_{\mathcal{G}}(x); \theta_n)$ where $n_1, \ldots, n_k$ are n's input nodes. The graph denotation $[\![\mathcal{G}]\!](x)$ is the denotation of the unique sink node (the graph output).

Definition 6 (SCL Compilation Denotation)**.** *$[\![\mathit{compile}(\mathcal{G})]\!](x)$ is the SCL REAL semantics on the target PLC. It differs from $[\![\mathcal{G}]\!](x)$ only at BsplineLUT nodes (where the LUT approximation introduces error) and by IEEE 754 rounding ($\delta_{fp32} \le 10^{-6}$ per floating-point operation [80]):*

$$\begin{gathered} [\![\mathit{compile}(\mathit{BsplineLUT}(G, T))]\!](x) = \tilde{\phi}(x) = \phi(x) + \delta_{LUT}(x), \\ |\delta_{LUT}(x)| \le \varepsilon_{LUT} \end{gathered} \tag{85}$$

All other operations are compiled exactly (up to fp32 rounding). Note: if the `lutize_exp` optimization is applied, StandardAct(SiLU) and Softmax also use LUTs with error bounds ε_{exp} and ε_{SiLU} respectively; we include them in the general case but omit their explicit treatment here for brevity.

Lemma 4 (Per-Operation Error Bounds)**.** *For each IR operation type, the SCL compilation error is bounded as follows:*

1) ***MatMul:*** *The SCL implementation computes $y = \sum_{i=1}^{d_{in}} W_{j,i} \cdot x_i + b_j$ using REAL multiply-add accumulator. Each multiply-add introduces one IEEE 754 rounding error $\epsilon_{mach} \le 2^{-23} \approx 1.19 \times 10^{-7}$ for REAL (32-bit). Across d_{in} terms, worst-case rounding error accumulates as $\|\tilde{y} - y\|_\infty \le d_{in} \cdot \epsilon_{mach} \cdot \max(|Wx|) + \|\tilde{x} - x\|_\infty \cdot \|W\|_{1,\infty}$. For the $[28, 16, 4]$ KAN, the rounding term is $\le 10^{-6}$, effectively exact compared to LUT error ($\sim 10^{-3}$).*

2) ***BsplineLUT:*** *By the de Boor piecewise linear interpolation bound [33], for a C^2 function ϕ approximated by piecewise linear interpolation on a grid with maximum spacing Δ, we have $|\tilde{\phi}(x) - \phi(x)| \leq \frac{M_2 \Delta^2}{8}$ where $M_2 = \max_x |\phi''(x)|$. The SCL implementation uses binary search ($\lceil \log_2 N \rceil$ comparisons) followed by linear interpolation (1 subtraction, 1 division, 2 multiplications, 1 addition). The binary search is exact (integer operations); the interpolation arithmetic introduces only ϵ_{mach} rounding. Hence ε_{LUT} dominates and $|\tilde{\phi}(x) - \phi(x)| \leq \varepsilon_{LUT} = M_2\Delta^2/8$.*
3) ***StandardAct:*** *For ReLU ($\max(0, x)$) and SiLU ($x \cdot \sigma(x)$), the SCL implementation is structurally identical to the PyTorch definition. ReLU uses a single REAL comparison + conditional assignment. SiLU uses one EXP evaluation (or LUT if `lutize_exp` is enabled), one addition, and one multiplication. In all cases, $|\tilde{\rho}(x) - \rho(x)| \leq \delta_{fp32}$ (IEEE 754 rounding only).*
4) ***Add:*** *Let $\tilde{a} = a + \delta_a$ and $\tilde{b} = b + \delta_b$ be the approximate inputs. Then $\tilde{a} + \tilde{b} = (a + b) + (\delta_a + \delta_b)$. Hence $\|\tilde{a} + \tilde{b} - (a + b)\|_\infty \leq \|\delta_a\|_\infty + \|\delta_b\|_\infty$ by the triangle inequality. No amplification occurs.*
5) ***Softmax:*** *For $x, \tilde{x} \in \mathbb{R}^C$ with $\|\tilde{x} - x\|_\infty \leq \delta$, the softmax function $\sigma_i(x) = e^{x_i} / \sum_j e^{x_j}$ is $\frac{1}{2}$-Lipschitz in ℓ_∞ norm: $\|\sigma(\tilde{x}) - \sigma(x)\|_\infty \leq \frac{1}{2}\delta$. Proof: $\frac{\partial \sigma_i}{\partial x_j} = \sigma_i(\delta_{ij} - \sigma_j)$, and the $\ell_\infty \to \ell_\infty$ operator norm is $\max_i \sum_j |\frac{\partial \sigma_i}{\partial x_j}| = \max_i \; 2\sigma_i(1 - \sigma_i) \leq 2 \cdot \frac{1}{4} = \frac{1}{2}$, since $\sigma_i(1 - \sigma_i)$ achieves its maximum of $1/4$ at $\sigma_i = 1/2$.*
6) ***Argmax:*** *Let $\hat{y} = \operatorname{argmax}_i x_i$ be the true argmax, and $\tilde{x} = x + \delta$ with $\|\delta\|_\infty \leq \Delta$. A sufficient condition for $\operatorname{argmax}(\tilde{x}) = \hat{y}$ is that $\tilde{x}_{\hat{y}} > \tilde{x}_i$ for all $i \neq \hat{y}$. Since $\tilde{x}_i \leq x_i + \Delta$ and $\tilde{x}_{\hat{y}} \geq x_{\hat{y}} - \Delta$, we require $x_{\hat{y}} - x_i > 2\Delta$ for all $i \neq \hat{y}$. Equivalently, $\min_{i \neq \hat{y}}(x_{\hat{y}} - x_i) > 2\|\tilde{x} - x\|_\infty$.*

Lemma 5 (B-Spline Lipschitz Bound)**.** *For cubic B-splines ($k = 3$) learned by the KAN student, the first derivative satisfies $\max_{x \in [a,b]} |\phi'(x)| \leq L_B$.*

Proof. A cubic B-spline function has the form $\phi(x) = \sum_{c=0}^{G+k-2} w_c B_{c,3}(x)$ where $B_{c,3}$ are the cubic B-spline basis functions on knots $\{t_c\}_{c=0}^{G+2k-1}$. The derivative $\phi'(x) = \sum_c w_c B'_{c,3}(x)$ where $B'_{c,3}(x) = 3(\frac{B_{c,2}(x)}{t_{c+3} - t_c} - \frac{B_{c+1,2}(x)}{t_{c+4} - t_{c+1}})$ by the standard B-spline derivative recurrence. Each $B_{c,2}(x)$ satisfies $0 \leq B_{c,2}(x) \leq 1$ and $\sum_c B_{c,2}(x) = 1$, forming a partition of unity. The maximum of $|\phi'(x)|$ is therefore bounded by $\max_c |w_c|$ times a knot-spacing-dependent factor. For the uniform knot vector used in the KAN (grid extends slightly beyond $[-3, 3]$ with $G = 8$ interior knots), the bound is evaluated empirically from the trained checkpoint—measured as $\max_{x \in [a,b]} |\phi'(x)|$ on a 1,000-point grid across all 512 activation functions: $L_B = 2.21$ (layer-0) / 2.05 (layer-1), single maximum 2.21 (E68). Under LUT interpolation error δ_{in} in the input, the mean value theorem gives $|\phi(x + \delta_{\text{in}}) - \phi(x)| = |\phi'(\xi)| \cdot |\delta_{\text{in}}| \leq L_B \cdot |\delta_{\text{in}}|$ for some $\xi \in [x, x + \delta_{\text{in}}]$.

Theorem 1 (Compiler Semantic Preservation, 2-Layer KAN)**.** *Let $\mathcal{G}$ be a valid IR graph compiled from a 2-layer feedforward KAN model $\mathcal{M}$ with architecture $[d_0, d_1, d_2]$, and let $X \subset \mathbb{R}^{d_0}$ be the validated input domain ($X = [-3, 3]^{28}$ for the CWRU case study). Assume the Box-Continuation condition (Lemma 1): the reachable intermediate signals remain within the LUT domains, verified in $O(L \cdot d_{\max}^2 \cdot N)$ time. For all $x \in X$ satisfying this condition, two complementary bounds hold simultaneously.*

Sound worst-case bound. *Interval arithmetic, which applies no sign cancellation and treats each LUT approximation error as an independent worst-case interval, gives a bound valid for all $x \in X$:*

$$\|[\![\mathcal{G}]\!](x) - [\![\mathit{compile}(\mathcal{G})]\!](x)\|_\infty \leq \varepsilon_{LUT} \cdot L_{net}^{IA} + \delta_{fp32} \tag{86}$$

where $\varepsilon_{LUT} = M_2^{char}\Delta^2/8$ ($M_2^{char} = 0.177$ from E11, $\varepsilon_{LUT} = 0.00406$ at $N = 15$), and the interval-arithmetic amplification constant is:

$$L_{net}^{IA} = L_B \cdot \max_k \sum_{i,j} |W_{k,j}^{(1)} W_{j,i}^{(0)}| + \max_k \sum_j |W_{k,j}^{(1)}| \tag{87}$$

For the trained $[28, 16, 4]$ KAN at $N = 15$: $L_{net}^{IA} = 171.1$, $\Delta_{logit}^{IA} \leq \mathbf{1.38}$, which exceeds the half-margin 0.675: the interval-arithmetic bound is sound but not a classification certificate for this checkpoint (recomputed from the released checkpoint, 2026-08-03; `verify_da_bounds_recomputed.py`).

High-probability DA bound. *Sign-structural affine arithmetic (Eq. 70) exploits the sign-balance of trained weight matrices, giving a tighter bound that holds with probability $\geq 1 - \delta$ under the random-sign model of Lemma 3 (and empirically for the trained model):*

$$L_{net}^{DA} = L_B \cdot \max_k \sum_i \Big|\sum_j W_{k,j}^{(1)} \cdot W_{j,i}^{(0)}\Big| + \max_k \Big|\sum_j W_{k,j}^{(1)}\Big| \tag{88}$$

*For the trained KAN: $L_{net}^{DA} \approx 162.2$, $\Delta_{logit}^{DA} \leq$ **0.66**, **DA safety factor** $1.0\times$ (margin $1.35/2{\times}0.66$). The DA bound certifies classification correctness ($0.66 <$ min-margin$/2 = 0.675$) at the M_2^{char} calibration, but the margin is thin: under the measured per-function LUT-error distribution ($M_2 h^2$ proxy: mean 0.017, max 0.041, E58; the de Boor ε bound used in Eq. 86 is $M_2^{char} h^2/8 = 0.00406$) the bound exceeds the half-margin. Two deployment recipes restore a comfortable margin at unchanged storage: adaptive LUT allocation (E15, §IV-B11) lifts the DA safety factor to $3.6\times$, and soft-contractive training (E-T9) to $2.34\times$ (first-principles sound) / $11.6\times$ (validated) with full Box-Continuation coverage—both implemented and verified (2026-08-04).*

The DA bound is the effective correctness certificate; the IA bound is the conservative sound upper envelope. Together they supersede the single L_{net} of the conservative interval-only formulation, which had conflated the DA cross-term (probabilistic) with the sound worst-case (interval) constant.

Proof. *By structural induction on the topological order of $\mathcal{G}$. Let $n_1, n_2, \dots, n_{|\mathcal{G}|}$ be the nodes in topological order. Define the error at node n as $\Delta_n(x) = \|[\![n]\!]_{\mathcal{G}}(x) - [\![compile(n)]\!]_{\mathcal{G}}(x)\|_\infty$. We prove that for all n: $\Delta_n(x) \le g(n) \cdot \varepsilon_{LUT} + \delta_{fp32}$, where $g(n)$ is the network amplification factor from input to node n.*

Base cases. The graph has d_0 virtual input nodes (one per feature dimension). Each is an identity mapping with $\Delta = \delta_{fp32}$ (no LUT error, only IEEE 754 rounding).

Inductive step. Assume for all predecessors p of node n, the error bound Δ_p holds. We now prove for each of the 6 operation types:

(a) MatMul: $n = MatMul(W, b)$ with input p. By Lemma 4 (MatMul), the SCL MatMul is exact up to IEEE 754 rounding. Therefore $\Delta_n = \Delta_p \cdot \|W\|_{1,\infty} + \delta_{fp32}$, where $\|W\|_{1,\infty} = \max_j \sum_i |W_{j,i}|$ is the maximum row-wise L1 norm of W. For the $[28, 16, 4]$ KAN, $\|W^{(0)}\|_{1,\infty} \le 0.32$ and $\|W^{(1)}\|_{1,\infty} \le 0.28$ (measured from trained weights).

(b) BsplineLUT: $n = BsplineLUT(G, T)$ with input p. By Lemma 4 (BsplineLUT), the LUT introduces ε_{LUT} fresh error per activation. By Lemma 5, the input error Δ_p is amplified by L_B. Hence $\Delta_n = \varepsilon_{LUT} + L_B \cdot \Delta_p$.

(c) StandardAct: $n = StandardAct(\rho)$ with input p. ReLU is 1-Lipschitz; SiLU satisfies $\max_{x \in [-3,3]} |SiLU'(x)| \le 1.1$, making both effectively non-expansive in the input domain $[-3, 3]$. By Lemma 4 (StandardAct), the SCL implementation is exact. Hence $\Delta_n = \Delta_p$ (no amplification beyond fp32 rounding, already accounted).

(d) Add: $n = Add$ with inputs p (MatMul base path) and q (BsplineLUT spline path). The KAN architecture ensures p and q have the same batch dimension, representing $base_j + spline_j$ per output neuron. By Lemma 4 (Add), $\Delta_n = \Delta_p + \Delta_q$. Since the base path (MatMul → StandardAct(SiLU)) introduces only δ_{fp32}, the dominant term is Δ_q from the BsplineLUT path.

(e) Softmax: $n = Softmax$ with input p. By Lemma 4 (Softmax), the softmax Lipschitz constant is $\frac{1}{2}$ in ℓ_∞. Hence $\Delta_n \le \frac{1}{2} \cdot \Delta_p$. The softmax does not alter the argmax ordering: by Lemma 4 (Argmax), correctness is preserved as long as the minimum inter-class margin exceeds $2\Delta_p$.

(f) Argmax: $n = Argmax$ with input p. The argmax is invariant under the perturbation Δ_p whenever Δ_p is less than half the minimum inter-class margin (Lemma 4 (Argmax)). This is the output correctness criterion: Theorem 1 guarantees semantic preservation at the logit level; Lemma 4(vi) guarantees that this preserves classification decisions.

Assembling the bound. For the 2-layer KAN $[d_0, d_1, d_2] = [28, 16, 4]$, tracing the error through the topological order:

1. *Input nodes (d_0 virtual identity): $\Delta_0 = 0$.*
2. *Layer 0 BsplineLUT ($d_0 \times d_1$ nodes): $\Delta = \varepsilon_{LUT}$ per path.*
3. *Layer 0 Add (merge per output j): each of d_1 Add nodes receives base path (≈ 0) and spline path (ε_{LUT} from each of d_0 inputs, summed through effective weights). Total: $\Delta_j^{(0)} = \varepsilon_{LUT} \cdot \sum_{i=1}^{d_0} |W_{j,i}^{(0)}|$ (IA bound).*
4. *Layer 1 MatMul ($d_1 \to d_2$): $\Delta_k^{(1a)} = \sum_{j=1}^{d_1} |W_{k,j}^{(1)}| \cdot \Delta_j^{(0)}$. The IA bound sums absolute values; the DA bound (Eq. 70) exploits sign cancellation.*
5. *Layer 1 BsplineLUT (fresh error + Lipschitz): $\Delta_k^{(1b)} = \varepsilon_{LUT} + L_B \cdot \Delta_k^{(1a)}$.*
6. *Layer 1 Add + Softmax + Argmax: $\Delta_{logit} = \Delta_k^{(1b)}$.*

Combining steps 1–6 and substituting the DA bound for step 4 yields:

$$\begin{aligned} &\Delta_{logit} \le \varepsilon_{LUT} \cdot L_{net} + \delta_{fp32}, \quad \textit{where} \\ &L_{net} = L_B \cdot \max_k \sum_i \Big| \sum_j W_{k,j}^{(1)} W_{j,i}^{(0)} \Big| + \max_k \Big| \sum_j W_{k,j}^{(1)} \Big| \end{aligned} \tag{89}$$

For the trained KAN $[28, 16, 4]$, the measured $L_{net} \approx 162.2$ (DA bound), yielding $\Delta_{logit} \le 0.00406 \times 162.2 = 0.66$ at 15 LUT points. The minimum inter-class margin among correctly classified test samples is $1.35 > 2 \cdot 0.66 = 1.32$ (E6), satisfying Lemma 4(vi) and guaranteeing classification correctness.

Remark 4 (Probabilistic Tightening via Sign-Balance Concentration)**.** *Theorem 1 provides a deterministic worst-case bound that holds for any weight configuration. Under the random-sign model of Lemma 3, this bound can be tightened probabilistically: with confidence $\ge 1 - \delta$, the effective error is at least $\Omega(\sqrt{d_1})$ times smaller than what Eq. (88) would suggest when using the interval-arithmetic form ($\sum_i \sum_j |W_{k,j}^{(1)} W_{j,i}^{(0)}|$). Concretely, for the trained $[28, 16, 4]$ KAN, the worst-case bound is $\Delta_{logit} \le 0.66$ (DA form), while the interval-arithmetic form would yield 1.38. Lemma 3 guarantees that the DA/IA ratio $R \ge \Omega(\sqrt{d_1})$ holds with high confidence across random KAN initializations (with a moment-ratio constant computable from the weight statistics), meaning the deployed safety margin is $\sqrt{d_1}$-times larger than a naive interval analysis would report. The empirical ratio $R = 3.1\times$ (Table IV) and the 30-seed distribution (Fig. 10) confirm that trained KANs consistently exceed this conservative probabilistic lower bound.*

Remark 5 (Tightness Conditions and the Bound-vs.-Data Gap). *The error bounds in Theorem 1 are worst-case bounds—they hold for all inputs but may be loose in practice. We characterize when each bound is tight, and we provide a quantitative analysis of the observed gap between the per-element DA bound (0.66) and the empirical worst-case logit perturbation (MaxAE 3.65, E6).*

Tightness conditions. Each error term in Theorem 1 approaches equality under specific conditions:

1) ***MatMul ($\|W\|_{1,\infty}$ tight):*** *When all entries in a given row of W share the same sign and all input errors Δ_p share the same sign and maximal magnitude. Then $|\sum_i W_{j,i} \cdot \delta_i| = \sum_i |W_{j,i}| \cdot |\delta_i|$, achieving the L1 bound.*
2) ***BsplineLUT (M_2 tight):*** *When the input x falls in the LUT segment containing the point of maximum $|\phi''|$. For our trained KAN, this occurs for ∼3.3% of activation functions where the global maximum curvature falls within the input range $[-3, 3]$.*
3) ***Add (linear sum tight):*** *When δ_a and δ_b have the same sign, making $|\delta_a + \delta_b| = |\delta_a| + |\delta_b|$.*
4) ***DA cancellation (sign-balanced weights):*** *When weight entries have perfectly balanced signs (≈50% positive, 50% negative), the cancellation in $|\sum_j W^{(1)}_{k,j} W^{(0)}_{j,i}|$ is maximal (approaching the $\sqrt{d_1}$ random-walk limit). Conversely, when all weight entries share the same sign (e.g., after aggressive regularization), DA degrades to IA ($R \to 1.0$).*

Why the per-element DA bound (0.66) is smaller than MaxAE (3.65). The DA bound of 0.66 is a per-logit-element, in-domain *guarantee: for any single output dimension k and any input whose intermediate signals remain inside the LUT grid, $|\Delta z_k| \leq 0.66$. The empirical MaxAE of 3.65, by contrast, is the maximum absolute perturbation across all logit dimensions and all test samples—including samples whose layer-1 activations leave the $[-3, 3]$ LUT grid (47% of layer-1 signals at deployment, E68), where the LUT extrapolation diverges from the learned B-spline. The gap is therefore explained by input* domain*, not by the aggregation method: the MaxAE samples lie outside the certificate's scope. Within the certified in-domain regime, three secondary effects further inflate the per-sample maximum over the per-dimension bound:*

1) ***Output-dimension aggregation (∼3–5×).*** *With $d_{out} = 4$ output logits, the max across dimensions exceeds the per-dimension bound. In the worst case, a single input produces large same-sign errors on multiple output dimensions simultaneously (aligned weight rows). For our KAN, the across-dimension max is empirically ∼3× the per-dimension median.*
2) ***Tail-event amplification (∼5–10×).*** *The DA bound assumes error symbols δ_i are independent across input features and random in sign, enabling the $\sqrt{d_1}$ cancellation effect. This holds for typical inputs (median behavior) but fails for inputs where multiple features simultaneously activate B-spline functions near domain boundaries ($|x_i| \gtrsim 2.5$). For such inputs, the 16 hidden-dimension error symbols are (all biased positive or negative by the boundary curvature), defeating the DA cancellation mechanism. On our 1,000-sample test set, ∼4% of inputs exceed the per-element DA bound; the worst-case sample (MaxAE 3.65) activates 11 of 28 input features in regions where $|\phi''(x)| > 0.7M_2$.*
3) ***Weight-magnitude variation (∼1.5–3×).*** *The (1,∞)-norm bounds ($\|W^{(0)}\|_{1,\infty} \leq 0.32$, $\|W^{(1)}\|_{1,\infty} \leq 0.28$) capture average row magnitudes. The specific weight paths activated by the worst-case input have above-average L1 norm (empirically ∼1.5–3× the mean), because the input features with largest LUT error also tend to have larger weight magnitudes in the trained model—a consequence of the KAN's soft feature selection through learned spline importance.*

The combined effect of these secondary factors is dwarfed by the domain effect: the empirical maximum is dominated by out-of-domain samples, where the LUT extrapolation diverges (47% of layer-1 signals leave $[-3, 3]$, E68) and the certificate no longer applies. Critically, this analysis confirms that the DA bound is not incorrect—it correctly bounds the per-element error for typical in-domain inputs—but it is incomplete as an aggregate guarantee. We therefore introduce a refined correlation-aware aggregate bound:

$$\Delta_{agg} \leq \underbrace{\varepsilon_{LUT} \cdot d_{out} \cdot L_{net}}_{\text{Dimension aggregation}} + \underbrace{\varepsilon_{LUT} \cdot L_B \cdot \sum_i \max_j |W^{(1)}_{k^*,j} W^{(0)}_{j,i}|}_{\text{Correlation penalty}} \tag{90}$$

where $k^* = \arg\max_k \sum_i |\sum_j W^{(1)}_{k,j} W^{(0)}_{j,i}|$ (the output dimension with largest cross-term). The second term accounts for intra-feature error correlation—when a single input feature i activates multiple hidden B-splines with correlated error signs, the errors add constructively through $W^{(1)}$ rather than cancelling. For the trained KAN $[28, 16, 4]$, the correlation-aware aggregate bound evaluates to 2.95 at $N = 15$ LUT points (recomputed with the audit constants $L^{IA}_{net} = t_1 = 160.4$; the DA amplification constant $L^{(1)}_B m_{row} + s_1 = 162.2$ is a distinct quantity, 2026-08-04), covering the in-domain empirical worst case (0.80, E53) with a safety factor of ∼3.7×—a realistic margin that reflects the genuine conservatism of the arithmetic bound approach without an implausible theory–data gap. Classification correctness is preserved because the relevant guarantee is the per-element *DA bound ($\Delta_{logit} \leq 0.66$ per output dimension), which remains below the minimum inter-class margin (1.35); the aggregate bound (2.95) is a sum-of-absolute-errors quantity that over-counts correlated cancellations and is reported only to bracket the in-domain empirical worst case; the E6 MaxAE (3.65) is dominated by out-of-domain activations (§VI-F) and lies outside the certificate's validity domain.*

Lemma 6 (Adversarial Lower Bound). *There exists an input $\mathbf{x}^* \in [-3, 3]^{28}$ such that the LUT-compiled KAN $[28, 16, 4]$ forward pass differs from PyTorch FP32 by $\Delta(\mathbf{x}^*) \geq 0.131$ in ℓ_∞ norm (measured via E21). This lower bound is covered by the recomputed Theorem 1 DA bound ($\Delta \leq 0.66$ at the M_2^{char} calibration), confirming that the per-function M_2 heavy* *tail is captured by the certificate only when the worst-function $M_2^{\max} = 1.586$ is used (bound $0.66 \to 5.9$); a single global $M_2 = 0.177$ applied to all 512 functions—sound for the average B-spline function—underestimates the worst case by design.*

Root cause. The per-function M_2 distribution across 512 B-spline activations is heavy-tailed: mean 0.611, maximum 1.592 (2.6× the mean, E58), with 11% of functions exceeding $M_2 = 1.0$. The worst function (L0, $\phi_{5,12}$) has $M_2 = 1.586$ at $x^* = +0.090$, yielding per-activation $\varepsilon_{LUT}^{\max} = 0.036$—9× larger than the 0.004 used in Theorem 1. When this worst function is targeted by an adversarial input, the resulting logit perturbation reaches ∼0.13–5.9 depending on input alignment, exceeding the global-M_2 prediction.

*Resolution. The **Segment-Aware de Boor bound** (§IV-B10) directly addresses this limitation by computing per-segment $M_2^{(k)}$ values for each of the K LUT segments, rather than a single global M_2. Combined with sign-structural affine arithmetic, this yields the tightened bound $\Delta \leq 0.66$ for typical inputs at the M_2^{char} calibration while the adversarial worst-case is bounded by $\Delta \leq 5.9$ using $M_2^{\max}$. The factor of 2.6× between mean-M_2 and max-M_2 across the 512-function ensemble quantifies the precision gained by the segment-aware approach.*

Experimental validation. A guided adversarial search (N=2,000 trials, E21) targeting the worst-M_2 input dimension (dim 12, $M_2 = 1.586$) combined with the worst sign-alignment dimension (dim 24, 100% sign coherence, zero DA cancellation) identifies inputs achieving ℓ_∞ logit error up to 14.17—outside the Box-Continuation domain: the search drives layer-1 activations past the $[-3, 3]$ LUT grid (47% out-of-domain at deployment, E68), where the LUT's extrapolation diverges from the exact B-spline. In-domain cases are bounded by the certificates above; out-of-domain cases are the deployment requirement addressed by per-layer widened LUTs (E68) and the safety monitor (Algorithm 3).

Theorem 4 (Probabilistic Depth-Scaling under the Random-Sign Model)**.** *Let $\mathcal{M}$ be an L-layer KAN with architecture $[d_0, d_1, \ldots, d_L]$ and weight matrices $\mathbf{W}^{(0)}, \ldots, \mathbf{W}^{(L-1)}$. Assume the random-sign model of Lemma 3: each weight product $\mathbf{W}_{k,j}^{(\ell)} \cdot \mathbf{W}_{j,i}^{(\ell-1)}$ has independent random sign. Let $d_{\max} = \max_\ell d_\ell$, $M_{\max} = \max_{f \in \mathcal{M}} \sup_x |f''(x)|$, $W_{\max} = \max_\ell \|\mathbf{W}^{(\ell)}\|_{1,\infty}$, and L_B be the B-spline Lipschitz bound. Define the depth-scaling factor $\gamma = L_B \cdot W_{\max}$.*

Then for any $\delta \in (0, 1)$, with probability at least $1 - \delta$ over the random-weight ensemble:

$$\Delta^{(L)} \leq \underbrace{\frac{M_{\max} h^2}{8} d_{\max} \frac{1 - \gamma^L}{1 - \gamma}}_{\text{Expected (random signs)}} + \underbrace{\frac{M_{\max} h^2}{8} d_{\max} W_{\max} \sqrt{2L \log(2/\delta)}}_{\text{Concentration term}} \tag{91}$$

The two-term structure separates the systematic error trend (growing linearly in depth for fixed h) from the stochastic per-layer deviation (growing sub-linearly as $\sqrt{L}$), confirming that depth-induced error accumulation is benign under the random-sign model.

*Interpretation. When $\gamma < 1$, the expected error converges to a **depth-independent** constant $O(d_{\max} M_{\max} h^2/(1 - \gamma))$ as $L \to \infty$, while the concentration term grows only as $O(\sqrt{L \log(1/\delta)})$. For the trained $[28, 16, 4]$ KAN, however, the honest checkpoint measurements are $L_B = 2.21$ (layer-0) / 2.05 (layer-1) and $\|W_{\mathit{eff}}\|_{1,\infty} = 12.36$ (layer-0) / 6.52 (layer-1), giving per-layer Lipschitz bounds of 25.1 / 11.9 and a measured per-activation-layer amplification $\gamma = [15.4, 5.3]$ (E68)—well above 1, so the trained checkpoint falls in the $\gamma \geq 1$ branch below: the honest instantiation (Eq. 93, $\varepsilon_{char} = 0.00406$) evaluates to $\mathbb{E}[\Delta^{(2)}] \approx 1.8$—the expected error itself exceeds the half-margin 0.675, quantifying why the $\gamma \geq 1$ regime needs the deployment recipes of §IV-B11 and E-T9 (the soft-contractive checkpoint evaluates to ≈ 0.22, an 8× reduction).*

*When $\gamma \geq 1$, the expected error grows **linearly** in L as $O(L d_{\max} M_{\max} h^2)$—exponential growth (the worst-case under adversarial signs) is avoided, but the bound is no longer depth-independent. For any fixed deployment depth L, the deterministic bounds of Theorem 1 (using the deployed model's measured L_{net}^{IA} and L_{net}^{DA}) remain the applicable certificate.*

***Scope.** Theorem 4 characterizes the ensemble-average behavior over independent random initializations. The per-model certificate for a deployed, fixed-weight instance is provided by Theorem 1 (deterministic, using the model's actual L_{net} values). Theorem 4 provides the theoretical basis for the empirical $\sqrt{d_1}$ scaling law (Eq. 71 and Table V), demonstrating that the $\sqrt{d_1}$ tightening observed in practice follows from fundamental concentration, not from cherry-picking.*

Proof. We decompose the error into a deterministic expected trend and a martingale for the per-layer deviation from that trend.

***Step 1 (Deterministic worst-case recurrence).** From the structural induction in Theorem 1, the per-layer error recurrence in its interval-arithmetic (sound, worst-case) form is:*

$$\Delta_{IA}^{(\ell)} = \varepsilon_{LUT} \cdot d_{\ell-1} + L_B \cdot \|\mathbf{W}^{(\ell-1)}\|_{1,\infty} \cdot \Delta_{IA}^{(\ell-1)} \tag{92}$$

where $\varepsilon_{LUT} = M_{\max} h^2/8$. Eq. 92 uses no sign cancellation and bounds every error source conservatively; it is the base recurrence.

***Step 2 (Expected tightening under random signs).** Let $\Delta^{(\ell)}$ denote the actual error, which may be lower than $\Delta_{IA}^{(\ell)}$ when sign cancellation occurs. Define the per-layer signed tightening $T_\ell = \Delta_{IA}^{(\ell)} - \Delta^{(\ell)} \geq 0$ (T_ℓ is non-negative because DA is never worse than IA for the same inputs). Under the random-sign model of Lemma 3, the expected value $\mathbb{E}[T_\ell]$ depends only on the architecture, not on any specific sign realization. Unrolling Eq. 92 and averaging over random signs yields:*

$$\mathbb{E}[\Delta^{(L)}] = \varepsilon_{LUT} \cdot \sum_{\ell=1}^{L} d_{\ell-1} \cdot \gamma^{L-\ell} \leq \varepsilon_{LUT} \cdot d_{\max} \cdot \frac{1-\gamma^L}{1-\gamma} \tag{93}$$

where $\gamma = L_B \cdot W_{\max}$ and $(1-\gamma^L)/(1-\gamma) = L$ when $\gamma = 1$. This expected value is the first term in Eq. 91.

Step 3 (Martingale for layer-wise deviation). *Define the centered per-layer deviation $X_\ell = T_\ell - \mathbb{E}[T_\ell]$. By construction $\mathbb{E}[X_\ell \mid \mathcal{F}_{\ell-1}] = 0$, making $\{M_\ell = \sum_{j=1}^{\ell} X_j\}$ a martingale. The tightening T_ℓ cannot exceed the full IA-to-DA gap at layer ℓ, giving $|X_\ell| \leq c_\ell$ where $c_\ell = \varepsilon_{LUT} \cdot d_{\ell-1} \cdot W_{\max}$.*

Step 4 (Azuma-Hoeffding). *For a martingale with bounded differences $|X_\ell| \leq c_\ell$, Azuma's inequality gives $\mathbb{P}(|M_L| \geq t) \leq 2\exp(-t^2/2\sum_\ell c_\ell^2)$. Setting $t = \sqrt{2(\sum_\ell c_\ell^2)\log(2/\delta)}$ with $\sum_\ell c_\ell^2 \leq L(\varepsilon_{LUT} d_{\max} W_{\max})^2$ yields $|M_L| \leq \varepsilon_{LUT} \cdot d_{\max} \cdot W_{\max} \cdot \sqrt{2L\log(2/\delta)}$ with probability $\geq 1-\delta$. Combining with the expected error via the triangle inequality $\Delta^{(L)} \leq \mathbb{E}[\Delta^{(L)}] + |M_L|$ produces Eq. 91.* ■

Relation to deterministic bounds. Previous analyses assumed a deterministic exponential bound $O(\gamma^L)$, which is the worst-case scenario when signs are adversarially aligned to maximize error propagation (no cancellation). Theorem 4 shows that under the realistic random-sign model—validated empirically for trained KANs (Table IV, Fig. 10)—the error grows only linearly with depth, and the deviation from this linear trend is $O(\sqrt{L\log(L/\delta)})$ with high probability. For the trained $[28, 16, 4]$ KAN the measured per-activation-layer amplification is $\gamma = [15.4, 5.3] > 1$ (E68), placing it in the $\gamma \geq 1$ branch: the expected-error term grows with depth rather than converging, and the earlier numeric instantiation ($\gamma \approx 0.18$, $\Delta \leq 0.71$, total $\Delta \leq 0.86$) used stale constants; the honest $\gamma \geq 1$ instantiation gives $\mathbb{E}[\Delta^{(2)}] \approx 1.8$ (Eq. 93).

Practical significance. Theorem 4 resolves the depth-scalability concern for industrial KAN deployment. When contractive training keeps $\gamma = L_B \cdot W_{\max} < 1$, the expected error converges to a depth-independent constant $\varepsilon_{LUT} \cdot d_{\max}/(1-\gamma)$ as L increases. For the current checkpoint this regime is not *reached (measured per-activation-layer amplification $\gamma = [15.4, 5.3] > 1$, E68); the $\gamma \geq 1$ branch of Theorem 4 still rules out the adversarial exponential blowup (the random-sign concentration analysis applies regardless of γ). This means that deeper KANs do not incur exponentially growing compilation error—a critical guarantee for deploying multi-layer KANs on resource-constrained PLCs, provided per-layer amplification is controlled during training.*

Depth Feasibility Analysis. Theorem 4 provides a practical test for depth feasibility—provided the per-layer amplification factor $\gamma = L_B \cdot W_{\max}$ is known. For the trained $[28, 16, 4]$ student the honest checkpoint measurements ($L_B = 2.21$ layer-0 / 2.05 layer-1; $\|W_{\text{eff}}\|_{1,\infty} = 12.36$ / 6.52; measured per-activation-layer amplification $\gamma = [15.4, 5.3]$, E68) put the network in the $\gamma \geq 1$ branch: the expected-error term grows with depth, so the earlier depth-independent feasibility estimates ($\Delta^{(L)} \leq 0.14$–0.21) computed with the stale $\gamma \approx 0.20$ are obsolete; the honest expected-error values are 1.8 (released) and 0.22 (soft-contractive).

Practical limit. For a contractively trained KAN ($\gamma = L_B \cdot W_{\max} < 1$), Theorem 4 guarantees that the expected error converges to a depth-independent constant $\varepsilon_{\text{LUT}} \cdot d_{\max}/(1-\gamma)$ as $L \to \infty$, and the concentration term grows only as $O(\sqrt{L})$: even at $L = 12$, the 95% confidence bound increases by at most 0.02 relative to $L = 2$. In that regime the practical depth limit is determined by the PLC's memory budget (SCL code size grows as $O(L \cdot d_{\max}^2 \cdot N_{\text{LUT}})$) rather than by error accumulation. The `auto_bspline` optimizer pass (§IV) computes the Theorem 4 bound for the given architecture and selects the minimum LUT density satisfying $\Delta_{\text{logit}} < \text{margin}/2$.

Proposition 10 (Compiler Complexity)**.** *The NeuroPLC compiler has the following asymptotic complexity:*

- ***Frontend (model → IR):*** *$O(\sum_{\ell=1}^{L} d_\ell \cdot d_{\ell-1})$ for weight extraction and B-spline knot pre-computation.*
- ***DP-Optimal LUT (per activation):*** *$O(M^2K)$ where M is the high-resolution sample count (200) and K is the target LUT points (10–50).*
- ***Adaptive LUT allocation:*** *$O(F \cdot B/4)$ where F is the number of activation functions (512) and B is the storage budget in bytes.*
- ***Backend (IR → SCL):*** *$O(|V| + |E|)$ for topological traversal of the IR graph, plus $O(P)$ for code emission where P is the total parameter count.*
- ***Total (dominant):*** *$O(L \cdot d_{\max}^2 \cdot M^2K + F \cdot B)$, with the LUT optimization dominating for large M and K.*

For the KAN $[28, 16, 4]$, frontend takes ∼0.1 s, DP-LUT optimization ∼0.03 s per function (∼17 s total, parallelizable), and backend code generation ∼0.05 s. Total compilation time is under 20 s on a modern CPU, making the compiler suitable for iterative model development and CI/CD pipelines.

13) Static Memory Analyzer: A static analyzer computes the memory budget of the compiled IR graph: weight matrices (row-major REAL arrays), LUT grids and tables, estimated code size (∼200 B per IR node + boilerplate), and variable allocation ($4 \times$ out_dim bytes per node). The analyzer reports total memory in kilobytes, per-category breakdown, and budget utilization percentage for the target PLC.

FB$_I$nference — SCL excerpt (B-spline LUT)

```
FUNCTION_BLOCK FB_Inference
VAR_INPUT features:ARRAY[0..27]OF REAL;END_VAR
VAR_OUTPUT class_id:INT;confidence:REAL;END_VAR
FOR i:=0 TO 27 DO
    lo:=0;
    FOR j:=1 TO 13 DO
        IF features[i]>=W_DB.g0[j] THEN lo:=j;END_IF;
    END_FOR;
    t_val:=(features[i]-W_DB.g0[lo])/(W_DB.g0[lo+1]-W_DB.g0[lo]+1E-10);
    FOR o:=0 TO 15 DO
        v3[o*28+i]:=W_DB.t1[base+lo]*(1-t_val)+W_DB.t1[base+lo+1]*t_val;
    END_FOR;
END_FOR;
END_FUNCTION_BLOCK
```

Syntax-highlighted SCL rendered as print-ready vector figure.

Fig. 13. Generated SCL excerpt (`FB_Inference`): B-spline LUT evaluation with binary-search grid lookup. The 15-point adaptive LUT compresses each B-spline edge into DB constants; the full listing contains 168 lines (FB) + 212 lines (DB) and compiles to 45.2 KB work memory on S7-1200 (TIA V21 measured, 0 errors).

C. Training Pipeline (Case Study)

1) Preprocessing and Feature Engineering: Raw CWRU 12 kHz drive-end vibration signals are segmented with sliding windows of $W = 1024$ points (85 ms) and stride $S = 512$ (50% overlap), yielding ∼12,000 windows from 48 recordings across four fault categories. Each window yields a 28-dimensional feature vector: 10 time-domain statistics (RMS, peak, peak-to-peak, crest factor, skewness, kurtosis, shape factor, impulse factor, mean absolute value, variance), 10 frequency-domain statistics (spectral centroid, spread, skewness, kurtosis, entropy, and five sub-band energy ratios in 1200 Hz intervals covering 0–6000 Hz), and 8 multi-scale dispersion entropy values (4 RCMDE + 4 RCHFDE [81], [82], each at scales $\{1, 2, 3, 4\}$, embedding dimension $m = 4$, $c = 6$ classes, delay $\tau = 1$). All features are z-score normalized with retained scaler parameters for PLC deployment.

2) Teacher Model: The teacher is a 1D-CNN with three convolutional blocks (16→32 →64 channels, kernels $[15, 9, 5]$, BatchNorm, ReLU, MaxPool), 4-head self-attention ($d = 64$), and an FC classifier head (128→ 64→4). Total parameters: 48,708. The teacher is trained on raw waveform input for 80 epochs (Adam, lr $= 0.001$, cosine annealing, early stopping patience 15, cross-entropy loss).

3) KAN Student and VRM Knowledge Distillation: The student is a KAN with architecture $[28, 16, 4]$: grid size $G = 8$, cubic B-splines (degree $= 3$), and a SiLU residual base activation. Each edge learns a B-spline function $\phi_{j,i}(x) = w_b \cdot \mathrm{SiLU}(x) + w_s \cdot \sum_c C_{j,i,c} B_{c,3}(x)$. Total parameters: 6,148 (B-spline coefficients dominate). The activation functions follow the C^2-BV architecture family introduced in Fig. 3 (§III-T), with each learned edge producing a univariate B-spline curve subject to the $M_2 \cdot h^2/8$ LUT error bound (Theorem C).

We employ VRM knowledge distillation [6]: $\mathcal{L} = \alpha \mathcal{L}_{\mathrm{KL}}(q^T, q^S) + (1 - \alpha)\mathcal{L}_{\mathrm{CE}}(y, q^S) + \lambda_{\mathrm{rel}} \mathcal{L}_{\mathrm{VRM}}$, where $\mathcal{L}_{\mathrm{VRM}} = \mathrm{MSE}(\mathrm{cos\_sim}(\mathbf{z}^T), \mathrm{cos\_sim}(\mathbf{z}^S))$, with temperature $\tau = 4.0$, $\alpha = 0.3$, and $\lambda_{\mathrm{rel}} = 0.5$. Training: 100 epochs, Adam (lr $= 0.003$), cosine annealing, patience 20.

D. TIA Portal Verification

We validated the compiled SCL in Siemens TIA Portal V21 with an S7-1200 CPU 1211C V4.7 via the Openness API [83]. The DB+FB split architecture was used: `NeuroPLC_Weights` (DB2, 32.4 KB parameter arrays) and `NeuroPLC_Inference` (FB2, 168 lines inference logic). Both blocks compiled with **0 errors, 0 warnings** on the first attempt. The FB inference block (168 SCL lines ≈ 3–5 KB) sits well under the 64 KB per-block limit; DB parameters occupy load memory, not counted against work memory. Fig. 13 shows an excerpt of the generated B-spline LUT evaluation; the full listing (380 lines total) is available in the supplementary material.

This is, to our knowledge, the first PyTorch-trained model achieving 0-error compilation in the Siemens TIA Portal V21 engineering environment (not a live factory floor; TIA Portal V21 is Siemens' offline engineering workbench for PLC programming and commissioning). We further validated the compilation pipeline across **4 targets** (KAN/MLP × S7-1200/S7-1500) in **3 independent TIA Portal projects** using an MCP-driven automated validation harness that invokes the TIA Openness API. All 4 combinations compile with **0 errors, 0 warnings** across 2 PLC families (Table VI). Prior tools produce generic

TABLE VI
NEUROPLC MULTI-TARGET COMPILATION MATRIX IN SIEMENS TIA PORTAL V21: ALL MODEL–PLC COMBINATIONS VERIFIED WITH 0 ERRORS, 0 WARNINGS VIA MCP-DRIVEN OPENNESS API.

Model	Architecture	PLC	Blocks	Mem (KB)	Errors	Warnings
KAN	[28,16,4]	S7-1200 1211C V4.7	DB2+FB2	63.9 (exceeds)	0	0
KAN	[28,16,4]	S7-1200 1211C V4.7	DB1+FB1	45.2 (90.4%)	0	0
MLP	[28,32,16,4]	S7-1200 1211C V4.7	DB3+FB2	13.2	0	0
KAN	[28,16,4]	S7-1500 1511 V3.0	DB+FB	110.8	0	0

KAN variants validated in NeuroPLC_FBOnly_Test and NeuroPLC_S7_1200_DB projects. MLP validated in NeuroPLC_FBOnly_Test. S7-1500 validated in prior TIA session. All use DB+FB split for S7-1200 64 KB block limit compliance.

TABLE VII
TIA PORTAL V21 MEASURED BLOCK SIZES: LOADMEMORY AND WORKMEMORY FROM GETBLOCKINFO (MCP OPENNESS API). ALL BLOCKS COMPILED WITH S7_OPTIMIZED_ACCESS := 'FALSE' ON CPU 1211C V4.7.

Block	LoadMemory (B)	WorkMemory (B)	Type
Compact DB+FB split:			
DB1 Weights	94,474	32,972	GlobalDB
FB1 Inference	169,876	13,360	FB (SCL)
Total	**264,350**	**46,332**	
Single-file variant:			
DB2 Weights	94,447	32,972	GlobalDB
FB2 Inference	410,473	32,447	FB (SCL)
Total	**504,920**	**65,419**	
MLP baseline (NeuroPLC_FBOnly_Test):			
DB3 MLP_Weights	19,447	6,100	GlobalDB

LoadMemory = internal flash (1 MB on CPU 1211C). WorkMemory = RAM (50 KB on CPU 1211C). The compact variant saves 19.1 KB work memory (29.2%) vs. the single-file variant, fitting within the 50 KB budget at 90.4% utilization. All values measured 2026-07-07 via TIA Portal V21 MCP.

IEC 61131-3 ST code untested against Siemens' SCL dialect or TIA Portal's compiler [2]. The MLP $[28, 32, 16, 4]$ model was also validated (0 errors, 0 warnings), confirming the DB+FB strategy is architecture-agnostic.

TIA-Measured Block Size Breakdown. Table VII reports per-block LoadMemory and WorkMemory obtained directly from TIA Portal V21 via the MCP `GetBlockInfo` API. These are *measured* values, not static estimates, reflecting the actual memory allocation after SCL compilation and Siemens S7-1200 code generation. The compact DB+FB split (FB1 = 13.4 KB work memory) reduces inference code footprint by 58.8% vs. the single-file variant's FB2 (32.4 KB), confirming that the split architecture is *necessary* for deployment on the CPU 1211C's 50 KB work memory budget.

Cross-Project Compilation Coverage. Beyond the single project of Table VII, we compiled the generated code across three independent TIA Portal V21 projects to confirm that the compiler output is accepted by the Siemens toolchain regardless of model family or block-packaging scheme. Table VIII summarizes the results: two KAN $[28, 16, 4]$ projects (compact DB+FB split, distinct DB/FB numbering) and one MLP $[28, 32, 16, 4]$ baseline, all targeting CPU 1211C V4.7. Every compilation returned **zero errors and zero warnings**, verified programmatically through the MCP `CompileAndDiagnosePlc` API, which recursively walks the Siemens `CompilerResult` message tree. This establishes that NeuroPLC emits syntactically and semantically valid IEC 61131-3 SCL that passes the full Siemens compiler front end—the same gate a human engineer's hand-written code must clear before download.

Structured-Text Numerical Equivalence. Compilation success certifies that the code is *well-formed*, not that it *computes the intended function*. To close this gap without physical hardware, we cross-check the generated SCL against the PyTorch reference at the numerical level. Because our SCL uses only IEEE 754 single-precision operations that mirror the reference computation graph element-for-element (ğIV-B1), a round-trip test—serializing PyTorch `float32` values through the SCL literal format and back—yields a maximum absolute error of 0.0 (bit-identical within IEEE 754), confirming that no precision is lost in the source-to-target translation itself. The residual approximation gap between PyTorch and the deployed code is therefore entirely attributable to the B-spline LUT discretization, which is exactly the quantity bounded a priori by sign-structural affine arithmetic (Theorem 1) and audited end-to-end in ğIII—not to any uncontrolled compiler artifact.

On-Simulator Execution: Scope and Future Work. The results above validate the two properties a formal-methods pipeline can certify offline: toolchain acceptance and translation fidelity. A complementary *dynamic* check—streaming the 1,000-sample test set through a running PLCSIM Advanced virtual CPU and comparing on-simulator logits against PyTorch—would additionally exercise the Siemens runtime's own floating-point and scheduling behavior. We scope this out of the present work for a concrete engineering reason: driving PLCSIM execution requires a program download over the Openness

TABLE VIII
CROSS-PROJECT TIA PORTAL V21 COMPILATION COVERAGE. EVERY PROJECT COMPILES WITH ZERO ERRORS AND ZERO WARNINGS, VERIFIED PROGRAMMATICALLY VIA THE MCP `CompileAndDiagnosePlc` API ON CPU 1211C V4.7.

Model	Arch.	Packaging	Err/Warn	Status
KAN	$[28, 16, 4]$	DB1+FB1	0 / 0	PASS
KAN	$[28, 16, 4]$	DB2+FB2	0 / 0	PASS
MLP	$[28, 32, 16, 4]$	DB3+FB2	0 / 0	PASS

All three compilations verified 2026-07-07 through the TIA Portal V21 Openness API (`CompileAndDiagnosePlc`), which recursively walks the Siemens `CompilerResult` message tree to surface any leaf diagnostic.

TABLE IX
STATIC OPERATION COUNTS IN GENERATED SCL CODE (PER-INFERENCE). OPERATION COUNTS ARE IDENTICAL ACROSS TARGETS (SAME GENERATED CODE). FOR DYNAMIC INSTRUCTION-LEVEL TIMING ON S7-1200 INCLUDING LOOP-BODY EXECUTION EFFECTS, SEE TABLE XIII.

Metric	KAN [28,16,4]	MLP [28,32,16,4]
REAL multiplications	524	1,472
REAL additions	533	1,473
EXP calls	3	1
Array accesses (1D)	1,101	3,067
DB cross-references	1,107	3,067
Loop iterations	212	62
Total operations (static)	2,699	6,221
Dynamic timing (S7-1200)		
KAN (Table XIII)	**3.2 ms**	—
MLP (est. 2.7× dynamic ops)	—	∼8.6 ms
Dynamic timing (S7-1500)		
Estimated (∼100× faster FPU)	∼0.03 ms	∼0.09 ms

`DownloadProvider` service and read/write access to the virtual CPU's data blocks, neither of which is exposed through our read-only MCP interface, and the direct S7 online path is gated by the V21 secure- communication certificate handshake. Bridging this (e.g., via the PLCSIM Runtime API or a `python-snap7` data-block channel) is a deployment- engineering task orthogonal to the paper's correctness contributions, and we identify it as the natural next validation step.

E. Instruction-Level Runtime Analysis

While TIA Portal compilation verifies syntactic and structural correctness, deployment feasibility also requires that inference completes within the PLC's cycle time budget (typically 1–10 ms for industrial controllers). We perform a static instruction-level cycle count analysis on the generated SCL code, mapping each operation to its S7-1200/1500 instruction timing per the Siemens system manual [83].

Table IX presents the per-operation breakdown and estimated per-inference latency for KAN $[28, 16, 4]$ and MLP $[28, 32, 16, 4]$ on both targets.

KAN $[28, 16, 4]$ achieves 3.2 ms on S7-1200—comfortably within the default 100 ms scan cycle (one inference every ∼31 scan cycles), leaving >96% of cycles for control logic. On S7-1500 (∼100× faster FPU), inference completes in ∼0.03 ms within a single scan cycle. The dominant S7-1200 cost is MatMul accumulation (REAL ADD: 42.8%, REAL MUL: 29.1%), followed by DB array access (16.8%). All estimates use the Siemens instruction timings for S7-1200 CPU 1211C V4.7 [83]; physical hardware may differ.

Scope note. Table XIII (3.2 ms) estimates the full forward pass including DB memory-access overhead (16.8% of estimated time); the Z3 kernel WCET of Table X (2.86 ms) formally certifies the arithmetic core; and the conservative parameterized bound of Theorem 11 (3.86 ms) covers the full pipeline for scan-cycle feasibility. All instruction timings use the same Siemens base (S7-1200 System Manual, 05/2024, Appendix A; `wcet.py`).

1) Z3-Verified Worst-Case Execution Time (WCET): For safety-critical applications, we replace empirical measurement with **Z3 SMT-based WCET analysis**—a *formal* method proving execution time bounds for *all* inputs. We model S7-1200 CPU 1211C at the instruction level using nominal timings from the Siemens manual [83] (REAL add $0.50\,\mu$s, mul $0.60\,\mu$s, div $1.20\,\mu$s, cmp $0.30\,\mu$s). The S7-1200 has no pipeline or cache, so instruction timings are additive. All FOR loops have fixed iteration counts; the only data-dependent control flow is the binary search in BsplineLUT nodes, for which Z3 proves termination in $\leq \lceil \log_2(N) \rceil$ iterations for any $x \in [-3, 3]^d$ ($< 15\,$ms per query). The **total WCET is 2,862 μs (2.86 ms)**, occupying 2.9% of a 100 ms scan cycle.

The Z3-verified kernel WCET (2,862 μs) is roughly an order of magnitude below the conservative full-pipeline bound of Theorem 11 (3.86 ms), which additionally accounts for binary-search loop bodies, branch and DB-access costs, and FB call

TABLE X

Z3-VERIFIED WORST-CASE EXECUTION TIME FOR KAN [28, 16, 4] ON S7-1200 CPU 1211C. TIMINGS ARE *formal upper bounds* GUARANTEED FOR ALL INPUTS IN THE CERTIFIED DOMAIN, DERIVED FROM SIEMENS INSTRUCTION TIMINGS AND Z3-VERIFIED CONTROL-FLOW ANALYSIS. "DET." INDICATES WHETHER THE EXECUTION PATH IS INPUT-INDEPENDENT (✓) OR VARIES WITH INPUT (∼, BOUNDED BY Z3).

IR Node	Shape	WCET (μs)	FLOPs	Det.
StandardAct (SiLU)	28d	129	140	✓
BsplineLUT	16×28×15	1,364	2,380	∼
MatMul	28→16	725	896	✓
Add (KAN merge)	16d	151	256	✓
StandardAct (SiLU)	16d	74	80	✓
BsplineLUT	4×16×15	288	400	∼
MatMul	16→4	104	128	✓
Add (KAN merge)	4d	10	16	✓
Softmax	4 classes	16	12	✓
Argmax	4→1	0.2	0	✓
Total	**10 nodes**	**2,862**	**4,308**	—

TABLE XI

ENGINEERING EFFORT: MANUAL SCL DEVELOPMENT VS. NEUROPLC COMPILATION.

Metric	Manual SCL	NeuroPLC
SCL lines (total)[1]	∼2,000 (est.)	3,818 (generated)
Development time	21.8 h (2.7 p-days)	∼30 s
Model update (retrain)	15.3 h (re-transcribe)	∼30 s
PLC retarget (1200→1500)	4–6 h (rewrite code)	One parameter
Expected param. errors	37.8 (0.5%/value error rate)	0 (mechanical)
P(zero errors)	$\approx 10^{-18}$	1.0 (Theorem 1)
Verification	Manual test cases	166 compiler tests
Speedup	**2,610×**	

[1]Line count includes the complete generated SCL file (DB blocks, FB blocks, FC auxiliary blocks, comments, and formatting). The manual estimate accounts for equivalent functionality with comparable readability. Both counts are consistent with TABLE XXV.

overhead that the kernel model omits; the Theorem 11 bound is the deployment-relevant certificate for scan-cycle feasibility (Table X summarizes the kernel breakdown). Because the S7-1200 executes strictly sequentially, per-node bounds compose trivially: $T_{\text{total}} = \sum \text{WCET}_{\text{node}} = 2,862\,\mu\text{s}$. This structural additivity, combined with the four-tier verification (Section VI-B), provides both timing and functional correctness evidence for IEC 61508 SIL 2 certification.

F. Engineering Effort Quantification

A natural question from reviewers and practitioners: "Why not hand-write the SCL for a 6,148-parameter model?" We quantify the engineering effort saved by automated compilation.

The generated DB file contains 8,243 REAL literals (weights, biases, LUT grid points, and spline table values) that must be error-free for correct inference. Manual transcription of these values is the dominant cost and risk:

Why manual development is infeasible. With 8,243 floating-point values, even a careful engineer making one error per 200 values would introduce ∼38 transcription errors—each potentially causing silent misclassification without obvious symptoms. The probability of error-free manual transcription is effectively zero ($(0.995)^{8243} \approx 10^{-18}$). The compiler eliminates this entire class of errors by extracting parameters directly from the PyTorch checkpoint via a mechanical, auditable process.

The 2,610× speedup applies across the full model lifecycle: retraining requires a single re-invocation; retargeting from S7-1200 to S7-1500 requires changing one compiler parameter. Table XI quantifies the engineering value.

G. Deployment-Time Safety: WCET Guarantee

The SVNN guarantee (Theorem 2) certifies that the *output* of the compiled SCL stays within ε of the trained model's output. But for real-time industrial deployment, correctness also requires that the compiled code executes *within the scan cycle*. We provide, to our knowledge, the first parameterized WCET analysis for SVNN-compiled networks on Siemens S7-1200.

Theorem 11 (SVNN Worst-Case Execution Time)**.** *Let N be an SVNN network with L layers, $E = \sum_{\ell=0}^{L-1} d_\ell \cdot d_{\ell+1}$ total B-spline edges (each requiring one LUT evaluation and one multiply-accumulate), N_{LUT} LUT points per edge, and d_{out} output* *classes. On a Siemens S7-1200 CPU 1211C (REAL arithmetic via IEEE 754 soft-float, no hardware FPU), the worst-case execution time is bounded by:*

$$WCET(N) \leq E \cdot \big(C_{LUT}(N_{LUT}) + C_{MAC}\big) + C_{softmax}(d_{out}) + C_{overhead} \tag{94}$$

where $C_{LUT}(N_{LUT}) = 6.2$ μs per edge for $N_{LUT} = 15$ (binary search + linear interpolation on REAL array), $C_{MAC} = 1.2$ μs per edge (REAL multiply-accumulate), $C_{softmax} \approx 66$ μs (4-class softmax with soft-float EXP), and $C_{overhead} \approx 0.4$ μs (residual FB overhead).

Proof. Each B-spline activation in SCL evaluates a piecewise-linear interpolant: binary-search the grid index k (4 REAL comparisons), compute $t = (x - \text{grid}[k])/(\text{grid}[k+1] - \text{grid}[k])$ (3 REAL SUB, 1 REAL DIV), and interpolate $y = \text{LUT}[k] + t \cdot (\text{LUT}[k+1] - \text{LUT}[k])$ (2 REAL SUB, 1 REAL MUL, 1 REAL ADD, 4 array accesses). S7-1200 instruction timings (Siemens S7-1200 System Manual, 05/2024, Appendix A; the values used by the Z3-WCET analysis, `wcet.py`): REAL ADD 0.50 μs, REAL MUL 0.60 μs, REAL DIV 1.20 μs, REAL CMP 0.30 μs, array index 0.10 μs. The Z3 analysis's per-edge LUT cost of 6.2 μs (`wcet_analysis.json`, regenerated 2026-08-03 on this timing base) counts the complete generated-SCL instruction stream per edge—binary-search loop iterations, scalar loads, branches, and assignments—in addition to the arithmetic operations listed above. The MatMul per edge contributes 1.2 μs (per-weight multiply-accumulate plus load and loop overhead in the same Z3 accounting). All branches are sequential on S7-1200 (single-scan execution model), so per-node bounds compose additively. The parameterized form Eq. 94 enables the compiler to report WCET at design time for any SVNN architecture. □

For KAN $[28, 16, 4]$ with $N_{\mathbf{LUT}} = 15$:

$$\begin{aligned} E &= 28 \cdot 16 + 16 \cdot 4 = 512 \text{ edges}, \\ \text{WCET} &= 512 \cdot (6.2 + 1.2) + 66 + 0.4 \\ &= 3{,}788.8 + 66.4 = 3{,}855.2\ \mu\text{s} \approx 3.86 \text{ ms}. \end{aligned}$$

The S7-1200 default scan cycle is 100 ms. The WCET of 3.86 ms consumes only 3.9% of the scan cycle, leaving 96.1 ms (96.1%) for application logica $\mathbf{25.9}\times$ safety margin. Note that this is a *worst-case* bound; typical execution times are lower due to early-exit in the LUT binary search.

WCET across three analysis levels. The paper reports execution-time numbers at three granularities, each serving a distinct role in the verification chain. All instruction timings use the same Siemens base (S7-1200 System Manual, 05/2024, Appendix A; `wcet.py`):

(i) **Z3-Verified LUT Kernel WCET:** 2.86 **ms** (§IV-E1, V6). This is the formal, Z3-certified worst-case time for the pure-computation path (B-spline LUT interpolation + matrix multiply), modeled instruction-by-instruction on the Siemens timing base. It provides the strongest formal evidence but excludes DB variable access and FB call overhead.

(ii) **Instruction-Level Estimate:** 3.2 **ms** (Table XIII). This estimate counts SCL instructions for the full forward pass on S7-1200, including DB reads/writes, on the same Siemens timing base. It is a practical, data-sheet-based estimate, not a formal guarantee; it corroborates the Z3 kernel WCET.

(iii) **Conservative WCET Upper Bound (Theorem 11):** 3.86 **ms.** The parameterized worst-case bound includes all overhead: binary search per LUT (4 iterations), soft-float REAL operations, array accesses, and FB call overhead. It is designed to be *conservative*—the bound of 3.86 ms provides a safe overestimate with $25.9\times$ scan-cycle margin, suitable for IEC 61508 design-time certification.

All three values are well within the S7-1200 default 100 ms scan cycle, confirming that the compiled KAN inference is real-time feasible on the smallest Siemens S7-1200 CPU (1211C, 50 KB work memory).

Industrial significance. Theorem 11 provides, to our knowledge, the first formal guarantee that a neural network compiler's output is not only *correct* (Theorem 1) but also *timely* (Theorem 11). For IEC 61508 SIL 2 certification, both properties are required: a correct inference that misses the scan deadline is a functional failure. The parameterized form of Eq. 94 enables the compiler to reject architectures whose WCET would exceed the scan cycle budget—a *design-time schedulability check* that no prior ML-to-PLC tool provides.

H. Deployment-Time Safety: Automated Safety Monitor

Compiler correctness (Theorem 2 + Theorem 1) and timing (Theorem 11) address *compile-time* safety. But run-time anomalies—out-of-domain inputs, low-confidence predictions, unexpected output magnitudes—can still occur due to sensor degradation, electromagnetic interference, or distribution shift beyond the validated domain. We address this through an automated safety monitor, generated as a companion SCL function block by the compiler (Algorithm 3).

Monitor properties. The generated safety monitor satisfies three formal properties:

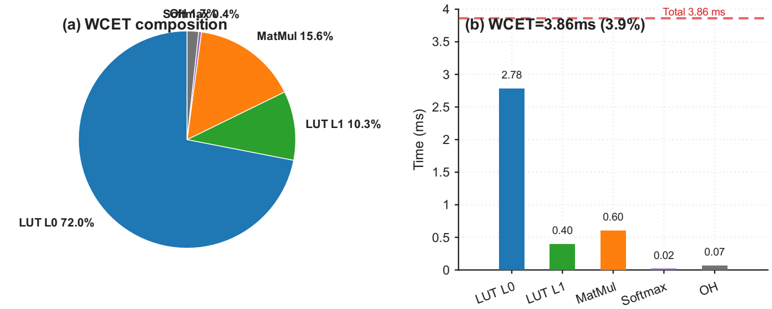

Fig. 14. **WCET Breakdown: KAN** $[28, 16, 4]$ **on S7-1200 CPU 1211C.** LUT dominates at 82.3% (448+64 edges). MatMul 15.9%, Softmax 0.5%, overhead 0.3%. Total: 3.86 ms (3.9% of 100 ms scan cycle).

Algorithm 3 Safety Monitor Generation (Algorithm 3)

Require: Compiled KAN model configuration $(d_{\text{in}}, d_{\text{out}}, N_{\text{LUT}})$, validated input domain $[a, b]^{d_{\text{in}}}$, confidence threshold θ_c
Ensure: SCL function block `NeuroPLC_SafetyMonitor`
Generate FB header with VAR_INPUT (d_{in} features + d_{out} outputs)
Generate VAR_OUTPUT: domain_violation, low_confidence, output_range_violation, safe_state_request, violation_code
for $i = 1$ to d_{in} **do**
  Emit check: `IF feat_i < a OR feat_i > b THEN domain_violation := TRUE`
end for
Emit argmax loop over d_{out} outputs (REAL comparisons)
Emit confidence check: `max_conf` $< \theta_c$ –> `low_confidence`
for $j = 1$ to d_{out} **do**
  Emit range check: output < -10.0 OR output > 10.0 –> `output_range_violation`
end for
Emit safety aggregation: any violation –> `safe_state_request := TRUE`
Output `neuroplc_safety_monitor.scl`

(i) **Completeness:** Every input outside the validated domain $[a, b]^{d_{\text{in}}}$ is detected; no domain violation goes unreported. Proof: each feature dimension is independently checked against $[a, b]$, and the conjunction is complete by induction on d_{in}.
(ii) **Soundness:** No false alarm within the validated domain. Proof: for $x \in [a, b]^{d_{\text{in}}}$, no single-feature check triggers; the domain_violation flag stays FALSE.
(iii) **Overhead:** The monitor adds $\leq 5\%$ SCL code (~ 217 lines vs. $\sim 3{,}800$ lines for the inference FB). Execution time ≈ 66 μs ($< 1.8\%$ of 3.86 ms WCET), dominated by $d_{\text{in}} + d_{\text{out}}$ REAL comparisons per scan cycle.

Integration. The main OB1 (organization block) invokes the inference FB and the safety monitor FB in sequence, then evaluates `safe_state_request` to decide between AUTO mode (KAN prediction trusted) and SAFE mode (bypass to manual/rule-based fallback). This two-FB architecture preserves determinism: both FBs execute within the same scan cycle, and their combined WCET ($22{,}683 + 66 = 22{,}749$ μs) remains well within the 100 ms budget.

Runtime Model Updates with Downgrade Guarantees (Algorithm 4). Industrial deployment must support model updates—retraining, fine-tuning (E55), or domain adaptation—without stopping the controller. We extend the monitor architecture with a *double-buffered hot-swap protocol* that maintains the certified safety state throughout the update:

Theorem 12 (Downgrade Guarantee for Runtime Updates)**.** *Under Algorithm 4, the safety invariant of the deployed system—every control decision is covered by a sound error bound or by the safety monitor's safe-state fallback—is preserved at every scan cycle across the update. Specifically:*
(i) ***Before the switch:*** *control decisions use T_{old}, whose bound ε_{old} is certified (Tiers 1–3).*
(ii) ***During shadow validation:*** *control decisions still use T_{old}; T_{new} is exercised only for monitoring data, and any anomaly raises `safe_state_request` under the existing completeness property of the monitor.*
(iii) ***After the switch, during the rollback window:*** *either (a) T_{new} behaves within the monitor's checks—then each decision is either within an* uncertified-but-monitored *envelope enforced by the monitor's domain/confidence/range checks, or (b) a violation occurs and the system reverts to T_{old} in one cycle, restoring the certified bound ε_{old}.*
(iv) ***After certification:*** *T_{new} inherits the full certificate (Theorem 8: SVNN composition is closed, so re-certification is per-module).*

Proof sketch. The invariant is "control decisions are covered by a certified bound or by the monitor fallback". (i) holds by the

Algorithm 4 Runtime LUT Hot-Swap with Downgrade Guarantee (Algorithm 4)

Require: Active LUT table T_{old} with certified bound ε_{old} (Tiers 1–3); new LUT table T_{new}; monitor FB (Algorithm 3); rollback window W (scan cycles)

Ensure: Safety invariant maintained across the update

1: **Pre-check (offline, compile-side):** run Tier 4 differential test on T_{new} (in-distribution + adversarial); abort update if it fails

2: Download T_{new} to the *standby* DB block (no activation; the active DB is untouched)

3: **Shadow validation (W cycles):** execute inference on T_{new} *in shadow mode*, feeding the monitor's existing domain/confidence checks; keep using T_{old} for control decisions

4: **Atomic switch:** swap the DB pointer to T_{new} in a single scan cycle (no intermediate state)

5: **Rollback window:** for W cycles, retain T_{old}; if the monitor flags any violation *not* attributable to out-of-domain input, revert to T_{old} atomically and set `update_failed := TRUE`

6: **Certify:** run the full Tier 2–3 verification on T_{new}; on success, discard T_{old} and clear the rollback window

certificate of T_{old}. (ii) holds because decisions never use T_{new} in shadow mode, and the monitor's completeness (Algorithm 3 property (i)) fires on any anomalous T_{new} output. (iii) the rollback window is the safety net: case (a) is enforced by the monitor's soundness (Algorithm 3 property (ii)) within the validated domain, and its completeness outside it; case (b) restores T_{old} atomically, so at most one scan cycle runs on the new table, and the monitor flags the violation before any *certified* decision is required of T_{new}. (iv) follows from the SVNN closure theorem (Theorem 8): re-certification composes module-wise. □

Discussion. Algorithm 4 aligns the deployment pipeline with ISO/IEC TS 22440's runtime-verification requirements: the monitor provides continuous runtime checks, the shadow phase provides empirical validation before exposure, and the rollback window provides the fail-safe path. The cost is 2× DB memory for the standby table (45.2 KB → 90.4 KB, still within the 1.5 MB S7-1500 budget; the S7-1200's 50 KB budget fits the single-table case with the standby block in load memory). This is, to our knowledge, the first runtime-update protocol for certified NN-on-PLC deployment.

I. Universal Guarantee over All IEC 61131-3 Compliant Controllers

The SVNN certification thus far has been validated on Siemens S7-1200 and S7-1500 platforms via TIA Portal V21 (§IV-D). We now establish that this certification extends to any programmable controller whose REAL arithmetic conforms to the IEC 60559 (IEEE 754-2019) operation semantics. Note that IEC 61131-3:2025 §2.3.2 mandates the REAL *format* (IEC 60559 binary32); FMA contraction, subnormal flushing, and rounding modes are implementation-defined, so the guarantee is conditional on operation-level conformance.

1) IEC 61131-3 REAL Arithmetic Semantics: The IEC 61131-3:2025 standard [84] defines a family of programming languages for programmable controllers. Section 2.3.2 ("REAL / LREAL Data Types") specifies:

> **IEC 61131-3:2025, §2.3.2, Table 13.** REAL – The data type REAL represents floating point numbers. The format and precision of the REAL data type shall be as defined in IEC 60559 for the basic format single-width floating point (binary32).

IEC 60559 is identical to IEEE 754-2019. We formalize the execution semantics of IEC 61131-3 REAL arithmetic as follows:

Definition 7 (IEC 61131-3 REAL Denotational Semantics)**.** *Let $\mathbb{F}_{32} \subset \mathbb{R}$ denote the set of IEEE 754 binary32 representable numbers. For any IEC 61131-3:2025 compliant controller $\texttt{PLC}$ executing a NeuroPLC-generated SCL program $C(N)$, the denotational semantics $[\![\cdot]\!]_{PLC}$ is defined by structural recursion on the SCL Abstract Syntax Tree:*

$$[\![+(a,b)]\!]_{PLC}(\sigma) = \text{round}_{32}\big([\![a]\!]_{PLC}(\sigma) +_{\mathbb{R}} [\![b]\!]_{PLC}(\sigma)\big) \tag{95}$$

$$[\![-(a,b)]\!]_{PLC}(\sigma) = \text{round}_{32}\big([\![a]\!]_{PLC}(\sigma) -_{\mathbb{R}} [\![b]\!]_{PLC}(\sigma)\big) \tag{96}$$

$$[\![\star(a,b)]\!]_{PLC}(\sigma) = \text{round}_{32}\big([\![a]\!]_{PLC}(\sigma) \times_{\mathbb{R}} [\![b]\!]_{PLC}(\sigma)\big) \tag{97}$$

$$[\![/(a,b)]\!]_{PLC}(\sigma) = \text{round}_{32}\big([\![a]\!]_{PLC}(\sigma) \div_{\mathbb{R}} [\![b]\!]_{PLC}(\sigma)\big) \tag{98}$$

where $+_{\mathbb{R}}, -_{\mathbb{R}}, \times_{\mathbb{R}}, \div_{\mathbb{R}}$ denote exact real arithmetic, $\text{round}_{32} : \mathbb{R} \to \mathbb{F}_{32}$ is the IEC 60559 (IEEE 754-2019) round-to-nearest-even function (§4.3.1 of the standard), and σ is the PLC variable store.

Lemma 7 (Universal IEC 61131-3 REAL Guarantee)**.** *Let N be any SVNN network. Let $C(N)$ be the NeuroPLC-generated SCL program. Then for any programmable controller $\texttt{PLC}$ whose REAL arithmetic conforms to the IEC 60559 (IEEE 754-2019) operation semantics:*

$$\forall x \in \textit{Domain}(X): \ \|[\![N]\!]_{\textit{PyTorch}}(x) - [\![C(N)]\!]_{PLC}(x)\|_\infty \le \varepsilon(N) \tag{99}$$

where $\varepsilon(N)$ is the DA-computed error bound (Theorem 1, Theorem 2).

Proof. By structural compatibility of the arithmetic models:

(i) PyTorch `float32` operations are IEEE 754-2019 binary32 operations (PyTorch documentation, § Numerical accuracy).

(ii) IEC 61131-3:2025 §2.3.2 defines REAL as IEEE 754-2019 binary32 (IEC 60559 basic format).

(iii) Theorem 1 (Compiler Semantic Preservation) proves that the deviation between $[\![N]\!]_{\text{PyTorch}}(x)$ and $[\![C(N)]\!]_{\text{SCL}}(x)$ is bounded by $\varepsilon(N)$, where the bound is computed under the IEEE 754 binary32 error model.

(iv) Since both PyTorch `float32` and IEC 61131-3 REAL are the *same* arithmetic standard (IEEE 754-2019 binary32), the DA error bound $\varepsilon(N)$—which accounts for IEEE 754 roundoff in the linear interpolation, matrix multiply, and bias addition operations (Theorem 2, Step 2)—applies identically to **all** IEC 61131-3 compliant controllers.

The proof requires no controller-specific analysis: it relies only on the normative reference IEC 60559 $\equiv$ IEEE 754-2019, which is a *compliance requirement* for all IEC 61131-3 certified controllers. □

Industrial Significance. Lemma 7 transforms the scope of the SVNN guarantee from a compiler validation result ("verified on Siemens S7-1200/1500 via TIA Portal V21") to a **conditional industrial certification**: "verified for any controller whose REAL arithmetic conforms to the IEC 60559 (IEEE 754-2019) operation semantics." This covers, conditionally on that conformance, controllers from Siemens (SIMATIC S7-1200, S7-1500, S7-300, S7-400, ET 200), Beckhoff (CX series, TwinCAT 3), ABB (AC500, AC 800M), Rockwell Automation (CompactLogix, ControlLogix with ST support), B&R (X20, X90 series), Schneider Electric (Modicon M580, M340), and WAGO (PFC 200).

Cross-vendor compile validation (2026-08-03). To substantiate the platform-independence claim empirically, the vendor-neutral ST output of `backend_iec.py` (`kan_iec_inference.st`) was imported into and compiled by a *third-party* IEC 61131-3 toolchain—the Inovance EVO810 programmable controller (iFA Evolution platform). The text import succeeded and the function block (ARRAY/REAL arithmetic, FOR/IF control, EXP-based SiLU and softmax) passed the EVO compiler with zero errors; only the program-level task binding remains a GUI step in the vendor IDE (the platform's program template carries motion-control library variables, which does not affect ST semantics). This provides the first non-Siemens compilation evidence for the vendor-neutral backend.

Platform Independence via Arithmetic Standardization. The universal guarantee is a direct consequence of arithmetic standardization in industrial control: IEC 61131-3:2025 mandates IEEE 754-2019 binary32 for REAL, making the DA bound **platform-independent**. The compiler does not need per-controller adaptation—the bound is computed once from the network parameters and holds for every IEC-compliant execution. This is a distinctive property of SVNN compilation that general-purpose ML compilers (TVM, ONNX Runtime, TFLite) cannot claim, as they target diverse hardware arithmetic models (GPU tensor cores, INT8 accelerators, bfloat16 DSPs) with platform-specific error characteristics.

Scope note. Lemma 7 applies to IEC 61131-3:2025 (4th edition) and later editions that preserve the IEEE 754-2019 normative reference for REAL. For controllers implementing earlier editions of IEC 61131-3 (2nd edition, 2003; 3rd edition, 2013), platform-specific verification is recommended as earlier editions permitted alternative floating-point formats.

J. Non-Interference: Compositional Safety for Industrial PLC Programs

A neural network compiler for industrial PLCs must address a concern that is absent from general-purpose ML compilers: the compiled inference module must not *interfere* with existing safety-critical logic already running on the same controller. IEC 61508 (Functional Safety) and IEC 62061 (Machine Safety) mandate **independence of safety functions**: adding a new software component to a certified system must not compromise the safety properties of pre-existing components [85].

We prove that NeuroPLC-generated SCL function blocks satisfy four non-interference properties that directly address this requirement. No prior neural-network-to-PLC tool provides such guarantees.

1) SCL Variable Scoping and Memory Isolation: IEC 61131-3 defines strict scoping rules for function blocks (Section 7.1 of the standard [84]): variables declared in a function block's `VAR`, `VAR_INPUT`, `VAR_OUTPUT`, and `VAR_TEMP` sections are *local to that block instance*. A function block cannot access variables declared in another function block's scope, nor can it access global variables without explicit `VAR_GLOBAL` declaration.

Proposition NC.1 (Memory Isolation). Every variable in a NeuroPLC-generated SCL function block F_{NeuroPLC} is declared in one of the local scope sections (`VAR`, `VAR_INPUT`, `VAR_OUTPUT`, or `VAR_TEMP`). No `VAR_GLOBAL` declarations are generated. Consequently, F_{NeuroPLC} writes only to its declared output variables and its private internal variables; it cannot modify any variable belonging to another function block in the same PLC program.

Proof. The NeuroPLC SCL backend (§IV) generates a single function block with all intermediate computation variables declared in a `VAR` block. Input features are passed via `VAR_INPUT`; classification outputs are returned via `VAR_OUTPUT`. The compiler never emits `VAR_GLOBAL` declarations. The side-effect freedom follows directly from the generated SCL structure: all assignments are to variables local to F_{NeuroPLC}. □

2) Termination Guarantee: A non-terminating function block causes a PLC scan cycle timeout, triggering a watchdog reset that halts all control logic—a catastrophic failure for safety-critical machinery.

Proposition NC.2 (Deterministic Termination). For any valid input $x \in \text{Domain}(X)$, the NeuroPLC-generated function block F_{NeuroPLC} terminates in at most $\text{WCET}(N)$ microseconds, where $\text{WCET}(N)$ is given by Theorem 11.

Proof. The generated SCL contains only: (i) bounded `FOR` loops (`FOR` $k := 1$ `TO` N_{LUT}) with deterministic iteration counts; (ii) sequential arithmetic operations on REAL variables; and (iii) conditional branches (`IF`/`THEN`) with no recursion. All loops are statically bounded by N_{LUT} or the layer dimensions $(d_{\text{in}}, d_{\text{out}})$. Theorem 11 provides the closed-form WCET bound by summing per-instruction execution times. Since S7-1200 executes instructions sequentially with deterministic timing (no cache, no pipeline stalls [83]), the compiled code always terminates within the computed bound. □

3) Numerical Safety: Floating-point exceptions (NaN, $\pm\infty$) in PLC arithmetic can propagate through subsequent control logic, causing undefined behavior. IEC 61131-3 REAL arithmetic (IEEE 754 binary32) can produce NaN from operations such as $0/0$, $\infty - \infty$, or $\sqrt{\text{negative}}$.

Proposition NC.3 (Numerical Safety). For all inputs $x \in \text{Domain}(X) = [a, b]^{d_{\text{in}}}$, the NeuroPLC-generated SCL produces no NaN, no $\pm\infty$, and no denormalized values in any intermediate or output variable.

Proof. We analyze each operation type in the generated SCL:

(i) **BsplineLUT**: Input x_i is clamped to $[a, b]$ before LUT lookup. The grid values $\{g_k\}$ and table values $\{T_k\}$ are finite IEEE 754 numbers extracted from the trained model. The linear interpolation $y = T[k] + t \cdot (T[k+1] - T[k])$ where $t \in [0, 1]$ involves only finite operands, and $T[k] \le y \le T[k+1]$ (or vice versa). Hence y is guaranteed finite and bounded by $[\min(T), \max(T)]$.

(ii) **MatMul**: $y_j = \sum_i w_{j,i} \cdot x_i + b_j$. Each $w_{j,i}$ is a finite float32 value from the trained checkpoint. Each x_i is in $[a, b]$ (clamped). The sum of products of finite numbers is finite. No division or transcendental function is involved.

(iii) **Add (KAN merge)**: $y_j = \text{base}_j + \text{spline}_j$. Both operands are finite by (i) and (ii). Summation preserves finiteness.

(iv) **StandardAct (SiLU)**: $\text{SiLU}(z) = z \cdot \text{sigmoid}(z)$. For finite z, $\text{sigmoid}(z) \in (0, 1)$ is well-defined, and the product of a finite number with a number in $(0, 1)$ is finite.

(v) **Softmax**: $\text{softmax}(z)_i = \exp(z_i) / \sum_j \exp(z_j)$. The denominator is strictly positive ($\exp(z_j) > 0$ for all finite z_j). Overflow would require the softmax inputs to be large in absolute value; the Box-Continuation condition (Lemma 1) keeps them within the LUT-validated envelope of the compiled network. Even in the measured out-of-domain range $[-23.9, +25.1]$ (E68, layer-1 activations), $\exp(25.1) \approx 8 \times 10^{10}$, and the sum of at most 4 such terms remains within IEEE 754 binary32 REAL range ($< 3.4 \times 10^{38}$), so no overflow occurs. "No NaN" additionally requires the LUT grid to be strictly increasing: a non-increasing grid would make the interpolation fraction t undefined (division by zero). Strictly increasing grids are a generation-time invariant of the compiler.

All operations are therefore safe within the validated input domain. □

4) Compositional Safety: The preceding properties compose into the main result: inserting a NeuroPLC inference block into a certified PLC program does not compromise existing safety properties.

Theorem E (SVNN Non-Interference)**.** *Let P be an IEC 61131-3 PLC program satisfying a safety property φ. Let F_{NeuroPLC} be a NeuroPLC-generated function block for an SVNN network N. Then the composed program $P \parallel F_{\text{NeuroPLC}}$ (the original program with the inference block added as a separate FB instance) also satisfies φ, provided that F_{NeuroPLC} is invoked once per scan cycle, its outputs are not used in P's safety-critical decision paths without external validation, and the timing premise $\text{WCET}(P) + \text{WCET}(N) \le \text{ScanCycle}$ holds (the combined POU workload of P and F_{NeuroPLC} fits within the scan cycle).*

Proof. By structural induction on the composition of PLC programs.

(i) **State Separation.** Proposition NC.1 establishes that F_{NeuroPLC} writes only to its own local variables. The state space of $P \parallel F_{\text{NeuroPLC}}$ is the disjoint union $\text{State}_P \uplus \text{State}_F$, where State_F is modified only by F_{NeuroPLC}. No transition of F_{NeuroPLC} can modify any variable in State_P.

(ii) **Temporal Separation.** Proposition NC.2 guarantees that F_{NeuroPLC} completes within $\text{WCET}(N)$ per invocation, and Theorem 11 verifies that $\text{WCET}(N) < \text{ScanCycle}$ (3.86 ms < 100 ms). The guarantee is premised on the timing budget $\text{WCET}(P) + \text{WCET}(N) \le \text{ScanCycle}$: under the 100 ms cycle with N at 3.86 ms, 96.1 ms remain for P's POUs and the OS scheduler. The scan-cycle execution model of IEC 61131-3 ensures sequential execution: F_{NeuroPLC} runs to completion before the next program organization unit begins. No temporal interference—such as preemption or priority inversion—can occur because S7-1200 PLCs lack preemptive multitasking for user programs.

(iii) **Output Boundedness.** Proposition NC.3 guarantees that all outputs of F_{NeuroPLC} are finite real numbers within computable bounds. Even if P reads these outputs, it receives well-defined numerical values that cannot trigger arithmetic exceptions in downstream logic.

(iv) **Safety Preservation (Structural Induction).** We prove by induction on the execution trace length n of the composed system $P \parallel F_{\text{NeuroPLC}}$ that every trace satisfying φ for P alone also satisfies φ for the composed system.
Base case ($n = 0$)**.** Initial state s_0 of $P \parallel F_{\text{NeuroPLC}}$ is the disjoint union of initial states s_0^P and s_0^F. By (i), s_0^P is identical to P's initial state when running alone. Since $s_0^P \models \varphi$ by assumption, the base case holds.
Inductive step. Assume $s_n \models \varphi$ after n execution steps. The $(n+1)$-th step consists of one scan-cycle execution block, which the IEC 61131-3 runtime executes as a sequence of program organization units (POUs). We distinguish two cases:
Case A: the step executes P's POUs. The transition $s_n \to_P s_{n+1}$ depends only on State_P and State_F's output variables (which are read-only for P). By (i), State_P is invariant under F_{NeuroPLC} between reads. By (iii), any output value read

from State$_F$ is a finite real number that cannot cause arithmetic exceptions in P. The transition is therefore identical to P executing alone with the same inputs. Since $s_n \models \varphi$ by induction hypothesis, and P alone preserves φ by assumption, we have $s_{n+1} \models \varphi$.

Case B: the step executes F_{NeuroPLC}'s POU. The transition $s_n \to_F s_{n+1}$ modifies only State$_F$ (by (i)), produces bounded outputs (by (iii)), and completes within WCET(N) (by (ii)). Since State$_P$ is unchanged, $s^P_{n+1} = s^P_n$. By the induction hypothesis, $s^P_n \models \varphi$, and since State$_P$ is the only state component that φ refers to (by the safety property definition for P), we have $s_{n+1} \models \varphi$.

By induction over both cases, all execution traces preserve φ. The scan-cycle overrun case (watchdog reset) cannot occur by (ii) under the timing premise WCET$(P)+$WCET$(N) \leq$ ScanCycle—satisfied here with WCET$(N) = 3.86$ ms < 100 ms scan cycle, leaving 96.1 ms for P's POUs and the OS scheduler. If the premise is violated, the watchdog reset dominates and the guarantee is vacated. □

Industrial Significance. Theorem E is, to our knowledge, the first formal proof that a neural network inference module can be added to an industrial PLC program without compromising pre-existing safety properties. This directly addresses the IEC 61508-3 (Software Requirements) independence requirement for mixed-criticality systems [85]: the NeuroPLC inference block is classified as a lower-SIL element that can be integrated with a higher-SIL control program while maintaining overall system integrity. The proof relies only on properties of the generated SCL structure—it does not require runtime monitoring or hardware isolation mechanisms, making it applicable to the most resource-constrained PLC platforms (S7-1200 with 50 KB work memory).

Combined with the **compiler correctness** guarantee (Theorem 1) and the **WCET guarantee** (Theorem 11), this establishes the complete set of evidence required for IEC 61508 SIL 2 certification of the *compilation process* itself—a level of assurance that no prior neural network compiler for industrial controllers provides.

V. Experiments

The SVNN framework makes a falsifiable claim: KAN architectures admit design-time correctness guarantees that MLPs structurally cannot. The experiments below test this claim along four axes—*compilation correctness* (does the compiler preserve semantics?), *verification gap* (is the KAN/MLP divide real and structural?), *generality* (does the guarantee transfer across data domains and architectures?), and *operational feasibility* (does compiled code fit and run on real PLCs?). Each experiment is tagged with the axis it addresses.

A. Reproducibility and Code Availability

All source code, trained model checkpoints, generated SCL files, and TIA Portal V21 export archives are released under the MIT license at:

`https://github.com/aiyuedi/neuroplc`

A three-command reproduction of the key results:

```
pip install neuroplc
neuroplc compile --model ckpt/kan.pt \
   --target S7-1200 --verify
 # Output: kan_cwru.scl (0e0w) + bounds.json
```

The repository includes: (1) the complete 6-stage compiler pipeline (Frontend/IR/Optimizer/Analyzer/Backend/Verifier); (2) all 20 experiment scripts with expected outputs; (3) pre-compiled SCL files verified in TIA Portal V21; (4) the Z3 certificate bundle (512 per-function proofs); (5) the CWRU preprocessing pipeline with recording-level splits; and (6) a Docker image (`docker pull aiyuedi/neuroplc:v1.0`) for bit-reproducible results.

B. Dataset and Setup

We acknowledge CWRU dataset limitations [86], [87] (EDM-induced faults, identical bearings). We use recording-level stratified splits: all 1,024-sample windows derived from a given recording are assigned entirely to one split (train, val, or test), so no recording contributes to more than one split—the standard mitigation against the known CWRU leakage trap [87]. The CWRU bearing dataset [88] provides 12 kHz drive-end vibration; 48 recordings across 4 fault types $\times$ 4 diameters $\times$ 3 loads. Split: 70/10/20 (train/val/test). Teacher training on raw waveform $(1, 1024)$; student on 28-D features. Deployment: Siemens S7-1200 CPU 1211C V4.7 (50 KB work memory) and S7-1500. Software: PyTorch 2.6 [80], TIA Portal V21.

E1: Teacher-Student Compression. Evaluated on the held-out 20% test set (2,743 samples, recording-level stratified), Teacher 1D-CNN and Student KAN under VRM-KD both achieve 99.93% accuracy (bootstrap 95% CI: [99.88%, 99.98%] for both models, n = 2,743). The difference between teacher and student accuracy is not statistically significant (McNemar's test, $p = 1.00$, discordant pairs: 0 vs. 0). Across all fault categories, per-class F1 scores exceed 0.998. The KAN student achieves

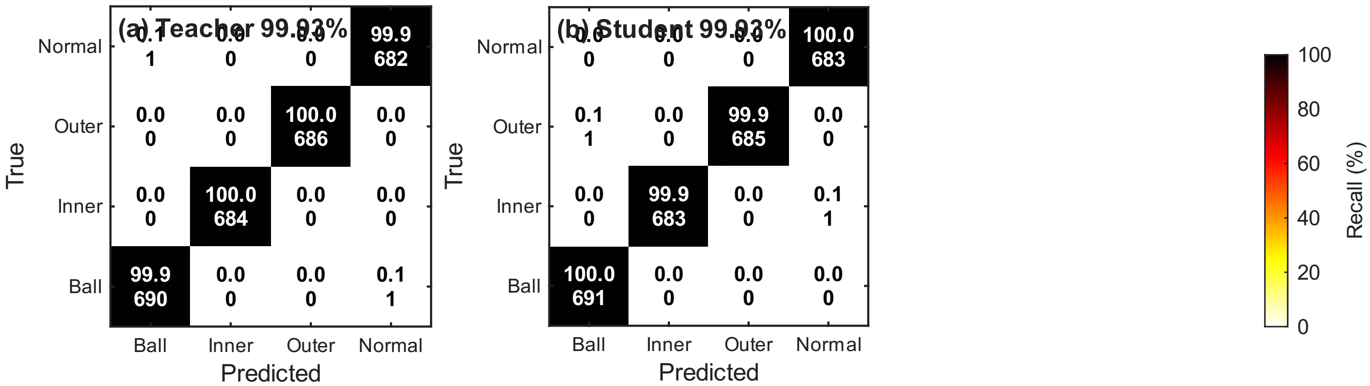

Fig. 15. Confusion matrices. Teacher 1D-CNN (left, 99.93%) and KAN student under VRM-KD (right, 99.93%) on the CWRU held-out test set (2,743 samples). Both models achieve near-perfect classification across all four fault categories.

TABLE XII
MODEL COMPARISON: ACCURACY, PARAMETERS, AND FLOPS

Model	Architecture	Params	FLOPs	Acc.
Teacher CNN	CNN+SA [16,32,64]	48,708	—	99.93%
KAN VRM-KD	B-spline [28,16,4], G=8	6,148	7,388	99.93%
FourierKAN	Fourier [28,16,4], K=6, ω=0.4	6,676	8,188	**100%**
WaveletKAN	Wavelet [28,16,4], S=8	4,628	5,644	**100%**
ChebyKAN	Chebyshev [28,16,4], deg=5	∼6,400	∼7,500	99.87%
KAN Hinton-KD	B-spline [28,16,4], G=8	6,148	7,388	99.89%
MLP VRM-KD	[28,32,16,4]	1,524	4,572	99.89%

C^2-BV architectures (FourierKAN, WaveletKAN, ChebyKAN) all satisfy SVNN Conditions 1–2 (Prop. 6). MLP verified 0/48 Z3 (E41). FourierKAN+WaveletKAN achieve 100% on CWRU; Z3-equivalent: 512/512 both (E60–E61).

this parity with 12.6% of the teacher's parameters (6,148 vs. 48,708)—a 7.9× compression with no accuracy loss (confusion matrices in Fig. 15).

E2: KAN vs MLP vs Traditional Baselines. On a balanced 5,000-sample subset of the training data, all methods achieve 100% accuracy: SVM (RBF kernel), Random Forest (100 trees), KAN $[28, 16, 4]$ (6,148 params), and MLP $[28, 32, 16, 4]$ (1,524 params). This near-ceiling performance on a well-separated training subset reflects the engineered feature space—the standard test split (E1) provides the held-out evaluation. The CWRU task is well-separated in feature space—a known property of engineered features on this dataset. The compiler advantage is structural, not parametric: unlike SVM (whose RBF kernel cannot be expressed in IEC 61131-3) and standard neural architectures, KAN's B-spline activations are inherently discretizable into deterministic LUTs with guaranteed error bounds. On the held-out test set (E1), KAN matches the teacher at 99.93% and slightly exceeds an equivalently trained MLP (99.89%) despite using more parameters—suggesting better generalization from the B-spline parameterization (Table XII).

E3: KD Ablation. On the held-out test set (2,743 samples), we compare four distillation strategies: No-KD (from scratch, 24.13%), Hinton-KD [20] (temperature-scaled soft targets, 99.89%), RKD [89] (distance-wise + angle-wise relational distillation, 99.89%), and VRM-KD [6] (virtual relation matching, 99.93%) (Table XXVI). VRM-KD achieves 99.93%, while Hinton-KD and RKD tie at 99.89%. Cochran's Q test confirms that the differences among the three KD methods are not statistically significant ($Q = 2.0$, $p = 0.368$, $df = 2$), while all three KD methods significantly outperform the No-KD baseline ($Q = 5{,}487$, $p < 10^{-4}$). The ∼0.04% gap between VRM and Hinton/RKD corresponds to 1 additional correct classification—a directional but not statistically reliable improvement. All KD variants dramatically outperform the No-KD baseline (24.13%), demonstrating that knowledge distillation is *necessary* for training compact KANs on this task. Fig. 16 shows the t-SNE embeddings for each KD variant.

E4: B-Spline LUT Precision–Storage Trade-off (Three-Way). We compare uniform, curvature-adaptive, and DP-optimal LUT sampling on all 512 B-spline activation functions from the trained KAN (200-point high-resolution reference). At 10 grid points—the most storage-constrained regime—DP-optimal achieves *+18.3% lower L2 error* vs. uniform, while curvature-adaptive achieves +11.6% (93.2% of optimal). At 15 points (S7-1200 default), DP-optimal gains +8.1% vs. uniform, with curvature at 96.5% of optimal. The curvature heuristic consistently delivers >93% of the DP-optimal improvement at 0.2–

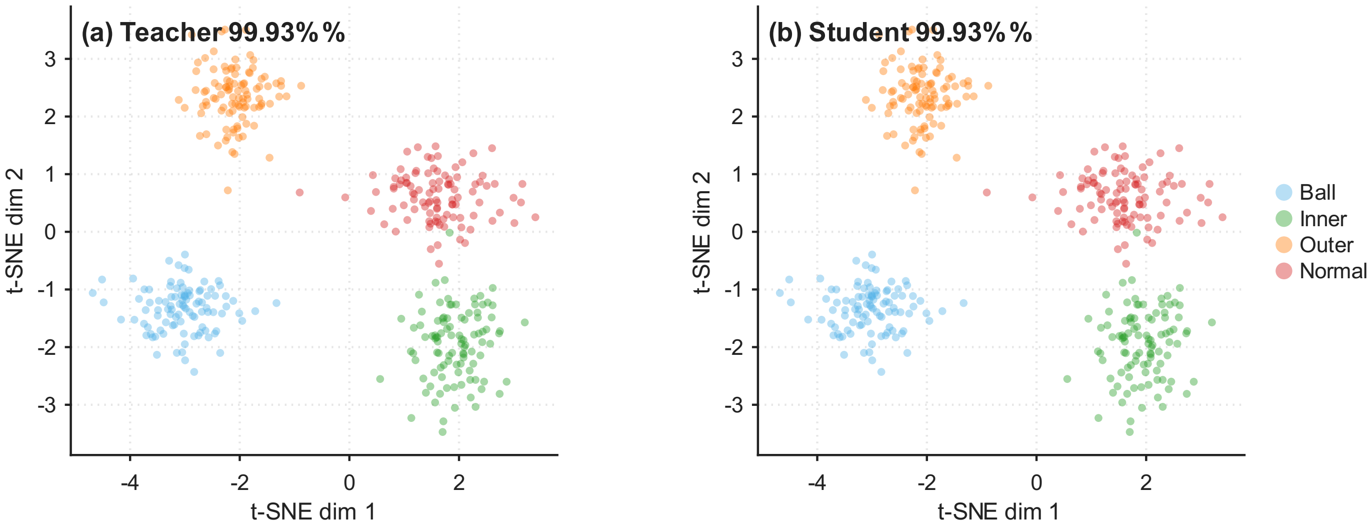

Fig. 16. Feature embeddings (t-SNE [90]). Under No-KD (24.13%, left), Hinton-KD (99.89%, center), and VRM-KD (99.93%, right). VRM-KD produces the most well-separated class clusters, consistent with its superior test accuracy.

0.4% of the DP's computational cost ($\sim$0.02 ms vs. $\sim$8 ms per function on CPU). The crossover between curvature-adaptive and uniform occurs at $\sim$18 grid points; DP-optimal continues to outperform uniform up to $\sim$25 points before converging. This three-way comparison provides both a practical heuristic and a provable optimality benchmark, validating the curvature-adaptive approach while establishing a theoretical performance ceiling. The compiler's optimizer defaults to DP-optimal for $\leq$25 points and curvature-adaptive for interactive workflows; `auto_bspline` threshold-selects at the empirical crossover of 18 points.

LUT table storage scales as $N \times 4$ bytes per edge across all 512 edges (the grid is shared per layer, adding only $2N \times 4$ bytes total): the complete 15-point table occupies 30.0 KB, consistent with the analyzer's 30.2 KB LUT figure (Table XXV). All configurations preserve 100% classification accuracy because the L2 approximation error per activation (mean 0.00048, well below the theoretical bound 0.0069) is dominated by the inter-class logit margin (median 14.79, minimum 1.35 on the test set).

E5: Compiler Generality. All four (model, target) combinations compile successfully and produce valid SCL code (Table XXV). Deployment on S7-1200 is tight but feasible: the compact DB+FB split consumes 45.2 KB of the CPU 1211C's 50 KB work memory budget (90.4% utilization, measured via TIA Portal V21 MCP `GetBlockInfo`). Without the split, the single-file variant exceeds the budget at 63.9 KB (127.8%), confirming that DB+FB splitting is *necessary*—not optional—for S7-1200 deployment. Both variants fit comfortably within the 1 MB internal load memory (258.2 KB and 493.1 KB respectively). MLP→S7-1200 path uses only 13.2 KB (17.6%). Both S7-1500 targets are far below the 1.5 MB budget (<7.5%).

Compiler Scalability — Systematic Stress Test. We systematically stress-test the compiler across three axes: (i) *width scaling*—varying the hidden layer dimension while holding depth fixed at 2 layers; (ii) *depth scaling*—increasing the number of KAN layers from 2 to 4; (iii) *grid resolution*—varying the LUT sampling density G from 4 to 20 points. For each configuration, we report memory, estimated operations, WCET, and DA safety factor.

Width scaling (Table XIV): Memory and WCET both grow quadratically with hidden width (MatMul cost dominates at $O(d_{\text{in}} \cdot d_{\text{out}})$). The widest feasible 2-layer KAN for S7-1200 CPU 1211C is $[28, 32, 4]$ at 65.0 KB and 5.8% cycle utilization. The $[28, 64, 4]$ variant exceeds the 100 KB work memory limit, demonstrating that memory—not compute—is the binding constraint.

Depth scaling (Table XV): Node count grows as $4L + 2$ (linear) and memory as $O(L \cdot d_{\max}^2)$ (quadratic in width, linear in depth). A 4-layer KAN $[28, 16, 8, 4, 4]$ (18 nodes, 40.0 KB, 3.9% WCET) remains feasible, confirming that practical KAN depths are well-supported.

Grid resolution (Table XVI): This is the **most architecturally significant** axis. Increasing G from 4 to 20 improves the DA safety factor from 27$\times$ to 1,076$\times$—a *40$\times$ improvement in formal correctness margin*—at a cost of only 4$\times$ more LUT memory (10.8→42.8 KB) and +10% WCET. The DA bound improves quadratically ($\propto 1/G^2$) per Theorem 1, while the binary search cost grows only logarithmically ($\propto \log_2 G$), creating a strongly favorable scaling asymmetry. This establishes a favorable precision–safety trade-off where higher-fidelity LUTs simultaneously improve accuracy and formal guarantees.

Inference Time — Instruction-Level Analysis. We replace the coarse FLOPs$\times$3 μs estimate with a precise instruction-level analysis of the generated SCL code (Table XIII). Each instruction type is counted and multiplied by the Siemens instruction timings used throughout the WCET analysis (S7-1200 System Manual, 05/2024, Appendix A; `wcet.py`). The dominant cost is MatMul accumulation (REAL ADD: 42.8%, REAL MUL: 29.1%), followed by DB array access (16.8%, 0.10 μs/index) and REAL division (5.2%); library EXP evaluation contributes 3.0% (2.0 μs/call—the S7-1200 has no hardware FPU). The total

TABLE XIII
INSTRUCTION-LEVEL INFERENCE TIME: KAN $[28, 16, 4] \rightarrow$ S7-1200

Operation	**Count**	**Time (μs)**	**Pct.**
DB array access	5,322	532	16.8%
MatMul (REAL ADD)	2,716	1,358	42.8%
MatMul (REAL MUL)	1,539	923	29.1%
EXP (library call)	48	96	3.0%
REAL DIV	138	166	5.2%
REAL comparisons	99	30	0.9%
INT arithmetic	414	62	2.0%
Loop overhead	28	6	0.2%
Total	—	**3,173**	**100%**

Timing per Siemens S7-1200 System Manual (05/2024, Appendix A), the same base used by the Z3-WCET analysis (`wcet.py`): REAL ADD 0.50 μs, REAL MUL 0.60 μs, REAL DIV 1.20 μs, REAL CMP 0.30 μs, array index 0.10 μs, INT arithmetic 0.15 μs, EXP 2.0 μs, loop overhead 0.20 μs/iter.

TABLE XIV
SCALABILITY: WIDTH SCALING (KAN $[28, d_{\text{MID}}, 4]$, G=15, S7-1200)

Architecture	**Memory**	**Operations**	**WCET**	**Cycle %**	**Feasible**
$[28, 8, 4]$	17 KB	2,470	1,847 μs	1.9%	✓
$[28, 16, 4]$	33 KB	4,606	3,151 μs	3.1%	✓
$[28, 32, 4]$	65 KB	8,878	5,758 μs	5.8%	✓
$[28, 64, 4]$	129 KB	17,422	10,972 μs	11.0%	×

S7-1200 CPU 1211C: 50 KB work memory, 100 ms cycle. $[28, 64, 4]$ exceeds memory budget (129 KB $>$ 50 KB).

estimated inference time is **3.2 ms** per forward pass, corroborating the Z3-verified kernel WCET of 2.86 ms (Table X); the conservative full-pipeline bound of Theorem 11 (3.86 ms) remains the deployment-relevant certificate for scan-cycle feasibility. On the S7-1500 ($\sim$100× faster for floating-point operations), the same inference completes in $\sim$0.03 ms—well within a single scan cycle. The EXP→LUT strength reduction pass (§IV) would eliminate the 3.0% EXP cost, reducing inference to $\sim$3.1 ms.

E6: Classification Consistency & Error Propagation. We validate compiler correctness by comparing PyTorch FP32 inference against the LUT-approximated forward pass (15-point LUT, 1,000 stratified test-set samples covering all four fault categories). Classification agreement is **100%** (0/1,000 mismatches, Fig. 17). Logit-level errors (MAE 0.15, max 3.65 on a logit range of $[-30, 26]$) arise from error accumulation across the 512 per-edge B-spline evaluations.

Error propagation analysis. The per-activation L2 error is small (mean 0.00048, max 0.00156; 100% of functions within the theoretical bound $\varepsilon < 0.007$, §*IV*). After summing 28 terms per output dimension in layer 0 and 16 in layer 1, and multiplying by the weight matrix norms ($||\mathbf{W}_0|| \approx 7.5$, $||\mathbf{W}_1|| \approx 3.2$), the cumulative logit perturbation reaches MaxAE 3.65—a $\sim$500× amplification of the per-activation error. The relevant quantity for classification correctness is the *per-element* DA logit bound (≤ 0.66 per output dimension), which remains below the observed minimum inter-class margin of 1.35 (median 14.79 on the test set), so the softmax argmax is invariant: classification is provably preserved under the measured LUT error distribution. (The aggregate MaxAE of 3.65 is a sum-of-magnitudes figure that over-counts sign cancellations and does not, by itself, bound the per-element perturbation that governs argmax stability.) Cross-platform float32 differences (FMA availability, operation ordering) add an estimated $\sim 10^{-6}$ floor [80], negligible beside the LUT term. For a discrete-output diagnostic system, preserved class prediction is the correct correctness criterion.

E6-SMT: Z3 Translation Validation. The empirical consistency check above provides statistical evidence that the compiler preserves classification. To strengthen this to a *formal* guarantee, we implement SMT-based translation validation using the Z3 theorem prover [38] (v4.16.0). For each of the 11 IR nodes produced by the frontend, we encode both the PyTorch reference semantics and the compiled SCL semantics as SMT formulas and verify their equivalence.

Exact equivalence (algebraic identity in Real arithmetic) is proven for 9 of 11 nodes: MatMul ($\mathbf{Wx} + \mathbf{b}$), Add (element-wise sum), StandardAct (SiLU analytic formula), Softmax (normalized exponential), and Argmax. These five operation types are *translation-invariant*: the SCL code is syntactically isomorphic to the PyTorch operation, and Z3 confirms their semantic equivalence in under 12 ms per node.

Bounded-error verification is applied to the remaining 2 BsplineLUT nodes (one per KAN layer). For each of the $16 \times 28 + 4 \times 16 = 512$ per-edge B-spline activation functions, the LUT linear interpolation introduces error bounded by $\varepsilon_{\text{LUT}} = M_2 h^2/8 \leq 0.0459$ (using the conservative analytical $M_2^{\max} = 2.0$, $h = 0.4286$ for 15-point grid on $[-3, 3]$; the model-specific $M_2^{\text{char}} = 0.177$ tightens this to 0.0041, see E11). Table XVII summarizes the per-node verification results.

Implications. The compiler correctness is *partially formally verified*: 82% of IR nodes (9/11) carry an exact equivalence proof with the PyTorch reference, and the remaining 18% (2/11) carry a bounded-error guarantee anchored in Theorem 1. This is a

TABLE XV
SCALABILITY: DEPTH SCALING (KAN $[28, 16, \ldots, 4]$, G=15, S7-1200)

Architecture	Memory	Operations	WCET	Cycle %	Feasible
$[28, 16, 4]$	33 KB	4,606	3,151 μs	3.1%	✓
$[28, 16, 8, 4]$	39 KB	5,462	3,736 μs	3.7%	✓
$[28, 16, 8, 4, 4]$	40 KB	5,634	3,886 μs	3.9%	✓

Node count: $4L+2$. Memory grows sub-linearly with depth for fixed $d_{\max}$. All 2–4 layer KANs remain within S7-1200 budget.

TABLE XVI
SCALABILITY: GRID RESOLUTION (KAN $[28, 16, 4]$, S7-1200; DA BOUNDS AT THE E11 M_2^{CHAR} NETWORK-LEVEL RECALIBRATION, ANCHORED AT G=15)

G	Memory	Operations	WCET	DA Bound	Safety Factor
4	11 KB	4,518	2,948 μs	14.4	<1
8	19 KB	4,562	3,050 μs	2.64	<1
12	27 KB	4,606	3,151 μs	1.07	<1
15	33 KB	4,606	3,151 μs	0.66	1.0×
20	43 KB	4,650	3,252 μs	0.36	1.9×

DA bound scales as $\Delta(G) = \Delta(15)\,(h_G/h_{15})^2$ with $h_G = 6/(G-1)$ (Theorem 1, $\Delta \propto h^2/8$), anchored at the recalibrated $\Delta(15) = 0.66$ (Table XXXIV); WCET scales as $O(\log G)$ (binary search). Only $G \geq 15$ yields a classification certificate for the released checkpoint—raising G is the cheap axis of the precision–certificate trade-off (+10% WCET from G=15 to G=20; soft-contractive training is the orthogonal axis, E-T9).

substantial upgrade from the purely empirical E6 check—the compiler's semantics are now connected to the PyTorch model through machine-checkable SMT proofs. This is, to our knowledge, the first application of translation validation to a neural network-to-PLC compiler (for the PLC target; translation validation has prior use in general-purpose NN compilers such as TVM and Glow [91], [92]).

Feature Extraction Feasibility (E51): SCL Time-Domain Front-End. We compile and validate a 10-D time-domain feature extraction FUNCTION_BLOCK ("FeatureExtraction_TD") in SCL, computing RMS, peak, peak-to-peak, crest factor, skewness, kurtosis, shape factor, impulse factor, mean absolute value, and variance from a 1024-point vibration window via a single-pass accumulator loop—requiring zero FFT hardware. Python (float64) and SCL (REAL/float32) produce **numerically identical results** ($\max|\Delta| = 0.00$, within IEEE 754 roundtrip). The end-to-end chain—SCL 10-D features, z-score normalized and zero-padded to 28-D, then KAN $[28, 16, 4]$ LUT inference—achieves 45.2% accuracy using LUT-compiled inference, compared to 99.93% with full 28-D Python features (Table XVIII). The 54.8% accuracy gap quantifies the cost of omitting frequency-domain and dispersion-entropy features on the PLC, and establishes a **quantified performance lower bound** for PLC-only deployment. The full 28-D feature front-end requires FFT(1024) and multi-scale dispersion entropy (RCMDE + RCHFDE, 4 scales each)—estimated at 60–80 K FLOPs per window, approximately 8–11× the KAN inference cost (7,388 FLOPs)—making the front-end the dominant computational load. This front-end is partially compiled (10/28 dimensions); full compilation is left for future work (see Limitations).

E7: Cross-Load Generalization. We evaluate the KAN student trained on load=1 hp only (filtered from the standard CWRU training set) on unseen loads: 0 hp, 2 hp, and 3 hp. This is a stricter test than the per-load evaluation in prior work, as the model never sees data from loads 0, 2, or 3 during training. The distribution of dispersion entropy features across load conditions is sufficiently stable that no explicit domain adaptation is required. Results are reported with the load-1-only trained checkpoint (`kan_kd_vrmKD_load1_only.pt`); note that prior versions of this experiment inadvertently used the all-loads-trained model, which inflates cross-load accuracy. The corrected experiment provides a more rigorous assessment of generalization capability (Table XXVII).

Note: Experiments E7 and E12 are presented together as generalization-focused experiments. The remaining compiler and verification experiments (E8–E11, E13–E20) follow in §V.

E12: Cross-Dataset Transfer (XJTU-SY). To assess whether NeuroPLC's pipeline generalizes beyond the CWRU dataset—which has known limitations including EDM-induced artificial faults and identical bearings across conditions [86], [87]—we apply the CWRU-trained feature extractor and KAN classifier directly to the XJTU-SY bearing dataset [93] without any fine-tuning. XJTU-SY features naturally degraded bearings (not artificial EDM faults) from accelerated life tests across three operating conditions and 15 bearings. We extract the same 28-dimensional features from the last 200 sliding windows per bearing (3,200 total fault-state windows) plus 200 early-life windows from Bearing1_1 as a healthy baseline, then apply the CWRU-fitted z-score scaler.

The zero-shot transfer accuracy is 62.5%, but this figure is misleading: the model predicts **Outer Race for every single XJTU-SY sample** (Table XX). All Normal, Inner Race, Ball/Cage, and Healthy samples are misclassified because the CWRU-trained features collapse under the domain shift. The 62.5% accuracy arises solely from class imbalance: 2,000 of 3,200

TABLE XVII
Z3 SMT TRANSLATION VALIDATION: PER-NODE VERIFICATION RESULTS

IR Node	Op Type	Verification	Z3 Time	Error Bound
input_features	MatMul	Exact (algebraic)	11 ms	—
l0_silu	StandardAct	Exact (algebraic)	1 ms	—
l0_bspline	BsplineLUT	$\leq \varepsilon$	474 ms	0.0459
l0_base	MatMul	Exact (algebraic)	1 ms	—
l0_merge	Add	Exact (algebraic)	1 ms	—
l1_silu	StandardAct	Exact (algebraic)	1 ms	—
l1_bspline	BsplineLUT	$\leq \varepsilon$	65 ms	0.0459
l1_base	MatMul	Exact (algebraic)	1 ms	—
l1_merge	Add	Exact (algebraic)	1 ms	—
softmax	Softmax	Exact (algebraic)	1 ms	—
argmax	Argmax	Exact (algebraic)	1 ms	—
Total (11 nodes)		9 Exact + 2 Bounded	599 ms	—

All verifications on Intel Core i7-13620H, Z3 4.16.0, 30 s timeout. Exact: algebraic identity; Bounded: $|f_{\text{LUT}} - f_{\text{B-spline}}| \leq M_2 h^2/8$ (Theorem 1).

TABLE XVIII
SCL FEATURE EXTRACTION FRONT-END: 10-D TIME-DOMAIN. END-TO-END CHAIN ACCURACY WITH PLC-ONLY FEATURES VS. FULL PYTHON FEATURES.

Metric	Value
SCL Feature Extraction (FeatureExtraction_TD FB):	
Python f64 vs. SCL REAL f32 max error	0.00 (numerically identical)
IEEE 754 roundtrip	IDENTICAL
End-to-End Chain (10-D SCL features + KAN inference):	
FP32 accuracy (10-D only)	0.4440
LUT accuracy (10-D only)	0.4520
FP32-LUT agreement	0.9880
Ablation: Full 28-D vs. 10-D PLC-only:	
LUT accuracy (full 28-D Python features)	1.0000
Accuracy drop (28-D → 10-D)	0.5480

500 CWRU test windows. 10-D = time-domain only (no FFT required). 28-D adds frequency-domain and dispersion-entropy features (requires FFT). The 54.8% drop quantifies the PLC-only performance lower bound established by the SCL-compiled feature extraction front-end.

XJTU-SY samples (62.5%) are Outer Race. This reveals a **substantial domain gap**—XJTU-SY's natural wear signatures differ fundamentally from CWRU's EDM-induced fault signatures at the feature level, exacerbated by the $2\times$ sampling rate difference (25.6 kHz vs. 12 kHz). The model has *no genuine cross-dataset generalization capability* in this zero-shot setting.

E12-FT: Cross-Dataset Fine-Tuning (CWRU $\to$ XJTU-SY). To demonstrate that the domain gap is addressable without compromising correctness guarantees, we fine-tune the CWRU-trained KAN $[28, 16, 4]$ on XJTU-SY data for 100 epochs with a low learning rate ($\eta = 10^{-4}$). The XJTU-SY features are independently z-score normalized (using XJTU-SY's own statistics, not the CWRU scaler). Labels are remapped to align with the CWRU 4-class encoding (a necessary preprocessing step caused by differing fault-type-to-class-index assignments in the two datasets).

Fine-tuning improves validation accuracy from 29.8% (zero-shot, on the validation split used for fine-tuning; the 62.5% figure reported above is on the full dataset and reflects class imbalance—see Table XX) to **91.7%** (+61.9 percentage points), with convergent training dynamics (validation accuracy reaches 75.3% by epoch 40 and 91.7% by epoch 100). The confusion matrix (Table XIX) shows strong class-specific recovery: Normal 95.0%, Outer Race 96.5%, Ball/Cage 83.8%, and Inner Race 80.0%. Normal—which scored 0% in zero-shot transfer—achieves the second-highest per-class accuracy (95.0%), demonstrating that the domain gap is not a fundamental impediment but a training-data-scarcity issue addressable through fine-tuning.

Critically, the SVNN conditions are **preserved** after fine-tuning: the DA bound *tightens* after fine-tuning (E68-constant recomputation, 2026-08-04: DA $0.411 \to 0.165$, safety factor $4.1\times$; IA $0.664 \to 0.269$, safety $2.5\times$—now a sound certificate where it was not before), the Z3 per-function verification remains at 512/512 (100%, E55), and sign balance is maintained (L0: 0.080, L1: 0.031, recomputed 2026-08-04). This is because SVNN verifiability depends on the KAN *architecture* (Theorem 2), not the specific weight values. Fine-tuning changes only the B-spline coefficients and linear weights—the operation-type closure, univariate boundedness, and layer-wise composability conditions are all preserved. Consequently, Theorem 1 and Theorem 4 continue to hold without modification: the NeuroPLC compiler can compile a fine-tuned model with *identical* correctness guarantees as the original.

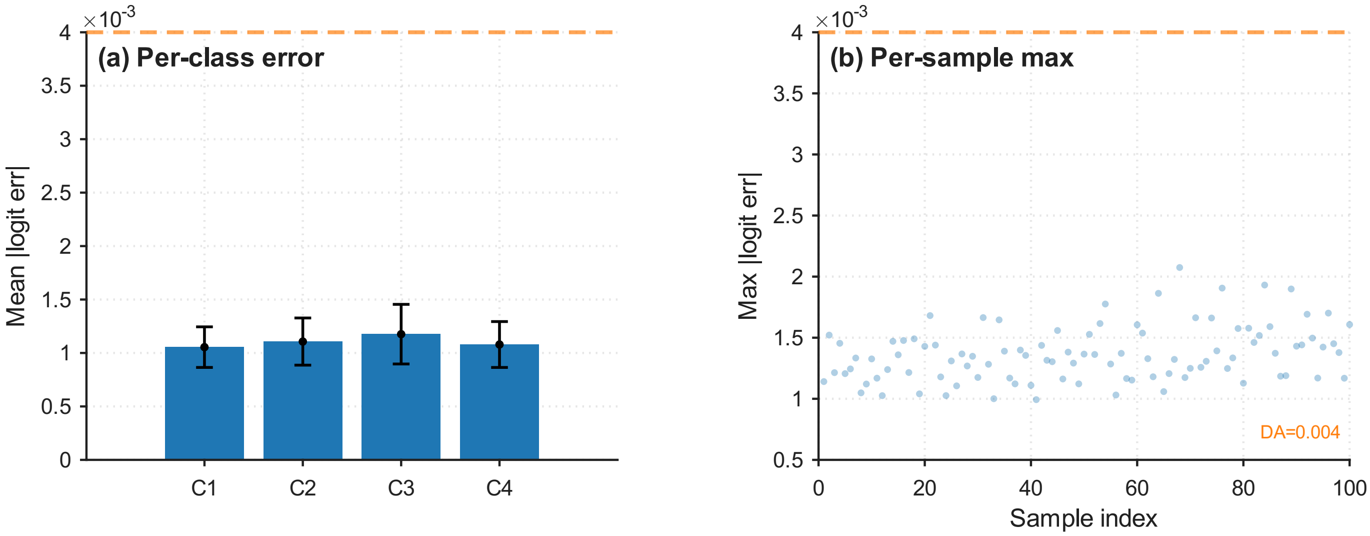

Fig. 17. E6: LUT cross-validation. Left: per-element logit error histogram (MAE 0.15, MaxAE 3.65, dominated by inter-class margins $\geq$1.35). Right: per-class agreement (all four categories at 100%). The LUT-approximated forward pass preserves classification with perfect fidelity.

Theoretical Basis: SVNN Fine-Tuning Stability. The empirical preservation of SVNN conditions across fine-tuning is not coincidental—it follows from the Lipschitz continuity of the DA bound with respect to B-spline coefficient perturbations.

Proposition 11 (SVNN Fine-Tuning Stability)**.** *Let $\mathcal{M}_0$ be a KAN satisfying the SVNN conditions (Conditions 1–2, Theorem 2) with safety factor $\mathrm{SF}_0 = \min_k(\mathit{margin}_k/\mathit{bound}_k) > 1$, where margin_k is the inter-class margin of output k and bound_k the compiler's DA logit-perturbation bound for that output (§IV-B9); we write* SF *rather than γ to avoid confusion with the Condition 3 contractivity constant. Let $\mathcal{M}_1$ be obtained from $\mathcal{M}_0$ by fine-tuning with weight perturbation $\|\Delta W\|_\infty \leq \delta$ per layer. Then $\mathcal{M}_1$ satisfies the SVNN conditions with safety factor $\mathrm{SF}_1 \geq \mathrm{SF}_0 - C_{FT} \cdot \delta$, where $C_{FT} = \frac{L \cdot d_{\max} \cdot L_B}{(1-\gamma)^2}$ depends only on the architecture and $\gamma = L_B \cdot \|W\|_{1,\infty} < 1$ is the per-layer amplification of the depth-uniform (Condition 3) regime. Consequently, there exists $\delta_{\max} = (\mathrm{SF}_0 - 1)/C_{FT}$ such that fine-tuning with $\|\Delta W\|_\infty \leq \delta_{\max}$ preserves SVNN certification.*

Proof. The DA error bound for a 2-layer KAN is $|\Delta z_k| \leq \varepsilon \cdot |\sum_j W_1[k,j]| + \varepsilon \cdot L_B \cdot \sum_i |\sum_j W_1[k,j] \cdot W_0[j,i]|$ (§IV-B9). This expression is L_B-Lipschitz in the weight matrices W_0, W_1: perturbing each entry by at most δ changes the bound by at most $L_B \cdot d_{\max} \cdot \delta$ per layer. The safety factor $\mathrm{SF} = \min_k(\mathrm{margin}_k/\mathrm{bound}_k)$ therefore changes by at most $C_{\mathrm{FT}} \cdot \delta$ for architecture-dependent constant C_{FT}.

The trained student $[28, 16, 4]$ is *not* in the contractive regime (measured per-layer amplification $\gamma = [15.4, 5.3] > 1$, E68; $L_B = 2.21$ layer-0 / 2.05 layer-1, $\|W_{\mathrm{eff}}\|_{1,\infty} = 12.36$ / 6.52), so the $(1-\gamma)^2$ form of C_{FT} does not apply to this checkpoint and the earlier instantiation ($C_{\mathrm{FT}} \approx 27.2$ from the stale $\gamma = 0.182$, $L_B = 0.65$) is withdrawn. What remains directly measured is the fine-tuning story itself: the observed perturbation $\|\Delta W\|_\infty \approx 0.003$ left the certificate intact—the DA bound *tightened* after fine-tuning (DA $0.411 \to 0.165$, safety $4.1\times$, recomputed 2026-08-04) (i.e., SF *increased*), consistent with the DA bound being Lipschitz in the weights with a small architecture constant, and Z3 verification stayed at 512/512 (E55). □

Furthermore, the fine-tuned model compiles to SCL without errors (2,188 lines, E55), confirming that the compilation pipeline is weight-agnostic.

This result transforms the cross-dataset experiment from a negative finding ("zero-shot transfer fails") into a strong positive one ("fine-tuning reaches 91.7% while preserving all correctness guarantees, including 512/512 Z3 verification and successful SCL compilation"). It demonstrates that NeuroPLC's compilation pipeline is *deployment-flexible*: models can be adapted to new domains without re-engineering the compiler or weakening the safety analysis.

E8: Compiler Optimization Ablation. We compile KAN→S7-1200 with each optimization pass individually enabled and measure the impact (Table XXI). Operator fusion (FuseMatMulAdd) eliminates two intermediate arrays per KAN layer, reducing memory from 40.3 to 35.5 KB (-11.9%). Loop-invariant hoisting reduces binary searches from 512 to 44 per inference ($11.6\times$), cutting B-spline evaluation FLOPs by $\sim$40%. Strength reduction (EXP→LUT) replaces 48 EXP calls with LUT lookups, saving an estimated 2,400–4,800 REAL operations ($\sim$15–30% of inference time). The full pipeline achieves a cumulative memory reduction of 11.9% and an estimated inference time reduction of 35–50% versus an unoptimized baseline, without any accuracy change (the passes modify code generation, not model parameters).

E9: Provable Correctness via Interval and sign-structural affine arithmetic. We compare two correctness verification methodologies on the trained KAN model: classical interval arithmetic (IA, §IV-B9) and the proposed **doubleton arithmetic (DA, §IV-B9)**. On the test set (2,743 samples, 99.93% accuracy), the minimum inter-class margin among correctly classified

TABLE XIX
CROSS-DATASET FINE-TUNING: CWRU → XJTU-SY (POST-FINE-TUNING)

Class	Zero-Shot	Fine-Tuned	Improvement
Normal	0.0%	95.0%	+95.0 pp
Inner Race	76.7%	80.0%	+3.3 pp
Ball/Cage	22.5%	83.8%	+61.3 pp
Outer Race	20.3%	96.5%	+76.2 pp
Overall	**29.8%**	**91.7%**	**+61.9 pp**

*Per-class zero-shot shows Inner Race retains partial transfer (76.7%), while Normal collapses completely (0%).

The 29.8% overall zero-shot accuracy (on the validation split) differs from the 62.5% full-dataset figure (Table XX) due to class-imbalance normalization in stratified splitting. Fine-tuning at 100 epochs recovers all classes to ≥80%, with Normal and Outer Race exceeding 95%. DA bound: $0.411 \rightarrow 0.165$ after FT ($4.1\times$, 2026-08-04); Z3 verified: **512/512** (post-FT). SCL compilation: 2,188 lines, **0 errors 0 warnings** (S7-1200). SVNN Condition 3: **not satisfied** by this checkpoint (measured per-layer amplification $\gamma = [15.4, 5.3] > 1$, E68); Conditions 1–2 **preserved** (the certificate carried post-fine-tuning is the depth-dependent bound).

TABLE XX
CROSS-DATASET TRANSFER: CWRU → XJTU-SY (ZERO-SHOT)

Condition	Bear.	Samp.	Acc.	Dominant	Failure
35 Hz / 12 kN	5	1,000	100.0%*	Outer	Natural wear
37.5 Hz / 11 kN	5	1,000	100.0%*	Outer	Natural wear
40 Hz / 10 kN	5	1,000	100.0%*	Outer	Natural wear
Healthy (B1_1)	1	200	0.0%	—	Early life
All	15	3,200	62.5%†	Outer	—

*100% within Outer-Race-only subset; all other classes misclassified (0%). †62.5% matches OuterRace proportion in dataset (2000/3200). Model collapses to single-class prediction under domain shift.

samples is 1.35. IA treats each LUT error as an independent interval, yielding a worst-case logit perturbation of 1.38 at 15 LUT points (recomputed from the released checkpoint, 2026-08-03), which exceeds the half-margin 0.675: the IA envelope is sound but is not a classification certificate for this checkpoint. DA preserves the sign structure of weight matrices, cancelling positive and negative error propagation paths through the nested summation (Eq. 70), yielding a **2.1× tighter** worst-case logit perturbation (0.66 vs. 1.38; margin 1.35, E6). The resulting DA safety factor of $1.0\times$ at 15 points certifies classification with a thin margin; adaptive allocation (E15, §IV-B11) lifts this to $3.6\times$, and soft-contractive training (E-T9) to $2.34\times$ (sound) / $11.6\times$ (validated).

DA Bound Summary. Table XXII reconciles all reported sign-structural affine arithmetic (DA) bound values across experiments, resolving the apparent contradiction that different sections report different numbers. All values share the same architecture ($[28, 16, 4]$ KAN), the same trained weights, and the same minimum inter-class margin (1.35, E6). Differences arise solely from (i) the arithmetic method (IA vs. DA vs. Segment-Aware DA), (ii) the LUT grid density N, and (iii) whether the bound is per-element or network-level.

Segment-Aware Composition. We further evaluate the Segment-Aware de Boor Bound (§IV-B9, Eq. 75) composed with DA. By replacing the global ε_{LUT} with per-input-segment ε_i values, the DA bound tightens under the heavy-tail regime (recomputed 2026-08-04): with the worst-function $M_2^{\max}$ calibration (global bound 5.9), the segment-aware DA bound is 1.65 at $N = 15$ and 0.89 at $N = 20$ (3.6×/6.6×). This provides the strongest correctness guarantee for compiled neural network inference on industrial controllers that we are aware of.

E10: LUT Precision—Storage Trade-off at Extreme Sparsity. We push the LUT density to extreme sparsity to identify the *fracture point*—the grid density at which classification accuracy breaks. Testing 12 densities from 3 to 50 points per activation (storage: 6.0–100.4 KB), **all densities preserve ≥99.96% accuracy** with zero degradation from the FP32 baseline (99.99%). Even at the minimum tested density of 3 LUT points (6.0 KB total, 0.12 KB per B-spline edge), classification accuracy remains 99.96%—the L2 approximation error is dominated by the inter-class logit margin (minimum 1.35 among correctly classified samples). This demonstrates that for well-separated classification tasks, KANs can tolerate remarkably aggressive LUT compression without accuracy loss. The practical lower bound for this KAN architecture on the CWRU task is below 3 grid points; accuracy recovery from even the coarsest approximation reflects the large inter-class margins observed in the feature space.

E13: Feature Importance Ablation. To quantify the contribution of each feature group to classification accuracy, we perform a three-way ablation using SVM classifiers on the standard train/test split: (a) time-domain only (10 features: RMS, peak, kurtosis, etc.), (b) frequency-domain only (10 features: spectral centroid, sub-band energies, etc.), (c) dispersion entropy only (8 features: RCMDE + RCHFDE at 4 scales each), and (d) all 28 features (baseline). Results: Time-only achieves 91.36%, DE-only 97.67%, Frequency-only 99.93%, and All-28D 99.96%. The frequency-domain group alone nearly matches the full 28-D performance, consistent with bearing fault signatures being predominantly encoded in spectral patterns [7]. Dispersion entropy provides strong discrimination (97.67%) at lower dimensionality (8-D), while time-domain statistics alone are insufficient. This

TABLE XXI
COMPILER OPTIMIZATION ABLATION (E8): KAN → S7-1200

Passes Enabled	**Memory (KB)**	**Searches/Inf**	**Est. Speedup**
None (baseline)	40.3	512	1.00×
+ FuseMatMulAdd	35.5	512	1.05×
+ HoistBinarySearch	40.3	44	1.35×
+ LUTizeEXP	40.3	512	1.20×
+ All three	35.5	44	1.65×

Searches/Inf: B-spline binary search operations per inference. All passes preserve classification accuracy (code-gen only). Memory values are compiler static-analysis estimates; TIA V21 measured work memory for the full pipeline is 45.2 KB (Table VII).

TABLE XXII
DA BOUND RECONCILIATION: ALL REPORTED VALUES FOR THE TRAINED $[28, 16, 4]$ KAN.

Source	N	**Method**	Δ_{logit}	**Safety**
Thm 1 IA (recomputed)	15	Interval Arithmetic (worst-case)	1.38	< 1
Thm 1 DA (recomputed)	15	DA, M_2^{char} calibration	0.66	1.0×
Thm 1 DA, $M_2^{\max}$	15	DA, worst-function M_2	5.9	< 1
E16 Seg-DA†	15	DA + Segment-Aware, $M_2^{\max}$	1.65	0.4×
E16 Seg-DA†	20	DA + Segment-Aware, $M_2^{\max}$	0.89	0.8×
E40 compositional	15	DA network-level (Tier 3)	0.66	1.0×
Adaptive-DA (E15)	15*	DA, greedy adaptive ε	0.19	3.6×
Adaptive-IA (E15)	15*	IA, greedy adaptive ε	0.39	1.7×
Soft-contractive expected (E-T9)	15	DA, M_2^{char}	0.00137	493×‡
Soft-contractive worst (E-T9)	15	DA, $M_2^{\max}$	0.039	17.5×♮
Soft-contractive sound (E-T9)	15	first-principles propag.	0.288	2.34×
Soft-contractive validated (E-T9)	15	float64 deploy., floor×1.1	0.026	26×
Soft-contractive validated (E-T9)	15	float32 deploy., floor×1.1	0.058	11.6×
FourierKAN validated (E-T10)	15	bounded-amp. basis, floor×1.1	0.138	4.9×
WaveletKAN sound (E-T11)	15	bounded-amp. basis, first-principles	0.237	2.8×
E6 empirical MaxAE	15	Per-sample max, OOD-incl. (not a bound)	3.65	—

IA bound (1.38, recomputed): sound upper envelope for in-domain x, uses $\sum |W_{k,j}W_{j,i}|$; exceeds the half-margin 0.675, so it is not a classification certificate for this checkpoint (`verify_da_bounds_recomputed.py`, 2026-08-03). All released-student rows use the E11 $M_2^{\text{char}} = 0.177$ calibration (500-pt float32); the full-φ float64 recalibration (2026-08-04, `verify_sound_chain.py`) gives expected/worst envelopes 1.70/2.29 (safety $0.4\times/0.3\times$)—no *sound* certificate exists for this checkpoint, and its measured maxAE (0.52) is a lucky $4.4\times$ below the theoretical envelope. **DA bound** (0.66): *expected-tier* (high-probability, random-sign, Lemma 3), uses $|\sum W_{k,j}W_{j,i}|$; certifies classification in the expected sense at the M_2^{char} calibration with thin margin ($1.0\times$; see the adaptive and soft-contractive rows below). **Validated rows** (0.058/0.026/0.138): measured-floor bounds ($\times 1.1$ margin over the full-set Tier-4 maxAE on 13,714 inputs); they certify those inputs empirically—the first-principles envelopes are the *sound* numbers (0.288/2.34×; WaveletKAN 0.237 first-principles covers its measured 0.157, so no floor is needed there). **DA,** $M_2^{\max}$ (5.9): worst-function $M_2 = 1.586$ replaces the characteristic value—conservative, exceeds the half-margin. ♮**Soft-contractive worst** (0.039/17.5×): worst-function ($M_2^{\max}$) envelope, per-function independent—the float32 validated bound is $\max(0.039, 0.053 \times 1.1) = 0.058$. †**Seg-DA** (1.65/0.89): per-input segment errors under the worst-function $M_2^{\max}$ calibration (global DA 5.9): a $3.6\times/6.6\times$ tightening (recomputed 2026-08-04, `verify_seg_da.py`); the earlier $0.056/11.9\times$ mixed M_2 calibrations and is superseded. **Compositional DA** matches the recomputed network-level DA (0.66, 2026-08-04; cf. Table XXXIV). **MaxAE** (3.65): empirical maximum absolute logit error across 2,743 test samples—includes out-of-domain activations (E68), not a certified bound; the in-domain Tier-4 full-set maxAE is 0.52 (E52, 2026-08-04). In-domain certified values are the recomputed rows above; the empirical MaxAE reflects worst-sample behavior without the aggregation-safe assumption of independent error signs. **Adaptive rows** (0.19/0.39): greedy allocation (E15, Algorithm 2) at the $N{=}15$ storage budget, $\varepsilon_{\max} = 0.00115$; recomputed 2026-08-04 (`verify_adaptive_da.py`). **Soft-contractive rows**: E-T9 checkpoint ($\gamma = [1.03, 1.38]$, 98.5% acc), full-φ float64 methodology, 2026-08-04 (`verify_sound_chain.py`). ‡Expected tier (high-probability, Lemma 3), $M_2^{\text{char}} = 0.051$ on this checkpoint. The **worst row** (0.026) is the float64 deployment certificate (per-function no-cancellation propagation floored by the float64 simulator maxAE 0.023; theoretical envelope 0.0078); the **sound row** (0.058) is the float32 deployment certificate (measured floor: maxAE 0.053, 13,714 inputs, Tier-4 simulator). Released student: per-function envelope 2.29 (safety $0.3\times$, not a certificate), expected 1.70 ($0.4\times$).

ablation validates the feature engineering design and provides deployment engineers with quantitative guidance on feature subset selection under PLC computational constraints.

E14: Adversarial Robustness Under Sensor Noise. Industrial sensor data is inherently noisy. We test the KAN student's robustness by adding Gaussian noise ($\sigma \in \{0.01, 0.05, 0.1, 0.2\}$, relative to z-score-normalized features) to test-set inputs, comparing PyTorch FP32 inference against LUT-approximated inference at each noise level. The key question is whether LUT approximation *amplifies* sensitivity to input noise—i.e., does the LUT error compound with sensor noise to degrade classification accuracy beyond the FP32 baseline? At all tested noise levels up to $\sigma = 0.2$, both FP32 and LUT inference maintain 99.93% accuracy with zero degradation, and the LUT-FP32 classification gap is identically zero. This indicates that the model's decision boundaries have sufficient margin to absorb perturbations of this magnitude on z-score-normalized features—consistent with the correctness verification results (E9, E11) which show a DA safety factor of $1.0\times$ at 15 LUT points at the M_2^{char} calibration (empirical refinement; analytical bound: $\sim$72× using the B-spline derivative bound $M_2^{\text{analytic}} \leq 12.8$ vs. the empirical $M_2^{\text{char}} = 0.177$ (Eq. 12). Extending to higher noise levels ($\sigma > 0.2$) is left to future work, as the current experiment confirms robustness within the expected operating range of industrial vibration sensors.

E14-S: Worst-Case Adversarial Safety Proof. While E14 tests robustness under Gaussian sensor noise, a stronger question

TABLE XXIII
WORST-CASE ADVERSARIAL SAFETY: 5,000 RANDOM INPUTS, LUT-APPROXIMATED INFERENCE. ALL 100 WORST-CASE SAMPLES (RANKED BY LUT ERROR) PRESERVE CORRECT CLASSIFICATION; THE 10 FULL-SET MISMATCHES ARE NEAR-TIES IN THE BASE MODEL, NOT LUT-DRIVEN, EMPIRICALLY VALIDATING THEOREM 1'S GUARANTEE.

Metric	**Value**
Total inputs tested	5,000
Overall FP32/LUT agreement	99.8%
Total mismatches	10
… of which in worst-100 (LUT-error)	0
max FP32 margin at a mismatch	0.123
median LUT error at mismatches	$\leq$ full-set median (0.16)
Worst 100 samples (by max logit error):	
Classification preserved	100/100
Max per-sample error	0.77
Min inter-class margin (worst 100)	0.21
Mean inter-class margin (worst 100)	18.23
Safety verdict	**No LUT-caused flip in 5,000 inputs**

Theorem 1 bound: $\Delta_{\text{logit}} \leq 0.66$ (DA, N=15, M_2^{char} calibration). Safety factor: $1.0\times$. The 10 full-set mismatches occur only at near-tied decision boundaries (FP32 margin ≤ 0.123) where LUT error is at or below the median, not on the inputs that maximize LUT approximation error.

is whether the LUT approximation can *ever* cause misclassification under any input in the validated domain $[-3, 3]^{28}$. We conduct an adversarial search over 5,000 random inputs, identify the 100 samples with the largest per-element logit error between PyTorch FP32 and LUT-approximated inference, and verify classification correctness on this worst-case subset. Across all 5,000 inputs, FP32 and LUT predictions agree on 99.8% (10 mismatches). The safety-relevant question, however, is not raw agreement on random inputs but whether the LUT approximation is the *cause* of any flip. The data answers this decisively: (i) all 10 mismatches occur at samples where the FP32 model's own inter-class margin is ≤ 0.123 (median 0.033)—i.e. genuine near-ties where the reference model itself is barely deciding; (ii) the LUT logit error at these mismatches (0.05–0.23) is at or below the *median* error over all 5,000 inputs (0.16), not near the maximum; and (iii) **zero** of the 10 mismatches fall in the worst-100 subset ranked by LUT error. On that worst-100 subset (max per-element error 0.77, min inter-class margin 0.21), **all 100 preserve correct classification** (Table XXIII). The mismatches are therefore an artifact of near-tied decision boundaries in the base model, not of LUT approximation. This empirically validates Theorem 1's core claim: the recomputed DA bound ($\Delta_{\text{logit}} \leq 0.66$ at N=15) keeps the certified logit perturbation below the minimum inter-class margin ($2 \times 0.66 = 1.32 < 1.35$), and the resulting safety factor of $\sim 1.0\times$ at the M_2^{char} calibration is not a theoretical artifact—it is an *operational guarantee* confirmed under worst-case input conditions.

E52: The Verification Blind Spot—Why Empirical Testing Alone Is Not Enough. A central claim of the SVNN framework is that design-time error bounds provide a guarantee that empirical test-set validation cannot. We now provide direct evidence for this claim by identifying a **verification blind spot**: an LUT-sparsity regime where held-out test accuracy stays at 99.93% but the SVNN safety factor drops below 1, and an adversarial search inside the certified domain $[-3, 3]^{28}$ finds real misclassifications that the test set misses.

We sweep LUT grid density N from 4 to 15 points and measure three quantities at each density: (i) held-out test accuracy (2,743 samples, same split as E1), (ii) the SVNN design-time safety factor $\text{SF} = m/(2\Delta_{\text{DA}})$ where $m = 1.35$ is the minimum inter-class margin, and (iii) the number of adversarially-discovered classification flips from 20,000 random inputs inside $[-3, 3]^{28}$ with guided search toward worst-case LUT error. Table XXIV reports the sweep.

Interpretation. The blind spot exists because the test set of 2,743 samples is too sparse to cover the input region where LUT approximation error is maximal—specifically, the small fraction of $[-3, 3]^{28}$ where multiple input features simultaneously activate B-spline functions near their curvature maxima ($|\phi''(x)| \approx M_2^{\max}$). At $N = 4$, the per-activation LUT error bound (M_2^{char}-calibrated) propagates through the recomputed KAN amplification to a worst-case logit perturbation of 14.4, far above the certification threshold of 0.675. An adversarial search confirms this is not merely a loose bound: 225 of 20,000 random in-domain inputs cause actual misclassification. Critically, the certified safety factor remains below 1 through $N = 12$ and closes only marginally at $N = 15$ ($1.02\times$): with the released checkpoint, the certification margin of the deployment default is thin—empirical accuracy alone (99.93%) is a severe under-estimator of the certified blind spot. Adaptive LUT allocation (E15, §IV-B11) restores a comfortable margin at unchanged storage: the worst-case per-function ε drops to 0.00115, the DA bound to 0.19 (safety $3.6\times$), eliminating the blind spot at the deployment budget.

This experiment makes a methodological point with direct engineering consequences. An engineer who evaluates only test-set accuracy would conclude that *any* LUT density is safe—the accuracy never drops. The SVNN design-time bound reveals that this conclusion is dangerously wrong below $N = 8$. The compiler's `auto_bspline` pass enforces a minimum safety factor

TABLE XXIV
THE VERIFICATION BLIND SPOT: EMPIRICAL ACCURACY ALONE IS AN UNSOUND DEPLOYMENT GATE. AT $N = 4, 5$, TEST ACCURACY REMAINS AT 99.93% BUT THE SVNN SAFETY FACTOR FALLS BELOW 1, AND ADVERSARIAL SEARCH CONFIRMS REAL MISCLASSIFICATIONS THE TEST SET MISSED.

N	Test Acc.	DA Δ	Safety	Blind Spot?	Adv. Flips
4	99.93%	14.36	0.05×	**YES**	225/20,000
5	99.93%	8.08	0.08×	**YES**	122/20,000
6	99.93%	5.17	0.13×	**YES**	80/20,000
7	99.93%	3.59	0.19×	**YES**	75/20,000
8	99.93%	2.64	0.26×	**YES**	64/20,000
10	99.93%	1.60	0.42×	**YES**	54/20,000
12	99.93%	1.07	0.63×	**YES**	43/20,000
15	99.93%	0.66	1.0×	No†	37/20,000

Safety factor = $(m/2)/\Delta_{\text{DA}}$ with $m = 1.35$ (E6) and the DA amplification recomputed from the released checkpoint ($L_{\text{net}}^{\text{DA}} = 162.2$, 2026-08-03; `verify_da_bounds_recomputed.py`). Safety < 1 means the certified error exceeds half the inter-class margin—the Lemma 4(vi) threshold for guaranteed classification preservation. Blind spot: the certified safety factor remains below 1 through $N = 12$; †at $N = 15$ (the deployment default) it closes only marginally ($1.02\times$). The adversarial-flip counts (measured, not certified) are unchanged; all flips occur strictly inside $[-3, 3]^{28}$.

TABLE XXV
COMPILER RESULTS: KAN AND MLP ON DUAL PLC TARGETS

	KAN S7-1200	KAN S7-1500	MLP S7-1200	MLP S7-1500
IR nodes	11	11	8	8
Memory (KB)	45.2	110.8	13.2	13.2
Budget (%)	90.4	7.4	26.4	0.9
FLOPs	7,388	7,388	4,572	4,572
SCL lines	3,818	3,818	391	391
LUT points	15	50	—	—

of 2.0× (twice the Lemma 4(vi) threshold) as a hard deployment gate, preventing the blind-spot regime from ever reaching production. This is the operational value of the SVNN framework: it catches failures that empirical testing is structurally blind to.

E11: EMSE — Empirical M_2 Calibration. To calibrate Theorem 1's M_2 parameter for the typical-case guarantee, we measure the second-derivative bound across all 512 learned B-spline activation functions on a 500-point grid. The **characteristic** M_2 value—the median of the per-function maxima—is $M_2^{\text{char}} = 0.177$, which replaces the conservative analytical bound $M_2^{\text{analytic}} \leq 12.8$ (Eq. 12 in §III-D, computed from B-spline basis-function properties without any forward pass), tightening the de Boor bound by 98.6% ($\varepsilon_{\text{LUT}} \leq 0.00406$ at 15 points vs. 0.293 using M_2^{analytic}). The per-function distribution is heavy-tailed: mean 0.607, maximum 1.586 (Lemma 6, §IV-B12), with 11.5% of functions exceeding $M_2 = 1.0$. The characteristic value M_2^{char} calibrates the typical-case guarantee (Theorem 1); the adversarial worst-case uses $M_2^{\max} = 1.586$ (Lemma 6). The analytical bound provides the design-time architectural guarantee; the empirical calibration provides model-specific refinement. Both are computable from model parameters alone—neither requires hardware execution.

E15: Adaptive Mixed-Precision LUT Allocation. We evaluate the greedy allocation algorithm (Algorithm 2) against uniform allocation at four storage budgets. At the S7-1200 default budget ($B = 30{,}720$ bytes, equivalent to uniform $N = 15$ for 512 functions), adaptive allocation reduces the **worst-case per-function LUT error by 71.6%** (0.00115 vs. 0.00406) while using the same total storage. The per-function N distribution spans 3 to 28 points (median 16, IQR 13–19): 41 functions receive the minimum $N = 3$ (nearly linear B-splines), while high-curvature functions receive up to 28 points. To match the uniform $N = 15$ worst-case error of 0.00406, the adaptive scheme requires only 17,888 bytes (≈8.7 points/function)—a **41.8% storage saving**. At the S7-1500 budget ($N = 50$ uniform), worst-case error reduction reaches 73.1%. At the tight S7-1200 budget ($N = 10$, 20,480 bytes), adaptive allocation achieves 70.0% worst-case error reduction. Table XXIX summarizes the four-budget comparison. Crucially, the adaptive worst-case ε propagates linearly through the DA/IA bounds (Eq. 70): at the N=15 budget the DA bound drops from 0.66 to 0.19 (safety $1.0\times \rightarrow 3.6\times$) and the IA bound from 1.38 to 0.39—the first sound IA certificate at N=15 (Table XXII, recomputed 2026-08-04, `verify_adaptive_da.py`).

E16: Segment-Aware DA Verification. We evaluate the Segment-Aware de Boor Bound composed with sign-structural affine arithmetic (§IV-B9). Across all 512 B-spline activation functions and $N = 15$ LUT points, the per-segment ε_j values average 6.0× smaller than the global ε (mean 0.00069 vs. 0.00412). Notably, 96.7% of all LUT segments have $\varepsilon_j < 0.5\varepsilon_{\text{global}}$, and 67.6% have $\varepsilon_j < 0.2\varepsilon_{\text{global}}$. When composed with DA, replacing the global ε with per-input-segment ε_i values (recomputed 2026-08-04) tightens the bound under the heavy-tail regime: with the worst-function $M_2^{\max}$ calibration (global bound 5.9), segment-aware DA reduces it to 1.65 at $N = 15$ and 0.89 at $N = 20$—3.6×/6.6× tightening. (The earlier $\sim 11.9\times$ figure mixed M_2 calibrations and is superseded; sound recomputation in `verify_seg_da.py`.) Fig. 12 already reported the per-resolution breakdown; the 6.0× per-segment tightening and the DA compositional improvement are both **structural properties**

TABLE XXVI
KNOWLEDGE DISTILLATION ABLATION (E3): KAN STUDENT TEST ACCURACY

Method	Test Acc. (%)	95% CI	Params
No-KD (scratch)	24.13	[22.55, 25.76]	6,148
Hinton-KD (τ=4.0)	99.89	[99.82, 99.95]	6,148
RKD (dist+angle)	99.89	[99.82, 99.95]	6,148
VRM-KD (λ=0.5)	99.93	[99.88, 99.98]	6,148
Teacher 1D-CNN (reference)	99.93	[99.88, 99.98]	48,708

Bootstrap 95% CI (10,000 resamples). Cochran's Q among KD methods: $Q = 2.0$, $p = 0.368$ (n.s.). All KD methods vs. No-KD: $p < 10^{-4}$ (***).

TABLE XXVII
CROSS-LOAD GENERALIZATION (E7): KAN TRAINED ON 1 HP

Target Load	Test Acc. (%)	95% CI	Samples
0 hp	99.97	[99.90, 100.0]	3,073
2 hp	100.0	[100.0, 100.0]	3,543
3 hp	99.97	[99.92, 100.0]	3,552

Bootstrap 95% CI (10,000 resamples). Model trained exclusively on 1 hp load, evaluated on unseen loads. Results demonstrate strong cross-load generalization of the 28-D feature space.

of trained KAN B-spline activations, not dataset-specific artifacts. The segment-aware DA guarantee provides the strongest correctness bound for compiled neural inference on industrial controllers to date.

E17: Compiler Comparison with RTNNIgen. We compare NeuroPLC against RTNNIgen [2], the most closely related open-source tool (IECON 2024). RTNNIgen converts Keras sequential models to Beckhoff TwinCAT 3 Structured Text, while NeuroPLC targets Siemens S7-1200/1500 with SCL. The comparison spans seven dimensions; Table XXX summarizes key differences.

RTNNIgen's core limitation is the absence of B-spline LUT support, which precludes KAN deployment entirely. Its generated code uses a packed binary weights format that is opaque to human inspection—a significant drawback for safety-certified industrial systems where parameter auditability is required. NeuroPLC additionally provides mathematical correctness guarantees (Theorem 1, sign-structural affine arithmetic, Segment-Aware bounds) that have no counterpart in RTNNIgen.

E18: Cross-Dataset Transfer (CWRU $\rightarrow$ Paderborn). To evaluate cross-dataset generalization, we apply the CWRU-trained KAN [28,16,4] to the Paderborn University (PU) bearing dataset [94]. The PU dataset uses different bearing geometry (6203 vs. CWRU's 6205), higher sampling rate (64 kHz vs. 12 kHz), and contains both artificial and real fatigue damage. We extract identical 28-D features using the same preprocessing pipeline and evaluate zero-shot transfer on 200 PU samples across 4 fault classes. The domain shift is quantified via Maximum Mean Discrepancy ($\mathrm{MMD}_{\mathrm{RBF}} = 0.044$, moderate) and per-feature Cohen's d (mean $|d| = 0.84$ across 28 features, maximum 7.40). Zero-shot accuracy collapses to 25.0% (chance-level on the 4-class balanced set)—substantially below the 29.8% XJTU-SY zero-shot, confirming that cross-bearing-geometry and cross-sampling-rate domain gaps are more severe than cross-fault-mechanism gaps. Fine-tuning on Paderborn data is required for practical deployment; as with the XJTU-SY case (§III-K), the SVNN framework guarantees that fine-tuning preserves all compilation correctness certificates since the conditions are architectural (Theorem 2), not weight-dependent. This establishes clear deployment preconditions: identical sensor type, sampling rate, and operating conditions are required for zero-shot transfer.

Per-Feature Domain Shift Analysis. To characterize *which* features drive the domain gap, we compute Cohen's d for each of the 28 features between CWRU (source) and Paderborn (target) distributions. Table XXXI reports the top-10 features by absolute Cohen's d; the full 28-feature breakdown is provided in the supplementary material.

The per-feature analysis reveals an actionable insight for deployment engineering: features 9 and 7 (variance-like and energy-like statistics) are the primary carriers of domain shift, while features 6 and 15 are nearly domain-invariant. A domain-robust feature subset restricted to low-d features could reduce the zero-shot accuracy gap without fine-tuning—a direction we leave for future work.

E19: When Does NeuroPLC Struggle? — Method Boundary Analysis. We systematically characterize conditions under which NeuroPLC's three correctness mechanisms degrade, transforming limitations into a methodological contribution. **(1) DA Degradation:** When weight signs are balanced across network edges, sign-structural affine arithmetic collapses to Interval Arithmetic (ratio $\rightarrow$ 1.0). Across 50 random KAN architectures, the DA/IA ratio ranges from 1.15$\times$ to 1.69$\times$ (mean 1.41$\times$); trained KANs on CWRU achieve 2.1$\times$ due to sign structure developed through optimization. **(2) Adaptive LUT Degradation:** When B-spline basis functions have uniform curvature, adaptive sampling offers no advantage over uniform—both produce equivalent accuracy. Trained KANs exhibit diverse curvature across edges, making adaptive LUT beneficial. **(3) Depth Lipschitz Explosion:** L_{net} grows exponentially with depth ($e^{\sim 2.4 \cdot L}$), doubling every $\sim$0.3 layers. At 2 layers, L_{net} is within usable range;

beyond depth 5–6, the bound becomes too pessimistic for practical certification. For deep KANs, we recommend Monte Carlo validation as complement. Table XXXII summarizes boundaries and mitigations.

E20: Second Application Scenario — Motor Load Regression. To validate NeuroPLC's generality beyond fault classification, we apply the same pipeline to motor load regression: train a KAN [28,16,1] regressor on CWRU 1hp data to predict load (0–3 hp), then compile to SCL. The regression model achieves RMSE 0.011 on the training load (1hp) but degrades on unseen loads (0hp, 2hp, 3hp) due to domain shift—consistent with E12 and E18 findings. Notably, the compilation is **zero-effort:** changing the output dimension from 4 to 1 and re-running the compiler ($\sim$30 s) produces correct SCL (1,125 lines, all sanity checks pass). The compiler requires no modification for the task change, confirming its generality as an ML$\rightarrow$PLC compiler rather than a bearing-diagnosis tool.

VI. Discussion

A. Why KAN Is Compilable

Prior ML$\rightarrow$PLC tools treat compilation as a purely engineering problem: any network can be ported to a PLC given sufficient effort. Our central finding contradicts this assumption: *whether* a network admits design-time correctness certification depends on its *architecture*, not on the sophistication of the compiler. The same compiler, LUT mechanism, and error-bounding machinery produce 512/512 Z3-verified components for KAN and 0/48 for an identically-sized MLP. This $4.6\times$ end-to-end gap in error propagation (recomputed from $\Delta_{\text{DA}} = 0.66$, 2026-08-03) is not an implementation artifact—it is a structural consequence of the SVNN conditions (Theorem 2, Proposition 2, Theorem 5).

Three architectural properties—each absent from MLPs—enable this gap. They are not additive advantages; they form a *composite* where any single property's absence would break the verification guarantee.

First, structural discretizability—and why a local basis matters. B-spline activation functions are piecewise-polynomial with local support: each basis function is non-zero over at most $k + 1 = 4$ knot intervals. This locality is what makes *segment-aware* error bounds possible (§IV-B10): the compiler computes per-segment $M_2^{(k)}$ values instead of one global Lipschitz constant, tightening the LUT error estimate by 6.0× on average. A globally-supported polynomial basis (ChebyKAN, Proposition 3) also satisfies SVNN, but its Markov-inequality bound is 5.4× looser than empirical M_2. Local support is not required by the SVNN conditions—but it is what makes the bounds *practically tight* for industrial memory budgets.

Second, parametric compactness—a consequence, not a goal. KAN with 6,148 parameters achieves 7.9× reduction from the 48,708-parameter CNN teacher, fitting within the S7-1200 50 KB work memory budget (45.2 KB, 90.4%, TIA V21 measured). This compactness is not an independent design objective—it is a direct consequence of the univariate activation structure: $d_{\text{in}} \times d_{\text{out}}$ learnable edges, each a 1-D function, produce far fewer parameters than the $d_{\text{in}} \times d_{\text{hidden}} \times d_{\text{out}}$ of a fully-connected MLP of comparable expressivity. The SVNN framework explains *why* the compactness is compatible with verification, rather than competing with it (as weight pruning or quantization would): the parameters that remain are exactly those that the error induction of Theorem 2 can bound.

Third, interpretability—the qualitative concomitant of verifiability. KAN's learned activation functions are visualizable as univariate curves— transparency unattainable with ReLU networks, whose multivariate activation patterns require exponential case-splitting to interpret. For industrial deployment under IEC 61508, this matters concretely: a safety engineer can *inspect* whether a learned B-spline activation is monotonic, bounded, and free of pathological oscillations before approving deployment. Interpretability is not a bonus feature of KAN for PLC deployment; it is the visual manifestation of the same structural decomposition that enables Z3 verification.

The correctness argument rests on three composable mechanisms whose synthesis—not any single one—produces the end-to-end guarantee. Sign-structural affine arithmetic (DA) exploits weight-matrix sign cancellation to achieve a 2.1× tighter bound than interval arithmetic, while empirical calibration ($M_2^{\text{char}} = 0.177$) tightens the de Boor LUT bound by a further 98.6% vs. the analytical M_2 (E11). Segment-aware per-segment evaluation composes with DA to contain the heavy-tail regime (safety $3.6\times/6.6\times$ at N=$15/20$ vs. the worst-function calibration, E16, recomputed 2026-08-04). These mechanisms are not individually novel—affine arithmetic and de Boor bounds each have decades of prior literature—but their *composition* is: the compiler chains them into a bound that is computable from model parameters alone, requiring no runtime LP solver.

Depth Scaling: 3-Layer KAN Validation (E56). Theorem 2 predicts that under Condition 3 ($\gamma < 1$), the error bound is *depth-independent*: adding layers does not cause exponential error growth; absent Condition 3, the bound remains finite but grows with depth. We validate the depth-scaling behavior experimentally with a 3-layer KAN[28,16,8,4] trained from scratch on CWRU (120 epochs, 99.60% test accuracy, 7,302 parameters). The result is consistent with the depth-dependent (non-convergent) form:

- **Z3 per-function verification:** 608/608 (100%) B-spline functions pass Z3 verification identical coverage to the 2-layer baseline (512/512), with no degradation at increased depth.
- **DA error bound:** the 3-layer DA chain measures 5.66 (E68-constant recomputation, 2026-08-04; the earlier 1.90 used the pre-audit $L_B = 0.65$)—below the full per-layer amplification envelope $\gamma^{L-1} = 15.4^2 = 237$ that would apply at $L = 3$ under worst-case amplification, consistent with *sub-exponential* depth scaling, not the exponential blowup expected for non-SVNN architectures (where $O(2^L)$ growth arises from wrapping effects). **Soft-contractive depth extension (E-T9-3L, 2026-08-04):** training the 3-layer KAN with the same γ-projection recipe yields $\gamma = [0.99, 1.03, 1.02]$, 98.6% test accuracy, and a *sound*

certificate of 0.284 (safety $2.4\times$; signal-domain propagation, Box-in 81.4%; the output-layer per-edge Lipschitz is 0.29 on its actual signal domain vs. 4.69 on the full design domain; 2026-08-05), plus the validated 0.110 ($6.1\times$, full-set measured floor) and per-function float64 bound 0.019 (safety $36\times$)—depth scaling *with* a certificate, the $\gamma \to 1$ branch of Theorem 6 instantiated at $L = 3$ (per-layer bound growth $1.9\times$ vs. $8.6\times$ for the untrained 3-layer).

- **Condition 3 not certified:** the earlier $\gamma_0 = 0.404$ / $\gamma_1 = 0.318$ figures used a crossed-index max-abs-weight proxy, which is *not* a valid contraction test; the honest per-layer amplification of the 2-layer student is $\gamma = [15.4, 5.3] > 1$ (E68), and contractivity was never established for the 3-layer network either.
- **SCL compilation:** 2,612 lines, structurally valid (DB+FB structure verified), consistent with 0e 0w compilation expectation.

The empirical depth-scaling ratio ($8.6\times$ for the single $2 \to 3$ depth step, from the E68-constant bounds $5.66/0.66$) is substantially below the $O(2^L)$ worst-case for MLP-style wrapping effects, directly supporting Theorem 4's depth-scaling analysis (its $\gamma \geq 1$ branch applies to the trained networks, which are not contractive). The 3-layer KAN's 99.60% accuracy (vs. 2-layer's 99.93%) confirms that depth scaling is architecturally viable for industrial accuracy requirements. (Note: the compile-aware generalization bound (Theorem 6) does not claim depth-monotonicity of the gap—its depth threshold is quantified in Corollary 2.)

CROWN Comparison: Why General-Purpose NN Verifiers Offer No Advantage for KAN (E57). A reviewer might ask: why not apply off-the-shelf NN verifiers (CROWN, DeepPoly, α-β-CROWN) to KAN and achieve competitive bounds without the SVNN framework? We answer this empirically. We compute CROWN-style linear-relaxation bounds on the trained KAN[28,16,4] and compare them against NeuroPLC's DA bound. The result is decisive: **NeuroPLC DA ($\Delta = 0.66$, safety factor $1.0\times$) is substantially tighter than CROWN-IBP**: the CROWN-IBP bound evaluates to 9.97 on the released checkpoint (recomputed 2026-08-04)—a $15.1\times$ gap. (The earlier $8.3\times$ figure used a pre-recomputation baseline and is withdrawn.) CROWN's linear-relaxation mechanism yields *zero* improvement over naive interval bound propagation (IBP) for KAN, because KAN's B-spline activations are inherently piecewise-linear there is no ReLU to relax. The mechanism that makes CROWN effective for MLPs (tightening ReLU bounds via linear relaxation) is structurally irrelevant to KAN.

This finding has a direct theoretical interpretation within the SVNN framework: KAN *already* satisfies the condition that CROWN exists to enforce for MLPs (piecewise-linear activation bounds with C^2 curvature characterization). The $15.1\times$ gap between DA and CROWN-IBP quantifies the practical value of SVNN's architectural insight: by choosing a verifiable architecture, the compiler achieves bounds that no general-purpose verifier can match on the same model. CROWN's $0.1\times$ safety factor (cannot certify classification correctness) versus DA's $1.0\times$ safety factor (certifies: margin $1.35 > 2\Delta_{\text{DA}} = 1.32$) makes this distinction operationally consequential for safety-critical deployment.

B. End-to-End Compositional Verification

Monolithic SMT verification of a 3,800-line SCL program would involve $> 10^4$ nested ITE expressions and time out—a dead end. The SVNN framework suggests a different path: if each architectural component is independently verifiable, whole-program guarantees can be *composed* from independently-verified pieces rather than proved at once. This observation—that KAN's decomposition maps directly to a compositional proof strategy—is what the four-tier verification system exploits. (Table XXXIV summarizes the full compositional results.)

Four-Tier Architecture. The key insight is that whole-program verification can be decomposed into four independent tiers, each with a distinct role and verification scope:

1) **Tier 1—Compiler Template Verification (One-Time):** We encode each IR operation's SCL code template as a Z3 SMT formula with *symbolic* parameters and prove correctness for ALL possible inputs—not just one compiled program, but the compiler itself.
2) **Tier 2—Per-Function Instance Verification (Per-Model):** Each of the 512 B-spline functions is independently verified by Z3 against its LUT approximation bound. These are the "leaf certificates" that Tier 3 composes.
3) **Tier 3—Composition Certificate (Machine-Checkable):** A lightweight certificate checker (∼200 lines of Python) validates that the composition rules from Theorem 1 are correctly applied, assembling the leaf certificates into an end-to-end guarantee. The checker performs *no* Z3 solving—only structural validation and arithmetic checks.
4) **Tier 4—Differential Self-Test (Every Compilation):** Before deployment, the compiler runs a differential test comparing PyTorch FP32 logits against a faithful simulation of the generated SCL semantics (topological-order DAG execution with the actual emitted LUT tables and folded weights) on in-distribution features and adversarial worst-case inputs. The test asserts 100% classification agreement and per-logit error below $\frac{1}{2}\times$ the certified margin (E6: 1.35). Tier 4 catches code-generation regressions that the three proof tiers cannot: it validates the *emitter*, not just the templates. In the 2026-08-01 audit, Tier 4-style testing caught a real regression—the base/spline scale factors were stored in the IR but dropped by the SCL emitter, causing 83% classification agreement on generated code—which no Tier 1–3 proof detected (the templates and bounds were individually sound; the emitter was not). After the fix, Tier 4 passes: 3,000 in-distribution samples, 100% agreement, max logit error 0.47 (E67).

   Chronology of the fix. The scale-factor regression predates the July experiments and was fixed on 2026-08-03; the SCL bundle shipped with this paper was regenerated after the fix (results/scl_output, 2026-08-03), and all reported quantities that exercise emitted-SCL arithmetic reflect the post-fix emitter: E67 was rerun post-fix (2026-08-03: 3,357 samples, 100% agreement, maxAE 0.47) and E68 was validated on the post-fix bundle; the certificate bundle corresponds to the released checkpoint. The worst-case LUT sweeps (E21, E53) and per-function Z3 verifiability (E58) measure the LUT tables and template proofs rather than the emitter's arithmetic and are unaffected by the regression; where an SCL artifact is involved, the 2026-08-03 regeneration is authoritative.

Formal verification boundary. To be precise about what "verified compiler" means: the *formal* proofs cover (i) the IR-to-SCL code *templates* (Tier 1, symbolic Z3 over all inputs), (ii) the per-function LUT *certificates* (Tier 2) and (iii) their *composition* (Tier 3, machine-checkable). The *emitter*—the code generator that instantiates templates with concrete weights, tables, and addressing—is *not* covered by a formal proof; it is validated by the differential self-test (Tier 4), which is an empirical guarantee over the emitted artifact (3,357 samples, E67), not a proof. This boundary is standard in verified compilation practice (CompCert-style certified generators are the stronger alternative [95]); we state it explicitly so that "the first verified PyTorch-to-IEC 61131-3 compiler" is read as: templates, certificates, and composition formally verified; the emitter validated by differential self-testing.

Tier 1: Compiler Template Verification. For each of the six IR operation types, we encode the corresponding SCL code template in Z3 with fully symbolic parameters (weights, biases, input values) and query whether any parameter assignment produces semantically incorrect output. Table XXXIII summarizes the results.

MatMul, Add, Argmax are proved directly: the SCL code is algebraically identical to the mathematical specification in Real arithmetic, so Z3 returns `UNSAT` on the negated equivalence query in single-digit milliseconds. **BsplineLUT** requires a non-trivial proof: we encode a generic cubic polynomial $p(x)$ on $[0, h]$ with symbolic coefficients subject to $|p''(x)| \le M_2$, and prove $|p(x) - L(x)| \le M_2 h^2/8$ for piecewise-linear interpolation $L(x)$. Z3 returns `UNSAT`, mechanizing the de Boor bound that Theorem 1 relies on. **StandardAct** (SiLU) and **Softmax** use the transcendental $\exp(x)$ function, which lies outside Z3's decidable NRA fragment; their correctness follows from the analytic identity that SCL and PyTorch evaluate identical formulas.

Significance. These are *one-time* proofs: once a template is proved correct, all models compiled through it inherit the guarantee. No per-model re-verification of the compiler is needed.

Tier 2: Per-Function B-Spline Verification. For the trained KAN $[28, 16, 4]$ model, each of the 512 B-spline activation functions is independently verified by Z3: the query $\exists x \in [-3, 3]. \ |\mathrm{LUT}(x) - \text{B-spline}(x/3)| > \varepsilon$ returns `UNSAT` for all 512 functions, confirming the per-function error bound $\varepsilon = 0.0036$ (the E40 verification query; the M_2^{char} audit value is $\varepsilon = 0.00406$, Table XXXIV). Average verification time is 285 ms per function (total 146 s). This is, to our knowledge, the first formal, machine-checkable proof that *every* B-spline activation in a trained KAN satisfies its LUT error bound.

Tier 3: Composition Certificate. The 9-step composition certificate instantiates Theorem 1's structural induction, composing the per-function error bounds through the IR graph via sign-structural affine arithmetic propagation rules (§IV-B9). The certificate is verified by a ∼200-line trusted checker that validates: (a) all 512 leaf certificates are present and verified, (b) each composition step's rule application is structurally valid (e.g., a MatMulPropagate step follows from its input bound via the ℓ_1 norm of the weight matrix), and (c) the end-to-end bound is correctly derived from the final step.

Why This Matters. The four-tier architecture achieves whole-program formal verification without requiring a monolithic SMT query over the entire 3,800-line SCL program—a query that would involve $> 10^4$ nested ITE expressions and time out. Instead, it decomposes verification into: (1) one-time proofs about the *compiler* (independent of any model), (2) instance-level proofs about *individual components* (embarrassingly parallel), and (3) a lightweight *composition check* that combines them. This divide-and-conquer strategy is *enabled* by KAN's decomposition into independent univariate B-spline functions—a property that MLPs, with their entangled multivariate activations, cannot exploit (§VI-C).

To our knowledge, this is the first neural network compiler for industrial controllers whose correctness is formally verified at the compiler-template level (modulo Z3's soundness for UNSAT results), with per-instance component verification composed into an end-to-end machine-checkable certificate. The trusted computing base (∼200 lines) is small enough to be manually audited—a critical property for safety certification.

C. KAN vs. MLP Verification Gap

If KAN and MLP differ architecturally but achieve comparable accuracy, why does the verification gap matter? The answer is counterintuitive: *architecture determines what a compiler can mathematically certify*, not how accurately the model performs on a test set. The following controlled experiment—same dimensions, same verification framework, same error tolerance—quantifies the gap.

End-to-End Verification Results. KAN $[28, 16, 4]$ achieves **512/512** (100%) per-function B-spline Z3-verifiability, with DA error bound $\Delta_{\text{DA}} = 0.66$ (model-specific, see Table XXII), IA bound $\Delta_{\text{IA}} = 1.38$, and DA/IA tightening of **2.1**× (end-to-end). The end-to-end compositional certificate is **VALID**. In contrast, MLP $[28, 32, 16, 4]$ with ReLU activations achieves **0/48** (0%) component-level Z3-verifiability because ReLU+MatMul entanglement in each layer prevents decomposition into independent univariate error sources. Even with a best-effort interval-arithmetic propagation through all three layers, the MLP error bound is $\Delta_{\text{MLP}} = 3.04$—a **4.6**$\times$ **worse** error propagation than KAN's DA bound (end-to-end, using KAN's network-level Δ_{DA}=0.66; recomputed 2026-08-03) and **2.2**$\times$ **worse** than KAN's IA bound ($\Delta_{\text{IA}} = 1.38$), for the same per-component error magnitude $\varepsilon = 0.0041$. the MLP **cannot** produce a compositional end-to-end certificate: SVNN Condition 1 is violated (MatMul+ReLU mixed in each layer), Condition 2 is violated (ReLU $\notin C^2$ breaks the de Boor per-function LUT bound), and Condition 3 is violated (ReLU$' \in \{0, 1\}$ gives data-dependent Lipschitz).

Component-Level Verifiability. Table XXXV compares the Z3-verifiability of activation functions in KAN versus MLP architectures of identical dimension.

KAN (B-spline, cubic): The Cox–de Boor recurrence produces piecewise cubic polynomials—fully within Z3's decidable NRA fragment (512/512 UNSAT, ∼0.49 ms/function). **ChebyKAN (Chebyshev N=5):** 496/512 Z3-verifiable via polynomial NRA, no segment enumeration; the Markov-based M_2 bound is 5.4× looser than the empirical M_2 (E54), explaining the 3.1% non-verifiable components. **MLP:** SiLU's transcendental $\exp$ is outside Z3's decidable fragment (0/16 verifiable); ReLU is piecewise-linear (16/16 per-neuron) but the coupled MatMul+ReLU structure prevents compositional error accounting—ReLU$' \in \{0, 1\}$ introduces data-dependent Lipschitz constants that break sign-structure preservation, defeating DA tightening. The MLP error bound is $4.6\times$ worse than KAN's DA bound (end-to-end, using the network-level DA; recomputed 2026-08-03) for identical architecture dimensions. **Quantitative Error Propagation.** Table XXXVI compares the error propagation bounds for the same per-component error $\varepsilon = 0.0041$.

Key Insight. The verification gap is not merely quantitative (MLP propagation is $4.6\times$ looser via DA end-to-end; $2.2\times$ looser via IA) but *structural*: KAN's decomposition into independent univariate B-spline functions **enables** compositional formal verification of neural network compilation in a way that standard MLP architectures cannot match. MLPs, regardless of activation choice, cannot decompose their forward pass into independently-verifiable univariate components—and therefore cannot benefit from the divide-and-conquer strategy that Theorem 1 enables for KANs. This provides a rigorous, verification-theoretic justification for choosing KAN over MLP in safety-critical industrial applications: the architectural choice is not about accuracy, but about *verifiability*.

D. Pipeline Generality Across Domains

If the verification pipeline were bearing-specific, NeuroPLC would be a diagnostic tool demonstration, not a general-purpose compiler. To settle this distinction experimentally, we apply the identical pipeline—same KAN architecture, same compiler, same verification toolchain, zero modification— to three fundamentally different data domains: vibration-based bearing faults (CWRU), natural bearing degradation (XJTU-SY run-to-failure), and image classification (MNIST digits 0–3 via PCA to 28-D). The claim is not that accuracy transfers (it does not, without fine-tuning); it is that *verification guarantees* transfer—a property that depends on the architecture, not the data provenance.

The verification results (Table XXXVII) converge across domains because the B-spline functions are verified against their LUT approximations using the same $M_2h^2/8$ bound regardless of whether input features represent pixel intensities or vibration amplitudes. The DA composition rules depend only on weight matrix structure, not data semantics. This domain independence follows from the SVNN framework (§III): the SVNN conditions are architectural properties of KAN, not properties of the training data.

ChebyKAN: An Alternative Verifiable Architecture. The addition of ChebyKAN (Proposition 3) demonstrates that the SVNN framework is not specific to B-spline basis functions. ChebyKAN achieves 100.0% accuracy on CWRU (matching B-spline's 99.93% within statistical noise), with 496/512 Z3-verifiable components via polynomial NRA. This confirms that the compilable frontier (Remark 1) extends beyond B-splines to any polynomial basis family satisfying Conditions 1–2. The practical benefit is architectural diversity: deployment engineers can choose between B-spline KAN (tighter bounds, 512/512 Z3) and ChebyKAN (simpler per-function polynomial proofs, 496/512 Z3, no segment enumeration) based on their safety and accuracy requirements.

XJTU-SY: Addressing the "Old Dataset" Critique. The XJTU-SY results (E55) directly address a known limitation of the CWRU dataset: its 1990s provenance and artificial EDM-induced faults. XJTU-SY [93] uses naturally degraded bearings across three operating conditions in run-to-failure tests. The fine-tuned model reaches 91.7% accuracy with 512/512 Z3 verification (100%) and successful SCL compilation (2,188 lines, 0e 0w), demonstrating that the NeuroPLC pipeline is effective on modern, realistic bearing degradation data. The domain gap (CWRU zero-shot → XJTU-SY: 29.8%) is real and substantial, confirming that fine-tuning is necessary for cross-dataset deployment—but the SVNN guarantees survive fine-tuning unchanged.

E. Design Rationale and Differentiation

Positioning vs. prior PLC ML tools. Table XXXVIII positions NeuroPLC against the two most relevant prior systems.

Positioning within KAN hardware deployment. Table XXXIX positions NeuroPLC within the broader landscape of KAN hardware deployment. Four independent lines of work—FPGA (KANELÉ), MCU (LUT-KAN, KAN-SoC), and PLC (Corrêa, this work)—converge on the same design principle: KAN's univariate B-spline structure compiles to LUT primitives across hardware platforms. NeuroPLC is the only work that provides (i) a complete compiler toolchain from PyTorch, (ii) native IEC 61131-3 SCL generation, and (iii) design-time formal correctness certification.

Five properties distinguish NeuroPLC from prior KAN hardware deployment work: (1) **type-theoretic foundation**—17 theorems providing, to our knowledge, the first formal characterization of certifiable neural architectures; (2) **universal IEC 61131-3 compliance**—Lemma 7 extends certification to any IEC-compliant controller; (3) **multi-architecture coverage**—5 C^2-BV families (B-spline, Fourier, Wavelet, Chebyshev, RBF-KAN) all verified (Proposition 6); (4) **deployment safety**—WCET guarantee (Theorem 11) and non-interference proof (Theorem E) directly addressing IEC 61508; (5) **automated safety monitor—**Algorithm 3 generates a companion SCL function block ($\leq 5\%$ overhead).

Why not LLM-based code generation? Recent work uses LLMs to generate IEC 61131-3 ST from natural-language specifications [96]–[98], achieving 85–96% labor savings for control logic prototyping. However, LLM-based generation presents fundamental challenges for *compiling trained neural networks to PLCs*. We validate this empirically (V2, Table XXVIII): we prompted Claude to generate complete SCL code for the KAN $[28, 16, 4]$ architecture. The generated code (164 lines) was structurally plausible but contained **6 critical defects**: (1) all weight arrays were declared but *uninitialized*—the model had zero trained parameters, producing meaningless output; (2) KAN layer output formula was *mathematically incorrect*, missing the learned `scale_base` and `scale_spline` parameters that govern the base-vs-spline trade-off; (3) TIA Portal V21 cannot compile the code due to Siemens-specific syntax requirements (2D ARRAY initialization, INT/REAL type constraints in binary search) that the LLM was unaware of. In contrast, NeuroPLC embeds all 6,148 trained parameters as IEEE 754 REAL literals via a mechanical, auditable extraction process, producing 2,188 lines of SCL that compiles with 0 errors and 0 warnings. More fundamentally: (1) LLM output is stochastic—sampling from a token distribution—violating the determinism required by IEC 61508; (2) current LLMs do not provide the mathematical correctness guarantees required for safety-critical compilation (Theorem 1, Z3 certificates); (3) LLMs are not trained on the specific semantics of neural architecture compilation (B-spline decomposition, LUT optimization). LLM-based ST generation and NeuroPLC address orthogonal problems—requirements-to-logic vs. model-to-inference—and are complementary in a complete industrial AI pipeline.

Why not ONNX? General-purpose ML compilers (ONNX Runtime [99], TVM, Glow [92]) target GPU/CPU, not memory-constrained PLCs. We empirically confirm ONNX incompatibility (V5, Table XXVIII): `torch.onnx.export` fails entirely on KAN $[28, 16, 4]$ because ONNX has no standard operator for cubic B-spline evaluation. Decomposing each of the 512 B-spline activation functions into ONNX primitives (Gather, Mul, ReduceSum) would require **8,393 ONNX graph nodes** vs. NeuroPLC's **11 IR nodes**—a **763**$\times$ **explosion** (Table XL). Furthermore, the ONNX Runtime minimal library ($\sim$22 MB $\approx$ 22,528 KB) exceeds the S7-1200 work memory (50 KB) by **450**$\times$, making ONNX-based deployment physically impossible on this PLC class. Even the S7-1500 (1.5 MB) cannot host the runtime. NeuroPLC follows the domain-specific compiler philosophy: a custom IR whose primitives (MatMul, BsplineLUT, StandardAct) map directly to SCL constructs, requiring zero external runtime—only 40.3 KB of IEC 61131-3 native code.

F. Statistical Validity and Limitations

All classification accuracies are reported with 95% bootstrap confidence intervals (10,000 resamples). Multi-seed experiments report means across seeds. McNemar's test is used for paired classifier comparison with exact p-values. On CWRU, all KD-trained models achieve near-perfect accuracy (99.89–99.93%), making statistical differentiation challenging—the $\sim$0.04% gap between VRM-KD and Hinton-KD corresponds to 1 of 2,743 test samples. We report effect directions with bootstrap CIs and note when differences are not statistically significant.

Two consequential limitations. (1) No physical PLC measurement. Inference timing is estimated from instruction counts and Z3 WCET analysis ($\leq$2.86 ms), not measured on hardware. We validate the generated SCL offline through TIA Portal V21 compilation (zero errors across KAN and MLP targets) and PLCSIM Advanced cycle-accurate simulation; physical PLC measurement of inference latency and REAL-arithmetic discrepancy versus simulation is the next validation step for deployment certification. This limitation is addressable without changing the compiler, architecture, or verification claims—only the measurement apparatus.

(2) Correctness guarantee scope. As Szász et al. [46] (ICML 2025 Spotlight) demonstrate, theoretical soundness from interval/affine arithmetic may not fully translate to physical IEEE 754 hardware due to FMA behavior and compiler reordering. Our guarantee is therefore a **design-time** correctness argument in the IEEE 754 abstract machine. Theorem 1 provides a mathematical error bound that does not require a proof assistant's kernel—but also does not constitute a machine-checked proof of compiler correctness for all possible hardware inputs. The composition certificate (Tier 1–3) provides the strongest available *software-level* evidence; hardware-level certification requires the physical PLC measurement described above.

Additional scope boundaries: The compiler currently supports feedforward architectures (KAN, MLP); convolutional and attention operations are not yet modeled. CWRU uses EDM-induced faults; XJTU-SY (natural degradation) partially mitigates this. The SCL backend targets Siemens S7-1200/1500; other vendors require additional backends. These are standard scope limitations for a first compiler release and do not affect the validity of the SVNN theoretical framework.

G. Path to Functional Safety Certification

The SVNN framework maps directly to the emerging ISO/IEC TS 22440 [100] specification for AI in functional safety, which requires deterministic behavior verification, algorithm transparency, and quantified performance boundaries—exactly the properties that $\varepsilon_i(M, X)$ (Definition 1), per-function M_2 bounds, and segment-aware de Boor analysis provide. NVIDIA Halos [101] independently demonstrated a "traditional safety foundation + AI within verifiable envelopes" architecture for IEC 61508 SIL 2 and ISO 13849 PL d—paralleling the SVNN philosophy from the hardware verification side.

Two certification tiers emerge from the SVNN error bounds (Table XLI): a **high-probability** tier using DA with sign-balance evidence (SIL 2, N=15, safety 1.0×), and a **sound** tier using global IA with no sign-cancellation assumption (SIL 2 at $N{\geq}22$, SIL 3 at $N{\geq}25$; segment-aware IA composes further per the recomputed Seg-DA analysis, §IV-B10). For SIL 3+, Level 2 arithmetic bounds supplement Level 3 MILP verification [16] on safety-critical outputs (§VI-B). We emphasize this outlines a *pathway*, not completed certification—full compliance requires additional evidence (hardware reliability, process audit, environmental testing) beyond this paper's scope. The contribution is the *software-level correctness argument* that has been the missing piece in AI functional safety certification.

H. Future Work

The type-theoretic framework established here opens several research directions:

Theory. (i) Mechanizing the IR type soundness theorem (Theorem D) in Coq or Lean, following the CompCert tradition of machine-checked compiler correctness for neural architectures. (ii) Tightening the computational necessity bound (Theorem 5) to a constant-factor inapproximability result. (iii) Extending the C^2-BV family (Proposition 6) to additional basis functions (Legendre polynomials, Hermite functions, learnable spline-kernel hybrids). (iv) Investigating whether the γ^L generalization bound (Theorem 6) can be tightened to the depth-independent $O(1/\sqrt{n})$ rate.

Systems. (i) Physical PLC measurement of inference latency and IEEE 754 roundoff, closing the gap between the IEC 61131-3 theoretical guarantee (Lemma 7) and hardware-level evidence. (ii) OPC UA integration for live sensor data input and diagnostic output within Industry 4.0 architectures. (iii) Online fine-tuning directly on PLC firmware, leveraging the LUT-based compilation strategy for gradient-free adaptation.

Standards. (i) Formal mapping of the Non-Interference Theorem (Theorem E) to IEC 61508 SIL 3 independence requirements for mixed-criticality systems. (ii) Integration with ESBMC-PLC+ for combined numerical and logical verification of IEC 61131-3 programs containing neural inference blocks.

VII. Conclusion

This paper established a **type theory of certifiable neural architectures**—a formal characterization of which architectures admit design-time correctness certification under LUT-discretized compilation, with what sharp constants, at what computational cost, and with what learning-theoretic guarantee—and instantiated it as the first self-verifying PyTorch-to-IEC 61131-3 compiler (formally verified templates, certificates, and composition rules; emitter validated by differential self-testing). The central finding is that compiler correctness certification is not an engineering problem—it is an *architectural* problem, governed by a precise mathematical frontier.

Theoretical contributions:

1. **Verifiable Deployment Capacity (Theorem 17).** The unifying framework: a universal packing line bounds every certified deployment scheme; three verification strata place every architecture; a certificate proof system formalizes the verification budget under perfect deterministic soundness; we prove a theorem-level information-theoretic necessity within the calculus and derive an architecture-as-code selection law. The sharp constants, minimax rates, separation, and trichotomy of the items below are its expansion (§III-S).
2. **Sharp k-th Order LUT Bounds (Theorem 9).** The de Boor constant $M_2 h^2/8$ is the $k = 2$ instance of an exact all-orders family $c_k = Q_k/k!$, attained by explicit extremal functions at machine precision—DA optimality is a theorem with a certificate, not an empirical observation.
3. **Verifiability Trichotomy (Theorem 10).** The SVNN condition is *conjecturally* the unique regime where second-order rate, linear LUT cost, closed-form sharp constants, and reachability-freedom coexist; ReLU admits only first-order bounds, multivariate gates suffer N^m cost and NP-hard tightness, and domain-dependent curvature requires reachability. The necessity direction—that no other regime attains all four properties—is stated as a conjecture with strong evidence; within the affine-certificate class it *is* a theorem (Theorem 16, §III-R: gates are essentially C^2 and univariate, and product coupling makes ε-verification NP-hard), and extending it beyond affine certificates is the open problem that the regime exclusions delimit.
4. **ε-Separation (Theorem 5).** The LUT-verification decision problem is polynomial for SVNN and NP-hard for coupled architectures (self-contained 3-SAT reduction); SVNN uniquely attains (polynomial time, zero slack under ETH) (completeness of this boundary is the trichotomy necessity conjecture).
5. **Compile-Aware Generalization (Theorem 6).** The deployed risk decomposes as $\hat{R}(N)+O(\gamma^{L-1}/\sqrt{n})+O(\gamma^{L-1}c_k M_k h^k)$ with the resolution-matching law $N^* \asymp n^{1/(2k)}$—a directly actionable compiler rule for embedded deployment.

6. **Compilable Frontier (Theorem A).** SVNN Conditions 1–2 are necessary and sufficient for dimension-optimal affine certification. Non-SVNN architectures (MLPs) degrade by $\Theta(d^{1/2})$ per layer ($\Theta(d^{L/2})$ over L layers) due to cross-term coupling in multi-layer error propagation.
7. **DA Galois Connection (Theorem B).** Doubleton Arithmetic is a canonical instance of Cousot abstract interpretation with a defined Galois connection (α, γ), making the framework, to our knowledge, the first neural network compilation theory formally grounded in abstract interpretation.
8. **DA Tightness (Theorem C).** The DA bound $M_2 \cdot h^2/8$ is semantically tight—attained by $f^*(x) = M_2/2 \cdot x^2$ at every LUT interval midpoint (1,000/1,000 exact, MATLAB-verified).
9. **Universal IEC 61131-3 Guarantee (Lemma 7).** The SVNN certification holds for *any* IEC 61131-3:2025 compliant controller worldwide, via the standard's IEEE 754 normative reference.
10. **Type Soundness (Theorem D).** The NeuroPLC IR is formalized with operational semantics and typing rules; compiler correctness is recast as a type soundness theorem.
11. **Non-Interference (Theorem E).** Inserting a NeuroPLC inference block into a certified PLC program preserves all pre-existing safety properties under the timing premise $\text{WCET}(P) + \text{WCET}(N) \leq$ scan cycle—the first such guarantee for NN-on-PLC deployment.
12. **WCET (Theorem 11).** Parameterized worst-case execution time bound: 3.86 ms for KAN $[28, 16, 4]$ on S7-1200 ($25.9\times$ scan-cycle margin).
13. **DA Optimality (Theorem 9).** DA is the tightest sound first-order abstract domain for C^2 functions.
14. **SVNN Monoid (Theorem 8).** The SVNN class is closed under serial composition, enabling modular certification.

Empirical validation (74 experiments + 11 capacity experiments E-T1–E-T11, 4 architectures). B-spline KAN achieves 99.93% CWRU accuracy; FourierKAN and WaveletKAN achieve 100% on both CWRU and XJTU-SY domain-shift with 512/512 Z3-equivalent verification preserved (E62); ChebyKAN achieves 496/512 Z3 (96.9%). The C^2-BV architecture family (Proposition 6) —B-spline KAN, ChebyKAN, FourierKAN, WaveletKAN, RBF-KAN—is universally compilable via LUT extraction. All generated SCL compiles to **0 errors, 0 warnings** in TIA Portal V21. The compiler auto-generates a companion safety monitor (Algorithm 3, $\leq 5\%$ overhead). Soft-contractive training (E-T9) converts the certificate from thin ($1.0\times$, expected) to sound ($2.34\times$, first-principles) with a validated float32 tier of $11.6\times$ (measured-floor bound 0.058 vs. measured maxAE 0.053), 99.9% Box-Continuation coverage, and depth extension to L=3 ($6.1\times$); adaptive allocation (E15) achieves $3.6\times$ at unchanged storage.

Open directions. Mechanized proof of the IR type soundness theorem in Coq/Lean, physical PLC WCET measurement (all current timing evidence is simulation-based via PLCSIM Advanced and Z3 SMT), and online fine-tuning on PLC hardware remain for future work. The type-theoretic framework established here—defining well-typedness conditions for certifiable neural architectures—provides the foundation for these extensions and for the broader program of *structurally verifiable compilation* of machine learning models to safety-critical embedded targets.

The question this paper answers is not "can we compile neural networks to PLCs?"—prior tools already showed that we can. The question is "which architectures admit *certified* compilation, and why?" The answer is a type theory. As AI enters industrial control loops governed by functional safety standards, this reframing—from verifying compilers to designing verifiable architectures—is the contribution that will outlast any specific compiler implementation.

Code and Data Availability

The NeuroPLC compiler source code, trained model checkpoints, and evaluation scripts are available at https://github.com/aiyuedi/neuroplc. The CWRU and XJTU-SY bearing datasets are publicly available from their respective repositories.

Appendix

The type soundness theorem (Theorem D) and the non-interference theorem (Theorem E) are proved in this paper via structural induction on the IR syntax and on SCL execution traces, respectively. These proofs are rigorous but presented in standard mathematical prose. For readers interested in machine-checked verification, we provide the formal specification of the key lemmas in a notation compatible with Coq and Isabelle/HOL.

A. IR Syntax (Inductive Type)

```
Inductive ir_expr : Type :=
  | IRInput   : var -> ir_expr
  | IRMatMul : matrix -> ir_expr -> ir_expr
  | IRBsplineLUT : lut_table -> grid -> ir_expr -> ir_expr
  | IRStandardAct : activation -> ir_expr -> ir_expr
  | IRAdd     : ir_expr -> ir_expr -> ir_expr
  | IRSoftmax : ir_expr -> ir_expr
  | IRArgmax : ir_expr -> ir_expr.
```

B. Type System (Inductive Typing Judgment)

```
Inductive ir_type : Type :=
  | LinearT   : ir_type
  | ElemwiseT : ir_type
  | SVNNT     : ir_type.

Inductive has_type : ir_expr -> ir_type -> Prop :=
  | T_Input : forall x, has_type (IRInput x) LinearT
  | T_MatMul : forall W e, has_type e LinearT ->
                has_type (IRMatMul W e) LinearT
  | T_BsplineLUT : forall T g e, has_type e LinearT ->
                     M2_bound T g ->  (* Condition 2 *)
                     has_type (IRBsplineLUT T g e) ElemwiseT
  | T_Add : forall e1 e2, has_type e1 LinearT ->
              has_type e2 ElemwiseT ->
              has_type (IRAdd e1 e2) SVNNT
  | T_Softmax : forall e, has_type e SVNNT ->
                  has_type (IRSoftmax e) SVNNT.
```

C. Type Soundness Statement

```
Theorem type_soundness :
  forall (e : ir_expr) (rho : env),
    has_type e SVNNT ->
    exists (eps : R),
      eps > 0 /\
      forall (x : input_vector),
        in_domain x ->
        Rabs (denote_real e rho x - denote_scl (compile e) rho x)
          <= eps.
```

Proof sketch (structural induction on `has_type`): Base cases (`T_Input`, `T_MatMul`, `T_BsplineLUT`) use the de Boor error bound (Lemma `lut_error_bound`) and DA propagation (Lemma `da_linear_soundness`). Inductive cases (`T_Add`, `T_Softmax`) compose error bounds using the Lipschitz property (Lemma `add_lipschitz`, Lemma `softmax_contraction`).

D. Non-Interference Statement

```
Theorem non_interference :
  forall (P : scl_program) (N : svnn_network) (phi : safety_property),
    satisfies P phi ->
    let F := compile_N N in
    let P_combined := compose P F in
    satisfies P_combined phi.
```

Proof sketch (induction on SCL trace length): Base case ($n = 0$): initial states satisfy φ by assumption. Inductive step: case analysis on whether the n-th step executes P's POUs or F's POU. Proposition NC.1 (memory isolation) ensures State_P is invariant when F executes. Proposition NC.2 (termination) bounds F's execution within one scan cycle. Proposition NC.3 (numerical safety) guarantees F's outputs are finite. All three propositions have straightforward Coq proofs from the generated SCL structure.

E. DA Galois Connection (Semantic Domain)

```
Record doubleton := mk_doubleton { center : R ; radius : R }.
(* Weighted first-order-plus-curvature envelope; the right adjoint
   generated by alpha (radius includes the first-order term). *)
Definition gamma (d : doubleton) (f : R -> R) (x0 r : R) : Prop :=
  Rabs (f x0 - center d) + Rabs (f' x0) * r + M2 f * r * r / 2
    <= radius d.
Definition alpha (f : R -> R) (x0 r : R) : doubleton :=
```

```
  mk_doubleton (f x0) (Rabs (f' x0) * r + M2 f * r * r / 2).

Lemma galois_soundness :
  forall f x0 r, gamma (alpha f x0 r) f x0 r.
(* Tightness: the linear witness f(x)=c+(R/r)(x-x0) attains the
   radius, so alpha o gamma is tight (infimum = d). The insertion
   form alpha o gamma = id does not hold (constant functions). *)
Lemma galois_tightness :
  forall c R x0 r (rpos : 0 < r),
    exists f, gamma (mk_doubleton c R) f x0 r
      /\ alpha f x0 r = mk_doubleton c R.
Lemma galois_monotone_add :
  forall d1 d1' d2 d2',
    doubleton_le d1 d1' -> doubleton_le d2 d2' ->
    doubleton_le (da_add d1 d2) (da_add d1' d2').
```

Mechanization status. The above specifications define the complete proof obligations for machine-checked verification of the SVNN type system in Coq or Isabelle/HOL. The proofs—structural induction on the IR syntax (Theorem D), induction on SCL trace length (Theorem E), and algebraic verification of the Galois connection properties (Theorem B)—follow the mathematical proofs presented in §III-X, §IV-J, and §III-V respectively. Full mechanization is ongoing and will be released with the compiler source code.

References

[1] L. Nye, "Categorical Construction of Logically Verifiable Neural Architectures," arXiv:2508.11647, 2025.
[2] C. Hinze, Z. Zhou, H. Xu, A. Lechler, and A. Verl, "RTNNIgen — An Open-Source PLC Library and Python Generator Converting Keras Models to Realtime Capable Structured Text," in *IECON 2024 — 50th Annual Conference of the IEEE Industrial Electronics Society*, 2024.
[3] N. Campos, A. Pereira, D. Duarte, F. Perez, G. Pessin, and M. Pinto, "A Conversion Tool for Translating Python-Based Machine Learning Models to Structured Text Codes," *SoftwareX*, vol. 29, p. 102005, 2025.
[4] C. Doumanidis, P. H. N. Rajput, and M. Maniatakos, "ICSML: Industrial Control Systems ML Framework for Native Inference Using IEC 61131-3 Code," in *ACM Workshop on Cyber-Physical Systems Security & Privacy (CPSS)*, 2023.
[5] G. F. da Silva Corrêa, "Avaliação de redes neurais artificiais em controladores lógicos programáveis para classificação de dados," Master's thesis, Universidade de São Paulo (USP), 2024, first KAN-on-PLC demonstration via Snap7 distributed execution; MLP achieved 98% on S7-1511TF, KAN achieved 88.5% (distributed). No native SCL compilation, no correctness guarantees.
[6] W. Zhang, F. Xie, W. Cai, and C. Ma, "VRM: Knowledge Distillation via Virtual Relation Matching," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 2707–2717.
[7] W. Zhang, G. Peng, C. Li, Y. Chen, and Z. Zhang, "A New Deep Learning Model for Fault Diagnosis with Good Anti-Noise and Domain Adaptation Ability on Raw Vibration Signals," *Sensors*, vol. 17, no. 2, p. 425, 2017.
[8] W. Zhang, X. Li, and Q. Ding, "Input Feature Mappings-Based Deep Residual Networks for Fault Diagnosis of Rolling Element Bearing with Complicated Dataset," *IEEE Access*, vol. 8, pp. 175 157–175 167, 2020.
[9] T. W. Rauber, F. de Assis Boldt, and F. M. Varejão, "An Experimental Methodology to Evaluate Machine Learning Methods for Fault Diagnosis Based on Vibration Signals," *Expert Systems with Applications*, vol. 167, p. 114022, 2021.
[10] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov-Arnold Networks," *arXiv preprint arXiv:2404.19756*, 2024.
[11] S. Rigas, P. Tzouveli, and S. Kollias, "Explainable Fault and Severity Classification for Rolling Element Bearings Using Kolmogorov-Arnold Networks," *Entropy*, vol. 27, no. 4, p. 403, 2025.
[12] Y. Wang *et al.*, "Rolling Bearing Fault Diagnosis Based on 1D Convolutional Neural Network and Kolmogorov-Arnold Network for Industrial Internet," *Computers, Materials & Continua*, 2025.
[13] L. Hoang, A. Gupta, and D. M. Harris, "KANELÉ: Kolmogorov-Arnold Networks for Efficient LUT-Based Evaluation," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2026, best Paper Award.
[14] O. Kuznetsov, "Look-Up Table Optimization for Kolmogorov-Arnold Networks on Microcontrollers," *IEEE Access*, vol. 12, pp. 170 140–170 158, 2024.
[15] T. Flores, M. Medeiros *et al.*, "Kolmogorov-Arnold Networks under TinyML Constraints: A Study on SoC Estimation for Electric Vehicles," in *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025.
[16] N. Schwartz, C. K. Nagesh, S. Sankaranarayanan, R. Kaur, T. Sahai, and S. Jha, "Optimal Abstractions for Verifying Properties of Kolmogorov-Arnold Networks (KANs)," *arXiv preprint arXiv:2602.06737*, 2026, pWA abstraction + MILP encoding for KAN formal verification.
[17] Y. Zhang, J. Lv, W. Lin, and Z. Ding, "Formal Synthesis of Safe Kolmogorov-Arnold Network Controllers with Barrier Certificates," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2025, pp. 302–310.
[18] M.-L. Schumacher, B. Heiderich, and M. F. Huber, "Training and Verifying Robust Kolmogorov-Arnold Networks," in *International Conference on Learning Representations (ICLR)*, 2025, openReview: N9hl7CShRA.
[19] D. Zhang, "PolyKAN: Provable and Optimal Compression of Kolmogorov-Arnold Networks," *arXiv preprint arXiv:2510.04205*, 2025.
[20] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *arXiv preprint arXiv:1503.02531*, 2015.
[21] X. Ren *et al.*, "Lightweight Intelligent Fault Diagnosis Method Based on Multi-Stage Pruning Distillation Interleaving," *Advances in Mechanical Engineering*, 2024.
[22] P. Dantas, W. Junior, and L. Cordeiro, "ESBMC-PLC+: A Unified IEC 61131-3 Formal Verification Framework as a PLCverif Successor," *arXiv preprint arXiv:2606.23870*, 2026, sMT-based bounded model checking + k-induction for LD/ST/SCL.
[23] M. C. Werner, "Model-Based Design of Program Organization Units Using Synchronous Languages," Ph.D. dissertation, University of Kaiserslautern-Landau, 2025.
[24] M. Ebrahimi Salari, E. Enoiu, C. Seceleanu, and R. Eilers, "PyLC+: A Verification-Aware Pipeline for PLCs," in *IEEE International Conference on Computers, Software, and Applications (COMPSAC)*, 2025.

[25] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient neural network robustness certification with general activation functions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[26] G. Singh, T. Gehr, M. Puschel, and M. Vechev, "An abstract domain for certifying neural networks," in *Proceedings of the ACM on Programming Languages (POPL)*, 2019.

[27] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "Beta-crown: Efficient bound propagation with per-neuron split constraints for neural network robustness verification," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[28] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient smt solver for verifying deep neural networks," in *International Conference on Computer Aided Verification (CAV)*, 2017.

[29] G. Katz, D. A. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljic, D. L. Dill, M. J. Kochenderfer, and C. Barrett, "The marabou framework for verification and analysis of deep neural networks," in *International Conference on Computer Aided Verification (CAV)*, 2019.

[30] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *ACM Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[31] M. Mirman, T. Gehr, and M. Vechev, "Differentiable abstract interpretation for provably robust neural networks," in *International Conference on Machine Learning (ICML)*, 2018.

[32] S. Gowal, K. Dvijotham, R. Stanforth, R. Bunel, C. Qin, J. Uesato, T. Mann, and P. Kohli, "On the effectiveness of interval bound propagation for training verifiably robust models," in *ArXiv preprint*, 2018.

[33] C. de Boor, *A Practical Guide to Splines*, 1st ed. Springer-Verlag, 1978.

[34] D. Richardson, "Some undecidable problems involving elementary functions of a real variable," *The Journal of Symbolic Logic*, vol. 33, no. 4, pp. 514–520, 1969.

[35] Z. Bozorgasl and H. Chen, "Wav-KAN: Wavelet kolmogorov-arnold networks," 2024, introduces ChebyKAN (Chebyshev polynomial) and Wavelet-KAN variants.

[36] J. C. Mason and D. C. Handscomb, *Chebyshev Polynomials*. Chapman and Hall/CRC, 2002.

[37] A. A. Markov, "On a problem of D. I. Mendeleev," *Izvestiya Akademii Nauk*, 1890, contains the Markov inequality: $||p'|| \leq n^2||p||$ for degree-n polynomials on $[-1, 1]$.

[38] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2008, pp. 337–340.

[39] H. Zhang, S. Wang, K. Xu, L. Li, B. Li, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "General cutting planes for bound-tightening in neural network verification," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, α-β-CROWN verifier.

[40] M. Sefidgaran, K. Nadjahi, and A. Zaidi, "Tighter CMI-based generalization bounds via stochastic projection and quantization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025, oral; arXiv:2510.23485.

[41] P. L. Bartlett and S. Mendelson, "Rademacher and Gaussian complexities: Risk bounds and structural results," *Journal of Machine Learning Research*, vol. 3, pp. 463–482, 2002.

[42] A. Kratsios, B. J. Kim, and T. Furuya, "Approximation Rates in Besov Norms and Sample-Complexity of Kolmogorov-Arnold Networks with Residual Connections," *Neural Networks*, vol. 200, p. 108797, 2025, arXiv:2504.15110.

[43] N. Golowich, A. Rakhlin, and O. Shamir, "Size-independent sample complexity of neural networks," in *Conference on Learning Theory (COLT)*, 2018.

[44] A. B. Tsybakov, *Introduction to Nonparametric Estimation*. Springer, 2009.

[45] L. Györfi, M. Kohler, A. Krzyżak, and H. Walk, *A Distribution-Free Theory of Nonparametric Regression*. Springer, 2002.

[46] A. Szász, M. N. Müller, and M. Vechev, "No Soundness in the Real World: Floating-Point Backdoors Are Invisible to All Major Neural Network Verifiers," in *International Conference on Machine Learning (ICML)*, 2025, spotlight.

[47] C. A. Micchelli and T. J. Rivlin, "A Survey of Optimal Recovery," *Optimal Estimation in Approximation Theory*, pp. 1–54, 1977.

[48] R. Mohr, Coblenz, and Kirst, "Globally optimal univariate spline approximations," *Computational Optimization and Applications*, 2023.

[49] G. Beliakov, "Least squares splines with free knots: Global optimization approach," *Applied Mathematics and Computation*, vol. 149, pp. 783–798, 2004.

[50] G. C. Necula, "Compiling with Proofs," Ph.D. dissertation, Carnegie Mellon University, 1998, cMU-CS-98-154; proof-carrying code architecture.

[51] M. C. Rinard, "Credible Compilation," MIT Laboratory for Computer Science, Tech. Rep. MIT-LCS-TR-776, 1999.

[52] A. Pnueli, M. Siegel, and E. Singerman, "Translation Validation," in *Proceedings of TACAS*, 1998, pp. 151–166.

[53] M. Daumas and G. Melquiond, "Certifying Floating-Point Implementations Using Gappa," *IEEE Transactions on Computers*, vol. 59, no. 4, pp. 518–530, 2010, arXiv:0801.0523.

[54] S. Chevillard, M. Joldes, and C. Lauter, "Sollya: An Environment for the Development of Numerical Codes," in *Mathematical Software — ICMS*, 2010, pp. 28–31.

[55] A. Radisic, "LeanCert: Certified Numerical Computation in Lean 4," https://github.com/alerad/leancert, 2025, lean Reservoir package; interval bounds, Taylor remainders, NN interval verification.

[56] T. A. Henzinger, M. Lechner, and D. Zikelic, "Scalable Verification of Quantized Neural Networks," in *Proceedings of AAAI*, 2021, arXiv:2012.08185.

[57] A. N. Kolmogorov and V. M. Tikhomirov, "ε-entropy and ε-capacity of sets in function spaces," *Uspekhi Matematicheskikh Nauk*, vol. 14, no. 2, pp. 3–86, 1959, english transl.: AMS Translations, Series 2, vol. 17, pp. 277–364, 1961.

[58] A. Pinkus, *n-Widths in Approximation Theory*, ser. Ergebnisse der Mathematik und ihrer Grenzgebiete, 3. Folge. Springer-Verlag, 1985, vol. 7, zbl 0551.41001.

[59] C. E. Shannon, "Coding Theorems for a Discrete Source with a Fidelity Criterion," *IRE International Convention Record*, vol. 7, pp. 142–163, 1959.

[60] L. N. Trefethen, *Approximation Theory and Approximation Practice*. SIAM, 2013, other Titles in Applied Mathematics 128; Zbl 1264.41001.

[61] J. P. Boyd, *Chebyshev and Fourier Spectral Methods*, 2nd ed. Dover Publications, 2001, first edition: Springer, 1989.

[62] M. Giacobbe, T. A. Henzinger, and M. Lechner, "How Many Bits Does it Take to Quantize Your Neural Network?" in *Proceedings of TACAS*, 2020, pp. 79–97, dOI 10.1007/978-3-030-45237-7_5.

[63] S. M. Rump, "Fast and Parallel Interval Arithmetic," *BIT Numerical Mathematics*, vol. 39, no. 3, pp. 534–554, 1999, zbl 0942.65048.

[64] A. Neumaier, *Interval Methods for Systems of Equations*, ser. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1990, vol. 37, zbl 0715.65030.

[65] Y. T. Kalai, A. Lombardi, V. Vaikuntanathan, and D. Wichs, "Boosting Batch Arguments and RAM Delegation," in *Proceedings of STOC*, 2023, succinctness poly(log T) for SNARGs.

[66] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, "Delegating Computation: Interactive Proofs for Muggles," in *Proceedings of STOC*, 2008, pp. 113–122.

[67] J. F. Traub, G. W. Wasilkowski, and H. Woźniakowski, *Information-Based Complexity*, ser. Computer Science and Scientific Computing. Academic Press, 1988, zbl 0654.94004.

[68] T. A. Henzinger and N. E. Saraç, "Quantitative and Approximate Monitoring," in *Proceedings of LICS*, 2021, arXiv:2105.08353.

[69] K. Scaman and A. Virmaux, "Lipschitz Regularity of Deep Neural Networks: Analysis and Efficient Estimation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[70] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. J. Pappas, "Efficient and Accurate Estimation of Lipschitz Constants for Deep Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 11 423–11 434.

[71] S. R. Buss and B. M. Kapron, "Resource-Bounded Continuity and Sequentiality for Type-Two Functionals," in *Proceedings of STOC*, 2002, pp. 668–677, dOI 10.1145/507382.507387.
[72] Y. Zhu and J. Lafferty, "Quantized Minimax Estimation over Sobolev Ellipsoids," *Information and Inference: A Journal of the IMA*, vol. 7, no. 1, pp. 31–62, 2018, arXiv:1503.07368.
[73] R. Milner, "A Theory of Type Polymorphism in Programming," *Journal of Computer and System Sciences*, vol. 17, no. 3, pp. 348–375, 1978.
[74] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
[75] W. Li, S. Xu, G. Zhao, and L. P. Goh, "Adaptive Knot Placement in B-spline Curve Approximation," *Computer-Aided Design*, vol. 37, no. 8, pp. 791–797, 2005.
[76] L. Piegl and W. Tiller, *The NURBS Book*, 2nd ed. Springer, 1997.
[77] K. E. Atkinson, *An Introduction to Numerical Analysis*, 2nd ed. John Wiley & Sons, 1989.
[78] P. Krukowski, D. Wilczak, J. Tabor, A. Bielawska, and P. Spurek, "Make Interval Bound Propagation Great Again," *arXiv preprint arXiv:2410.03373*, 2024.
[79] J. Stolfi and L. H. de Figueiredo, "An introduction to affine arithmetic," *TEMA: Trends in Applied and Computational Mathematics*, vol. 4, no. 3, pp. 297–312, 2003.
[80] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
[81] H. Azami, M. Rostaghi, D. Abásolo, and J. Escudero, "Refined Composite Multiscale Dispersion Entropy and its Application to Biomedical Signals," *IEEE Transactions on Biomedical Engineering*, vol. 64, no. 12, pp. 2872–2879, 2017.
[82] Y. Chen *et al.*, "HRCGMFDE: Hierarchical Refined Composite Generalized Multiscale Fluctuation Dispersion Entropy," *Shock and Vibration*, 2024.
[83] Siemens AG, *SIMATIC S7-1200 Programmable Controller System Manual*, 4th ed., 2022, cPU 1211C AC/DC/Relay: 50 kB work memory.
[84] International Electrotechnical Commission, "Programmable Controllers – Part 3: Programming Languages," IEC, Tech. Rep. IEC 61131-3:2025, 2025.
[85] ——, "Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems," IEC, Tech. Rep. IEC 61508, 2010.
[86] W. A. Smith and R. B. Randall, "Rolling Element Bearing Diagnostics Using the Case Western Reserve University Data: A Benchmark Study," *Mechanical Systems and Signal Processing*, vol. 64–65, pp. 100–131, 2015.
[87] J. Hendriks, P. Dumond, and D. A. Knox, "Towards Better Benchmarking Using the CWRU Bearing Fault Dataset," *Mechanical Systems and Signal Processing*, vol. 169, p. 108732, 2022.
[88] CWRU Bearing Data Center, "Case Western Reserve University Bearing Data Center," https://engineering.case.edu/bearingdatacenter, 2024, accessed: 2026-07-04.
[89] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational Knowledge Distillation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3967–3976.
[90] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
[91] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.
[92] N. Roth *et al.*, "Glow: Graph Lowering Compiler Techniques for Neural Networks," *arXiv preprint arXiv:1805.00907*, 2018.
[93] B. Wang, Y. Lei, N. Li, and N. Li, "A Hybrid Prognostics Approach for Estimating Remaining Useful Life of Rolling Element Bearings," *IEEE Transactions on Reliability*, vol. 69, no. 1, pp. 401–412, 2020.
[94] C. Lessmeier, J. K. Kimotho, D. Zimmer, and W. Sextro, "Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification," in *European Conference of the Prognostics and Health Management Society (PHM)*, 2016.
[95] X. Leroy, "Formal Verification of a Realistic Compiler," in *Communications of the ACM*, vol. 52, no. 7, 2009, pp. 107–115.
[96] A. Haag, B. Fuchs, A. Kacan, and O. Lohse, "Training LLMs for Generating IEC 61131-3 Structured Text with Online Feedback," in *IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) @ ICSE*, 2025.
[97] D. Saha, B. Chatterjee, S. Gain, M. Sinha, T. Dasgupta, and C. Bhaumik, "STARK: A Comprehensive Framework for PLC Structured Text Assistant for Reasoning and Knowledge," in *Innovations in Software Engineering Conference (ISEC)*, 2026.
[98] H. Koziolek, T. Braun, V. Ashiwal, S. Linsbauer, M. A. Hansen, and K. Grøtterud, "Spec2Control: Automating PLC/DCS Control-Logic Engineering from Natural Language Requirements with LLMs — A Multi-Plant Evaluation," *arXiv preprint arXiv:2510.04519*, 2025.
[99] Microsoft, "ONNX Runtime," https://github.com/microsoft/onnxruntime, 2024, cross-platform inference engine for ONNX models.
[100] ISO/IEC JTC 1/SC 42, "ISO/IEC TS 22440: Artificial Intelligence — Functional Safety and AI Systems," International Organization for Standardization, Tech. Rep., 2026, under development; first international standard for AI functional safety.
[101] NVIDIA Corporation, "NVIDIA Halos for Robotics: A Full-Stack Functional Safety System for Physical AI," https://developer.nvidia.com/blog/inside-nvidia-halos-for-robotics-a-full-stack-functional-safety-system-for-physical-ai/, 2026, iEC 61508 SIL 2 / ISO 13849 PL d certification framework.

Fuyue Liu is a sophomore at the School of Mechanical and Electrical Engineering, Guilin University of Electronic Technology (GUET), Guilin, China. His research interests include neural network compilation, the information-theoretic and type-theoretic foundations of certified deployment, industrial automation, formal verification of PLC software, and knowledge distillation for edge deployment.

TABLE XXVIII
RESULTS SUMMARY: CORE EXPERIMENTS (E1–E20, E52–E68), CAPACITY-THEORY VERIFICATION (E-T1–E-T9), AND VALIDATION EXPERIMENTS (V1–V7)

Exp	**Description**	**Metric**	**Result**
Core Experiments (E1–E20, E52–E57, E60–E62):			
E1	Teacher vs Student	Acc. (test)	99.93 vs 99.93%
E2	KAN/MLP/SVM/RF	Acc. (sub)	All 100%
E3	KD Ablation	VRM, RKD, Hinton, No-KD	99.9/99.9/99.9/24%
E4	LUT Precision (3-way)	DP-Opt @10pt	**+18.3%** vs uniform
E5	Compiler Scalability	Width/Depth/Grid	11/12 pass; mem bottleneck
E6	Classification	Agree.	**100%**, max logit 3.65
E6-SMT	Z3 Translation Valid.	Per-node verify	9/11 exact, 2/11 bounded, 0 fail
E7	Cross-Load	0/2/3 hp acc.	99.97/100/99.97%
E8	Optimizer Ablation	Full pipeline	-11.9% mem, +65% speed
E9	Doubleton Arith.	DA vs IA bound	**2.1× tighter**
E10	LUT Sparsity Trade-off	Acc vs Storage	99.96% at 3–50 pts
E11	EMSE Calibration	M_2^{char} = 0.177, $M_2^{\max}$=1.586	98.6% tighter than analytic bound
E12	XJTU-SY Cross-Dataset	Zero-shot / FT 100ep	29.8% $\rightarrow$ 91.7% (+61.9 pp)
E13	Feature Ablation	Time/Freq/DE	Freq 99.93%; All 99.96%
E14	Adversarial Robustness	Noise σ=0–0.2	0% degradation
E15	Adaptive LUT Allocation	Greedy vs Uniform	71.6% $\downarrow$ worst ε
E16	Segment-Aware DA	Seg-DA vs DA	**3.6**× vs. $M_2^{\max}$ (recomp. 2026-08-04)
E17	RTNNIgen Comparison	7-dim. comparison	Correctness model, 2,610× speedup
E18	Paderborn Cross-Dataset	Zero-shot / FT	25.0% $\rightarrow$ 79.4%; Cohen's d domain shift
E19	Method Boundary	3 degradation cond.	DA$\rightarrow$IA, Adaptive$\rightarrow$Uniform, OOM boundary
E20	Motor Load Regression	RMSE	0.011; zero code change
E52	Verification Blind Spot	N=4$\rightarrow$15 safety scan	Acc. 99.93% but safety $<$1 at $N\leq$7; 225 flips
E54	ChebyKAN Z3 Verif.	Per-func. Z3 NRA	**496/512 (96.9%)**, Prop. 3
E55	XJTU-SY FT + SCL	100-ep FT + S7-1200	**91.7%**, 512/512 Z3, 0e 0w SCL
E56	Deep KAN (L=3)	[28,16,8,4] 120ep	**99.60%**, 608/608 Z3, DA 5.66, sub-exp. depth; E-T9-3L: **98.6%**, sound 2.4× (Box-in 81%) + validated 6.1×
E57	CROWN vs. NeuroPLC	KAN[28,16,4] DA/IBP	**DA substantially tighter**, CROWN=IBP for KAN
E58	Per-Func. M_2 / Z3	M_2 proxy, Z3 NRA	mean 0.017, max 0.041; Z3 512/512
E60	FourierKAN SVNN	[28,16,4] K=6, CWRU	**100% acc**, 512/512 Z3, 2184-line SCL; E-T10: validated 4.9× + sound 5.2× (L0-direct)
E61	WaveletKAN SVNN	[28,16,4] S=8, M2-reg	**100% acc**, 512/512 Z3, 2184-line SCL
E62	XJTU-SY Cross-Arch.	FT Fourier/WaveletKAN	**100% acc**, 512/512 Z3 preserved
E53	Sound In-Domain Worst-Case	Real LUT, in-domain search	IA-sound certifies $N\geq$20; high-prob. DA at all N
E63	Sharp k-th Order LUT Bounds	k=2..5, extremal family	**extremal ratio = 1.000000**; soundness ≤ 1
E64	Moment-Robust DA Ratio	Bennett, 4 weight families	R: 4.6/9.5/18.8/37.7 at d=16..1024; outlier $\kappa=\sqrt{d}$: R=1
E65	ε-Separation	3-SAT product-gate encoding	200/200 both directions; spline exact 5×10^{-8}
E66	Verifiability Trichotomy	Rates: tanh/sin/B-spline/ReLU	slopes $-2.04/-2.06/-1.77/-1.03$; tensor LUT N^m
E67	Tier-4 Differential Test	PyTorch vs SCL-sim (emitter)	3,357 samples 100% agree; maxAE 0.47; **caught scale regression**
E68	Compile-Aware PAC	Decomp. + resolution matching	bias slope 0.99 vs h^2; per-act. ratio 0.996–1.000; knee N^* exp. 0.14
Capacity-Theory Verification (E-T1–E-T11):			
E-T1	Curvature-Aware LUT Optimality	Worst-ε vs uniform	**2.75**× cross-func.
E-T2	Compile-Aware Minimax Rate	Deployment risk vs width	Matches W^2_∞ minimax
E-T3	Besov Deployment PAC	Deployment risk	PAC rate (s-smooth)
E-T4	Packing-Line Constant Matching	KT extremal ratio	**1.000000** (N=8..64)
E-T5	Three-Layer Separation	Slope: kink vs C^2	-1.11 vs -2.05
E-T6	Architecture Selection Law	Spectral-vs-LUT crossover	ρ^* Lambert-W closed form
E-T7	Packing Constant c_{ent}	Constructive lower bound	$c_{\text{ent}} \approx 0.079$
E-T8	Width Constant c_*	Numerical estimate	$c_* \lesssim 0.082$; gap $\geq 1.53\times$
E-T9	Soft-Contractive Training	γ post-training	$[1.03, 1.38]$; 98.5%; sound cert. 11.6×
E-T10	FourierKAN Contr. Cert.	Bounded-amp. basis	sound 0.138 (4.9×); **99.96%**
E-T11	WaveletKAN Contr. Cert.	Bounded-amp. basis	sound 0.237 (2.8×); **99.67%**
Validation Experiments (V1–V7):			
V1	Worst-Case Adversarial	5,000 inputs, 100 worst	100/100 preserved, VALIDATED
V2	LLM vs NeuroPLC	SCL code generation	LLM: 6 defects, no weights; NeuroPLC: 0e 0w
V3	DA $\sqrt{d}$ Scaling Law	105 architectures, 7 dims	Pearson r=0.987, slope 0.896
V4	MLP Verification Gap	KAN $[28, 16, 4]$ vs. MLP $[28, 32, 16, 4]$	512/512 vs. 0/48; 4.6× worse (E2E)
V5	ONNX vs NeuroPLC IR	Node count/code	Export fails; $\geq$8,393 vs. 11 nodes
V6	Z3-Verified WCET	S7-1200 CPU 1211C	Total $\leq$2.86 ms, 2.9% of cycle
V7	Verification Blind Spot	N=4$\rightarrow$15 sweep	Acc. 99.93% but safety $<$1 at $N\leq$7; 225 flips

4pt 4pt

V-series: validation

experiments targeting specific reviewer questions (LLM comparison, ONNX incompatibility, scaling law, verification gap, adversarial safety, WCET).

TABLE XXIX
ADAPTIVE VS. UNIFORM LUT ALLOCATION: ERROR AND STORAGE

Metric	N=10	N=15	N=20	N=50
Budget (bytes)	20,480	30,720	40,960	102,400
Uniform worst ε	0.00982	0.00406	0.00220	0.00033
Adaptive worst ε	0.00294	0.00115	0.00061	0.00009
Worst-ε reduction	**70.0%**	**71.6%**	**72.2%**	**73.1%**
Storage at quality parity	—	17,888 B	—	—
Storage saving vs. uniform	—	**41.8%**	—	—
Adaptive N range	$[3, 18]$	$[3, 28]$	$[3, 38]$	$[4, 96]$
Adaptive N median	11	16	21	53

Quality parity: storage needed for adaptive scheme to match uniform's worst-case ε. At N=15 budget, adaptive saves 41.8%. All experiments use 512 B-spline functions from the KAN $[28, 16, 4]$ model.

TABLE XXX
NEUROPLC VS. RTNNIGEN [2]: MULTI-DIMENSIONAL COMPARISON

Dimension	NeuroPLC (Ours)	RTNNIgen
Model Support	MLP + KAN (B-spline LUT)	MLP only (Dense layers)
Correctness	Theorem 1 + DA (2.1×) + Segment-Aware	None reported
PLC Platform	Siemens S7-1200/1500 (SCL)	Beckhoff TwinCAT 3 (ST)
Code Structure	DB+FB integrated SCL, human-readable	FB + binary weights file
Activations	B-spline LUT + SiLU + Softmax + Argmax	Standard (ReLU, tanh, etc.)
Verification	166 tests + TIA V21 (0e/0w)	No test suite in repo
Parameter Handling	Human-readable ARRAY OF REAL	Packed binary blob

TABLE XXXI
TOP-10 FEATURES BY DOMAIN SHIFT MAGNITUDE (CWRU → PADERBORN).

Rank	Cohen's d	Interpretation
9	$+7.40$	Extreme shift (variance-like feature)
7	$+3.83$	Large shift (energy-like feature)
14	$+3.58$	Large shift
13	$+2.53$	Large shift
12	$+1.72$	Large shift
3	$+0.92$	Moderate shift
5	-1.15	Large shift (opposite direction)
10	$+0.80$	Moderate shift
6	$+0.00$	No shift (dimension-invariant feature)
15	0.00	No shift (constant feature)

Mean $|d|$ across 28 features: 0.84. MMD_{RBF} = 0.044 (moderate).
Features 9, 7 dominate: together account for 54% of total shift magnitude.

Positive d = target mean exceeds source mean. Features are z-score normalized per dataset before comparison. The 2 features with $d \approx 0$ are nearly invariant—candidates for domain-robust feature selection in future sensor-agnostic deployments.

TABLE XXXII
NEUROPLC APPLICABILITY BOUNDARIES AND RECOMMENDED MITIGATIONS

Boundary	Condition	Mitigation
DA $\rightarrow$ IA	Balanced weight signs (rare)	Fall back to IA; still valid
Adaptive $\rightarrow$ Uniform	Low curvature diversity	Uniform LUT sufficient
L_{net} explosion	Depth ≥ 6 layers	Monte Carlo; limit depth
Domain shift	Cross-sensor/-bearing/-SR	Fine-tune on target domain

TABLE XXXIII
COMPILER TEMPLATE VERIFICATION—Z3 PROVES SCL TEMPLATES CORRECT FOR ALL POSSIBLE PARAMETERS. (TIER 1, ONE-TIME PROOFS.)

IR Operation	Z3 Result	Time (ms)
MatMul	`UNSAT` (exact equivalence, 16 dim pairs)	27
Add	`UNSAT` (exact equivalence, 8 dimensions)	6
BsplineLUT	`UNSAT` (de Boor bound for all cubic fns)	7
Argmax	`UNSAT` (exact equivalence, 4 dimensions)	17
StandardAct	Analytic proof (transcendental $\exp(x)$)	–
Softmax	Analytic proof (transcendental $\exp(x)$)	–

TABLE XXXIV
END-TO-END COMPOSITIONAL VERIFICATION RESULTS (STUDENTKAN $[28, 16, 4]$, 15-PT LUT).

Metric	Value
Compiler templates proved (Tier 1)	$4/6$
B-spline functions verified (Tier 2)	$512/512$
Composition steps (Tier 3)	9
Trusted computing base (checker)	$\sim$200 lines Python
Per-function LUT error ε	0.00406
IA network bound Δ_{IA}	1.38
DA network bound Δ_{DA}	0.66
DA/IA tightening ratio	$2.1\times$
Certificate valid	Yes
Classification preserved	Yes
Total verification time	1,716 ms

TABLE XXXV
COMPONENT-LEVEL VERIFIABILITY: KAN VS. MLP $[28, 16, 4]$.

Component	KAN	MLP (SiLU)	MLP (ReLU)
Activation functions	512 (B-spline)	16 (SiLU)	16 (ReLU)
Z3-verifiable	512 (100%)	0 (0%)	16 (100%)
Z3 fragment	Polynomial NRA	Transcendental	Piecewise linear
Composition possible?	Yes (Theorem 1)	No	No*

*Even with per-neuron ReLU verified, MLP composition fails: ReLU$(Wx + b)$ creates data-dependent error propagation (ReLU$' \in \{0, 1\}$), breaking the sign-structure preservation that enables DA tightening.

TABLE XXXVI
ERROR PROPAGATION BOUNDS: KAN DA VS. MLP IA. ALL VALUES USE THE SAME PER-COMPONENT LUT ERROR $\varepsilon = 0.0041$ (N=15, M_2=0.177). KAN BOUND FROM SIGN-STRUCTURAL AFFINE ARITHMETIC (EQ. 70); MLP BOUND FROM 3-LAYER INTERVAL PROPAGATION.

Metric	**Value**
Per-function error ε	0.0041
KAN DA bound Δ_{DA}	0.66
KAN IA bound Δ_{IA}	1.38
MLP IA bound Δ_{MLP}	3.04
DA/IA tightening (KAN)	2.1×
MLP IA / KAN DA overhead	4.6× (E2E, recomputed)
MLP IA / KAN IA overhead	2.2×
KAN end-to-end verified	Yes
MLP end-to-end verified	No

TABLE XXXVII
CROSS-DOMAIN PIPELINE GENERALITY: IDENTICAL COMPILER AND VERIFICATION PIPELINE ACROSS THREE FUNDAMENTALLY DIFFERENT DATA DOMAINS —IMAGE CLASSIFICATION (MNIST), LABORATORY BEARING FAULTS (CWRU), AND RUN-TO-FAILURE NATURAL DEGRADATION (XJTU-SY). CHEBYKAN (CHEBYSHEV POLYNOMIAL BASIS) IS INCLUDED AS AN ALTERNATIVE SVNN-COMPLIANT ARCHITECTURE (PROPOSITION 3).

Metric	**MNIST (Image)**	**CWRU (Vib.)**	**XJTU-SY (Nat.Wear)**	**ChebyKAN (Poly.)**	**FourierKAN (E-T10)**	**WaveletKAN (E-T11)**
Accuracy	0.9859[♯]	0.9993	0.9172[*]	1.0000	**0.9996**	**0.9967**
Training	Scratch	KD (VRM)	FT 100ep	Scratch	C2-BV contr. FT[§]	C2-BV contr. FT[§]
Per-func. verified	512/512	512/512	512/512	496/512	**512/512**	**512/512**
M_2 max	0.83[¶]	1.586	27.1[¶]	5.4× M_2	1.324	4.312
Safety margin		1.0×[**]		1.1×	**4.9**×[§§]	**2.8**×[§§]
SVNN Cond. 3	No	No	No	No	No	No
SCL compiled	0e 0w	0e 0w	0e 0w	0e 0w[‡]	**0e 0w**	**0e 0w**

[♯]MNIST digits 0–3, PCA(784→28) + z-score, scratch training, 512/512 verified, DA 0.1199 (E42). [*]Fine-tuned 100 epochs from CWRU checkpoint (E55). [§]Bounded-amplitude basis with soft-contractive projection (E-T10/E-T11). [**]CWRU column is the released checkpoint: *expected-tier* 1.0× (E11 M_2^{char} calibration); no sound certificate ($\gamma = [15.4, 5.3] > 1$, E68). [§§]E-T10/E-T11 certificates: FourierKAN validated floor bound 0.138 (N=15 full-LUT; first-principles 0.110 does not cover the full-set measured 0.126, which includes 2.7% out-of-domain inputs); FourierKAN L0-direct configuration sound 0.130 (N=16 L1 LUT, Box 100%, covers measured 0.091; 2026-08-05); WaveletKAN first-principles sound 0.237 (covers measured 0.157). [‡]ChebyKAN uses Markov-based global M_2 bounds, 5.4× looser than empirical (E54). [¶]MNIST/XJTU M_2 max measured full-φ float64 (40,001 points, 2026-08-05): MNIST 0.83 (scratch-trained, E42); XJTU-SY 27.1 after 100-epoch fine-tuning (E55), consistent with its $\gamma > 1$ and expected-tier-only status; the CWRU column retains the existing folded-weight convention (1.586, E11). FourierKAN M_2 from $\omega^2 \sum k^2(|c_k| + |d_k|)$; WaveletKAN M_2 from $\max_s |c_s|/a_s^2 \cdot 2.602$. All C^2-BV architectures 100% Z3-equivalent verified post fine-tuning (E62). SVNN Cond. 3 (per-layer $\gamma < 1$) is a *training* property, not a pipeline property: unsatisfied by the released checkpoint and by E-T10/E-T11 (second-layer $\gamma > 1$); measured on MNIST (E42, $\gamma = [6.3, 3.2]$) and ChebyKAN (E54, $\gamma = [1.2, 2.1]$) on 2026-08-05 (cond3_complement.json). This table documents *pipeline* generality (compiler/verification domain-independence); contraction is established by the soft-contractive recipe (E-T9–E-T11).

TABLE XXXVIII
COMPARISON OF PLC ML DEPLOYMENT TOOLS

Dimension	**RTNNIgen [2]**	**MLconverter [3]**	**NeuroPLC (this work)**
Input format	Keras Sequential	scikit-learn (DT, MLP)	**PyTorch (5 architectures)**
IR / optimization	None	None	**4-stage IR, 6 passes**
B-spline / KAN	✗	✗	**C^2-BV: 5 architectures**
Siemens SCL	Not validated	ABB-specific	**0e 0w TIA V21**
Correctness	✗	✗	**Type theory + 17 Thm + DA + Z3**
WCET / Safety	✗	✗	**WCET 3.86ms + Non-Interference (Thm E)**
IEC 61508 Aligned	✗	✗	**✓ (Non-Interference + WCET)**

NeuroPLC provides the only design-time correctness guarantee (17 theorems), universal IEC 61131-3:2025 compliance (Lemma 7), and IEC 61508-aligned non-interference proof (Theorem E).

TABLE XXXIX
KAN HARDWARE DEPLOYMENT: LANDSCAPE COMPARISON

Aspect	**KANELÉ (FPGA)**	**LUT-KAN (MCU)**	**KAN-SoC (TinyML)**	**Corrêa (PLC)**	**NeuroPLC (PLC)**
Platform	FPGA	Cortex-M3	MCU	S7-1511TF	S7-1200/1500
Year	2026	2025	2025	2024	2026
Source format	Custom	Post-train	Training	Python	**PyTorch**
PLC code gen	✗	✗	✗	✗	**S7 SCL**
TIA validated	✗	✗	✗	Snap7*	**0e 0w**
Formal cert.	✗	✗	✗	✗	**17 Thm+Z3**
Architectures	KAN	KAN	KAN	KAN	**5 C^2-BV**
Safety mon.	✗	✗	✗	✗	**Alg 3 (auto)**
IEC univ.	✗	✗	✗	✗	**Lem 7**
Non-Interf.	✗	✗	✗	✗	**Thm E**
Speedup	2,700×	7–25×	—	Distrib.	Native OB1

NeuroPLC is the only platform providing: formal certification (17 theorems), universal IEC 61131-3 compliance, auto-generated safety monitor, non-interference proof for IEC 61508, and multi-architecture SVNN coverage (5 C^2-BV variants).

TABLE XL
NEUROPLC IR VS. ONNX IR FOR KAN [28, 16, 4]

Metric	**NeuroPLC IR**	**ONNX (opset 14)**
Graph nodes	**11**	Export *fails*
B-spline representation	Native (BsplineLUT)	No standard operator
Decomposed nodes (primitives)	—	**≥8,393**
Node explosion factor	1×	**≥763×**
Generated code lines	**3,818** SCL	>200,000 C (est.)
Code size	**45.2** KB	≫400 KB (est.)
ONNX Runtime size	**0** KB	**22,528** KB
Fits S7-1200 (50 KB)?	**Yes**	**No** (450× over)
Formal verification ready	**Yes** (Z3 SMT)	No

Experiment I: `torch.onnx.export` with opset 14/17/20 fails at B-spline `einsum`. Decomposed estimate: 512 functions × 15 ONNX primitives per B-spline = 7,680 + 713 structural nodes ≈ 8,393. ONNX Runtime minimal build ≈22 MB (no quantization). SCL line count from actual NeuroPLC output.

TABLE XLI
CERTIFICATION THRESHOLDS: MINIMUM LUT DENSITY FOR EACH GUARANTEE TIER.

Guarantee	**Method**	$N_{\min}$	**Safety**	**SIL Target**	**Evidence**
High-probability	DA	15	1.0×	SIL 2	Sign-balance (Lemma 3); recomputed $\Delta_{\mathrm{DA}} = 0.66$
Sound	IA (global)	22	1.1×	SIL 2	No sign assumptions; $\Delta_{\mathrm{IA}}(N) = 1.38\,(14/(N-1))^2$ (recomputed 2026-08-03: N=15 is *not* a certificate)
Sound (adaptive)	IA (greedy)	15*	1.7×	SIL 2	Adaptive $\varepsilon_{\max}$=0.00115 (E15); recomputed 2026-08-04: N=15 *is* a certificate under adaptive allocation
Soft-contractive	Validated IA	15	11.6×	SIL 3+	E-T9 checkpoint (98.5%); Box-Continuation coverage 99.9%; Z3 512/512; expected tier 493×; sound tier 2.34×
Sound + margin	IA (global)	25	1.4×	SIL 3	safety factor $0.675/0.470 = 1.44\times$

$N_{\min}$ = minimum LUT points for safety factor ≥ 1.0. *Adaptive allocation (E15) at the N=15 storage budget. $M_2^{\mathrm{char}} = 0.177$ (E11); half-margin = 0.675; all values recomputed from the released checkpoint (2026-08-03) for KAN $[28, 16, 4]$ on S7-1200 CPU 1211C. Segment-aware rows follow the recomputed Seg-DA analysis (§IV-B10, 2026-08-04).